VERSION 3.0 • DEFINITIVE LEARNING EDITION • PYTHON & AI SYSTEMS ARCHITECTURE

# HIGH-LEVEL DESIGN

## For Python & AI Infrastructure Engineers

An authoritative, end-to-end master textbook for designing, implementing, defending, and operating mission-critical distributed software and AI systems at planetary scale.

### 25 Deep-Dive Core Chapters

Foundations, Distributed Data, Consensus (Raft), Event Streams, Distributed Auth, Storage Engines, and Microservices.

### Dedicated AI Systems Perspectives

In-depth AI infra in every chapter: KV-cache memory math, PagedAttention, Continuous Batching, Vector Search, and GPU cluster economics.

### Production-Grade Python & SQL

Real distributed implementations: atomic Redis Lua scripts, durable Saga state machines, hybrid RRF search, and transactional outboxes.

### Symmetrical Architectural Diagrams

Publication-grade orthogonal SVG diagrams with balanced bus routing, VPC/AZ isolation boundaries, and zero collisions.

### 6 Full-Blueprint Production Workshops

Exhaustive architectures: E-Commerce, Chat & Presence, Video Streaming, Ride-Hailing, Distributed LLM Serving, and Enterprise RAG/Agents.

### Staff/Principal Interview Playbook

45-minute pacing guide, interviewer negotiation scripts, dimensional estimation formula sheets, FAANG rubrics, and 30-day curriculum.

FOUNDATIONAL PREFACE

# How to Use This Master Guide

*Moving from hand-waving buzzwords to battle-tested distributed engineering & AI infrastructure.*

Most system design preparation fails because candidates treat high-level design (HLD) as a multiple-choice vocabulary quiz. They throw product names like *"Kafka,"* *"Redis,"* and *"Cassandra"* at a whiteboard without understanding the underlying invariants, the network boundaries, the failure states, or the financial cost of their decisions.

In modern production, systems fail at the intersection of scale and state: a network partition delaying heartbeats, an unthrottled retry storm saturating connection pools, an asynchronous cache fill writing stale data over a transaction, or a GPU cluster suffering sudden CUDA Out-of-Memory crashes during high-concurrency LLM decoding.

### 💡 THE THREE-LEVEL LEARNING ARCHITECTURE

To make learning effortless without sacrificing engineering depth, every chapter is structured across three distinct tiers:

- **Level 1: Core Understanding.** First-principles mental models, intuitive analogies, and essential component architectures. Focuses on the *why* and the core invariant.
- **Level 2: Engineering Depth.** Fully typed Python implementations, internal data structures, mathematical derivations (Little's Law, KV-cache sizing), failure scenarios, and concrete trade-offs.
- **Level 3: Advanced Deep Dive.** Hardware physics (PCIe DMA, GPU HBM, NVLink), distributed consensus edge cases (Raft log matching, fencing tokens), and research-level systems papers.

### THE DUAL-TRACK ARCHITECTURE CURRICULUM

**Track 1: Mission-Critical Distributed Systems.** Master the core invariants of high-throughput web architecture: horizontal scaling, Little's Law concurrency, cache-aside consistency, consistent hashing rings, consensus under Paxos/Raft, linearizable storage, and durable event logs.

**Track 2: AI Infrastructure & Machine Learning Systems.** Understand the physical and mathematical realities of serving deep learning models: compute-bound prefill vs. memory-bandwidth-bound decode, GPU HBM budget calculations, PagedAttention virtual block tables, Radix tree prefix caching, HNSW vector search graph sharding, and transactional tool execution in autonomous agent workflows.

## The Master Syllabus & Learning Roadmap

| Section | Chapters | Core Systems Competency & AI Infrastructure Angle |
|---|---|---|
| **I. Scale & Ingress** | 01. Scalability<br>02. Load Balancing<br>03. Caching<br>04. CDN | Ingress routing, Little's Law for GPU inference, prefix-aware LLM routing, semantic vector caching, and model weights CDN caching. |
| **II. Distributed Data** | 05. Database Sharding<br>06. Database Replication<br>07. Consistent Hashing<br>08. CAP Theorem<br>09. PACELC<br>10. Consistency Models | Data placement, split-brain fencing epochs in PyTorch distributed training, quorum mechanics, vector DB sharding, and multi-agent memory consistency. |
| **III. Async Systems** | 11. Kafka & Pub/Sub<br>12. Event-Driven Architecture | Partitioned append-only logs, consumer group offset commits, streaming LLM token chunks over event buses, and transactional outbox for agent tool execution. |
| **IV. Services & Edge** | 13. Microservices<br>14. API Gateway<br>15. Rate Limiter Design | Domain-driven boundaries, BFF aggregation, dual RPM/TPM token bucket rate limiting, and prompt injection shield edge proxies. |
| **V. Resilience & Primitives** | 16. Bloom Filters<br>17. Circuit Breaker Pattern<br>18. Saga Pattern<br>19. Blob & Object Storage | Probabilistic hashing pre-checks for embeddings, cascading GPU worker breaker protection, multi-step LLM pipeline sagas, and Safetensors DMA zero-copy loading. |
| **VI. Quantitative Design** | 20. Back-of-the-Envelope Estimation | Dimensional analysis, Little's Law in-flight work, KV cache memory formulas, and GPU cloud economics. |
| **VII. Modern Primitives** | 21. Distributed Search<br>22. Consensus (Raft)<br>23. Stream Processing (Flink)<br>24. Distributed Auth & Zanzibar<br>25. Storage Engines (LSM vs B-Tree) | Inverted indexes & vector search (HNSW), Raft leader election, stateful event-time windows, Google Zanzibar ReBAC for RAG document ACLs, and LSM-tree write performance. |
| **VIII. Production Blueprints** | Workshops A through F | Exhaustive, multi-page blueprints: Global E-Commerce, Chat & Presence, Video Platform, Ride-Hailing Dispatch, Distributed LLM Serving Platform, and Enterprise Multi-Tenant RAG/Agents. |
| **IX. Interview Readiness** | Playbook, Rubrics, Practice Bank, Glossary, 30-Day Plan | Staff/Principal interview pacing, FAANG evaluation criteria, quantitative estimation cheat sheet, 70+ term glossary, and 4-week deliberate practice curriculum. |

MASTER SYLLABUS

# Table of Contents

*Complete architectural roadmap spanning 25 core chapters, 6 production workshops, and interview playbooks.*


## PART I: SCALING & INGRESS

**01. Scalability** — Stateless compute tiers, saturation boundaries • AI: LLM inference capacity, TTFT & continuous batching

**02. Load Balancing** — L4 Maglev vs L7 Envoy, health checks • AI: Prefix-aware LLM routing

**03. Caching** — Cache-aside, stale-fill race, Dogpile protection • AI: Semantic vector caching

**04. Content Delivery Networks (CDN)** — Edge POPs, origin shielding • AI: Model weights & embedding distribution

## PART II: DISTRIBUTED DATA

**05. Database Sharding** — Shard directory, horizontal partitioning • AI: Multi-billion vector DB sharding

**06. Database Replication** — Sync vs async followers, fencing epochs • AI: Distributed PyTorch training coordination

**07. Consistent Hashing** — Virtual nodes, clockwise continuum • AI: KV-Cache prefix affinity routing

**08. CAP Theorem** — Network partitions, linearizability trade-offs • AI: Agentic vector memory consistency

**09. PACELC** — Normal vs partitioned trade-offs • AI: Speculative decoding latency vs consistency

**10. Consistency Models** — Strict linearizability, causal order • AI: Multi-agent state synchronization

## PART III: ASYNCHRONOUS SYSTEMS

**11. Kafka & Publish/Subscribe** — Partitioned commit logs, consumer groups • AI: Token streaming pipelines

**12. Event-Driven Architecture (EDA)** — Event sourcing, idempotency • AI: Transactional outbox for agent tool execution

## PART IV: SERVICE ARCHITECTURE & EDGE

**13. Microservices Architecture** — Domain boundaries, service mesh • AI: Modular LLM agents & tool microservices

**14. API Gateway** — Edge routing, auth termination, BFF • AI: Prompt injection shields & token budgeters

**15. Rate Limiter Design** — Token bucket, sliding window in Redis • AI: Dual RPM / TPM rate limiting for LLMs

## PART V: RESILIENCE & PRIMITIVES

**16. Bloom Filters** — Probabilistic hashing, bit vectors • AI: Fast deduplication in pre-training corpora

**17. Circuit Breaker Pattern** — Closed, Open, Half-Open state machines • AI: Upstream LLM provider failover & breakers

**18. Saga Pattern** — Orchestrated workflows, compensations • AI: Multi-step agent tool rollbacks

**19. Blob & Object Storage** — Presigned multipart S3 uploads • AI: Safetensors zero-copy DMA model loading

## PART VI: QUANTITATIVE SYSTEM DESIGN

**20. Back-of-the-Envelope Estimation** — Dimensional analysis, Little's Law • AI: GPU HBM memory math & cluster economics

## PART VII: MODERN PRIMITIVES

**21. Distributed Search & Inverted Indexes** — BM25 scoring • AI: HNSW vector search & hybrid RRF ranking

**22. Consensus Protocols: Paxos & Raft** — Leader election, log replication • AI: GPU cluster node membership

**23. Stream Processing: Apache Flink** — Event-time watermarks, RocksDB state • AI: Real-time feature stores

**24. Distributed Auth: Google Zanzibar** — ReBAC relationship tuples • AI: Enterprise document ACL filtering for RAG

**25. Storage Engines: B-Trees vs. LSM-Trees** — Page splits vs SSTables • AI: High-throughput telemetry & vector stores

## PART VIII: PRODUCTION SYSTEM DESIGN BLUEPRINTS

**Workshop A: Global E-Commerce & Flash Sales** — 100k checkout RPS, atomic Redis inventory reservation.

**Workshop B: Real-Time Distributed Chat & Presence** — 50M concurrent WebSockets, ScyllaDB history logs.

**Workshop C: Global Video Streaming & Processing** — Direct S3 multipart ingest, FFmpeg transcoding.

**Workshop D: Ride-Hailing & Geospatial Engine** — 250k location pings/sec, Uber H3 hexagonal indexing.

**Workshop E (AI Infra): Production LLM Serving Platform** — vLLM continuous batching, PagedAttention block manager.

**Workshop F (AI Systems): Enterprise Multi-Tenant RAG & Agents** — Hybrid search, Zanzibar ReBAC ACL filter, tool outbox.

## PART IX: STAFF/PRINCIPAL INTERVIEW READINESS & APPENDICES

**Staff & Principal Interview Rubrics** — FAANG scoring criteria across Systems & AI tracks.

**Mid-Interview Curveball Transcripts** — Live negotiation scripts for 3 tough whiteboard challenges.

**Integrated Practice Bank** — 12 comprehensive end-to-end design prompts.

**Appendix A: Distributed Systems & AI Glossary** — 70+ authoritative architectural definitions.

**Appendix B: 30-Day Engineering Mastery Curriculum** — Deliberate practice syllabus.

EXECUTIVE PLAYBOOK

# The Repeatable 9-Step HLD Interview Method

*How Staff and Principal engineers steer a 45-minute whiteboard session through 9 distinct steps.*

System design interviews are not exams with one secret "right" answer. They are **collaborative technical consultations** where the interviewer evaluates how you navigate ambiguity, enforce invariants, bound resources, and defend engineering trade-offs across 9 disciplined, sequential steps.

### THE 45-MINUTE CLOCK ALLOCATION STRATEGY (9 DISCIPLINED STEPS)

**00:00 - 04:00 • Step 1: Clarify Scope & Functional Requirements.** Separate the core revenue-driving use-cases from secondary features. Establish explicit system boundaries, tenant isolation requirements, and user personas.

**04:00 - 08:00 • Step 2: Define Non-Functional SLOs & Invariants.** Establish hard Service Level Objectives: p99 latency targets (e.g. <100ms for web, <800ms TTFT for LLM inference), availability tiers (99.99%), and consistency models (linearizable vs. eventual). For AI systems, establish token generation budgets and safety boundaries.

**08:00 - 13:00 • Step 3: Back-of-the-Envelope Estimation & Dominant Constraints.** Calculate average and peak RPS, write-to-read ratio, egress bandwidth, storage growth over 3 years, and in-flight concurrency ($L = \lambda W$). For AI systems, calculate GPU HBM memory footprint ($M_{\text{weights}} + M_{\text{KV}} + M_{\text{activations}}$) and determine whether the system is compute-bound or memory-bandwidth-bound.

**13:00 - 17:00 • Step 4: Core Interface Contracts & Data Schemas.** Define exact REST / gRPC endpoints, idempotency keys, and asynchronous event schemas. Specify database entities, primary keys, and partitioning keys.

**17:00 - 25:00 • Step 5: Symmetrical High-Level Architecture.** Lay out the end-to-end topology: ingress routing (L4/L7), stateless application compute fleets, message queues, and partitioned data tiers. Keep data flow unidirectional and avoid premature complexity.

**25:00 - 32:00 • Step 6: Deep Dive into Critical Bottlenecks & Concurrency.** Zoom into the highest-risk component: distributed locking, cache stampede mitigation, consensus leader election, or GPU KV-cache radix prefix routing. Walk through concrete race conditions and state invariants.

**32:00 - 37:00 • Step 7: Failure Modes, Partitions & Self-Healing.** Systematically inject faults: primary node crashes, network partitions, split-brain scenarios, and dependency brownouts. Detail circuit breaker probe policies, dead-letter queues, and reconciliation workers.

**37:00 - 41:00 • Step 8: SRE Observability, Telemetry & Guardrails.** Enforce the Four Golden Signals (Latency, Traffic, Errors, Saturation). Detail synthetic canaries, distributed tracing (W3C TraceContext), replication lag alerts, and AI guardrail evaluation metrics.

**41:00 - 45:00 • Step 9: Trade-Off Defense, 10x Scale Evolution & Cloud Economics.** Explain why simpler alternatives were rejected. Describe architectural bottlenecks when traffic grows by 10x, and present estimated monthly infrastructure and GPU operational costs.

**THE "ANTI-HAND-WAVING" RULE**

Never say: *"We'll use a cache to make it fast."*
Say instead: *"We will place a Redis Cluster in front of Postgres for product detail reads using a cache-aside pattern with a 5-minute TTL and 20% jitter. To prevent stale-fill races on write invalidation, cache updates will enforce a monotonic row version via an atomic Lua script."*

PART I • SCALING & INGRESS • CHAPTER 01

# 01. Scalability

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

A small restaurant can add shifts, improve the kitchen, or open another branch. Software has the same choices: improve one machine, add machines, or change the work so that less capacity is needed.

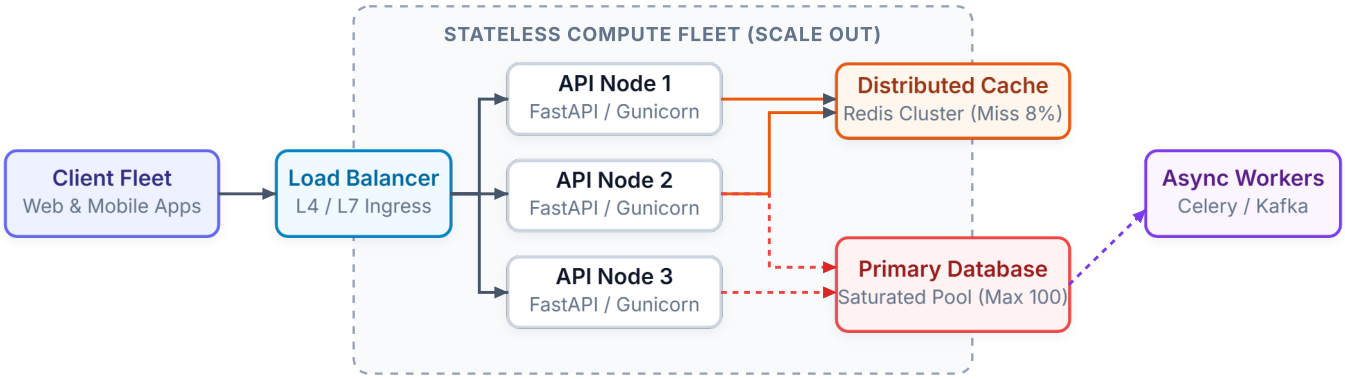
### 🤖 AI INFRASTRUCTURE LENS: LLM INFERENCE CAPACITY, TTFT & CONTINUOUS BATCHING

**Prefill vs. Decode Phase Disparity:** In traditional APIs, every request consumes relatively uniform CPU/memory. In Large Language Model (LLM) serving, requests execute across two fundamentally distinct operational regimes:

- **Prefill (Prompt Processing):** Compute-bound. The model ingests $P$ prompt tokens in parallel using General Matrix-Matrix Multiplications (GEMM), driving tensor cores near peak FLOPS.
- **Decode (Token Generation):** Memory-bandwidth-bound. The model generates tokens sequentially, loading all $W$ billion parameters from GPU High Bandwidth Memory (HBM) to compute just *one* token using General Matrix-Vector Multiplication (GEMV).

**Little's Law for GPU Inference Fleets:** Long-run concurrency $L = \lambda \times W$, where total execution time $W = \text{TTFT} + (N_{\text{out}} \times \text{TPOT})$. If an 8x H100 GPU pod processes an arrival rate of $\lambda = 200$ req/s with a Time-To-First-Token (TTFT) of 150 ms and generates 250 output tokens at a Time-Per-Output-Token (TPOT) of 15 ms ($W = 0.15 + 3.75 = 3.9$ s), in-flight concurrency is $L = 200 \times 3.9 = 780$ active sequences.

**The Static Batching Trap vs. Continuous Batching:** Naive static request batching waits for all sequences in a batch to reach their EOS (End-Of-Sequence) token. If one sequence generates 10 tokens and another generates 1,000, 99% of the batch slots sit idle as "GPU bubbles". Production engines (vLLM, TensorRT-LLM) use **Continuous Batching (Iteration-Level Scheduling)**: on every single forward token step, completed sequences are evicted from memory and new prompt prefill sequences are dynamically injected into the batch.



### 🎯 Learning Outcomes

- Distinguish scalability, performance, elasticity, availability and reliability.
- Translate product usage into workload: throughput, concurrency, payload, read/write mix and burstiness.
- Find bottlenecks with measurements rather than adding servers blindly.
- Choose vertical, horizontal, functional and data-layer scaling deliberately.
- Estimate capacity with Little's Law and explicit headroom assumptions.

the same workload.

## 1.1 What scalability means

Scalability is a system's ability to sustain a larger workload while meeting stated service objectives as capacity is increased or work is reorganised. The phrase "while meeting objectives" matters. A service that handles ten times the traffic but lets p99 latency grow from 200 ms to 20 seconds has not scaled successfully for users. Performance describes behaviour at a particular operating point: latency, throughput, resource use and efficiency. Scalability describes how those measures change as workload or resources change. Elasticity is the operational ability to add and remove capacity as demand changes. Availability describes whether the service can be used; reliability describes whether it continues behaving correctly over time. These qualities interact but are not synonyms. A useful engineering statement is measurable: "At 12,000 requests per second, with a 70:30 read/write mix and 2 KB median response, the service must keep p95 below 250 ms, errors below 0.1%, and sustain one availability-zone loss." This makes capacity and architecture decisions testable.

## 1.2 Describe the workload before drawing boxes

User count alone is not a capacity plan. Separate registered users, daily active users, peak concurrent users and requests per active user. Then distinguish average traffic from peak traffic. A service with a daily average of 1,000 RPS and a peak of 15,000 RPS has a very different capacity problem from a flat 1,000 RPS service. Characterise request cost. A small cacheable profile read, an image upload, a full-text search and a report generation endpoint are not equivalent requests. Record payload size, CPU time, database round trips, downstream calls, response size, and whether the operation can be asynchronous. Also identify traffic shape: smooth, bursty, periodic, flash-crowd or highly skewed around a few hot objects.

- Throughput: completed work per unit time, such as requests/second or messages/second.
- Latency: elapsed time for an operation. Report distributions such as p50, p95 and p99, not only the mean.
- Concurrency: work simultaneously in flight. It is not the same as requests per second.
- Saturation: how close a constrained resource is to its useful capacity.
- Workload mix: reads versus writes, small versus large payloads, synchronous versus asynchronous work.

## 1.3 Little's Law and why concurrency matters

Little's Law states L = lambda x W for a stable system: the long-run average number of items in the system equals arrival rate multiplied by average time spent in the system. If a service completes 800 requests/second and mean time in the service is 0.25 seconds, average in-flight work is about 200 requests. If latency rises to 1

second at the same arrival rate, in-flight work rises to about 800. This relationship explains why a slow dependency can exhaust connection pools and worker slots even when CPU is not saturated. Requests spend longer waiting, so concurrency grows. Queues then lengthen, timeouts occur, and retries can add more work. A design should bound queues and concurrency rather than allowing unbounded accumulation. L = lambda x W Example: 800 requests/s x 0.25 s = 200 average in-flight requests.

## 1.4 Vertical, horizontal and functional scaling

Vertical scaling (scale up) adds resources to a machine: CPU, RAM, disk IOPS or network capacity. It is operationally simple and often a good first step, especially when a workload benefits from shared memory or a larger database instance. It has a hardware ceiling, creates a large failure domain and can become expensive at the high end. Horizontal scaling (scale out) adds independent instances. It is effective when work can be partitioned or requests are independent. It requires service discovery, load balancing, stateless request handling, distributed coordination and careful treatment of shared dependencies. Ten API instances behind a load balancer do not create ten times the database capacity. Functional scaling decomposes different kinds of work so they can be scaled independently. API serving, image processing, search indexing and notifications may need separate worker pools. Data scaling can use replicas, partitioning or sharding. These methods can be combined; the right choice follows the measured bottleneck.

Approach What grows Good fit Primary cost Vertical Resources on one node Simple stateful service or DB Ceiling, expensive large nodes Horizontal Number of nodes Independent requests / partitionable work Coordination and shared-state complexity Functional Separate component capacity Different workload profiles More operational boundaries Data partitioning Data ownership across nodes Storage or write bottleneck Routing, rebalancing, cross-partition work Traffic spike Load balancer More API replicas DB connection pool

throughput limit.

## 1.5 Statelessness and shared state

Horizontal application scaling is easiest when any healthy instance can handle any request. An instance should not be the only place where a user session, uploaded file or workflow state exists. Store durable business state in an authoritative data store; use a shared cache or session store for ephemeral shared state; use signed tokens only when their revocation and expiry properties fit the requirement. Stateless does not mean "the system has no state." It means a replaceable application instance does not own irreplaceable state. A server can keep local caches, connection pools and temporary computation, provided requests remain correct when another instance handles the next request. Watch out: Sticky sessions can hide state coupling rather than solve it. They make failover and scale-in harder, and can create uneven load. Prefer externalised state unless there is a measured reason for affinity.

## 1.6 Bottleneck analysis and the saturation chain

A system is constrained by the narrowest critical resource on the path. A request may traverse an API server, Redis, a primary database, a payment provider and an object store. Each hop consumes capacity and contributes latency. A database connection pool of 100 connections can cap the system even if 30 API instances are idle. A third-party API quota can be the true throughput ceiling. Use the request path to build a resource budget. For

each dependency, record capacity, utilisation, queue depth, timeout, retry policy and failure mode. Test what happens when a component is slow, not only when it is completely down. High utilisation near a hard limit can cause disproportionate latency because queues grow rapidly as utilisation approaches 100 percent.

- CPU-bound: profile hot code, reduce algorithmic cost, use processes for CPU parallelism in conventional

CPython, or move compute to specialised workers.

- Memory-bound: profile retained objects, stream/batch data, cap caches and avoid loading entire datasets.
- I/O-bound: use asynchronous I/O or bounded thread pools, connection pooling and parallelise independent

calls carefully.

- Database-bound: inspect query plans, indexes, locks, connection saturation, read/write mix and cache

effectiveness.

- Network-bound: reduce payloads, use compression where appropriate, keep connections reusable and move

static delivery to a CDN.

## 1.7 Capacity estimation with headroom

Capacity tests should measure a service under a realistic workload mix. If one API instance sustains 250 RPS while meeting its latency/error SLO at 65 percent CPU, do not plan on its theoretical peak. If required peak is 8,000 RPS and the safe capacity target is 180 RPS per instance, start with ceil(8,000 / 180) = 45 instances, then add capacity for failure domains, deployment surge and uncertainty. Headroom is not an arbitrary percentage. Derive it from failure requirements and traffic variability. If one availability zone can be lost, the remaining zones must carry the peak load. If deployments temporarily run old and new versions together, budget for surge capacity. If traffic is forecastable, scheduled pre-scaling may be safer than waiting for CPU alarms. Required instances = ceil(peak RPS / measured safe RPS per instance) Example: ceil(8,000 / 180) = 45 instances before zone-failure reserve.

## 1.8 Backpressure, queues and graceful degradation

When arrival rate exceeds service rate, a queue grows. A queue absorbs bursts but cannot fix a permanent capacity deficit. If arrivals average 1,000 tasks/s and workers complete 700 tasks/s, backlog grows by 300 tasks/s. Eventually storage, latency or deadlines fail. Monitor queue age as well as depth: 10,000 recent tasks may be healthier than 500 tasks waiting for an hour. Backpressure makes overload explicit. Use bounded queues, concurrency limits, admission control, load shedding and clear responses such as 429 or 503 with Retry-After where appropriate. Degrade optional features first: return a cached recommendation, delay analytics, or omit a noncritical enrichment rather than allowing checkout to fail because an optional service is slow.

## 1.9 Autoscaling: signals, lag and oscillation

Autoscaling policies translate observed signals into capacity changes. CPU and memory are useful for compute-heavy services; queue age is useful for workers; request concurrency and latency can be better indicators for I/O-heavy APIs. A single metric can mislead: CPU can be low while all request handlers are blocked on a

downstream dependency. Scaling has delay. A new node must start, load configuration, warm connections and pass readiness checks. Reactive autoscaling cannot rescue a sudden flash crowd if current capacity is far below demand. Use minimum baseline capacity, predictive or scheduled scaling for known events, cooldowns and step limits. Avoid oscillation by smoothing signals and separating scale-out thresholds from scale-in thresholds.

## 1.10 Python lab: bounded concurrency and capacity

intuition The following sketch illustrates why a bounded semaphore can protect an expensive dependency. It is not a complete server admission-control implementation; production code should also propagate deadlines, return explicit overload responses and measure wait time.

### 💻 Python / implementation sketch

```
import asyncio
# At most 80 concurrent operations may use the dependency.
```

DB_SLOTS = asyncio.Semaphore(80)

```
async def read_customer(customer_id: str):
    async with DB_SLOTS:
        # Replace with an async database call.
        return await repository.fetch_customer(customer_id)
# Capacity is now bounded instead of allowing an unlimited
# number of requests to queue on the database connection pool.
```

## 1.11 Common design mistakes

- Sizing from registered users instead of peak request workload.
- Assuming adding application replicas solves a database, cache or provider bottleneck.
- Using average latency when p99 determines user-visible timeouts.
- Allowing unbounded queues and retries during overload.
- Scaling on CPU alone for an I/O-bound workload.
- Ignoring failure-domain capacity: a design that works only when every node is healthy is not resilient.
- Treating autoscaling as an alternative to capacity planning and load testing.

### 💻 Extended Python lab

```
from math import ceil
def required_instances(peak_rps, safe_rps_per_instance,
                       failure_reserve=0.20):
    if peak_rps <= 0 or safe_rps_per_instance <= 0:
        raise ValueError("rates must be positive")
    base = ceil(peak_rps / safe_rps_per_instance)
    return ceil(base * (1 + failure_reserve))
```

print(required_instances(8_000, 180)) # 54 # Estimate average in-flight requests using Little's Law. rps = 800 mean_latency_seconds = 0.25 print(rps * mean_latency_seconds) # 200 Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code.

Applied design walkthrough A photo-sharing product expects 4 million daily active users. Each user opens the feed 12 times/day, uploads

## 0.2 photos/day and scrolls 8 pages/day. A product campaign creates a 6x peak over average feed-read traffic.

Start by estimating average feed reads: 4,000,000 × 12 / 86,400 = about 556 RPS. Peak feed reads are about 3,334 RPS. This is not the final API RPS: each feed read may fan out to cache, ranking, profile and media metadata calls. Define whether the estimate counts external requests or internal service calls. Then load-test one feed-serving instance, model cache hit ratio and downstream query amplification, and size the bottleneck rather than simply multiplying API instances.

### Interview questions and answer expectations

**Q. Explain scalability versus performance and elasticity.**

**Q. A service is slow at 30% CPU. What do you investigate?**

**Q. Why does adding 10 API replicas sometimes reduce reliability?**

**Q. Estimate instances for a peak workload and one-zone failure.**

**Q. When would you choose vertical scaling even in a distributed system?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Calculate average and peak RPS for 2.5 million DAU, 18 reads per day and a 4x peak factor.
- A service receives 2,000 RPS at mean latency 350 ms. Estimate average in-flight work.
- Design a load-shedding policy for a report-generation service during a 5x traffic spike.

Operational metrics and signals

- RPS and completion throughput by endpoint
- p50/p95/p99 latency and timeout rate
- CPU, memory, disk and network saturation
- Queue depth and oldest-message age
- Database pool wait, query latency and lock time
- Per-dependency latency, error rate and retry volume

Chapter mastery checklist

- Scalability is workload growth while meeting explicit SLOs.

- Use Little's Law to reason about in-flight work.
- Scale the measured bottleneck, not the most visible tier.
- Horizontal scaling requires replaceable instances and explicit shared-state design.
- Bound queues, concurrency and retries; keep operational headroom.

## 🔬 Advanced Engineering Clinic

This clinic moves from the slogan “add more servers” to a capacity model. The key questions are: where does work queue, what resource saturates first, and what happens when a failure removes capacity while demand remains unchanged? Arrivals: peak rate λ Admission control priority + request budget Bounded queue max wait + max depth Worker / API fleet c instances × μ each Load shed / 429 / degrade queue full / deadline expired Downstream dependency Queue age • p99 • saturation rejection rate • dependency latency

Queueing theory beyond Little’s Law Little’s Law, L = λW, relates average in-flight work L, arrival rate λ and time in system W for a stable system over a suitable observation interval. It does not tell you whether the system is stable, nor does it predict tail latency. For that, compare arrival rate with service capacity and measure queue behaviour under realistic variability. For c identical workers, each completing μ jobs/second, nominal service capacity is cμ. Define utilisation ρ = λ/(cμ). As ρ approaches 1, there is almost no slack for bursts, retries or slow requests. In queueing systems, waiting time often rises non-linearly near saturation. A fleet running at 99% average CPU can have severe p99 latency even when average throughput appears adequate. Capacity planning should therefore target safe utilisation measured under the actual workload mix. The safe point is empirical: load-test with production-like payload sizes, downstream latency, connection pools and concurrency. Add headroom for a failed zone, rolling deployment surge and forecast error rather than multiplying by a random comfort factor. Utilisation: ρ = λ / (c × μ) In-flight work: L = λ × W Stable queue requires long-run arrival rate below sustainable service rate. Signal What it reveals Action Queue depth rising Arrival rate exceeds service rate over time Add capacity, reduce work or reject demand

Signal What it reveals Action Queue age rising Users wait longer even if depth is modest Enforce deadlines; prioritise work CPU low, latency high Likely I/O or downstream wait Inspect pool waits and dependency spans CPU high, queue stable Compute constrained but possibly healthy Benchmark safe per-instance throughput Concurrency limits are capacity controls Every downstream resource has a finite concurrency budget: database connections, HTTP connection pools, file descriptors, worker slots and provider quotas. If an API accepts 20,000 concurrent requests but the database pool has 100 connections, the excess requests wait somewhere. If that waiting room is unbounded, memory and latency grow while the service appears to accept traffic. Use bounded semaphores and deadlines at dependency boundaries. The timeout budget must be hierarchical: a request with a 1-second deadline cannot safely perform three sequential downstream calls each with a 1-second timeout. Propagate remaining time, not a fresh full timeout at every hop. A timeout should release resources and produce a defined outcome. Bulkheads isolate workload classes. A slow report export should not consume every request slot needed for login or checkout. Separate pools, queues and concurrency budgets by priority or dependency. Isolation is more reliable than hoping a clever load-balancer algorithm will fix contention inside the process. Advanced implementation note

```
import asyncio
class DependencyPool:
    def __init__(self, limit: int):
        self._slots = asyncio.Semaphore(limit)
    async def call(self, operation, timeout_s: float):
        try:
            async with asyncio.timeout(timeout_s):
                async with self._slots:
                    return await operation()
        except TimeoutError:
            raise RuntimeError("dependency budget exceeded")
# Give checkout and bulk exports separate pool instances.
```

checkout_pool = DependencyPool(limit=40) export_pool = DependencyPool(limit=5) Failure capacity and graceful degradation A system should be sized against the failure it promises to survive. If three zones each carry one-third of peak traffic, losing one zone leaves two zones to carry 150% of their normal load. The surviving fleet must have that headroom, or the system needs admission control and an explicit degraded mode. Degradation should be designed by business priority. Preserve authentication, checkout state transitions and core reads; defer recommendations, analytics and non-essential enrichment. Return a clear partial response rather than letting optional dependencies consume the entire latency budget. Every degraded response should be observable and have a recovery path. Autoscaling is not a substitute for overload protection. Provisioning has delay: metrics need to be collected, a decision made, instances started, health checks passed and traffic shifted. Bounded queues, rejection, pre-scaling and spare capacity bridge that delay.

Failure Unsafe reaction Safer response One zone lost Keep accepting full traffic until timeouts explode Route away, use reserved capacity, shed low priority work DB saturated Increase API replicas Cap DB concurrency; protect pool; diagnose query pressure Queue backlog grows Allow unlimited enqueue Bound queue; reject or defer according to contract Autoscaler lag Assume new nodes arrive instantly Keep baseline capacity and pre-scale predictable peaks

## ❓ Advanced Design-Review Questions

- Your service meets p95 at 55% CPU but times out during a provider slowdown. Which pool, queue and

deadline measurements would you inspect first?

- Design a one-zone-loss capacity plan when traffic is highly bursty and autoscaling takes three minutes.

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

PART I • SCALING & INGRESS • CHAPTER 02

# 02. Load Balancing

*CORE SYSTEM DESIGN*

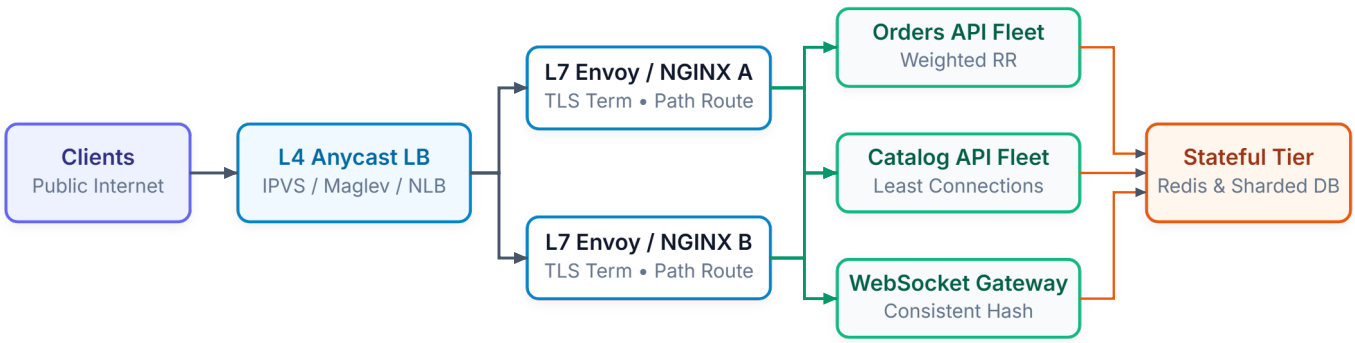
### 💡 INTUITIVE MENTAL MODEL

A dispatcher at a busy clinic does more than send every patient to the next room. It checks which clinicians are available, routes specialist cases correctly and stops assigning work to an unavailable room.

### 🤖 AI INFRASTRUCTURE LENS: GPU-AWARE & PREFIX-AFFINITY LOAD BALANCING

**Why Standard L7 Least-Connections Fails for LLMs:** Standard load balancers treat all backend connections as identical units of capacity. In LLM inference, routing a request without inspecting prompt structure destroys performance:

- **Prefix Cache Locality:** If Worker Node A already cached the Key-Value (KV) tensors for a 16k-token system prompt (e.g. standard developer instructions or a 50-page legal contract), routing a new query to Node A achieves an instant KV-cache hit, dropping TTFT from 800 ms to 12 ms! Routing to Node B forces a complete prefill recomputation.
- **Context-Aware Memory Balancing:** A request with a 64k context window consumes 16x the KV-cache HBM of a 4k query. LLM ingress routers (e.g. Radix-aware proxies, TensorRT-LLM cluster routers) balance by *available GPU HBM blocks* rather than active TCP connection count.
- **Streaming Token Ingress:** LLM output is streamed token-by-token over Server-Sent Events (SSE) or WebSockets. L7 proxies must disable HTTP response chunk buffering (`proxy_buffering off;` in NGINX) to prevent batching tokens into 4KB network buffers, which destroys real-time perceived latency.




### 🎯 Learning Outcomes

- Explain L4 and L7 load balancing and the information each can inspect.
- Choose algorithms based on server capacity, connection duration and request cost.

- Design health checks, draining, failover and load-balancer high availability.
- Understand reverse proxies, TLS termination, sticky sessions and global traffic steering.
- Implement a simple selector and configure an NGINX upstream for FastAPI.

Healthy backend pool Browsers / mobile clients DNS / anycast entry Regional load balancer TLS + routing FastAPI A FastAPI B FastAPI C Shared session / cache Database

backend instances.

## 2.1 What a load balancer does and does not do

A load balancer accepts traffic at an entry point and selects a backend from a configured pool. It can distribute connections or requests, check backend health, terminate TLS, apply routing policies and expose traffic metrics. It improves resource utilisation and can isolate a failed backend from new traffic. It does not automatically make the entire system highly available. The entry point, DNS, configuration, certificates, network, backend dependencies and data tier all remain part of the availability story. It also cannot decide whether a business operation is safe to retry unless it understands the application semantics.

## 2.2 Layer 4 versus Layer 7

Layer 4 balancing operates with transport information such as source and destination addresses, ports and TCP/UDP connection state. It can forward traffic without interpreting the HTTP request path. It is useful for high-throughput transport proxying, non-HTTP TCP services and cases where payload inspection is unnecessary. Layer 7 balancing understands application protocols such as HTTP. It can route by host, path, method, header or cookie; terminate TLS; enforce request-size rules; and split traffic between service versions. This added awareness costs processing and creates more protocol and security responsibilities. L4 and L7 are often composed: a network-level entry point can feed an L7 proxy tier. Dimension L4 L7 Decision inputs IP, port, connection state HTTP host, path, headers, cookies, method Protocols TCP / UDP forwarding HTTP / HTTPS and supported application protocols TLS Often passthrough or provider-specific termination Frequently terminates TLS for inspection and routing Routing richness Connection-level Request-level and content-aware Typical examples Network LB, IPVS, HAProxy TCP mode NGINX HTTP, Envoy, HAProxy HTTP, application LB

## 2.3 Scheduling algorithms and their assumptions

Round robin is appropriate when backends are similar and request costs are relatively uniform. Weighted round robin assigns more traffic to higher-capacity nodes. Least connections is useful when connection lifetimes vary, but connections are only a proxy for work: ten idle keep-alive connections may consume less CPU than one expensive report request. Least response time uses observed latency, but noisy measurements can cause oscillation. Consistent hashing is not a general substitute for a load scheduler. It maps a key, such as a user or cache object, to a stable backend and is useful when affinity or data ownership matters. IP hashing can create skew behind NAT because many users share one public address. Sticky cookies create session affinity but make failover and rebalancing harder. For unequal nodes, weights should reflect measured safe capacity rather than CPU count alone. Capacity can be constrained by memory, database connection budgets, network or downstream quota. A scheduler should also exclude unhealthy nodes and use deterministic tie-breaking where it improves testability. Algorithm Selection rule Good fit Watch out for Round robin Next eligible backend Similar

nodes and work Does not measure load Weighted RR Ratio based on weights Different backend capacities Weights can become stale Least connections Fewest active connections Long-lived or variable-duration connections Connections are not CPU load Weighted least connections Connections adjusted by weight Unequal backend capacities Still only an approximation

Algorithm Selection rule Good fit Watch out for Hash / affinity Stable mapping from a key Session or partition locality Skew and node-set changes

## 2.4 Health checks: liveness, readiness and startup

A liveness probe answers whether a process should be restarted. A readiness probe answers whether it can safely receive new traffic. A startup probe allows slow initialisation without being killed by an overly aggressive liveness timer. These signals should not be conflated. A process can be alive while its database pool is not ready, or ready to serve cached reads while an optional recommendation dependency is unavailable. Active checks periodically call a health endpoint or perform a TCP probe. Passive checks infer health from real request failures, timeouts or connection resets. Active probes detect failure without waiting for user traffic; passive checks observe realistic behaviour. Use thresholds and hysteresis so one transient error does not flap a backend in and out of service. Watch out: A readiness endpoint should be cheap and should represent the ability to serve the intended traffic. If it checks every downstream service synchronously, a failure in an optional dependency can remove every backend and create an outage. New request Health filter remove unhealthy nodes Scheduler RR / weighted / least-connections / hash Backend A Backend B Backend C excluded if unhealthy Active probes + passive errors health state

health/load signals.

## 2.5 Connection lifecycle, draining and long-lived protocols

HTTP/1.1 keep-alive, HTTP/2 multiplexing and WebSockets have different connection behaviour. A load balancer may pick a backend when a connection is established and keep that connection associated with it. A WebSocket can remain open for hours; a short request-per-connection balancing intuition is therefore misleading. Connection draining removes a backend from new assignment while allowing existing work to finish within a deadline. Graceful application shutdown should stop accepting new work, finish or cancel in-flight tasks, close database connections and exit before the orchestrator kills the process. Long-lived connections need reconnect logic and a strategy for re-authentication and message replay after disconnection.

## 2.6 Reverse proxy, gateway and load balancer boundaries

A reverse proxy is an intermediary that accepts client requests and forwards them to upstream servers. It may provide TLS termination, compression, caching, buffering and routing. Load balancing is one function a reverse proxy can provide. An API gateway usually adds API-specific concerns such as authentication, quotas, request transformation, API versioning and consumer policies. A gateway may itself be implemented as a reverse proxy and may use load balancing internally.

Do not draw three boxes merely because three terms appear in a textbook. In a small architecture, one Envoy or NGINX layer can provide several functions. In a larger organisation, separate edge gateway, regional load

balancer and service-mesh proxies may exist because they have different ownership, failure and policy boundaries.

## 2.7 TLS termination and client identity

TLS termination means the load balancer decrypts a client HTTPS connection. It can then inspect HTTP metadata and forward to the backend over a private network or re-encrypt with TLS. TLS passthrough forwards the encrypted stream to a backend that performs the handshake. Re-encryption protects the internal hop and is appropriate where the threat model requires it. When a proxy forwards traffic, the backend needs a trustworthy client identity. Headers such as X-Forwarded-For or Forwarded are only trustworthy when inserted by a trusted proxy and untrusted inbound copies are stripped or normalised. Otherwise clients can spoof their IP address and bypass IP-based rate limits or audit logic.

## 2.8 High availability of the load-balancing layer

A single self-managed load-balancer process can become a single point of failure. Redundancy may use active-active instances behind an anycast or managed entry point, or active-passive failover with a virtual IP. Managed cloud load balancers usually hide the appliance-level redundancy, but the service still has quotas, regional scope and configuration dependencies. Capacity planning must account for loss of one zone or one LB instance. If two active nodes each run at 80 percent capacity, losing one may overload the survivor. Failover also depends on health-detection time, connection reset behaviour and client retry strategy. Keep configuration consistent and deploy changes safely.

## 2.9 Python selection sketch and NGINX configuration

The code below demonstrates round-robin selection across a health-filtered pool. It is not a network proxy: a real load balancer must handle sockets, TLS, timeouts, health transitions, retries, backpressure and connection draining.

### 💻 Python / implementation sketch

```
from itertools import cycle
from threading import Lock
class RoundRobinPool:
    def __init__(self, backends):
        if not backends:
            raise ValueError("at least one backend is required")
        self._backends = list(backends)
        self._iterator = cycle(self._backends)
        self._lock = Lock()
    def choose(self):
        # Protect selector state when used by threads.
        with self._lock:
            return next(self._iterator)
```

pool = RoundRobinPool(["api-a:8000", "api-b:8000"]) print(pool.choose()) # NGINX upstream example: # upstream api_pool {

# least_conn; # server 127.0.0.1:8001 max_fails=3 fail_timeout=10s; # server 127.0.0.1:8002 max_fails=3 fail_timeout=10s; # } # server { listen 8080; location / { proxy_pass http://api_pool; } }

## 2.10 Failure scenarios to rehearse

- Backend crash: fail health checks, stop new assignments, preserve enough capacity on survivors.
- Backend slow but not dead: use timeouts, passive health and queue/concurrency controls; avoid routing more

work to a saturated node.

- LB failure: redundant entry point and tested failover are required.
- Bad deployment: readiness gate, canary traffic, rapid rollback and connection draining.
- Retry storm: bounded retries with exponential backoff and jitter; only retry operations safe under idempotency

rules.

- NAT skew: do not assume client IP is a unique user key.
- WebSocket reconnect: authenticate again and recover missed messages from durable sequence/offset state.

### 💻 Extended Python lab

# Weighted round-robin using a smooth accumulator is preferred over # expanding a list by weight when weights can be large or dynamic.

```
class SmoothWeightedRR:
    def __init__(self, weights):
        if not weights or any(w <= 0 for w in weights.values()):
            raise ValueError("weights must be positive")
        self.weights = dict(weights)
        self.current = {k: 0 for k in weights}
        self.total = sum(weights.values())
    def choose(self):
        for name, weight in self.weights.items():
            self.current[name] += weight
        selected = max(self.current, key=self.current.get)
        self.current[selected] -= self.total
        return selected
```

pool = SmoothWeightedRR({"small": 1, "medium": 2, "large": 4}) print([pool.choose() for _ in range(14)]) Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A chat product keeps 250,000 WebSocket connections across three zones. Connection counts are not uniform because some customers are more active and some connections are idle. A safe design uses an L4 or L7-capable edge tier with WebSocket support, distributes new connections across healthy gateway instances, sets appropriate idle and maximum connection lifetimes, and tracks per-instance file descriptors, memory and event-loop lag. The gateways publish presence and message-routing metadata to shared infrastructure. If a gateway fails, clients reconnect with jitter; messages missed during disconnection are recovered from durable conversation history or offsets. Do not assume the load balancer itself will relay messages between gateways.

### Interview questions and answer expectations

**Q. Compare L4 and L7 balancing for a public API.**

**Q. Why can least-connections perform poorly with HTTP keep-alive?**

**Q. Design health checks for a service with optional downstream dependencies.**

**Q. How should an LB handle a backend draining during deployment?**

**Q. What happens to WebSockets when a backend is removed?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Implement weighted round robin and test distribution over 70 selections.
- Write a health-state machine with healthy, suspect and ejected states.
- Design an edge for 3 regions with a complete region-loss scenario.

Operational metrics and signals

- Requests and bytes per backend
- Connection count, connection age and resets
- Backend p95/p99 and 5xx rate
- Health-check transitions and ejection duration
- TLS handshake rate and failures
- Per-zone active capacity and spare capacity

Chapter mastery checklist

- L4 selects at transport level; L7 can route using application semantics.
- Filter unhealthy nodes before selecting a backend.
- Pick algorithms based on request/connection workload, not fashion.
- Draining, retries, TLS and client identity are part of the design.
- The load-balancing tier itself needs capacity, redundancy and observability.

## 🔬 Advanced Engineering Clinic

A load balancer is a scheduler operating at a network boundary. A good design distinguishes connection assignment from request assignment, backend health from backend load, and retries from safe replay.

Client / DNS L4 entry TCP / UDP / connection L7 proxy HTTP route + policy Backend pool A • B • C Bounded retry / outlier policy failure / timeout Active probes + passive ejection eligible endpoints retry only if safe

Connection balancing versus request balancing At L4, a balancer commonly assigns a TCP or UDP flow to a backend. Once a TCP connection is established, many requests may travel over that same connection. With HTTP/1.1 keep-alive, HTTP/2 multiplexing or WebSockets, connection count is not the same thing as request

count or CPU cost. A server with fewer connections may still be doing more work. At L7, a proxy understands application protocol boundaries and can select a backend per HTTP request when the protocol and proxy mode allow it. This enables path, host, header, method and cookie routing. It also means the proxy parses more protocol state and may terminate TLS. Be precise in diagrams: “one request per backend” is not a property of every L4 balancer. Least-connections is a useful heuristic for long-lived connections but does not directly measure CPU, queue depth or downstream waits. Least-request or power-of-two-choices approaches can use active request counts. Latency-aware routing can help, but measurements must be smoothed; otherwise a transient slow sample can cause oscillation. Algorithm Good fit Caveat Round robin Similar backends and request costs Ignores current work and request duration Least connections Long-lived connections Connections are only a proxy for work Least requests HTTP requests with variable durations Needs accurate in-flight accounting Power of two choices Low-cost load-aware selection Still needs health and capacity signals Consistent hash Affinity / cache ownership Can skew; not a live-load balancing algorithm

Health, ejection and retry discipline Active health checks probe a configured endpoint. Readiness should answer whether the instance can serve the traffic class being routed to it. Liveness should answer whether the process is stuck and should be restarted. Passive health checks observe real errors and timeouts. A single failed probe should not necessarily eject a backend; thresholds, consecutive failures, recovery checks and outlier-ejection limits prevent flapping. Retries can improve success probability for transient failures but can amplify overload. A caller, gateway and service mesh that each retry three times can multiply one original operation into many attempts. Set a retry budget across the request path, use exponential backoff with jitter for delayed retries, and retry only when the operation is idempotent or protected by an idempotency key. Do not retry a payment POST simply because the response was lost. Connection draining removes an endpoint from new assignments while allowing existing work to finish up to a deadline. Long-lived WebSockets need a reconnect and resume protocol; graceful drain does not preserve a TCP connection through process termination. Advanced implementation note

```
def retry_allowed(method: str, has_idempotency_key: bool,
                  attempt: int, max_attempts: int = 2) -> bool:
    safe_method = method.upper() in {"GET", "HEAD", "OPTIONS"}
    replay_safe = safe_method or has_idempotency_key
    return replay_safe and attempt < max_attempts
# Also enforce a global retry budget and per-attempt deadline.
```

assert retry_allowed("POST", False, 0) is False assert retry_allowed("POST", True, 0) is True Worked routing decision Assume three backends have weights 1:2:3 and measured safe in-flight request capacities of 100, 200 and 300. Weighted round robin approximates a 1:2:3 arrival split, but that split is only sensible if request costs and health are comparable. If backend C has a queue-age alarm or fails readiness, remove it from eligibility; do not keep sending its weighted share merely to preserve the ratio. For heterogeneous instances, derive weights from benchmarked capacity under the same service-level target, not raw core count. Re-evaluate after code changes, instance-family changes and traffic-mix shifts. A balancer’s health and scheduling view is approximate; application-level admission control remains necessary. Question Decision evidence Can this backend receive new work? Readiness, outlier ejection, zone health Which backend is least loaded? In-flight requests, queue depth, resource cost Can this operation be replayed? HTTP semantics, idempotency key, downstream contract Can a connection move? Protocol and connection lifecycle; usually reconnect for WebSocket

## ❓ Advanced Design-Review Questions

- Explain why least-connections can route more work to a CPU-saturated server when HTTP/2 multiplexing is used.
- Design a retry policy for a POST payment endpoint where the connection can fail after the provider commits.

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 03. Caching

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

A frequently used reference notebook saves trips to a textbook. But if the textbook changes, the notebook can become wrong. Caching is a speed optimisation that creates a data-freshness responsibility.

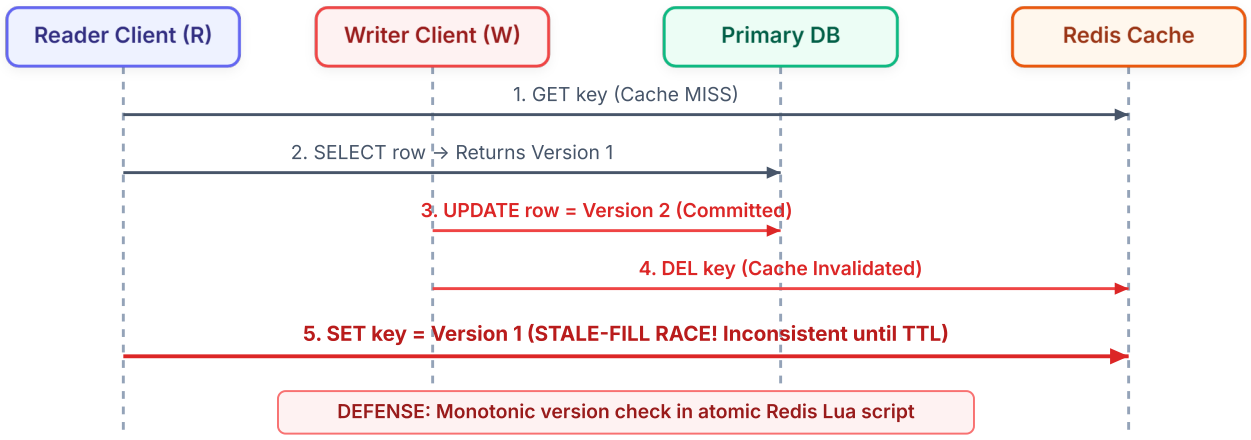
### 🤖 AI INFRASTRUCTURE LENS: SEMANTIC VECTOR CACHING & RADIX KV-CACHE

**The Exact-Match Cache Deficit:** Traditional caches (Redis/Memcached) use deterministic key hashing: `hash("What is Raft consensus?") != hash("Explain the Raft consensus algorithm")`. In AI applications, user intent is identical but exact string matching has a near-0% hit ratio.

**Dual-Tier Semantic Caching:**

- **Tier 1: Embedding Vector Similarity:** Generate a dense 768-d embedding for the incoming query $q$. Query an approximate nearest neighbor index (HNSW/IVF-PQ in Redis or Milvus). If cosine distance $1 - \cos(q, v_i) < \epsilon$ (typically $\epsilon ≤ 0.04$), return the cached response.
- **Tier 2: Fast Cross-Encoder Reranker:** A lightweight BERT cross-encoder scores the query against candidate cached questions. If confidence $≥ 0.95$, return the response in <10ms, bypassing a \$0.03 LLM generation call.

**Radix Attention (Prompt Prefix Caching):** Inside the serving engine (vLLM), KV-caches are modeled as a Radix Tree. Common prompt prefixes across distinct users (system prompts, few-shot examples, document attachments in chat) share identical physical memory pages via PagedAttention, reducing KV memory footprint by up to 60%.



### 🎯 Learning Outcomes

- Compare local, distributed, browser, CDN and database buffer caches.
- Design cache-aside and write patterns with correct keys and TTLs.
- Understand LRU/LFU eviction and Redis memory policies.
- Prevent stampede, penetration, avalanche and hot-key failures.
- Reason about invalidation, dual writes, stale reads and cache outages.

Client Application service Optional L1 process cache L1 hit L2 shared Redis miss L2 hit optional fill Authoritative database miss populate

observability work.

## 3.1 Cache economics and hit ratio

A cache is valuable when the cost of a hit plus the expected cost of misses is lower than going to the source for every request. For a simple two-path model, expected latency is H x Lcache + (1-H) x (Lcache + Lsource), where H is the hit ratio. This model ignores queueing, serialization and contention, but it forces assumptions into the open. Hit ratio is hits divided by lookups. It should be reported by key family and endpoint, not only as a single fleet-wide number. A high hit rate for static categories can hide a low hit rate for expensive personalised queries. Also monitor source-load reduction and the age of served data. A 99 percent hit ratio is harmful if the cached values are wrong for a business-critical operation. Expected lookup latency (simple model) = Lcache + (1 - hit ratio) x Lsource Database miss RPS = read RPS x (1 - hit ratio).

## 3.2 Cache placement and hierarchy

In-process caches offer very low access latency but are isolated per process and disappear on restart. Distributed caches such as Redis or Memcached are shared across application instances but add a network hop and an operational dependency. Browser caches reduce repeated downloads by one client. CDNs reduce origin traffic across a geography. Databases also maintain buffer caches to reduce disk reads.

Multi-level caching uses an L1 local cache and an L2 shared cache. On an L1 miss, the application checks L2; on an L2 miss, it reads the source and fills both layers. The difficult part is invalidation: an update must either reach every L1, use short L1 TTLs/version checks, or accept bounded staleness.

## 3.3 Cache-aside, read-through and write patterns

Cache-aside (lazy loading) is application-managed: read cache; if absent, fetch the database; populate cache; return. It avoids preloading cold data and is easy to adopt incrementally. Read-through delegates the miss loader to a cache abstraction. In both cases, miss behaviour is explicit and must be protected against duplicate concurrent loads. Write-through updates cache and source synchronously through a cache layer. Write-behind acknowledges a cache write before asynchronously persisting it, which can reduce write latency but creates durability and replay requirements. Write-around bypasses the cache on writes and lets the next read populate it. No pattern is universally best; choose based on read/write ratio, freshness requirements and whether the cache can safely own a pending write. Pattern Read miss Write path Primary trade-off Cache-aside App loads source and fills cache App updates source then updates/deletes cache Simple; app owns consistency Read-through

Cache abstraction loads source Independent write policy Centralises miss handling Write-through Reads from cache Cache layer synchronously writes source Fresh cache, extra write latency Write-behind Reads from cache Cache accepts then persists later Fast writes, durability complexity Write-around Miss loads and fills cache Writes bypass cache Avoids caching one-time writes

## 3.4 Keys, serialization and tenant boundaries

A cache key is part of correctness. Every input that changes the result should be represented in the key: tenant, user or permission scope, locale, pagination, sort order, feature version and query parameters. Omitting one dimension can return the wrong result or leak data between users. Use stable, readable prefixes such as tenant:acme:product:v2:101. Serialization is not free. JSON is inspectable but larger and slower than some binary formats. Pick a schema that tolerates deployments and version it when shape changes. Cache only data that the caller is authorised to receive. Avoid putting raw credentials or unnecessary personal data in keys; keys can appear in logs, metrics and diagnostic tools. Many concurrent reads Hot cache key expired All callers see MISS Database receives N identical reads Single-flight / lease OR early refresh protected path One loader refreshes key populate populate

creating one source request per caller.

## 3.5 Expiration and eviction are different mechanisms

TTL controls freshness or lifetime. Eviction controls what happens when memory pressure requires space. An entry can expire even when memory is abundant; it can also be evicted before its TTL if the configured max-memory policy requires space. Choose TTL based on the maximum acceptable staleness and refresh economics, not by copying one value across all data types. LRU approximates recency; LFU approximates frequency; FIFO evicts by insertion age; random eviction is simple but can remove a hot key. Redis offers policies such as allkeys-lru, allkeys-lfu, volatile-lru and noeviction. Redis implementations commonly use sampling/approximations rather than a perfect global ordering. Monitor evictions because an unexpected rise may indicate poor sizing or key churn. Policy Eviction signal Typical strength Typical weakness LRU Least recently accessed Adapts quickly to recency shifts Sequential scans can pollute cache LFU Lowest access frequency Retains consistently popular items Old popularity needs aging FIFO Oldest insertion Simple ordering Ignores reads TTL Expired lifetime Bounds staleness Synchronized expiry can stampede No eviction Reject write at limit Avoids silent key removal Writes fail under memory pressure

## 3.6 Invalidation and dual-write races

Common cache-aside write flow: commit database update, then delete the cache key. This makes the next read refill from the source. It is simpler than writing a second copy of the new value, but a race remains: a concurrent reader can fetch the old database value before the update and populate the cache after invalidation. Versioned values, short TTLs, locks, write ordering or event-driven invalidation can mitigate this depending on the required consistency. Updating database and cache is a dual write. They are independent systems and do not share one ACID transaction. If the database commits but cache invalidation fails, stale values remain. A durable outbox can record an invalidation event in the same DB transaction and publish it asynchronously. This gives reliable eventual propagation, not instantaneous cross-system atomicity. Watch out: Do not assume "delete cache after

DB commit" is a proof of strong consistency. State the accepted stale window and identify the race you are controlling.

## 3.7 Stampede, penetration, avalanche and hot keys

A stampede occurs when concurrent requests miss the same hot key and all load the source. Use request coalescing, a carefully designed distributed lease, early refresh or stale-while-revalidate. A lock needs ownership tokens and safe compare-and-delete; a plain SET then DEL can delete another caller's lock after the lease expires. Penetration is repeated lookup of nonexistent keys. Negative caching with short TTLs or a Bloom filter can reduce source load. Avalanche is a broad burst of expirations or a cache outage that sends traffic to the source. Use TTL jitter, prewarming, redundancy, admission control and a database-protection policy. A hot key

gets disproportionate traffic despite being cached; use L1 caching, replication, CDN, key splitting for aggregations, or workload-specific partitioning.

## 3.8 Redis operational architecture

Redis can store strings, hashes, lists, sets, sorted sets and streams. Use strings for serialized objects or counters, hashes for field-oriented records, sorted sets for rankings and streams for append-oriented event processing. Atomic Redis commands can update one key without a client-side read-modify-write race. Multi-key operations may have cluster slot constraints. Replication provides copies and can support failover/read distribution, but asynchronous replication can lag. Sentinel monitors a non-clustered primary/replica deployment and coordinates failover. Redis Cluster partitions keyspace across 16,384 hash slots and can combine sharding with replicas. None of these removes the need to decide what the application does when Redis is slow or unavailable.

## 3.9 HTTP caching and validators

HTTP caching uses Cache-Control, Expires, ETag, Last-Modified, Vary and conditional request headers. max-age defines freshness; s-maxage applies to shared caches; private restricts storage to private caches; public permits shared caching; no-store prohibits storage; no-cache permits storage but requires validation before reuse. These directives have precise semantics and should not be used interchangeably. ETag is a validator for a representation. A client can send If-None-Match; if the representation is unchanged, the server replies 304 Not Modified with no representation body. Last-Modified and If-Modified-Since provide time-based validation. Vary tells caches which request headers affect the response variant. Incorrect Vary or cache-key dimensions can serve one user's response to another.

### 💻 Python / implementation sketch

```
from fastapi import FastAPI, Response
```

app = FastAPI()

```
@app.get("/assets/app.8f3a.js")
def versioned_asset(response: Response):
    # Safe for immutable, content-hashed public assets.
    response.headers["Cache-Control"] = (
        "public, max-age=31536000, immutable"
    )
    return {"asset": "content-hashed bundle"}
@app.get("/account")
def account(response: Response):
    # Personalised data should not enter a shared public cache.
    response.headers["Cache-Control"] = "private, no-store"
    return {"account": "user-specific"}
```

## 3.10 Failure policy: fail-open, fail-closed and stale serving

Fail-open means that when the cache is unavailable, the application bypasses it and reads the source. This protects availability when the database has spare capacity, but can overload the database during a broad cache failure. Fail-closed means the request fails or is rejected rather than bypassing the cache; this protects the source but reduces availability. Stale serving returns a previously cached value when refresh fails and the data class permits it. There is no universal policy. A public product description may be served stale for a short period. A stock reservation, permission decision or payment state should use authoritative transactional data. A robust design

applies per-data-class policies and has a circuit breaker, concurrency cap or load shedding around source fallback.

### 💻 Extended Python lab

```
import json
from redis.asyncio import Redis
async def get_product(redis: Redis, product_id: int, db):
    key = f"product:v2:{product_id}"
    raw = await redis.get(key)
    if raw is not None:
        return json.loads(raw)
    # In production, add single-flight/lease coordination for hot keys.
    item = await db.fetch_product(product_id)
    if item is None:
        await redis.set(key, "__missing__", ex=30)
        return None
    await redis.set(key, json.dumps(item), ex=300)
    return item
# Note: a production lock must use a unique token and atomic
# compare-and-delete, not unconditional DEL of a shared lock key.
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A product catalog receives 40,000 read requests/second. A small fraction of SKUs account for most reads, prices change several times per day, and inventory changes continuously. Use cache-aside for product descriptions and categories with versioned

keys and TTL jitter. Consider CDN caching for public images and non-personalised detail fragments. Treat price and availability separately: use short freshness windows or authoritative reads at checkout; never authorise a purchase solely from a stale cache. Protect hot SKUs with request coalescing or stale-while-revalidate. During Redis failure, cap database fallback concurrency and shed optional traffic rather than allowing a cache outage to become a database outage.

### Interview questions and answer expectations

**Q. Compare cache-aside and write-through.**

**Q. Explain the DB-update/cache-invalidation race.**

**Q. How do stampede, penetration and avalanche differ?**

**Q. When is a local cache useful beside Redis?**

**Q. What does HTTP no-cache mean compared with no-store?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Build a FastAPI + Redis cache-aside endpoint with TTL and invalidation.
- Simulate 100 concurrent misses for one key and compare source calls with and without single-flight.
- Choose TTL and staleness budgets for product price, product description, inventory and public image.

Operational metrics and signals

- Hit/miss ratio by key family and endpoint
- Cache p50/p95/p99 latency and timeout rate
- Memory, fragmentation, evictions and expirations
- Hot-key command rate and per-node traffic skew
- Source DB miss RPS and query latency
- Stale-serving rate and invalidation propagation delay

Chapter mastery checklist

- Caching reduces repeated work; it does not automatically make the source faster.
- Cache-aside is a common default, but writes need explicit invalidation semantics.
- TTL and eviction are separate mechanisms.
- Protect the source from stampedes, penetration, avalanches and hot keys.
- Treat Redis as a performance layer unless the system deliberately designs it as a durable system of record.

## 🔬 Advanced Engineering Clinic

A mature cache design is not just GET then SET. It is a contract about freshness, admission, concurrent fills, key identity, failure policy and the authoritative source. Request for key K L1 local cache very fast / per-process L2

Redis shared / TTL / eviction L1 miss Single-flight / lease one loader per hot key L2 miss Authoritative database loader Refresh + version check result + version optional local fill populate

Single-flight, leases and stale-while-revalidate Single-flight coalesces concurrent loads for the same key inside one process: one coroutine performs the source query while other callers await the same future. This is often sufficient for one instance, but it does not coordinate across a fleet. Distributed leases can coalesce fills across instances, but leases need unique ownership tokens, bounded duration and compare-and-delete release so an expired owner cannot delete a newer owner's lock. Stale-while-revalidate stores a fresh-until time and a serve-stale-until time. Before fresh-until, serve the value. Between the two times, serve the stale value and trigger one refresh. After the hard stale boundary, wait for refresh or fail according to policy. This is suitable for catalog descriptions or public metadata, not for authorising a payment or allocating the last seat. Probabilistic early refresh spreads refresh work before a synchronized expiry boundary. A request near expiration may refresh with a probability that increases as expiry approaches and refresh cost grows. This reduces stampede risk without a single global timer, but it is an optimisation, not a correctness mechanism. Technique Scope What it prevents Async single-flight One process Duplicate concurrent loads in that process Distributed lease Multiple processes Duplicate loads across instances, if lock discipline is correct TTL jitter Many keys Synchronized expiration across keys Stale-while-revalidate One key / request path Blocking all callers during refresh Negative caching Missing keys Repeated source lookups for absent entities Prevent stale-fill races with versions A classic race occurs when reader R loads version 7 from the database, writer W commits version 8 and invalidates the cache, then R writes version 7 into the now-empty cache. The cache is stale even though the writer correctly deleted it. Deletion alone cannot prevent an older in-flight reader from repopulating stale data. One defence is version-aware cache writes. Store a monotonically increasing entity version with the cached payload and only accept a fill if it is not older than the version already recorded. Another is a per-key generation/tombstone that changes on invalidation. The comparison and write must be atomic at the cache boundary (for Redis, use a Lua script or transaction-like server-side operation). The authoritative database must supply a reliable version; wall-clock timestamps are unsafe substitutes when clocks can skew. For many systems, bounded staleness plus TTL and invalidation is an acceptable business contract. Strong cache coherence is expensive. Write down the requirement per data class instead of trying to make every cache behave like a linearizable database. Advanced implementation note # Conceptual Redis-side rule (execute atomically via Lua): # if incoming_version >= cached_version: replace payload + version # else: reject the stale fill. # The DB row version must increase transactionally on every update.

```
def cache_key(tenant_id: str, entity_id: str, schema_v: int) -> str:
    return f"v{schema_v}:tenant:{tenant_id}:product:{entity_id}"
# Tenant and schema dimensions avoid cross-tenant leakage
# and incompatible payload decoding.
```

Cache policy by data criticality Separate display data from decision data. A product description can often tolerate a minute of staleness. Inventory displayed on a product page may be approximate, while checkout must reserve against authoritative inventory. A cached account balance may be displayed with a timestamp, but a debit must validate and commit against the ledger. Cache is an optimisation and projection unless the system explicitly defines it as the authoritative store. Fail-open means bypass cache and query the source when Redis is unavailable. It is safe only if the database has protected capacity for the fallback load. Fail-closed rejects or degrades the request to protect the source. Stale serving uses a prior value within an explicitly allowed stale window. A robust design may combine them by data class and priority. Cache metrics should be segmented by

key family and tenant. A 95% aggregate hit ratio can hide a critical 20% hit ratio for one expensive endpoint. Track miss rate, source QPS, load time, evictions, memory fragmentation, hot keys, stale age and refresh failures. Data class Suggested policy direction Static/versioned asset Long TTL; immutable URL; CDN Catalog description Cache-aside; moderate TTL; invalidate on edit Inventory display Short freshness; show timestamp/approximation Checkout reservation Authoritative transactional check; cache not authority Personalised profile Identity-scoped key; private cache; explicit invalidation

## ❓ Advanced Design-Review Questions

- Show the stale-fill race where a reader repopulates version 7 after a writer invalidates version 8. How would

you prevent it?

- Define fail-open, fail-closed and stale-serving policies for a product catalogue versus checkout inventory.

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

PART I • SCALING & INGRESS • CHAPTER 04

# 04. Content Delivery Networks (CDN)

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

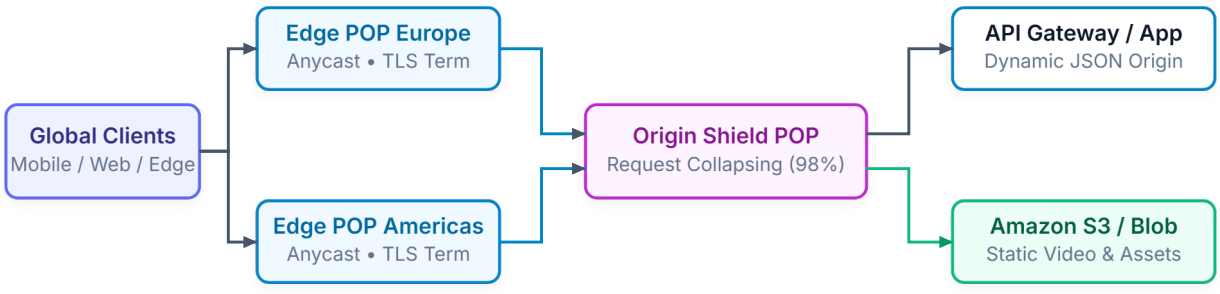
A global retailer does not ship every small, common item from one central warehouse. It places popular stock in regional distribution centres. A CDN does something similar for bytes: it serves copies from edge locations near users.

### 🤖 AI INFRASTRUCTURE LENS: MODEL WEIGHT DISTRIBUTION & EDGE VS. CLUSTER INFERENCE

**Distributing 140GB Checkpoints to Multi-Region Fleets:** Cold-starting a 70B parameter model cluster across 50 GPU instances requires transferring over 7 Terabytes of model weights (Safetensors format). If every instance fetches directly from an S3 bucket, S3 throughput thresholds and NAT gateway egress saturate, blowing startup times to 45 minutes.

**The Peer-to-Peer & Origin Shield Architecture:**

- **P2P Mesh Distribution (Dragonfly / Kraken):** A CDN origin shield caches model weight shards in the local cluster datacenter. The first instance pulls from the shield; subsequent GPU nodes pull chunks directly from peer memory/NVMe over 100 Gbps RoCE (RDMA over Converged Ethernet) fabrics, downloading 140GB in under 35 seconds.
- **Edge vs. Cloud Partitioning:** Small quantized models (1B–3B parameters, INT4/W4A16 via ONNX Runtime / WebGPU) run directly inside client mobile apps or edge POPs for zero-cloud-cost instant responses. Complex reasoning queries fall back to centralized cloud GPU pods (e.g. 8x H100).




Edge Hit Ratio: 92-96% • Origin Shield Hit Ratio: 85% on Edge Misses • Sub-25ms Latency

## 🎯 Learning Outcomes

- Explain edge POPs, origin, cache hit/miss, shielding and cache hierarchy.
- Design cache keys and understand Cache-Control, Vary, ETag and purge semantics.

- Separate public static content from personalised or sensitive responses.
- Reason about media delivery, signed URLs, origin protection and cache poisoning.
- Estimate origin offload, bandwidth and latency impact.

User in Hyderabad Nearby CDN edge POP HIT: serve object Regional origin shield MISS Origin API / web server MISS Object storage fetch bytes Cache policy / purge control purge / config

geographic proximity.

## 4.1 CDN architecture and terminology

A CDN operates a fleet of edge points of presence (POPs). A user request is directed to an edge through DNS-based or anycast routing. If the requested representation is fresh in that edge cache, it is returned locally. On a miss, the edge fetches from an origin, may pass through an origin shield, and can store the response according to policy. The origin is the authoritative service that produces or stores the content: an application server, object store or media origin. An origin shield is an intermediate cache that consolidates misses from many edges and reduces duplicate origin fetches. A cache hierarchy can reduce origin load dramatically for globally popular objects.

## 4.2 Request path and cache states

A cache miss is not a failure; it is a normal path that fetches from origin. A stale response may trigger revalidation with If-None-Match or If-Modified-Since. A stale-while-revalidate policy can allow a stale response to be served while refresh occurs asynchronously. A stale-if-error policy can permit serving stale data when the origin fails, but only when the data type and business contract allow it. CDN metrics should distinguish edge hit, shield hit, origin fetch, revalidation and error. A high edge hit ratio can still hide poor cache design if a huge volume of unique query strings or cookie variations fragments the cache.

## 4.3 Cache keys, variants and Vary

A CDN cache key commonly includes the host and URL path, and may include selected query parameters, headers, cookies or device dimensions. Every input that changes the representation must either be part of the key or be normalised away. If a response varies by Accept-Encoding, language or an approved device header, the cache must account for that variation. Overly broad keys are dangerous: two users can receive the same cached personalised response. Overly specific keys reduce hit ratio: arbitrary tracking query parameters can create thousands of variants for one image. Normalise query parameters, exclude irrelevant tracking fields and never ignore a parameter that affects content or authorisation. Watch out: Cache poisoning can occur when a cache and origin disagree about how a request is parsed or which headers/parameters affect the response. Keep cache-key rules aligned with origin routing and reject ambiguous inputs.

## 4.4 HTTP caching directives for origin teams

Cache-Control is the primary policy language. public allows shared caching; private limits caching to private caches; no-store prohibits storage; no-cache allows storage but requires validation before reuse; max-age controls freshness for private caches; s-maxage controls shared-cache freshness. These are not

interchangeable. For authenticated responses, default to private or no-store unless a carefully designed shared-cache policy proves that responses are safe to share. ETag and Last-Modified reduce bandwidth through conditional validation. Immutable, content-hashed assets are ideal for long-lived caching: a changed file has a new URL, so clients do not need an emergency purge. For mutable HTML and API data, shorter freshness or validators are more appropriate. Cache-Control is a contract with intermediary caches, not just an optimisation hint. Browser / player Signed URL / cookie check CDN edge cache lookup HLS / DASH manifest Video segments Range / chunk requests Origin shield / media origin Object storage immutable segment objects CDN edge cache lookup object bytes cdn_edge CDN edge cache lookup manifest hit segment hit miss

delivers immutable segments from object storage.

## 4.5 Invalidation, purge and versioned assets

Invalidation can be performed by purging URLs, surrogate keys or tagged object groups, depending on CDN capability. Purge operations can be asynchronous and may not instantly erase every edge copy. Broad purges can create a miss storm at the origin, so use versioned URLs for static assets and targeted invalidation for mutable content. Surrogate keys allow multiple URLs to be associated with a product or article tag. When a product changes, the origin can purge every representation tagged product-101. Versioned filenames are even simpler for build assets: app.4f8c.js is immutable while app.9b2d.js is a new object.

## 4.6 Media delivery and large objects

Images and videos are commonly stored in object/blob storage and distributed through a CDN. Image transformation can create variants for thumbnails, responsive widths and formats. Video-on-demand often uses segmented delivery such as HLS or DASH: a manifest points to many short media segments that the player fetches independently. Segment naming, cache lifetime and encryption policy affect hit ratio and content protection. For large objects, support byte-range requests when clients need partial downloads or seeking. Ensure the CDN correctly caches or forwards Range and Content-Range behaviour. Avoid routing large media bytes through an application server unless the application must transform or authorise the stream; use signed access and direct-to-object-storage paths where appropriate.

## 4.7 Origin protection and security

If an origin can be accessed directly, attackers or misconfigured clients may bypass the CDN and overload it. Restrict origin access to trusted CDN networks or use authenticated origin pulls where available. Validate signed URLs or cookies at the correct layer, set short-lived tokens, and ensure cache policy does not turn one authorised response into a public object. CDNs can absorb volumetric traffic and provide WAF, bot and DDoS controls, but they do not replace application authorisation. A cache hit should not bypass tenant isolation or access-control decisions. If access is private, use an architecture in which the CDN verifies a signed capability or the origin performs an authorised response that is not shared across users.

## 4.8 Routing, POP selection and geographic trade-offs

DNS-based steering can choose an edge based on resolver location and policy, but DNS caching can make changes non-instantaneous and recursive resolver location may differ from the end user. Anycast advertises the

same IP from many locations and relies on network routing to select a nearby point, but the "nearest" network path is not always the lowest-latency or healthiest application path. CDN selection should consider regional coverage, peering, cache rules, origin integration, invalidation API, logs, egress price, TLS and security features. Test from real user networks; a geographically nearby POP can still have poor peering or congestion.

## 4.9 Worked offload and bandwidth estimate

Suppose a site serves 2 million image requests/day with average object size 180 KB. Total daily edge payload is about 360 GB before protocol overhead. At a 90 percent edge hit ratio, origin fetch volume is about 36 GB/day if every miss fetches the full object. The hit ratio should be measured in bytes as well as requests: a 90 percent request hit ratio can correspond to a much lower byte offload if misses are larger files. If peak traffic is 20 times the daily average for a short event, design for that burst and account for cold-cache behaviour. The first request after a regional cold start may go to the shield or origin. Prewarm only likely hot objects; loading the entire catalogue can waste bandwidth and origin capacity. Daily bytes = requests/day x mean object bytes. Origin bytes (simple model) = daily bytes x (1 - byte hit ratio).

## 4.10 CDN failure and operational patterns

- Origin outage: serve stale only when allowed; otherwise fail clearly and protect origin on recovery.
- Cache purge storm: use targeted invalidation and origin shielding.
- Cache poisoning: canonicalise URL and headers; test cache/origin parsing agreement.
- Private data leak: partition cache by identity or prohibit shared caching.
- Hot object: edge replication and shield caching can absorb request concentration.
- Regional POP issue: steer away through health and routing policy; test failover behaviour.
- Large upload: use direct object-storage upload or a specialised upload path rather than proxying bytes through

the API fleet.

### 💻 Extended Python lab

```
from fastapi import FastAPI, Response
from datetime import datetime, timezone
import hashlib
```

app = FastAPI()

```
@app.get("/public/catalog-version")
def catalog_version(response: Response):
    body = b"catalog-v17"
    etag = hashlib.sha256(body).hexdigest()
    response.headers["ETag"] = f'"{etag}"'
    response.headers["Cache-Control"] = "public, max-age=60, s-maxage=300"
    return {"version": "catalog-v17"}
# Production handlers should implement conditional request semantics
# (If-None-Match -> 304) and must not mark personalised data public.
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A learning-video platform has users in India, Europe and North America. Video files are stored in object storage; the API verifies entitlement and returns a short-lived signed manifest URL. A CDN serves HLS/DASH manifests and segments. Use immutable segment names so they can be cached for a long time; use a shorter manifest lifetime if the playlist changes. Design origin shielding and signed URL validation so a leaked URL has bounded exposure. Estimate bandwidth from view starts, average watch duration, bitrate and segment size, not merely page views. Observe rebuffering, time-to-first-frame and edge hit ratio by geography.

### Interview questions and answer expectations

**Q. How does a CDN reduce origin load?**

**Q. Explain cache key design and Vary.**

**Q. What is origin shielding?**

**Q. How do you invalidate static content without a global purge?**

**Q. How can a CDN safely serve private media?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Design HTTP cache headers for fingerprinted JS, product detail JSON and a private account endpoint.
- Estimate origin offload for 30 million requests/day at 240 KB and 88% byte hit ratio.
- Draw a video delivery path from authorisation to player segment requests.

Operational metrics and signals

- Edge hit ratio and byte hit ratio
- Origin fetches, shield hit ratio and origin bandwidth
- TTFB and cache age by region
- 5xx/4xx and TLS errors
- Purge propagation time
- Media rebuffering and time-to-first-frame

Chapter mastery checklist

- A CDN is a geographically distributed cache, not just a faster DNS.
- Cache keys, privacy and HTTP headers determine correctness.
- Use immutable URLs for static build assets; target purges for mutable content.
- Protect origins with shielding, access restrictions and surge planning.
- Media delivery is usually a direct CDN/object-storage path, not an API-server byte stream.

## 🔬 Advanced Engineering Clinic

CDN design is a problem of cache correctness and origin protection at global scale. A fast edge is useful only if the cache key separates representations correctly and never mixes private data.

Viewer Edge POP object cache + TLS GET object HIT Regional origin shield collapse misses MISS Access logs + analytics usage telemetry Origin / object storage MISS Auth service signed URL / cookie short-lived capability

Cache-key correctness and cache poisoning A CDN cache key is the identity of the representation being stored. If the response varies by path, query parameters, host, selected headers, language, device variant or authentication, the key policy must reflect those dimensions or deliberately exclude them. Ignoring a relevant dimension can serve the wrong representation; including irrelevant high-cardinality dimensions destroys hit ratio. Normalise URLs consistently: define query parameter ordering, decide which tracking parameters can be ignored, normalise case only where the origin treats it equivalently, and handle encoded path characters consistently. Cache poisoning occurs when the edge and origin disagree about request parsing or which headers affect output. An attacker may cause a malicious response to be cached under a key later requested by other users. Never put private personalised responses into a shared cache unless the identity is safely partitioned and the cache policy is explicitly designed for it. In many cases, mark the response private or no-store. Test cache keys with adversarial headers, duplicate query parameters and URL encoding edge cases. Question Correctness check Does language change the body? Include locale in key or use correct Vary policy Does query order change meaning? Canonicalise without changing origin semantics Is response user-specific? Private cache or identity-safe partition Can a header alter origin output? Include it in variant key or prevent shared caching Can an attacker influence cache key parsing? Test proxy/origin parser agreement

Origin shielding, tiering and range requests Without an origin shield, a popular object missing at many edge POPs can produce one origin fetch per POP. A shield layer centralises misses for a region or provider network, reducing duplicate origin work. Request collapsing (also called collapsed forwarding) lets concurrent requests for the same missing object wait on one origin fetch instead of all forwarding independently. Large media delivery often uses byte-range requests. The CDN and origin must agree on Range and Content-Range semantics, object validators and partial-response caching. For streaming video, immutable segments and manifests with versioned names are easier to cache safely than mutable objects whose identity never changes. Invalidation can be implemented with purge, surrogate keys or versioned URLs. Purging is useful for urgent changes but can create an origin thundering herd if every edge must refill at once. Immutable content with a new versioned path avoids most invalidation for static assets; mutable metadata can use shorter TTLs and targeted purge. Advanced implementation note # Example cache policy model for a public immutable asset. asset_policy = { "cache_control": "public, max-age=31536000, immutable", "key": "/assets/app.8f31c2.js", "purge_strategy": "publish a new content-hash URL", } # Do not apply this policy to a mutable price or private profile. assert "immutable" in asset_policy["cache_control"] Signed delivery and security boundary A signed URL is a time-limited bearer capability: possession grants access until expiry or revocation conditions are met. Use short

lifetimes appropriate to the user experience, scope the signature to one object or narrow prefix, bind required HTTP method and relevant constraints, and avoid logging full signatures. A signed cookie can be more convenient for access to many segments under one path policy. Private content caching requires deliberate separation. The CDN may cache the bytes while enforcing authorisation before serving, or it may use signed capabilities as part of the cache/origin access model. Do not blindly include a unique signature in the cache key if that prevents all sharing; equally, do not remove it from the key unless the CDN verifies the capability on every request before returning the cached object. Operationally monitor edge hit ratio, origin fetch rate, shield hit ratio, bytes egressed, 4xx/5xx by POP, purge propagation, stale serving and origin latency. A high hit ratio is not useful if it is achieved by caching the wrong representation. Failure Mitigation Origin outage Stale serving only for eligible public content Purge storm Targeted purge, versioned assets, origin shield Private object leak Signed access, cache partitioning, authorisation before hit Hot object Request collapsing, tiered cache, edge replication

## ❓ Advanced Design-Review Questions

- Design a cache key for a private, locale-specific video manifest. Which dimensions are security-sensitive?
- How would you prevent a global purge from overwhelming the origin when millions of edge objects refill?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

PART II • DISTRIBUTED DATA • CHAPTER 05

# 05. Database Sharding

*CORE SYSTEM DESIGN*

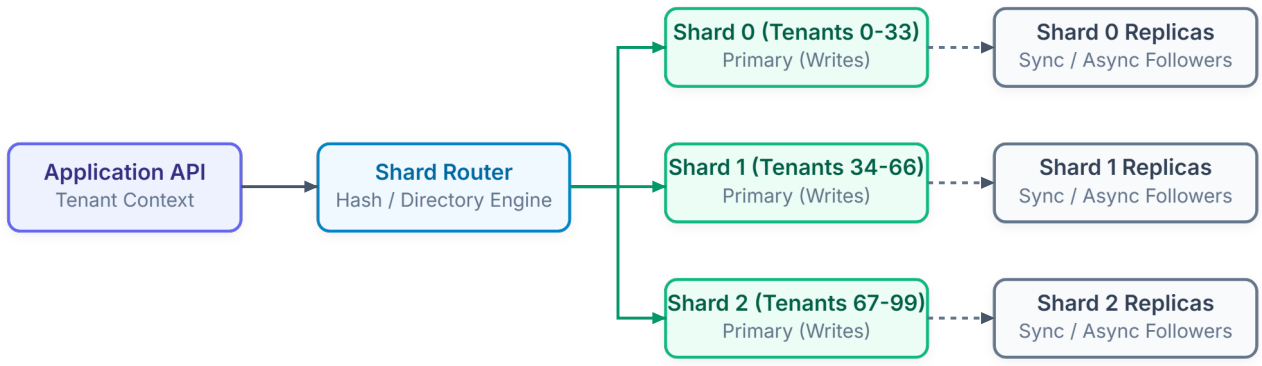
### 💡 INTUITIVE MENTAL MODEL

A library with one room eventually runs out of shelves and becomes difficult to search. It can divide books among buildings. The catalog must then know which building owns each book; that routing catalog is as important as the partitioning itself.

### 🤖 AI INFRASTRUCTURE LENS: SHARDING HIGH-DIMENSIONAL VECTOR DATABASES

**The Curse of Dimensionality in Vector Sharding:** Sharding traditional relational tables is straightforward (hash by `tenant_id` or range by date). Sharding billions of 1536-dimensional embeddings (OpenAI `text-embedding-3-large`) introduces unique architectural challenges:

- **Document-Based (Scatter-Gather) Sharding:** Each shard holds an independent HNSW (Hierarchical Navigable Small World) index for a subset of vectors. A nearest-neighbor query broadcasts to *all N shards*. Each shard returns its local top-*K* candidates, and an aggregator merges them into the global top-*K*. Scaling reads requires read replicas; scaling writes requires adding shards.
- **Centroid-Based Partitioning (K-Means Sharding):** Vectors are clustered into Voronoi cells. A routing service compares query vectors to cluster centroids and routes queries only to the top-*M* closest shards, eliminating scatter-gather amplification at the cost of potential boundary recall loss.
- **The Metadata Filter Trap:** Executing post-filtering (find top-100 nearest neighbors, then filter by `tenant_id = 'acme'`) collapses recall if only 1% of documents belong to 'acme'. Production vector systems enforce **Single-Stage Iterative Filtered HNSW**: the graph traversal evaluates boolean predicates directly during neighbor exploration.




## 🎯 Learning Outcomes

- Explain horizontal partitioning and choose a shard key from query patterns.

- Compare range, hash, directory and geographic sharding.
- Handle skew, hotspots, cross-shard queries and transactions.
- Plan online resharding and shard-key migration safely.
- Distinguish sharding from replication and partitioning within one database.

Independent data partitions Application / repository Shard router key → owner Shard A Tenant 0-99 Shard B Tenant 100-199 Shard C Tenant 200-299 Shard map / directory routing metadata

relevant records.

## 5.1 What sharding is and when to introduce it

Sharding horizontally partitions records across independent database nodes. Each shard owns only a subset of the logical dataset. It is used when one database can no longer meet storage, write-throughput, I/O or maintenance requirements, and simpler techniques such as indexes, query improvements, vertical scaling and read replicas are insufficient. Sharding is not a default optimisation. It complicates joins, unique constraints, transactions, migrations, backups, analytics and operations. The migration itself can create risk. Before sharding, identify the real constraint: a large table may be slow because of an unindexed query rather than because the dataset must be split.

## 5.2 Partitioning, sharding and replication

Partitioning is the broad idea of dividing a dataset. A single relational database can partition a table by range or list while still presenting one logical database instance. Sharding typically means distributing partitions across independent database servers or clusters, so capacity and failure domains are spread. Replication copies data to additional nodes; it does not by itself split ownership. A shard can itself have a primary and replicas. Thus a common architecture is partitioned ownership plus replication within each partition. For example, shard A owns users 1-1M and has two replicas; shard B owns users 1M-2M and has two replicas. Technique Data on nodes Primary motivation Table partitioning Partitions within one DB system Pruning, maintenance, manageability Sharding Different records owned by different DB nodes Storage and write scale-out Replication Copies of the same records Availability, recovery, read scale Sharding + replication Each shard has its own replicas Scale and resilience together

## 5.3 Shard-key selection: the defining decision

The shard key determines where a record lives. Choose it from the dominant access patterns, not merely from a field that is unique. A good key distributes load, is available on most requests, keeps related data together and avoids hot partitions. A poor key makes common queries scatter across every shard or puts most traffic on one shard. For multi-tenant SaaS, tenant_id is attractive because it co-locates one tenant's data and supports tenant-scoped queries. But one very large tenant can become a hotspot. For a social network, user_id may co-locate a user's profile and activity, but a feed that joins thousands of followed users can require fan-out. For time-series events, time range partitioning enables retention but can send all current writes to the newest partition.

- High cardinality is usually helpful, but uniqueness alone is not enough.
- Avoid monotonically increasing keys as the sole range key when they funnel writes to the newest shard.

- Check whether query predicates include the shard key.
- Consider co-location of records frequently joined or transacted together.
- Plan explicitly for very large tenants, celebrities, hot products and geographic skew.

## 5.4 Sharding strategies

Range sharding maps key intervals to shards. It makes range scans and ordered locality straightforward, but uneven key distribution and monotonically increasing writes can create hotspots. Hash sharding hashes the key before assigning a partition, which usually spreads keys more evenly but destroys natural range locality. Directory-based sharding stores an explicit mapping from tenant/key to shard, enabling flexible placement at the cost of a routing dependency. Geographic sharding places data near a region or residency boundary, but users and relationships can cross regions.

Consistent hashing can help reduce remapping when nodes are added, especially for cache ownership or distributed keyspaces. It is not a complete database sharding strategy: schema migration, transactions, replica placement, directory updates and hot-key handling remain necessary. Strategy Routing rule Strength Risk Range Key falls in interval Range queries and locality Skew / hot newest range Hash hash(key) maps to partition Even distribution Poor range locality; resharding Lookup key → shard Flexible placement Directory availability and cache coherence Geographic Region / residency rule Latency and data locality Cross-region workflows and imbalance Router v1 Bucket 0-999 → A Old shard A old owner Snapshot copy + change capture New shard D Checksum / count validation Versioned cutover router v2

rollback options.

## 5.5 Query routing and scatter-gather

A targeted query includes the shard key and can be routed to one shard. A scatter-gather query runs against many or all shards and merges results. Scatter-gather cost grows with shard count, network hops and tail latency. A global search or analytics query may therefore belong in a search index or analytical warehouse rather than on the transactional shard path. Pagination across shards is difficult. Each shard can return its local top K, then a coordinator merges sorted results, but the system must define stable ordering and a cursor that captures progress on each shard. Offset pagination can become increasingly expensive; keyset pagination with a deterministic sort key is often preferable.

## 5.6 Cross-shard transactions and unique IDs

A transaction contained within one shard can use the database's normal ACID guarantees. A transaction spanning shards requires coordination: distributed transaction protocols, application-level sagas, or a model that avoids cross-shard atomicity. Cross-shard joins and uniqueness constraints are similarly difficult because no single node sees all rows. Global IDs should be generated without a central round trip on every insert. Options include UUIDs/ULIDs, Snowflake-style time/worker/sequence identifiers, or a carefully designed ID service. IDs need uniqueness, ordering properties if required, collision handling and safe worker identity allocation. Do not assume an auto-increment sequence on each shard creates a globally unique ID.

## 5.7 Rebalancing and resharding

Resharding changes the mapping as data volume and load evolve. With modulo N, changing N can remap a large fraction of keys. A safer migration uses a routing map or virtual buckets. One approach: introduce a new routing version, copy a subset in the background, dual-read or shadow-compare, then switch ownership for a bucket after writes are reconciled. A practical online migration plan is: (1) define target layout and routing version, (2) start copying with a consistent snapshot, (3) capture concurrent changes using a change log or dual-write mechanism, (4) verify row counts and checksums, (5) briefly fence writes or use a precise cutover protocol, (6) flip routing, (7) retain rollback capability, and (8) remove old data only after a validation window. Watch out: Never treat "copy rows then change the router" as safe when writes are occurring. Without a change-capture or write-fencing plan, the migration can silently lose updates.

## 5.8 Hot partitions and skew

Even a mathematically even key distribution can be workload-skewed. A celebrity account, viral item, or very large tenant can receive a disproportionate number of requests. A shard key that balances stored bytes may not balance QPS, write rate or lock contention. Monitor both data distribution and workload distribution. Mitigations include splitting a hot tenant into sub-partitions, salting keys for append-heavy aggregates, caching hot reads, routing hot objects to dedicated capacity, or moving the hot tenant. Salting can make reads require fan-out or aggregation, so it trades write distribution for read complexity.

## 5.9 Python routing sketch and its limitation

The example below provides a stable hash assignment for a fixed shard count. It is intentionally not an online-res harding system. A production router needs a versioned mapping, topology discovery, retries, connection pools per shard and safe migration support.

### 💻 Python / implementation sketch

```
import hashlib
```

SHARDS = ["db-a", "db-b", "db-c", "db-d"]

```
def shard_for(tenant_id: str) -> str:
    digest = hashlib.sha256(tenant_id.encode()).digest()
    number = int.from_bytes(digest[:8], "big")
    return SHARDS[number % len(SHARDS)]
# Route repository operations using the tenant's shard key.
def get_tenant_repository(tenant_id, repositories):
    shard = shard_for(tenant_id)
    return repositories[shard]
# Warning: changing len(SHARDS) changes many assignments.
# Use bucket mapping or consistent hashing for controlled movement.
```

## 5.10 Operational practices

- Monitor per-shard CPU, IOPS, storage, lock waits, QPS and p99 latency.
- Keep schema migrations compatible across mixed software versions.
- Back up and restore shards independently, then test full logical reconstruction.
- Automate shard placement and capacity alerts before a shard is full.
- Use query tracing to reveal scatter-gather and cross-shard fan-out.
- Document ownership, shard-map versions, data residency and recovery procedures.

### 💻 Extended Python lab

# A versioned bucket directory avoids changing every tenant # assignment when the number of physical shards changes. BUCKET_COUNT = 4096

```
def bucket_for(tenant_id: str) -> int:
    digest = hashlib.sha256(tenant_id.encode()).digest()
    return int.from_bytes(digest[:8], "big") % BUCKET_COUNT
# Control-plane metadata maps stable buckets to physical shards.
# Example only; production metadata needs caching, versioning,
# atomic cutover and stale-map handling.
```

bucket_owner = {b: "db-a" for b in range(BUCKET_COUNT)} bucket_owner.update({b: "db-b" for b in range(2048, 4096)}) Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A multi-tenant SaaS has 2,000 tenants. Most tenants are small, but one tenant accounts for 25 percent of writes. A simple tenant_id hash spreads small tenants but does not split the large tenant. Start with directory-based tenant placement so tenants can be moved independently. For the large tenant, consider sub-partitioning by a secondary entity key or dedicated shard. Ensure queries include the routing key, co-locate tenant-local transactions and treat cross-tenant analytics as an asynchronous warehouse workload. Plan a versioned bucket map, snapshot plus change capture, checksum verification and cutover rehearsal.

### Interview questions and answer expectations

**Q. How do you choose a shard key?**

**Q. Compare range and hash sharding.**

**Q. How do you handle one extremely hot tenant?**

**Q. What becomes harder after sharding a relational database?**

**Q. Describe an online resharding migration that does not lose writes.**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Choose a shard key for a chat product with conversations, messages and users.

- Design a bucket directory and explain stale-router behaviour.
- Plan a 4-to-8-shard migration while writes continue.

Operational metrics and signals

- Per-shard QPS, bytes, storage and growth rate
- Read/write skew and hottest key/tenant
- Cross-shard query count and fan-out latency
- Replication lag for each shard
- Reshard copy lag, checksum mismatch and cutover duration
- Shard-map cache age and routing errors

Chapter mastery checklist

- Sharding partitions ownership; replication copies data.
- A shard key should follow access patterns and balance load.
- Scatter-gather, cross-shard transactions and uniqueness become harder.
- Plan hot-key mitigation and online resharding before the shard is full.
- Consistent hashing helps remapping but does not solve the whole database migration problem.

## 🔬 Advanced Engineering Clinic

Sharding becomes difficult during change, not during the happy-path lookup. This clinic focuses on shard-key consequences, online resharding, global queries and the operational control plane. Application Shard directory / router key owner + epoch Old shard range source of truth Epoch cutover route new reads/writes control-plane update Backfill + change capture checksums / lag snapshot + tail changes New shard range shadow writes new owner

Shard-key design as an access-pattern decision A shard key determines both data placement and which operations are cheap. Choose it from the dominant access patterns and invariants: records frequently read or updated together should ideally share a partition. A user_id key can make user profile and account queries local, but a globally popular tenant or celebrity account may create a hot shard. A time key supports range scans but can send all new writes to the newest partition. Estimate cardinality, distribution, growth, query locality and hot-key risk. Avoid keys with low cardinality (status, country, plan tier) unless combined with another dimension. Avoid mutable shard keys because

moving a row can require a distributed transaction. A compound key such as tenant_id plus entity_id can provide tenant locality, but a single large tenant may still require subpartitioning. Global secondary indexes are not free. If lookup by email must find a user whose shard is selected by user_id, you need a separate index, directory, scatter-gather or a different key strategy. The index itself needs consistency, sharding, uniqueness enforcement and recovery semantics. Candidate key Strength Risk user_id User-local access Large tenant/user hot spots; cross-user search created_at Time-range scans Newest partition receives concentrated writes tenant_id Tenant isolation/locality Noisy-neighbour risk for large tenants hash(entity_id) Even placement Range scans and locality are harder tenant_id + hash(id) Tenant-aware distribution Hot tenant can still dominate Online resharding without a big-bang migration An online migration usually has phases: prepare the new shard, take a consistent snapshot

from the old owner, copy rows, capture changes made during the copy, apply the change stream, compare counts/checksums, then move routing ownership. A directory or routing map should carry an epoch so stale clients cannot continue writing to the previous owner after cutover. Dual writes are dangerous if the two writes are not atomic. Prefer a source-of-truth write plus a durable change stream/outbox/CDC path to the destination. During migration, define which side serves reads and how writes are ordered. Shadow reads can compare results without exposing the new shard to users. Cut over gradually by tenant or key range, and keep a rollback window until consistency is verified. After cutover, do not immediately delete the old copy. Retain it read-only for a bounded period, monitor unexpected old-owner writes, then decommission after backup and recovery procedures are updated. A migration is complete only when operational runbooks, dashboards, backups and data-retention rules reflect the new topology. Migration correctness checklist: 1) snapshot at position p 2) copy snapshot 3) replay changes after p 4) verify count + checksums + invariants 5) advance ownership epoch 6) route traffic and reject stale-owner writes Cross-shard queries and transactions Scatter-gather sends a query to multiple shards and merges results. It can be acceptable for infrequent admin/reporting operations, but fan-out grows with shard count and the slowest shard dominates tail latency. Use bounded parallelism, deadlines, pagination and a separate analytical projection for broad scans. Cross-shard transactions are expensive because atomic commit requires coordination, often through distributed transaction protocols or an application-level saga. Keep invariants local where possible. Generate

globally unique IDs independently (for example, time/worker/sequence schemes or UUIDs) so inserts do not require a central auto-increment counter. Unique constraints across shards need a deliberate authority. A per-shard unique index cannot guarantee global uniqueness if the same value can land on different shards. A directory service, deterministic routing by the unique value, or a reservation protocol may be required. Operation Default design direction Fetch one user by user_id Route to a single shard Search all users by name Search projection / index, not blind scatter Transfer between shard-owned accounts Saga or carefully scoped distributed transaction Enforce global email uniqueness Deterministic ownership or central uniqueness directory

## ❓ Advanced Design-Review Questions

- A tenant grows to 40% of total traffic. How would you split its partition online without allowing two owners to

accept writes?

- How would you enforce globally unique email addresses when users are sharded by user_id?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

PART II • DISTRIBUTED DATA • CHAPTER 06

# 06. Database Replication

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

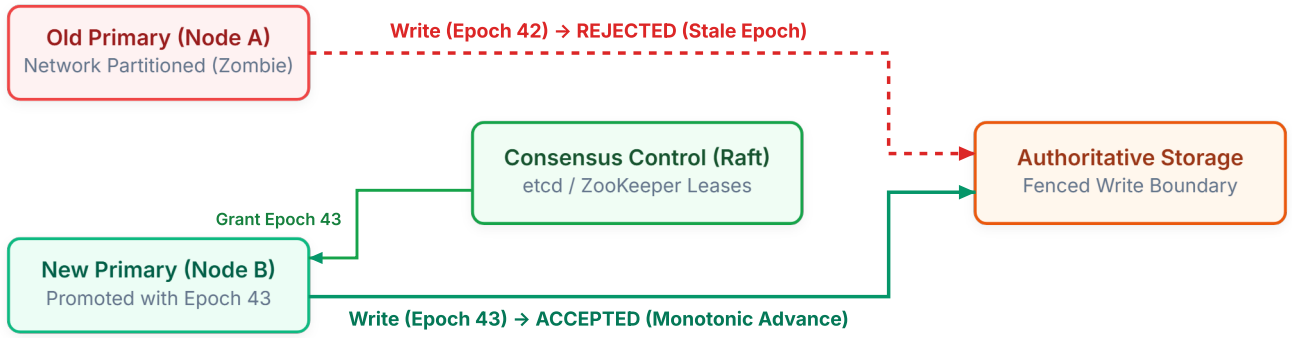
A chain of stores keeps copies of a product catalog. Each store can answer local questions, but if only the central store accepts updates, nearby copies may briefly be behind. Replication creates copies; it does not automatically make them identical at every instant.

### 🤖 AI INFRASTRUCTURE LENS: DISTRIBUTED TRAINING CHECKPOINTING & SPLIT-BRAIN FENCING

**Distributed PyTorch Training Failures:** In large-scale pre-training across 1,024 H100 GPUs (Megatron-LM / PyTorch FSDP), the Mean Time Between Failures (MTBF) of hardware (bad HBM, PCIe errors, InfiniBand link drops) is often less than 24 hours.

**Fencing Epochs in Distributed Checkpointing:**

- When a worker node experiences an InfiniBand partition or soft hang, the training orchestrator (Slurm/Kubernetes) triggers rank reallocation and promotes a hot-standby node.
- If the zombie node awakens and attempts to flush its half-written optimizer state to the distributed object store, it can corrupt the checkpoint.
- **Mitigation:** Monotonic Fencing Tokens tied to the global training epoch. Storage writes reject any payload signed with an epoch less than the coordinator's current committed rank iteration.




## 🎯 Learning Outcomes

- Compare leader-follower, multi-leader and leaderless replication.
- Explain synchronous and asynchronous replication, replication lag and quorum.
- Understand failover, split brain, RPO, RTO and replica promotion.

- Choose safe read-routing policies based on consistency requirements.
- Design recovery and test stale-read behaviour.

Application Primary / leader accepts writes writes Read router consistency-aware reads WAL / replication log Replica A read-only copy replicate Replica B read-only copy replicate strong / read-your-writes stale-tolerant stale-tolerant

## 6.1 Why replicate?

Replication maintains copies of data on multiple nodes. It can improve availability when a node or zone fails, support disaster recovery, distribute read traffic, and place data closer to users. Replication is distinct from backup: a replica can faithfully copy accidental deletion or corruption, while a point-in-time backup provides a separate recovery history. A replica is not automatically a safe read target. If replication is asynchronous, it may lag behind the primary. If an application reads from a lagging replica immediately after a write, it can violate read-your-writes expectations. Define which reads can tolerate staleness and which must use the leader or a stronger consistency mechanism.

## 6.2 Leader-follower replication

In leader-follower (primary-replica) replication, one node is the write authority for a data partition. It records changes in a durable log, and followers apply those changes. The log may be a write-ahead log, binlog or database-specific replication stream. Followers can serve read traffic if the application accepts their lag. The leader simplifies conflict resolution because concurrent writes are ordered at one authority. It can become a bottleneck and a failover dependency. A follower that is promoted must have an acceptable log position;

otherwise acknowledged writes can be lost. Promotion requires a control-plane decision and safe fencing of the old leader.

## 6.3 Synchronous and asynchronous replication

With asynchronous replication, the leader acknowledges a write without waiting for every follower to apply it. This reduces write latency and allows the leader to continue when followers are slow, but a leader failure can lose acknowledged changes that had not reached a promoted replica. The replication lag is a first-class operational metric. Synchronous replication waits for one or more replicas to acknowledge a defined stage of the write before acknowledging to the client. The exact guarantee varies: a replica may acknowledge receipt, durable log flush or apply. Synchronous replication reduces the risk of losing acknowledged writes but increases latency and makes write availability dependent on replica reachability. Always ask what "acknowledged" means in the database protocol. Mode Commit waits for Benefit Cost Async Leader commit only Low latency, follower independence Non-zero data-loss window on failover Sync to one replica A follower acknowledgement at a defined durability stage Smaller loss window Latency and availability depend on follower Quorum Enough replicas acknowledge Tolerance of some node failures Coordination and quorum availability

## 6.4 Replication lag and read routing

Replication lag is the distance between the leader's latest committed state and a replica's applied state. It can be measured in bytes, log sequence number, commit timestamp or elapsed time. Lag grows when writes spike, the network slows, a replica is under-provisioned or long-running queries delay apply. Applications can route stale-tolerant reads to replicas, but use the primary for read-your-writes or correctness-sensitive checks. A session can remember a commit token or log sequence number; a replica is eligible only once it has applied at least that position. Simpler systems route the user's immediate post-write reads to the primary for a bounded period. Application writes Promoted leader epoch 42 after routing update Old leader (partitioned) Control quorum lease / epoch authorise epoch 42 Fence old leader reject epoch < 42 prevent stale writes Follower with latest safe log promote prevent stale writes

returning as a competing writer.

## 6.5 Failover, split brain and fencing

Failover promotes a replica when the primary is unavailable. A failure detector cannot perfectly distinguish a dead leader from a network partition. If the old leader is still running but disconnected from the control plane, promoting a new leader can create two writers: split brain. Use quorum-based control, leases with carefully bounded assumptions, and fencing tokens or epochs so stale leaders cannot continue accepting writes. Promotion is not just changing a DNS record. The system must select a sufficiently up-to-date replica, ensure the old primary is fenced, reconfigure replicas, update routing, validate application connections and monitor recovery. An unsafe automatic failover can trade a brief outage for silent data divergence. Watch out: A health check that cannot reach the primary does not prove the primary is dead. Failover decisions need a coordination and fencing model.

## 6.6 RPO and RTO

Recovery Point Objective (RPO) is the maximum acceptable data-loss window after a failure. Recovery Time Objective (RTO) is the maximum acceptable time to restore service. Asynchronous cross-region replication may have a non-zero RPO; synchronous cross-region replication can reduce RPO but adds network latency and partition sensitivity. Backups and tested restores are required even when replicas exist. For a low-risk analytics cache, a larger RPO may be acceptable. For order records, the RPO may be near zero for acknowledged commits. Be precise: "zero RPO" depends on the failure set and commit acknowledgement semantics. No system can guarantee against every correlated failure merely by having replicas.

## 6.7 Multi-leader and leaderless replication

Multi-leader replication allows writes in multiple regions or sites. It improves local write availability and latency but creates conflict resolution problems when the same record is updated concurrently. Strategies include last-write-wins (with clock assumptions), application-level merge, conflict-free replicated data types for suitable data, or rejecting conflicts. Leaderless systems often use quorum reads and writes across replicas. If there are N replicas, write quorum W and read quorum R, the condition R + W > N ensures overlap under the model, but overlap alone does not guarantee full linearizability. Version vectors, read repair, hinted handoff and sloppy quorums affect observed behaviour. A quorum is a protocol, not a magic synonym for strong consistency.

## 6.8 Replication and backups are complementary

Replication improves live availability but propagates logical mistakes. A malicious update, accidental delete or corrupt migration may replicate quickly. Backups preserve historical recovery points, and point-in-time recovery can reconstruct to a time before the bad change. Test restore time and data integrity, not just backup job success. For disaster recovery, consider region loss, credential compromise, control-plane failure and corruption. Store backups in a separate failure domain with protected credentials and retention controls. A replica in the same region does not satisfy regional disaster recovery by itself.

## 6.9 Python consistency-aware read routing sketch

This pseudocode illustrates a simple policy: reads immediately following a write go to the leader; ordinary stale-tolerant reads may go to a follower. Production implementations should use driver-level replication status, retry budgets and a precise session-consistency token.

### 💻 Python / implementation sketch

```
from dataclasses import dataclass
@dataclass
class ReadPolicy:
    recently_wrote: bool
    requires_latest: bool = False
async def read_order(order_id, policy, leader, replicas):
    if policy.requires_latest or policy.recently_wrote:
        return await leader.fetch_order(order_id)
    # A production router should select only a healthy replica
    # whose replay position satisfies any session token.
    replica = replicas.choose_healthy()
    return await replica.fetch_order(order_id)
```

## 6.10 Failure and recovery checklist

- Leader process failure: detect, fence, promote, reconfigure and verify.
- Network partition: avoid two writers; decide which side retains quorum.
- Replica lag: remove lagging replicas from sensitive reads and alert.
- Replica corruption: restore from clean backup or rebuild from trusted leader.
- Region loss: execute documented RPO/RTO runbook and validate data.
- Read scaling: measure replica CPU and lag; adding readers can increase replication and storage pressure.

### 💻 Extended Python lab

# A small model for quorum arithmetic (not a replication protocol).

```
def quorum_intersection(n: int, read_q: int, write_q: int) -> bool:
    if not (1 <= read_q <= n and 1 <= write_q <= n):
        raise ValueError("quorums must be between 1 and N")
    return read_q + write_q > n
```

print(quorum_intersection(3, 2, 2)) # True # Intersection is necessary in common quorum designs, but the # consistency guarantee also depends on versioning and protocol rules. Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A SaaS database serves local reads in two regions and stores order writes in one primary region. Use asynchronous followers for analytics and stale-tolerant browsing, but route checkout and read-after-order requests to the primary or wait until a replica has replayed the session commit position. Specify the primary failure process, fencing and acceptable RPO. Backups must be separate from replicas. If regional writes are required, do not simply add a second leader; define a multi-region conflict and transaction model first.

### Interview questions and answer expectations

**Q. What is the difference between synchronous and asynchronous replication?**

**Q. Why can replicas return stale data?**

**Q. How do you prevent split brain?**

**Q. Can replication replace backups?**

**Q. Explain RPO and RTO with a concrete database failure.**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Design read routing that guarantees read-your-writes for a session.
- Model primary loss with one up-to-date and one lagging replica.
- Choose replication mode for analytics, shopping cart, and order ledger.

Operational metrics and signals

- Replication lag in time and log position
- Replica health and replay throughput
- Failover detection and promotion duration
- RPO observed during failover tests
- Read routing distribution and stale-read indicators
- Backup restore duration and integrity checks

Chapter mastery checklist

- Replication copies data; sharding partitions ownership.
- Sync replication trades latency/availability for a smaller loss window.

- Async replication has lag and potential failover data loss.
- Failover needs quorum/control and fencing to prevent split brain.
- Replicas improve availability but do not replace backups.

## 🔬 Advanced Engineering Clinic

Replication is a protocol for keeping copies useful under failure. The critical questions are what is acknowledged, what can be lost, how a new leader is fenced, and which reads may be stale.

Client write Primary commit log / WAL write + acknowledgement policy Replica A replay position LSN x WAL / log stream Replica B replay position LSN x WAL / log stream Failover controller lease + epoch / fencing fence old writer Promoted primary new epoch promote eligible replica

What acknowledgement actually promises An acknowledgement is meaningful only relative to a failure model. If a primary acknowledges after local disk flush, the write may survive a process crash but be lost if the entire machine or zone is destroyed before replication. If the primary waits for one or more replicas to flush, the acknowledged write may survive more failures, but write latency and availability depend on those replicas. Synchronous replication is not a magic guarantee. Define whether the replica has received bytes, written them to durable storage, or replayed the transaction so it can serve reads. A quorum acknowledgement must specify membership and failure assumptions. For N replicas and quorum sizes R/W, the condition R + W > N gives set intersection under a fixed membership, but does not alone establish linearizability or protect against every reconfiguration and sloppy-quorum case. RPO describes the acceptable amount of data loss; RTO describes the acceptable recovery time. A zero RPO claim needs a precise failure scope and durable acknowledgement rule. A small RTO requires tested promotion, endpoint discovery, connection refresh and client retry behaviour. Replication and backup are separate: replication can copy accidental deletion or corruption. Acknowledgement policy Latency tendency Possible loss on primary loss Local commit only Lowest Recent unreplicated commits Async replica send Low to moderate Commits not yet durable on replica Wait for replica flush Higher Lower within stated replica/failure assumptions Quorum durable acknowledgement Often highest Bounded by quorum and membership protocol

Failover, fencing and split brain Failure detection is uncertain: a slow node can look dead, and a network partition can isolate a healthy primary

```
from a controller. If two nodes both accept writes, the system has split brain. A safe design needs an authority
```

mechanism, not just a timeout. Leases, consensus-backed terms or epochs can grant one writer authority at a time. Fencing tokens are monotonically increasing epochs. Every write carries the current token; storage rejects writes from an older token after a new leader is elected. This protects against a paused old process waking up and continuing to write after its lease expired. The fencing check must be enforced at the resource being protected, not merely in the coordinator process. Promotion should select a replica according to a documented durability policy. If the most up-to-date replica is unavailable, the system must decide whether to wait (availability loss) or promote a lagging replica (possible data loss). After promotion, reconfigure followers, invalidate stale connection pools and verify that the old primary cannot rejoin as writer. Advanced implementation note # Conceptual fencing check at a write boundary.

```
class FencedStore:
    def __init__(self):
        self.epoch = 0
        self.value = None
    def promote(self, new_epoch: int):
        if new_epoch <= self.epoch:
            raise ValueError("epoch must increase")
        self.epoch = new_epoch
    def write(self, epoch: int, value):
        if epoch != self.epoch:
            raise RuntimeError("stale writer fenced")
        self.value = value
```

Read routing with consistency requirements Replicas are often used for read capacity, but routing every read to a replica can violate read-your-writes. After writing, the client may carry a commit position or version token; a replica is eligible only after it has replayed at least that position. Otherwise route to the primary or wait within a bounded deadline. Lag is not only milliseconds. Monitor lag in bytes, log sequence position, transaction age and replay queue. A replica can report a small time delay while missing a large transaction, or be caught up in bytes but unable to serve because replay conflicts or recovery is running. Read routing should use the database's actual replication progress signal. Failover runbooks should include backup integrity, point-in-time recovery, replica rebuild, topology changes and verification of application-level invariants. A green health endpoint does not prove that the promoted database contains every acknowledged transaction. Requirement Read route Latest committed value Primary or replica proven caught up to commit token Eventual feed / analytics Replica or projection with visible staleness Read-your-writes Sticky primary or version-aware replica wait Read after disaster failover Verify recovery point against declared RPO

## ❓ Advanced Design-Review Questions

- A primary fails after acknowledging a write that has not reached any replica. What can the promoted node

safely claim?

- Explain how a fencing token prevents a paused former primary from writing after failover.

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 07. Consistent Hashing

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

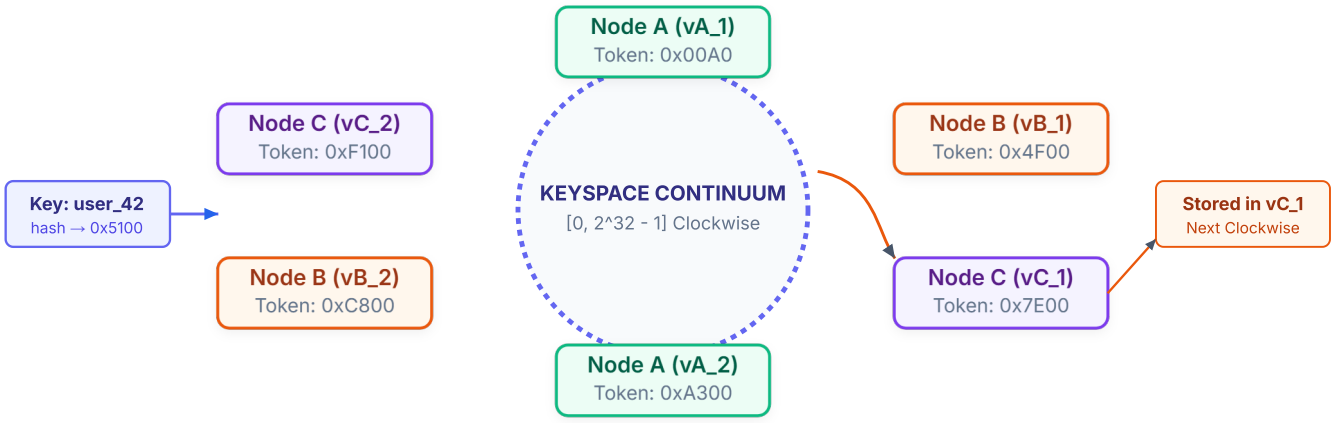
Imagine assigning books to a changing set of shelves. A naive rule based on shelf count can move most books whenever a shelf is added. Consistent hashing arranges keys and nodes on a ring so that a node change moves only a limited portion of keys.

### 🤖 AI INFRASTRUCTURE LENS: CONSISTENT HASHING FOR KV-CACHE AFFINITY ROUTING

**Eliminating Prefill Recomputation via Consistent Prefix Routing:** In multi-turn chat applications or agent workflows, users repeatedly pass the same growing conversation history or shared system prompt. If request *T* + 1 lands on a different GPU worker than request *T*, the entire context must undergo prefill GEMM computation from scratch, wasting compute and multiplying Time-to-First-Token (TTFT).

**Consistent Hash Affinity & Its Physical Boundaries:**

- **Prefix Fingerprint Hashing:** The ingress router hashes the SHA-256 fingerprint of the shared prompt prefix (system prompt + initial few-shot tokens) onto a consistent hash ring with virtual nodes (vnodes) to direct matching requests to the same GPU worker pool.
- **Cache Hit Dynamics & Eviction Pressure:** While prefix routing maximizes the probability of reusing cached Key-Value blocks, real-world hit rates depend heavily on GPU High Bandwidth Memory (HBM) headroom. Under high concurrency, serving engines (e.g., vLLM) reach the watermark threshold and evict older KV blocks under LRU or prefix-tree eviction policies, causing cache misses despite correct routing.
- **Hotspot Mitigation with Bounded Load:** Viral system prompts can concentrate disproportionate traffic onto a single GPU node. Production architectures implement **Bounded-Load Consistent Hashing** (Mirrokni et al.): if a worker's active KV memory or queue depth exceeds $c × ( ext{average load})$, the router diverts requests clockwise to the next available ring replica, balancing load while preserving locality.



### 🎯 Learning Outcomes

- Explain modulo hashing and why node-count changes cause remapping.
- Build a hash ring and map keys to the next clockwise node.
- Use virtual nodes to improve balance.
- Understand movement, hotspots, replica placement and failure behaviour.
- Distinguish cache routing from database shard ownership.

Logical hash ring (clockwise) Node A virtual points Key X clockwise order Node B Key Y Node C wrap around

## 7.1 The modulo hashing problem

A common simple placement rule is shard = hash(key) mod N. It distributes keys across N nodes when the hash is well behaved. But when N changes, most modulo results change. For example, a key mapped to node 2 out of 3 may map to node 0, 1, 2 or 3 after the node count becomes 4. Large remapping can cause cache misses, data movement and load spikes. For an ephemeral cache, mass remapping can temporarily overload the source database. For a persistent key-value store, it can require moving large amounts of data. A stable mapping strategy reduces unnecessary movement when the topology changes.

## 7.2 Ring construction and lookup

Choose a hash function that maps both node identifiers and data keys into a large numeric space, such as 0 through 2^m-1. Sort node positions on a ring. To place a key, hash it and move clockwise to the first node position greater than or equal to the key hash; wrap to the first node if no later position exists. This gives each node the interval since its predecessor.

When a node is added, it takes ownership of the interval immediately preceding its position. Only keys in that interval should move to the new node. When a node is removed, its interval is taken by its clockwise successor. The expected movement is roughly proportional to the fraction of ring space owned, assuming well-distributed hashes.

## 7.3 Virtual nodes and balance

One physical server placed at one ring position can own a disproportionately large interval. Virtual nodes (vnodes) place multiple points for each physical server. A key maps to a virtual point, which resolves to its physical server. More vnodes generally improve balance and make capacity weighting more flexible, at the cost of a larger ring and more metadata. Weighted consistent hashing can assign more virtual points to a more powerful node. The weights should represent safe capacity, and changes should be gradual. Vnodes reduce statistical imbalance but do not eliminate workload skew: one hot key can still dominate its owner.

## 7.4 Node joins, leaves and failure

On a planned join, add the node positions and move only the newly owned intervals. A migration process copies data and updates ownership. On an unplanned failure, the failed node's key intervals can be served by successor replicas if replication exists. Hash placement alone does not provide durability; it only tells where a key should

map. Use a topology version so clients do not route according to incompatible ring views. During transition, a request may need to query the old owner, new owner or a forwarding path. A cache can tolerate a miss and refill; a persistent store must guarantee that reads and writes are not lost during ownership transfer. Before: ring owns A, B, C Keys in interval (A,C] Add node D After: only interval split moves A/C ownership adjusts Other intervals remain mapped unaffected keys

distribution.

## 7.5 Replication and successor lists

Distributed stores often replicate a key to the next R distinct physical nodes in ring order. This spreads copies across failure domains. Do not count two virtual points belonging to the same physical server as two independent replicas. Placement should account for zone or rack diversity so one power or network failure does not remove every copy. Replica placement, quorum reads/writes, repair and hinted handoff are separate protocol choices. Consistent hashing can determine candidate owners, but it does not define conflict resolution or consistency guarantees.

## 7.6 Hash functions and practical choices

A hash used for placement should be deterministic across processes and versions, well distributed and stable. Python's built-in hash for strings is deliberately randomised between interpreter processes by default, so it must not be used for durable cross-process placement. Use a specified algorithm such as SHA-256, BLAKE2 or a fast non-cryptographic hash with fixed seeds and compatibility guarantees. A cryptographic hash may be slower than a specialised high-speed hash, but routing computation is often tiny compared with network access. Choose based on throughput, attack surface and ecosystem compatibility. If keys are attacker-controlled, predictable weak hashes may allow deliberate hotspotting.

## 7.7 Python implementation with virtual nodes

This implementation is suitable for learning and small simulations. It uses SHA-256 for deterministic mapping and bisect for ring lookup. A production implementation also needs node removal, duplicate-position handling, a versioned topology, replication placement and a migration protocol.

### 💻 Python / implementation sketch

```
import bisect
import hashlib
class HashRing:
    def __init__(self, virtual_nodes=100):
        if virtual_nodes < 1:
            raise ValueError("virtual_nodes must be positive")
        self.virtual_nodes = virtual_nodes
        self.positions = []
        self.owner = {}
    def _hash(self, value: str) -> int:
        digest = hashlib.sha256(value.encode()).digest()
        return int.from_bytes(digest[:8], "big")
    def add_node(self, node: str):
        for replica in range(self.virtual_nodes):
            point = self._hash(f"{node}#{replica}")
            if point in self.owner:
                raise RuntimeError("hash collision; use collision buckets")
            self.owner[point] = node
            bisect.insort(self.positions, point)
    def get_node(self, key: str) -> str:
        if not self.positions:
            raise RuntimeError("empty ring")
        point = self._hash(key)
        index = bisect.bisect_left(self.positions, point)
        if index == len(self.positions):
            index = 0
        return self.owner[self.positions[index]]
```

ring = HashRing(virtual_nodes=100) for node in ("cache-a", "cache-b", "cache-c"): ring.add_node(node) print(ring.get_node("tenant-42:profile"))

## 7.8 Consistent hashing versus database sharding

Consistent hashing is a placement technique. Database sharding is an operational data architecture. A database needs schema and index management, transaction boundaries, unique identifiers, backups, migrations, query routing and recovery. A hash ring may help route cache keys or partitioned records, but does not provide those capabilities.

For fixed database partitions, a directory or virtual-bucket map may be easier to operate than a ring. A directory can move a tenant explicitly and support exceptional large tenants. Consistent hashing is attractive when a relatively stable key-to-node mapping and incremental node changes are central.

## 7.9 Testing balance and movement

Test at two levels. Distribution test: generate millions of representative keys and measure key counts per node and per vnode. Movement test: record assignments, add or remove a node, and calculate the percentage that changes. Also test weighted nodes, skewed key popularity and failure-domain placement. A mathematically balanced ring does not guarantee balanced traffic if key popularity is uneven. Expected moved key share when

adding one equal-capacity node: approximately 1 / (N + 1) under uniform hashing. This is an expectation, not a guarantee.

## 7.10 Operational traps

- Using Python built-in hash() for cross-process or durable placement.
- Changing the ring without topology versioning.
- Assuming hash placement replicates or protects data.
- Counting virtual nodes on the same machine as independent replicas.
- Ignoring hot-key traffic skew.
- Moving persistent data without a copy/catch-up/cutover protocol.
- Failing to test key movement after membership changes.

### 💻 Extended Python lab

# Measure movement after adding a node.

```
def assignments(ring, keys):
    return {key: ring.get_node(key) for key in keys}
```

keys = [f"customer:{i}" for i in range(100_000)] old = assignments(ring, keys) ring.add_node("cache-d") new = assignments(ring, keys) moved = sum(old[k] != new[k] for k in keys) print(f"Moved {moved / len(keys):.1%} of keys") # For stable results, keep the hash, vnode count and topology fixed. Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A distributed cache has 12 nodes and 80 million keys. A node is added for capacity. Modulo hashing can remap a large share of keys, creating a cache miss wave. Use a consistent-hash ring with virtual nodes or a stable virtual-bucket directory. Estimate bytes to move, throttle migration, warm popular keys, and maintain a fallback to old owners until cutover. Monitor movement, per-node QPS and miss spikes. For a persistent data store, include replicas and a durable migration process; a cache can simply refill from source, but the source must be able to withstand it.

### Interview questions and answer expectations

**Q. Why does modulo hashing move many keys when N changes?**

**Q. How do virtual nodes help?**

**Q. Does consistent hashing solve hot keys?**

**Q. How do you combine ring ownership with replication?**

**Q. Why is Python hash() unsafe for distributed placement?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Simulate 100,000 keys across 3 nodes, then add a fourth. Compare modulo and ring movement.

- Add node removal and weighted virtual nodes to the Python ring.
- Design migration for a persistent key-value store with 3 replicas.

Operational metrics and signals

- Key and byte distribution per node
- QPS and p99 latency per node
- Movement percentage and bytes migrated
- Hot-key concentration
- Topology version mismatch and routing failures
- Replica health and read repair activity

Chapter mastery checklist

- Consistent hashing reduces remapping when membership changes.
- Vnodes improve statistical balance and enable weighting.
- A ring maps keys; it does not itself replicate, migrate safely or guarantee consistency.
- Stable, deterministic hash functions and topology versions are required.
- Measure traffic skew as well as key-count balance.

## 🔬 Advanced Engineering Clinic

Consistent hashing is best understood as an ownership function plus a migration protocol. The ring reduces remapping, but it does not eliminate skew, replication, failure detection or data movement.

Hash space 0 … 2^m1 clockwise successor ownership A: virtual nodes B: virtual nodes C: virtual nodes key hash h(k) first clockwise vnode Add D only interval before D moves ownership interval changes

Replication factor and ownership In a basic ring, each key maps to the first node encountered clockwise from its hash position. To survive node loss, systems often replicate a key to the next R distinct physical nodes, not merely the next R virtual points. The distinction matters because several adjacent virtual nodes may belong to the same machine. Replica placement should also consider rack or availability-zone diversity. When a node joins, it takes ownership of the arc between its predecessor and its position. When a node leaves, its arc transfers to the next owner. This limits the amount of key movement relative to modulo hashing, but data still has to move. A control plane must track ownership, migration state, replication acknowledgements and node epochs. Virtual nodes improve statistical balance by giving each physical node many positions. More vnodes reduce variance but increase metadata and ring-management work. Weighted capacity can be represented by assigning more virtual positions to stronger nodes, but capacity weight is only an approximation: memory bandwidth, network and request cost may differ. Concern Design requirement Node failure Replicate to distinct failure domains Node join Transfer newly owned intervals and verify data Weighted capacity Assign vnode count proportionally, then measure skew Membership churn Version ring configuration and coordinate clients Hot key Replicas alone do not split one hot key’s write load Consistent hashing versus rendezvous hashing Rendezvous (highest-random-weight) hashing scores each candidate node for a key and selects the node with the highest score. Adding or removing a node changes ownership for a limited subset of keys without explicitly maintaining a circular ring. It is easy to reason about and can support weighted variants, but a naive implementation scores every node for every

lookup, O(N) per key. A ring lookup is typically O(log V) with V virtual nodes using a sorted array and binary search. It is attractive when the node set is large and membership is stable enough to maintain a ring. Rendezvous hashing can be simpler for small node sets or when a deterministic per-key score is convenient. Both approaches require stable hash functions and consistent membership views.

Neither algorithm automatically gives balanced load when key popularity is skewed. Hashing balances key counts or key space, not necessarily bytes, CPU cost or request frequency. Measure hottest partition and rebalance by observed load when necessary. Advanced implementation note

```
import hashlib
def rendezvous_owner(key: str, nodes: list[str]) -> str:
    if not nodes:
        raise ValueError("no nodes")
    def score(node: str) -> int:
        raw = f"{key}|{node}".encode()
        return int.from_bytes(hashlib.sha256(raw).digest(), "big")
    return max(nodes, key=score)
# Keep node identifiers and hash algorithm stable across clients.
```

print(rendezvous_owner("customer:42", ["A", "B", "C"])) Data movement and migration protocol A new owner should not immediately serve reads merely because its hash interval changed. Copy the data, continue capturing writes, verify checksum and version ranges, then atomically advance the ring epoch. During transition, requests may be forwarded to the old owner or dual-read for verification. Define a single write authority at every instant to avoid divergent values. Clients may cache ring metadata. A stale client can send a request to the old owner after migration. The old owner should return a redirect or forward using the new epoch; the new owner should reject writes from stale epochs if the topology requires fencing. Keep migration idempotent and observable. Movement expectation for N equally weighted nodes: Adding one node to N existing nodes moves approximately 1/(N+1) of key space under an ideal balanced ring. This is an expectation, not a guarantee for skewed keys or vnode placement.

## ❓ Advanced Design-Review Questions

- Compare ring hashing and rendezvous hashing for 20,000 cache nodes. Discuss lookup cost, membership

updates and skew.

- A key is extremely hot but all keys are evenly distributed. Why does adding virtual nodes not solve the hot

key? Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

PART II • DISTRIBUTED DATA • CHAPTER 08

# 08. CAP Theorem

*CORE SYSTEM DESIGN*

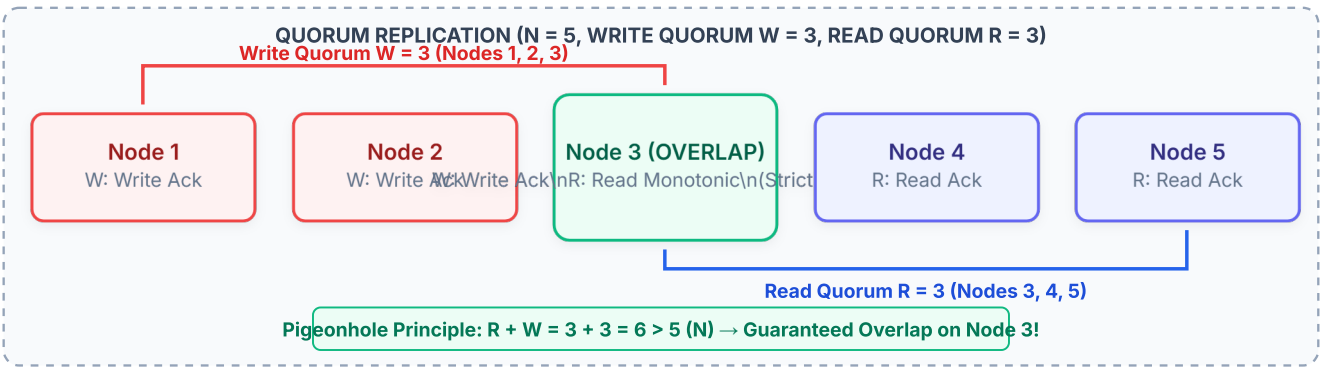
### 💡 INTUITIVE MENTAL MODEL

Two branches of a bank lose their communication link. They can each remain open and accept local activity, or they can stop some operations to avoid contradictory records. The network split forces an explicit choice; it is not merely a performance issue.

### 🤖 AI INFRASTRUCTURE LENS: CAP TRADE-OFFS IN AUTONOMOUS AGENT PLATFORMS

**The Duality of Speculation vs. Action Execution:** Autonomous AI agents (e.g. AutoGPT, LangGraph, CrewAI) navigate two distinct consistency regimes during problem-solving:

- **Thought / Planning Loop (AP - Availability & Partition Tolerance):** When the agent generates internal reasoning steps, reflects on observations, and queries vector memory, it prioritizes availability and low latency. Eventual consistency across agent memory stores is acceptable.
- **Tool / Action Execution (CP - Consistency & Partition Tolerance):** When the agent invokes an external tool (e.g. `transfer_funds()`, `drop_table()`, `send_client_email()`), the system MUST enforce strict linearizable consistency. A network partition cannot allow speculative duplicate execution. The action must be fenced by transactional locks.




## 🎯 Learning Outcomes

- Define consistency, availability and partition tolerance in CAP precisely.
- Explain why the trade-off appears during a partition.
- Distinguish CAP consistency from ACID consistency and ordinary cache freshness.
- Classify operations by whether they should reject or accept during a partition.
- Avoid simplistic CP/AP labels and apply CAP to a concrete data operation.

Partition A Partition B Client A Replica A Network link unavailable Client B Replica B

both linearizable consistency and availability for every request.

## 8.1 State the theorem carefully

CAP is commonly phrased as: in the presence of a network partition, a distributed system cannot guarantee both consistency and availability for every operation. Here, consistency means an atomic/linearizable view of a single-copy object: operations behave as if they occurred in one real-time order consistent with their invocation and completion. Availability means every request sent to a non-failing node eventually receives a response, and the response is not an error or a result from an operation that cannot be completed under the system's rules. Partition tolerance is not an optional feature in a networked distributed system. Messages can be delayed or dropped, and nodes can become unable to communicate. The practical decision is what the system does

during a partition: reject or block operations that cannot be made consistent, or continue to answer with potentially divergent state.

## 8.2 Why the conflict exists

Imagine two replicas, A and B, that hold a bank balance of 100. A network partition separates them. A client writes 80 to A and another client reads from B. If B must respond successfully using its local copy, it may return 100 even though the write to A has completed. A linearizable system cannot allow that response if the write completed before the read began. To preserve linearizability, B must know whether a write happened elsewhere or refuse/wait until it can safely establish the result. During a partition it cannot always do that. If it responds anyway, it sacrifices the strict consistency guarantee for that operation. This is the essential impossibility intuition.

## 8.3 CP and AP as operational choices

A CP-style choice prioritises consistency during a partition: a node that cannot confirm it can safely participate in the consistency protocol may reject or wait for some operations. This protects a single coherent history but can reduce availability in the isolated partition. A distributed lock service or a strongly consistent metadata store often chooses this behaviour. An AP-style choice continues accepting operations in each reachable partition, then reconciles later. This improves availability for operations that can tolerate conflicts or merge. A shopping wishlist or social reaction count may accept local updates; a bank debit generally cannot safely merge two independently accepted withdrawals without additional business rules. CP and AP are simplifications: real systems choose per operation, data type and failure mode. Choice during partition Behaviour Example fit Cost Consistency-prioritising Reject/wait when safe ordering cannot be established Unique reservation, lock ownership, account debit Reduced availability in isolated side Availability-prioritising Accept local operations and reconcile Likes, drafts, some collaborative state Conflicts, stale reads or merge complexity

## 8.4 CAP consistency is not ACID consistency

The word consistency is overloaded. ACID consistency means transactions preserve declared database invariants. CAP consistency refers to single-copy atomic/linearizable behaviour of a distributed object. A

database can be ACID-compliant on each node and still expose stale replicas across nodes. Conversely, an AP design can preserve local transactional invariants while allowing temporary conflicts between replicas. Do not confuse CAP consistency with the consistency of a cache entry or eventual consistency. CAP uses a specific model of availability and linearizable consistency under partition. When discussing a design, define the operation, failure scenario and guarantee rather than saying only "we choose consistency."

Network partition Consistency-prioritising path Availability-prioritising path Wait / reject if quorum or authority unavailable Accept local update; reconcile later

## 8.5 Partition is not the same as node failure

A node crash removes one participant. A network partition divides participants into groups that cannot communicate reliably. The remaining nodes may all be alive and serving local clients. This is why partition behaviour is so important: a health check from one side may not prove the other side is dead. Partial failure is a normal distributed-systems condition. Timeouts are imperfect suspicions, not proof. A design must state what quorum is needed, which side can make progress, how leaders are fenced and whether operations can be retried.

## 8.6 CAP through concrete operations

Consider four operations. Reading a public profile can often return slightly stale data, so an AP-like response may be acceptable. Adding an item to a wishlist can often accept local intent and reconcile. Reserving the last hotel room should not allow two isolated regions to both promise the same room; it needs a consistency-preserving authority or a reservation protocol. Debiting an account must preserve financial invariants and should fail safely when authority cannot be established. The system can make different choices for different data. A single product may use an AP-like view count, CP-like inventory reservation and an eventually consistent recommendation projection. This is more accurate than labelling an entire product CP or AP.

## 8.7 Quorums and the limits of a simple rule

In a common replicated-register model with N replicas, read quorum R and write quorum W, the condition R + W > N ensures read and write sets overlap. This is useful intuition for seeing a newer write. But overlap alone is not enough for linearizability: the protocol also needs versioning, write ordering, conflict handling and rules for concurrent writes. Some systems use consensus or a leader to provide stronger guarantees. Quorum availability depends on which nodes can communicate. A system configured for majority quorum may stop writes on a minority side of a partition. That is a deliberate consistency/availability choice. A system that accepts writes on both sides needs a conflict model and weaker consistency semantics.

## 8.8 CAP design method

- Name the data object and operation: read, write, reserve, increment, transfer or lock.
- Name the partition: which nodes cannot communicate, and which clients can still reach each side?
- State the required consistency property in precise terms.
- State what counts as an available response, including whether an error is allowed.

- Choose the behaviour during partition: reject/wait or accept/reconcile.
- Explain recovery: log replay, conflict resolution, deduplication and client-visible outcome.

## 8.9 Common CAP misconceptions

- "Choose any two of three" is an oversimplified mnemonic; partition tolerance is assumed in a real distributed

setting.

- CAP consistency is not the same as ACID constraint consistency.
- A network partition is not the same as a single server crash.
- A system is not permanently CP or AP for every operation and failure mode.
- High availability does not mean every request must succeed regardless of validity; the formal definition

concerns every request to a non-failing node under the model.

- Quorum intersection is not, by itself, a complete linearizability proof.

### 💻 Extended Python lab

# A deliberately simplified quorum-intersection check.

```
def intersects(n: int, read_q: int, write_q: int) -> bool:
    if min(n, read_q, write_q) < 1 or max(read_q, write_q) > n:
        raise ValueError("invalid quorum sizes")
    return read_q + write_q > n
```

assert intersects(3, 2, 2) assert not intersects(3, 1, 1) # This condition alone does not implement a linearizable register. Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough Design a ticketing service selling the final seat. A partition separates two regional write paths. If each side independently accepts a reservation, the same seat may be sold twice. A consistency-preserving approach routes reservation to a quorum or single fenced authority; the isolated region returns a retryable failure or accepts a non-binding waitlist intent. The read-only seat map may be stale, but final allocation must atomically claim the seat. Explain the customer experience during the partition and how abandoned reservations expire.

### Interview questions and answer expectations

**Q. State CAP without using the "pick any two" shortcut.**

**Q. Why is partition tolerance not an optional feature?**

**Q. Give one operation where AP behaviour is acceptable and one where it is dangerous.**

**Q. How does quorum intersection relate to consistency?**

**Q. Can one product contain both CP-like and AP-like components?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Classify profile view, wishlist add, payment debit and seat reservation during a partition.
- Draw a two-region partition and show what each side returns.
- Explain CAP consistency versus ACID consistency to a new engineer.

Operational metrics and signals

- Quorum availability and rejected writes
- Partition detection and duration
- Stale-read rate and conflict count
- Reconciliation backlog and resolution outcomes
- Client retry volume and timeout rate

Chapter mastery checklist

- CAP is a statement about guarantees during network partition.
- Consistency means an atomic/linearizable single-copy view in the CAP model.
- A CP-style path may reject or wait; an AP-style path may accept divergent updates.
- The right choice belongs to each operation and invariant.
- Quorum overlap is helpful but not a complete consistency protocol.

## 🔬 Advanced Engineering Clinic

CAP is not a product label and not a claim that a database is permanently “CP” or “AP”. It describes a choice that becomes unavoidable for an operation when replicas are partitioned and cannot coordinate. Client / region A Replica A reachable by A Client / region B Replica B reachable by B Network partition messages cannot cross CP choice reject / wait to preserve one value coordination unavailable AP choice accept locally; reconcile conflicts local acceptance policy

Formalise the guarantee before classifying the system In the classic CAP formulation, consistency means a single-copy, linearizable view: completed operations appear to take effect atomically in an order consistent with real time. Availability means every request received by a non-failing node eventually receives a non-error response. Partition tolerance means the system continues to face message loss or unbounded communication delay between groups; it cannot assume the network will always deliver messages. When no partition exists, a system may coordinate strongly and still be available, subject to ordinary failures and latency. CAP’s sharp trade-off is during a partition. If two sides must both answer every request, they may accept conflicting writes. If they must preserve one linearizable value, one side may need to reject or wait for coordination. “AP” does not mean every request always succeeds, and “CP” does not mean the database is unavailable for all operations. The theorem is a model of a particular operation and failure interval. A system can serve cached reads on one path, reject account debits on another, and accept independent wishlist additions on a third. Operation Possible partition policy Reason Read public article Serve local replica/cache Staleness may be acceptable Debit bank account Reject or route to authority Avoid double-spend / invariant violation Add wishlist item Accept locally with merge Independent set union may be safe Reserve last seat Coordinate a single authority Prevent overselling Update presence Accept ephemeral local state Short-lived inconsistency is tolerable Quorums: useful arithmetic,

incomplete proof For N replicas, write quorum W and read quorum R, the condition R + W > N ensures any read quorum intersects any write quorum when all quorums are drawn from the same fixed replica set. This is a set-intersection property. It does not by itself say which value wins, whether the intersecting replica has applied the newest write, or how membership changes are handled. If N=3, W=2 and R=2, a read set and write set must overlap. A versioning or consensus rule is still needed to identify the latest committed value. If writes are accepted independently on both sides of a partition using sloppy quorums, the normal intersection assumption may fail. Read repair, hinted handoff and anti-entropy affect eventual convergence but are not a substitute for a declared write-order guarantee. For linearizable operations, consensus protocols such as Raft or Paxos coordinate an authority/commit order. Majority quorum usually sacrifices progress on a minority partition to prevent two independent majorities from committing conflicting histories. Understand the protocol and failure model rather than relying on the shorthand "majority means consistent". Quorum intersection condition (fixed membership): R + W > N Example: N=3, R=2, W=2 ⇒ every read quorum intersects every write quorum. This is necessary for some quorum designs, but not sufficient for linearizability.

How to answer a CAP interview question Start with a concrete operation and a partition diagram. Ask whether stale reads are allowed, whether independent writes can merge, whether the operation must be linearizable, and what "available" means to the caller. Then state what each side does during the partition and how recovery reconciles or rejects history. Avoid saying "pick any two" as if a team chooses two permanent product properties. A better statement is: under a network partition, a particular operation cannot guarantee both linearizable consistency and availability at every non-failing partitioned node. The design chooses the contract that best preserves the business invariant. Weak answer Stronger answer "MongoDB is CP." "For this operation and configured write concern, a minority partition cannot commit writes; let us verify the exact deployment guarantee." "AP means eventual consistency." "Local operations are accepted; the system defines merge/conflict handling and convergence conditions." "Network failure means CAP." "A partition is inability to communicate across groups; a process crash is a different failure."

## ❓ Advanced Design-Review Questions

- During a partition, should a collaborative document accept independent text edits? What merge model is

required?

- Why is R + W > N not a complete proof of linearizability in a replicated database?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 09. PACELC

*CORE SYSTEM DESIGN*

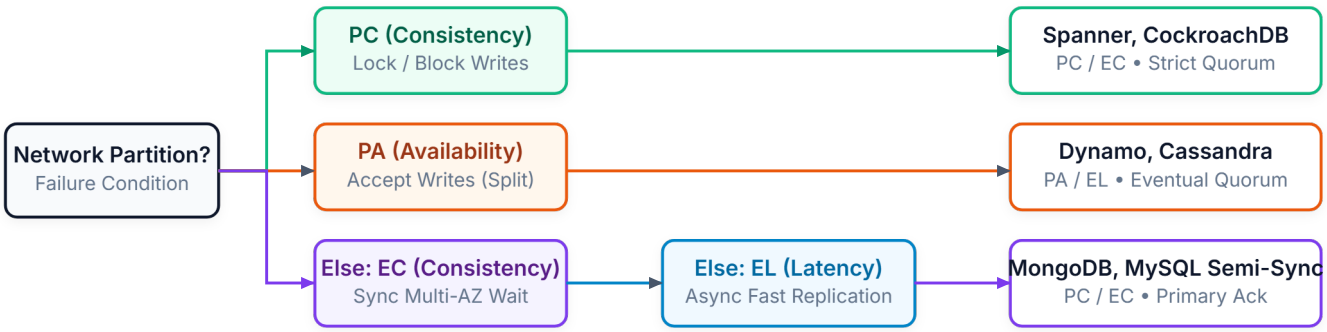
### 💡 INTUITIVE MENTAL MODEL

When roads are blocked, a delivery network chooses whether to stop or take an uncertain route. When roads are clear, it still chooses between a faster nearby warehouse and a slower central warehouse with a more coordinated inventory view. PACELC makes both decisions visible.

### 🤖 AI INFRASTRUCTURE LENS: PACELC THEOREM & SPECULATIVE DECODING

**PACELC in AI Systems:** Under network Partition (*P*), choose between Availability ($A$) and Consistency ($C$). Else ($E$), during normal operations, trade off Latency (*L*) versus Consistency/Accuracy ($C$):

- **Partition Mode (*P/A*):** When upstream LLM provider APIs (OpenAI/Anthropic) suffer network partitions or timeouts, the edge router fails over to local quantized open-source models (Llama-3-8B-Instruct on local GPUs), sacrificing maximum reasoning depth for immediate service availability.
- **Normal Mode (*E/L*): Speculative Decoding** embodies the PACELC latency trade-off. A small draft model (1B parameters) rapidly generates 5 speculative tokens (*L*). The large 70B model verifies all 5 tokens in a *single forward matrix-matrix pass* ($C$). If 4 tokens match, latency drops by 3x without altering the mathematically exact probability distribution of the 70B model.




### 🎯 Learning Outcomes

- State PACELC and distinguish it from CAP.
- Explain the Else case: latency versus consistency when no partition exists.
- Understand why synchronous coordination adds network round trips.
- Apply the model to regional reads, quorum settings and data placement.
- Avoid vendor labels and derive trade-offs from actual operation semantics.

No partition (E) Choose L or C Latency vs Consistency Partition (P) Choose A or C Availability vs Consistency PACELC decision else E if P

coordination can still increase latency.

## 9.1 Expand the acronym

PACELC means: if there is a Partition (P), a distributed system must trade off Availability (A) and Consistency (C); Else (E), when the system is not partitioned, it trades off Latency (L) and Consistency (C). It is an analytical framework rather than a protocol or a universal classification of databases. The useful insight is that consistency costs time even without a failure. A read that must contact a leader or quorum across regions can be slower than a local replica read. A synchronous write waits for remote acknowledgements. The network is healthy, but coordination still adds latency.

## 9.2 The P branch: partition behaviour

The P branch is the CAP situation. If nodes cannot communicate, a system can refuse or delay operations that need a coherent order, or it can continue local operations and reconcile. The choice depends on the data invariant and business contract.

For example, a local draft editor can accept updates offline and reconcile later. A globally unique username claim may require a single authority or quorum to avoid duplicate ownership. PACELC reminds us to explain both the partition response and the normal operating mode.

## 9.3 The E branch: latency versus consistency

Even with no partition, a distributed read can be served locally from a replica that may lag, or it can coordinate with a leader/quorum for a fresher result. Local read is fast but may be stale. Coordinated read is slower but can provide a stronger guarantee. Similarly, a write can wait for local durable commit only or for remote replication acknowledgements. Latency is not just geographic distance. It includes network round trips, queueing, storage flush, coordination, serialization and contention. A cross-region quorum can add tens or hundreds of milliseconds depending on topology. The design should put strong consistency only where the user-visible invariant needs it.

## 9.4 Read and write choices

For a read-heavy social feed, local replicas and caches may be acceptable if the product tolerates a short stale window. For an account balance or inventory reservation, the read may need a leader or a serializable transaction. A write to a user preference can be acknowledged locally and propagated asynchronously; a transfer between accounts needs a transaction or workflow that preserves invariants. Consistency is not a single switch for the entire database. Systems often expose consistency levels or route operations to different storage paths. Make the guarantee explicit: read-your-writes, monotonic reads, bounded staleness, linearizable read, or eventual convergence. Local replica read Low latency possible stale value Leader / quorum read Extra network round trip stronger freshness/order

stronger view.

## 9.5 Regional architecture implications

A multi-region design has choices: active-passive, single-writer with local read replicas, multi-writer, or partitioned ownership by region. Single-writer simplifies ordering but remote writes pay network latency. Multi-writer improves local writes but needs conflict resolution. Region-local partitions reduce coordination for most operations but cross-region transactions become workflows.

Global placement should follow data ownership and user journey. A profile may be owned by one home region while public content is replicated globally. A payment ledger may use a strongly consistent authority while recommendations use asynchronous projections. PACELC helps explain why a hybrid is rational.

## 9.6 A practical decision table

Operation Fast/local choice Stronger coordination choice Feed read Local cache/replica; bounded stale Leader/quorum read for a just-published post if required Inventory display Cached approximate availability Authoritative check at reservation/checkout Seat reservation Accept waitlist intent locally Quorum/authority for final seat claim Profile preference Local write + async propagation Synchronous owner write if immediate visibility required Account transfer Not safe as independent local writes Transactional/ledger authority with durable workflow

## 9.7 Avoid overgeneralising database products

It is tempting to label one database "CP" and another "AP" or to assume a product has one fixed PACELC position. Real guarantees depend on the configuration, topology, consistency level, replication mode, failure model and operation. A database may offer fast local reads in one mode and quorum reads in another. When comparing systems, examine the exact consistency contract, commit acknowledgement, replica placement, failover, partition behaviour and latency under the expected topology. Ask what happens to an acknowledged write if the region fails immediately afterward.

## 9.8 PACELC as a design conversation

- Name the operation and data invariant.
- In normal operation, state whether reads/writes are local or coordinated.
- Quantify the added round trips and expected latency range.
- During partition, state which requests continue and which fail.
- Explain how replicas converge, conflicts are resolved and clients observe results.
- State which operations intentionally use different policies.

### 💻 Extended Python lab

```
# A routing policy should be explicit per operation, not global.
```

```
from enum import Enum
class ReadGuarantee(Enum):
    EVENTUAL = "eventual"
    READ_YOUR_WRITES = "read_your_writes"
    LINEARIZABLE = "linearizable"
async def read_profile(user_id, guarantee, local_replica, leader):
    if guarantee is ReadGuarantee.EVENTUAL:
        return await local_replica.get_profile(user_id)
    # A real implementation must ensure the leader path actually
    # satisfies the requested guarantee.
    return await leader.get_profile(user_id)
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A collaboration product stores documents across three regions. Users expect typing to feel immediate, but final access-control changes must not be served from stale state. Use region-local presence and draft updates with asynchronous propagation or conflict-free merge where feasible. Use an authoritative policy/permission service for sensitive actions. For document edits, define ordering/version semantics and conflict resolution; for billing, use a stronger transactional path. Explain the normal-operation latency and the behaviour if a region is partitioned.

### Interview questions and answer expectations

**Q. What does the E in PACELC add beyond CAP?**

**Q. Why can consistency cost latency without a partition?**

**Q. How would you choose read consistency for an account balance versus a feed?**

**Q. Why is it misleading to label an entire product AP?**

**Q. How does multi-region topology affect the trade-off?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Classify five operations by their desired normal and partition behaviour.
- Estimate the latency impact of one extra cross-region round trip.
- Design a hybrid multi-region system with a local feed and global account ledger.

Operational metrics and signals

- Regional read/write latency
- Replica lag and consistency-level distribution
- Cross-region round trips per operation
- Partition duration and rejected operations
- Conflict and reconciliation rate

Chapter mastery checklist

- PACELC adds the normal-operation latency/consistency choice to CAP.
- Remote coordination costs latency even when no component has failed.
- Choose guarantees per operation and business invariant.
- Hybrid consistency is normal; avoid blanket CP/AP labels.
- Quantify the user-visible price of stronger coordination.

## 🔬 Advanced Engineering Clinic

PACELC extends the partition-time CAP discussion by making healthy-network latency a first-class design decision. The useful question is what coordination cost a read or write pays in normal operation. Distributed data operation Is there a partition? P: choose availability or consistency Yes E: choose lower latency or stronger consistency No Accept local / reconcile or reject / coordinate Local read / async replica or quorum / leader round trip

Quantify the latency cost of coordination In a single region, a local cache read may take hundreds of microseconds while a quorum read can require network round trips to replicas. Across regions, a leader-based write may pay at least one inter-region round trip before acknowledgement. A system that waits for multiple replicas to acknowledge can improve durability/consistency but increase tail latency and become sensitive to the slowest required participant. For a sequential request path, dependency latencies add. If an operation waits for a local database (4 ms), a remote quorum (70 ms) and a provider (120 ms), the total is approximately 194 ms plus processing and queueing. Parallel fan-out instead tends toward the slowest branch plus coordination overhead, but every required branch can still determine tail latency. Draw the critical path, not merely the number of services.

Quorum size is not a free dial. Larger acknowledgement sets may improve intersection/durability but increase latency and failure sensitivity. A local read from an asynchronous replica can be fast but stale. A leader read can be fresher but may cross a region. Choose according to the data invariant and user promise. Sequential critical path: $T \approx \Sigma$ dependency latency + processing + queueing Parallel fan-out: $T \approx \max(T1, T2, \ldots, Tn)$ + merge/coordination overhead Compare architecture choices for the same data Consider a user profile stored in three regions. Option A routes all writes to a home-region leader and allows local stale reads. It provides a clear write order but remote users may read stale data. Option B performs quorum reads/writes across regions for stronger guarantees, increasing latency. Option C accepts multi-leader writes and resolves conflicts, reducing write latency but changing the data model and conflict contract. The correct choice depends on what a profile field means. A “last active” timestamp may tolerate approximate merge. A legal name change may require a clear authoritative order. A balance or inventory reservation needs transactional authority, not just low-latency convergence. PACELC helps frame this conversation but does not pick the architecture by itself. Classify data and operations separately. The same service can use strongly coordinated writes for account ownership and eventually consistent projections for a home feed. It is misleading to label the entire application “PA” or “PC” without naming the operation and topology. Data / operation Healthy-network preference Partition preference Presence heartbeat Local low-latency update Accept ephemeral local state Account debit Coordinate authoritative order Reject or wait rather than double debit Social feed projection Fast local read Serve stale projection Inventory reservation Authoritative transactional write Protect no-oversell invariant Use a latency budget, not intuition For each user-facing operation, define an end-to-end deadline. Allocate a sub-budget to each synchronous dependency and include queueing, DNS/TLS, retries and serialization. A retry consumes the same overall deadline; it must not reset the clock. If a dependency cannot fit within the remaining budget, fail fast

or degrade rather than waiting beyond the user contract. Use p95 and p99 measurements, not only averages. PACELC trade-offs are especially visible in tail latency because waiting for a remote replica or the slowest quorum member increases variance. Consider hedged reads only where duplicate work is safe and the extra load cannot worsen congestion.

- Draw the normal path and partition path separately.
- Mark which read can be stale and how stale.
- Name the replica/region coordination required for each write.
- State the user-visible fallback when the coordination deadline is exceeded.

## ❓ Advanced Design-Review Questions

- Compare local leader reads with cross-region quorum reads using an explicit latency budget.
- Which fields of a user profile might tolerate asynchronous multi-leader updates, and which require authority?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

PART II • DISTRIBUTED DATA • CHAPTER 10

# 10. Consistency Models

*CORE SYSTEM DESIGN*

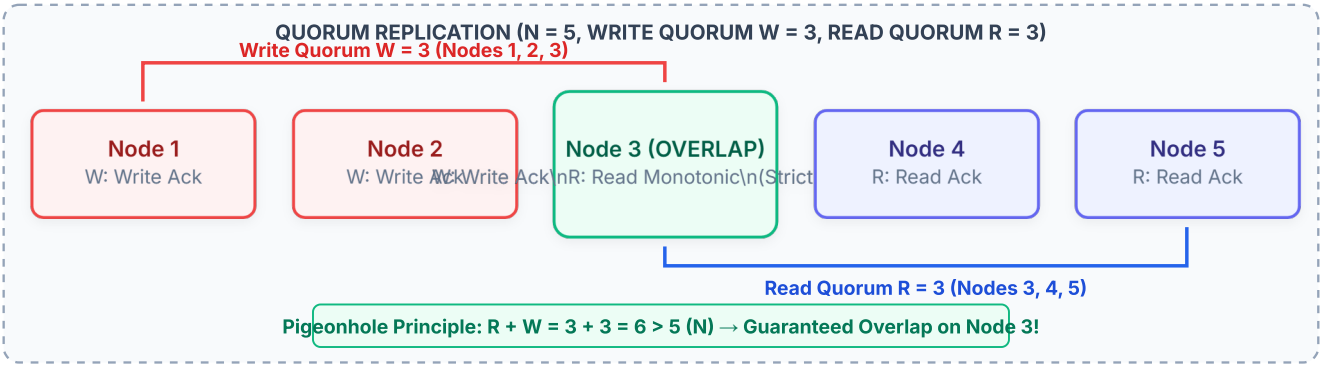
### 💡 INTUITIVE MENTAL MODEL

A shared whiteboard can have rules ranging from "everyone must see each marker stroke in one agreed order" to "each room updates locally and the boards converge later." A consistency model is the contract for what observers may see.

### 🤖 AI INFRASTRUCTURE LENS: MEMORY CONSISTENCY IN MULTI-AGENT SWARMS

**Causality Inversions in Agentic Systems:** In multi-agent architectures (Planner $ ightarrow$ Coder $ ightarrow$ Security Reviewer $ ightarrow$ Tester), agents exchange intermediate state through a shared message bus or vector database.

- If the Security Reviewer reads an eventually consistent memory store before the Coder's update has replicated, it evaluates stale code and passes a security vulnerability.
- **Causal Consistency Enforcement:** Every agent message carries a Lamport Vector Clock. No downstream agent can begin context ingestion until its local dependency vector reflects all parent agent message IDs ($ec{V}_{\text{local}} ≥ ec{V}_{\text{causal}}$), preventing hallucinated reviews of uncommitted code.




## 🎯 Learning Outcomes

- Differentiate linearizability, sequential consistency, causal consistency and eventual consistency.
- Explain session guarantees such as read-your-writes and monotonic reads.
- Understand quorum reads/writes, versioning and conflict resolution.
- Recognise clock and ordering pitfalls.
- Choose the weakest model that still preserves the business invariant.

Client A writes v2 Authoritative order / leader Replica 1 version 2 apply v2 Replica 2 version 1 (lagging) delayed replication Client B reads observes v2 may observe v1

concurrency and ordering matter even on one machine.

## 10.1 Why consistency models exist

Distributed components observe events at different times. A write may commit on one node before another replica applies it. Two clients can update the same record concurrently. Messages can be delayed or reordered. A consistency model defines the relationships an observer is entitled to rely on. The model should be expressed in user-visible terms. "Strong consistency" is vague until you say whether a read must return the latest completed write, whether a session must see its own writes, and how concurrent writes are ordered. Different operations in one application can legitimately use different models.

## 10.2 Linearizability

Linearizability requires each operation to appear to take effect at a single instant between its invocation and response, while respecting real-time order. If write X completes before read Y begins, Y must observe X or a later write. Concurrent operations may be ordered either way, but the resulting history must be equivalent to one legal sequential history that respects completed-before-started relationships. It is intuitive for a single authoritative object such as a lock, unique reservation or account balance. It usually requires coordination, a leader or consensus protocol, and can cost latency across regions. It is stronger than eventual consistency and is not obtained merely by writing to two replicas.

## 10.3 Sequential consistency

Sequential consistency requires operations to appear in one total order that is consistent with each process's program order, but the order need not respect real-time order between different processes. It is weaker than linearizability. A system could place two concurrent operations in a total order that differs from their wall-clock completion order, provided each client's own order is respected. The difference is subtle but important. Linearizability adds a real-time constraint. Sequential consistency is useful as a theoretical model for reasoning about shared memory and distributed operations, but application teams should verify the exact guarantee a database offers rather than infer it from a marketing term.

## 10.4 Causal consistency

Causal consistency preserves cause-and-effect relationships. If operation B depends on seeing operation A, every observer that sees B must also see A first. Independent concurrent operations do not need a single global order. For example, a user posts a comment and then replies to it; a reader should not see the reply without the comment that caused it. Causal metadata can be represented using logical clocks, dependency tracking or version vectors. This is often a useful compromise for social and collaborative systems: preserve meaningful order without coordinating every unrelated update globally. Operation history A: write x=2 completes at t2 B: read x begins t3 returns 1 completed before start C: read x begins t4 returns 2 completed before start

permit that stale observation.

## 10.5 Eventual consistency

Eventual consistency says that if updates stop and communication/reconciliation proceeds, replicas eventually converge. It does not specify how stale a read may be, which value wins during conflict, or whether a client sees its own write. Those are additional guarantees. A system can be eventually consistent but still have long or unbounded staleness under repeated failures. Conflict resolution is essential. Last-write-wins chooses by timestamp or version, but clock skew can discard a logically newer update. Application merge can preserve fields or domain-specific intent. CRDTs provide mathematically defined merge operations for suitable data types, but not every business object can be merged safely.

## 10.6 Session guarantees

Guarantee Meaning Example Read-your-writes A client sees its own completed writes After changing avatar, profile read shows new avatar Monotonic reads A client does not move backward to an older version A feed page does not revert to an earlier state Monotonic writes A client's writes are applied in order Rename then update profile description Writes-follow-reads A write is ordered after the data version it depends on Reply is created after reading a post Bounded staleness Replica lag is limited by a defined bound Dashboard may be at most 5 seconds behind

## 10.7 Quorums and versioned reads

Quorum systems choose a number of replicas that must respond to a read or acknowledge a write. With N replicas, read quorum R and write quorum W, R + W > N gives overlap in a simple set model. Version numbers or timestamps help identify the newest value. A read may contact multiple replicas and return the latest version, then repair older copies. Quorum intersection does not by itself guarantee linearizability. Concurrent writes, sloppy quorums, clock rules, replica membership changes and failure recovery matter. Always read the database documentation for the precise guarantee, and test it under node loss and network delay.

## 10.8 Clocks, timestamps and ordering

Wall clocks are not perfectly synchronised. Clock skew, clock adjustment and leap handling mean a larger timestamp does not necessarily mean a causally later event. Last-write-wins can lose data if it relies on imperfect clocks. Logical clocks and version vectors capture ordering relationships without assuming globally perfect time. Lamport clocks guarantee that if A causally precedes B, then L(A) < L(B), but the converse is not guaranteed: a larger Lamport timestamp does not prove causality. Vector clocks can identify concurrent versions but require metadata that grows with participants or groups. Hybrid logical clocks combine physical-time intuition with logical ordering, but still require a well-defined protocol.

## 10.9 Choose the model from the invariant

For a bank transfer, the invariant is that money is neither created nor lost and both account changes are coordinated. Use a transactional ledger or a durable workflow with explicit semantics. For a public view count, temporary divergence is often acceptable; aggregate asynchronously. For a comment thread, causal ordering can prevent a reply appearing before its parent. For a username claim, uniqueness requires one authority or a

globally coordinated uniqueness mechanism. Start from the business invariant, then choose the weakest consistency that preserves it. Stronger consistency everywhere increases coordination cost, but weaker consistency can create user-visible anomalies or impossible-to-repair business errors.

## 10.10 Python: versioned values and stale-read guard

This simple model illustrates rejecting a cached value that is older than a known minimum version. It is not a distributed consistency protocol, but the pattern is useful when a client carries a commit/version token.

### 💻 Python / implementation sketch

```
from dataclasses import dataclass
@dataclass(frozen=True)
class VersionedValue:
    version: int
    value: dict
class StaleReplica(Exception):
    pass
def accept_read(value: VersionedValue, min_version: int):
    if value.version < min_version:
        raise StaleReplica(
            f"need >= {min_version}, got {value.version}"
        )
    return value.value
```

latest = VersionedValue(42, {"status": "paid"}) print(accept_read(latest, min_version=40))

## 10.11 Common consistency mistakes

- Calling every stale read an eventual-consistency guarantee; eventual convergence needs assumptions.
- Assuming timestamps establish causality.
- Treating quorum intersection as a complete proof of linearizability.
- Using a cache hit for a correctness-critical authorization decision.
- Ignoring client session guarantees after routing reads to replicas.
- Choosing a stronger model than the business invariant requires, then paying unnecessary coordination

latency.

### 💻 Extended Python lab

# A tiny causal-dependency illustration.

```
from dataclasses import dataclass
@dataclass(frozen=True)
class Event:
    event_id: str
    depends_on: tuple[str, ...]
```

post = Event("post-1", ()) reply = Event("reply-1", ("post-1",))

```
def can_publish(event, visible_ids):
    return set(event.depends_on).issubset(visible_ids)
```

assert not can_publish(reply, set()) assert can_publish(reply, {"post-1"}) Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A social application has a post, comments and replies. Feed ranking can be eventually consistent; view counts can be approximate; but a reply should not appear without its parent post. A user who edits their own profile should see that edit immediately. Use causal/session guarantees for interactive flows, a fast asynchronous projection for ranking, and an authoritative write store for profile updates. Explain where a stale replica is acceptable and where the read must be routed to the primary or wait for a version.

### Interview questions and answer expectations

**Q. Define linearizability and contrast it with sequential consistency.**

**Q. What does causal consistency protect?**

**Q. What does eventual consistency not promise?**

**Q. Explain read-your-writes and monotonic reads.**

**Q. Why do wall-clock timestamps not prove causality?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Classify user profile, feed count, reply thread and bank transfer by required model.
- Construct a history that violates linearizability but could be eventual.
- Implement a version token check for read-your-writes.

Operational metrics and signals

- Replica staleness by time/version
- Read-your-writes fallback rate
- Conflict frequency and resolution outcome
- Quorum failure and retry rate
- Client-visible consistency anomalies

Chapter mastery checklist

- Consistency models define legal observations under concurrency and replication.
- Linearizability adds real-time order; sequential consistency does not.
- Causal consistency preserves cause/effect; eventual consistency promises convergence under assumptions.

- Session guarantees are often a practical product-level contract.
- Choose consistency from business invariants, not from a desire to label the whole system strong or weak.

## 🔬 Advanced Engineering Clinic

Consistency models describe allowed histories, not vague product labels. This clinic practices reading concurrent histories and choosing a guarantee from the invariant. Client A: write x=1 Authoritative order commit index 42 commit at 42 Client B: read x Replica applied index 40 stale read possible replication Read result x=1 at index 42 x=0 at index 40

Reason from histories and invariants A consistency model defines which concurrent operation histories are legal. Linearizability requires each operation to appear to take effect at one instant between invocation and response, preserving real-time order for non-overlapping operations. Sequential consistency requires a single total order consistent with each process's program order, but that order need not respect real-time order across clients. Causal consistency preserves cause-before-effect relationships while allowing concurrent independent updates to be seen in different orders. Eventual consistency says replicas converge if updates stop and anti-entropy/reconciliation succeeds. It does not bound staleness, define conflict resolution, or guarantee read-your-writes. Session guarantees strengthen the experience: monotonic reads, read-your-writes, monotonic writes and writes-follow-reads can be provided without requiring every operation to be globally linearizable.

Database isolation levels are related but not identical to distributed consistency models. Serializability and snapshot isolation describe transactional execution anomalies; linearizability describes operation histories relative to real time. A system may offer serializable transactions in one shard but stale reads from an asynchronous replica. Model Guarantee intuition Not guaranteed by the model alone Linearizability One real-time-respecting total order Availability during partition Sequential consistency One order respecting each client's program order Real-time order between clients Causal consistency Cause precedes effect everywhere One order for unrelated concurrent writes Eventual consistency Convergence after updates stop Bounded staleness or conflict policy Read-your-writes Client sees own completed writes Other clients immediately see them Causal metadata and conflict resolution If update B depends on update A, a causal system must not expose B while hiding A. A common way to represent causality is with version vectors or logical metadata that captures which updates a replica has observed. Two versions that are incomparable may be concurrent and require a merge or conflict policy. Last-write-wins based on wall-clock time is simple but can lose a concurrent update and is sensitive to clock skew. Domain-specific merge may be safer: a set can merge by union; a counter can use a CRDT design; an address or account owner may require explicit conflict resolution rather than automatic merging. Logical clocks do not tell physical time. A Lamport timestamp can order events consistently when combined with a node tie-breaker, but it does not by itself detect concurrency. Version vectors can identify causal relationships at higher metadata cost. Choose the representation that matches the required semantics and expected replica count. For version vectors V and W: $V \le W$ component-wise, with at least one strict inequality, means V happened-before W. If neither $V \le W$ nor $W \le V$, they are concurrent. Read tokens and session consistency A practical API can return a commit/version token after a write. The client sends that token on its next read. The service routes to a replica that has applied at least the token, waits briefly for one to catch up, or falls back to the leader. This provides read-your-writes without forcing all reads globally to a leader. The token should be opaque to clients and bound to the right tenant/data scope. It must not be treated as an authorisation credential. The backend validates and uses it as a minimum-version constraint. If no replica catches up before the deadline, return a defined error or leader read rather than silently violating the promised guarantee. Client

action Session guarantee Write profile, then read profile Read-your-writes Read version 10, later read same entity Monotonic reads: do not return version 9 Write A then write B in same session Monotonic writes

Client action Session guarantee Read A, then write dependent B Writes-follow-reads / causal dependency

## ❓ Advanced Design-Review Questions

- Construct a history that is sequentially consistent but not linearizable.
- How can an API implement read-your-writes while still serving most reads from replicas?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

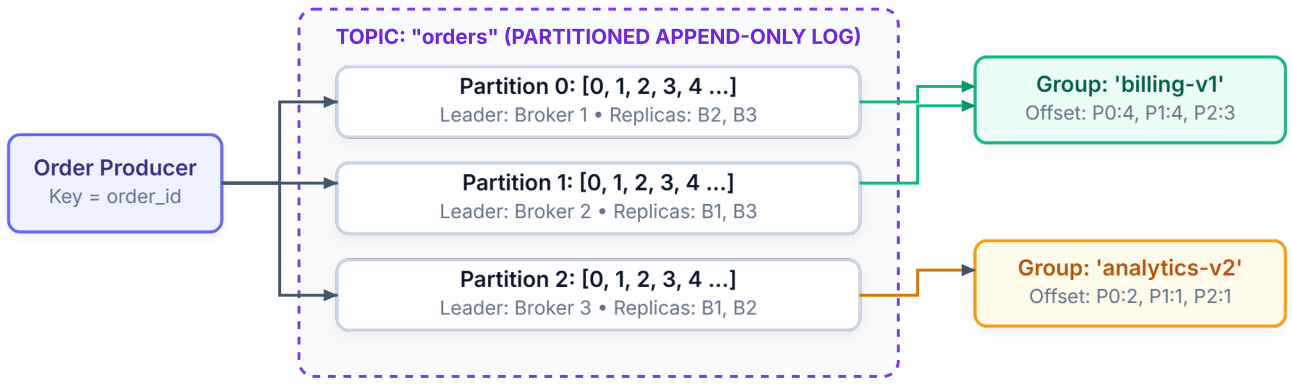
PART III • ASYNCHRONOUS SYSTEMS • CHAPTER 11

# 11. Kafka and Publish/Subscribe

*CORE SYSTEM DESIGN*

> 💡 **INTUITIVE MENTAL MODEL**
>
> A newspaper publisher prints an edition once, then many subscribers read it at their own pace. A traditional queue is more like a ticket handed to one worker; a log is more like a numbered journal that many independent readers can replay.

> 🤖 **AI INFRASTRUCTURE LENS: STREAMING TOKEN INGESTION & TRAINING TELEMETRY VIA KAFKA**
>
> **Decoupling Inference Generation from Downstream AI Observers:** Generating streaming tokens for 50,000 concurrent LLM sessions produces a high-frequency stream of token payloads. Coupling token generation to real-time safety classification, token billing, and analytics creates massive tail latency.
>
> - **Kafka Streaming Architecture:** The inference server emits raw token chunks to a Kafka topic `llm.token.stream` partitioned by `session_id`.
> - **Consumer Group 1 (Token Billing & Quotas):** Aggregates input/output token counts per user and flushes balances to PostgreSQL every 5 seconds.
> - **Consumer Group 2 (Asynchronous Safety & Guardrails):** Scans generated content asynchronously for PII leaks, policy violations, and toxicity, issuing session abort signals via WebSocket if a violation threshold is crossed.
> - **Consumer Group 3 (Fine-Tuning Ingestion):** Filters high-rated conversation turns into an append-only Parquet training corpus in S3 for future RLHF / DPO fine-tuning.




## 🎯 Learning Outcomes

- Understand topics, partitions, offsets, brokers, replication and consumer groups.
- Compare Kafka-style durable logs with queues and ephemeral Pub/Sub.

- Explain ordering, delivery semantics, retention and replay.
- Design idempotent consumers and safe offset commits.
- Implement a basic Python producer/consumer and reason about failure cases.

Topic: orders (partitioned log) Independent consumer groups Order service producer Kafka cluster P0 0 1 2 3 … P1 0 1 2 3 … P2 0 1 2 3 … Billing group Analytics group Email group

applications can read the same event stream at different speeds.

## 11.1 Queue, Pub/Sub and event log

A work queue usually distributes each task to one competing worker so the work is performed once logically, subject to delivery semantics and retries. Pub/Sub broadcasts a message to subscribers that are currently connected or have durable subscription semantics depending on the product. A Kafka topic is a retained partitioned log: consumers track offsets and can replay retained records independently. These models overlap but are not identical. Kafka can support queue-like consumer groups and broadcast-like independent groups. Redis Pub/Sub is low-latency but ephemeral: a disconnected subscriber misses messages. A durable message broker or Kafka is more appropriate when messages must survive consumer downtime.

Model Who receives a message? Retention / replay Common use Work queue One competing worker per task under queue semantics Broker-specific; often removed/acknowledged Background jobs Ephemeral Pub/Sub All currently subscribed listeners Usually no replay for disconnected listener Live notifications, transient signals Durable Pub/Sub Each durable subscription/group Stored until retention/ack policy Integration events Kafka log Each consumer group reads independently by offset Retained log, replayable Event streams, CDC, analytics, integration

## 11.2 Kafka core objects

A broker is a Kafka server. A topic is a named stream. A topic is divided into partitions, and each partition is an ordered append-only log. Each record has an offset within that partition. A producer publishes records; a consumer fetches records and manages progress. A consumer group lets multiple consumers share partitions so that one group can process in parallel. A partition has a leader replica and follower replicas. Producers write to the partition leader; followers replicate. The replication factor controls copy count. The in-sync replica set (ISR) represents replicas sufficiently caught up according to cluster rules. Producer acknowledgement settings and minimum in-sync replica policy affect durability and availability.

## 11.3 Partitions, keys and ordering

Kafka guarantees ordering within a partition, not a total order across a topic's partitions. A record key is commonly used to choose a partition. If customer_id is the key, events for one customer can be ordered in one partition, provided the partitioning strategy and partition count remain compatible. If the key is absent, the producer may distribute records using a sticky or other configured strategy. More partitions increase parallelism and can increase throughput, but they add broker metadata, file handles, replication work and consumer coordination. A consumer group cannot actively assign more consumers than there are partitions for one topic;

extra consumers remain idle. Increasing partition count can change key-to-partition mapping and complicate long-term ordering assumptions.

## 11.4 Consumer groups and offset management

Each consumer group tracks its own offset per partition. Different groups can read the same topic independently. Within one group, partitions are assigned to consumers; on a rebalance, assignments move. Consumers should stop cleanly, commit progress deliberately and handle reprocessing. A rebalance can pause consumption and cause duplicate processing if the offset commit and business side effect are not coordinated. Auto-commit can be convenient but may commit offsets before work is safely complete, or commit less often and reprocess more records after failure. Manual commit after a durable side effect is safer for at-least-once processing, but the side effect itself must be idempotent. Kafka offsets and a database transaction are not one atomic transaction by default.

Producer key = order_id Partition 0 Leader B1 replica B2 hash key Partition 1 Leader B2 replica B3 other keys Group: billing consumer A + B Group: analytics consumer C

independently read the same partition.

## 11.5 Delivery semantics: at-most-once, at-least-once,

exactly-once At-most-once means a record may be lost but is not intentionally redelivered: commit progress before processing. At-least-once means records are not lost under the stated durability assumptions, but can be processed more than once: process, then commit. Exactly-once is a narrowly defined end-to-end property. Kafka transactions can provide exactly-once processing for Kafka-to-Kafka flows when configured correctly, but an external database or payment provider needs its own idempotency/transaction protocol. A practical design often chooses at-least-once delivery plus idempotent effects. Use a stable event_id and a consumer inbox/deduplication table with a uniqueness constraint, or make the downstream operation naturally idempotent. Do not claim "exactly once" because a producer retries or a broker has transactions. Semantics Typical order Failure risk At-most-once Commit offset then process Crash after commit can lose work At-least-once Process then commit Crash before commit can repeat work Exactly-once within a defined boundary Transactional consume/process/produce Scope and external side effects must be explicit

## 11.6 Replication, durability and producer

acknowledgement Producer acks=0 does not wait for broker acknowledgement. acks=1 waits for the leader; if the leader fails before followers replicate, an acknowledged record can be lost. acks=all waits for the current in-sync replica set, and min.insync.replicas constrains how few replicas may acknowledge. Idempotent producer mode prevents duplicates from producer retries within its supported scope. These settings trade write availability and latency against durability. Durability also depends on replication factor, broker storage, unclean leader election policy, failure domains and operational discipline. Three replicas in one rack are not equivalent to copies across independent zones. Acknowledgement semantics should be documented alongside RPO and failover expectations.

## 11.7 Retention, compaction and replay

Retention keeps records for a configured time or size, independent of whether all consumers have read them. Consumers that fall behind beyond retention may be unable to resume from their old offsets. Log compaction keeps the latest value for each key over time and is useful for changelog/state reconstruction, but it is not a substitute for an event history where every transition matters. Replay is powerful for rebuilding projections and reprocessing bugs, but it can repeat external side effects. Consumers must have replay-safe semantics, deterministic transformations where possible, and a strategy for backfills that do not overload downstream databases. Use separate consumer groups or controlled offset resets for replay.

## 11.8 Schema evolution and event contracts

Events outlive the code version that produced them. Use explicit schemas, event type/version, stable field meanings and compatibility rules. Add optional fields before requiring them; avoid changing units or semantic meaning without a new version. Schema registries can enforce compatibility policies for Avro, Protobuf or JSON Schema. An event is a fact that happened, not a remote procedure call disguised as a message. Prefer OrderPlaced over PleaseCreateAnOrder on a shared event stream. Include event_id, aggregate_id, event_time, schema_version, trace context and causation/correlation identifiers as appropriate. Avoid placing secrets or excessive personal data in retained logs.

## 11.9 Python producer and consumer sketch

The example uses the confluent-kafka client. It is a practical starting point, but production code needs authentication, TLS, schema handling, structured errors, metrics and careful commit strategy.

### 💻 Python / implementation sketch

# pip install confluent-kafka

```
from confluent_kafka import Producer, Consumer
import json
```

producer = Producer({ "bootstrap.servers": "localhost:9092", "enable.idempotence": True, "acks": "all", })

```
def publish_order(order_id, event_id):
    event = {"event_id": event_id, "order_id": order_id,
             "type": "OrderPlaced", "version": 1}
    producer.produce(
        "orders", key=str(order_id),
        value=json.dumps(event).encode("utf-8")
    )
    producer.flush(10)
```

consumer = Consumer({ "bootstrap.servers": "localhost:9092", "group.id": "billing-v1", "auto.offset.reset": "earliest", "enable.auto.commit": False, }) consumer.subscribe(["orders"]) try: while True:

msg = consumer.poll(1.0) if msg is None: continue if msg.error(): raise RuntimeError(msg.error()) event = json.loads(msg.value()) process_idempotently(event) # durable dedupe + side effect consumer.commit(message=msg, asynchronous=False) finally: consumer.close()

## 11.10 Kafka versus simple Pub/Sub choice

Use ephemeral Pub/Sub for transient fan-out where missed events are acceptable or reconstructed elsewhere. Use a durable queue for jobs that need acknowledgement and retry. Use Kafka when multiple independent consumers need retained history, high-throughput partitioned ordering, replay and stream processing. Do not deploy Kafka for a handful of low-volume email tasks if a managed queue is simpler and operationally cheaper. Kafka is an operational platform, not just a library. Plan broker capacity, partitions, replication, retention, consumer lag, disk, upgrades, authentication, ACLs, schema compatibility and disaster recovery.

## 11.11 Failure patterns

- Consumer crash after side effect but before offset commit: event is reprocessed; deduplicate or make effect

idempotent.

- Offset committed before side effect: work may be lost.
- Poison message: isolate with retry/DLQ policy; avoid infinite blocking of a partition.
- Hot partition: improve key distribution or isolate hot aggregates.
- Consumer lag: add consumers only up to partition parallelism and check downstream capacity.
- Broker leader failure: verify ISR and producer acknowledgement policy.
- Schema break: deploy consumers that tolerate new fields before producers emit them.

### 💻 Extended Python lab

# Idempotent consumer idea: claim event_id and apply effect # in the SAME database transaction.

```
async def consume_order(event, db):
    async with db.transaction():
        inserted = await db.execute(
            "INSERT INTO processed_events(event_id) VALUES ($1) "
            "ON CONFLICT DO NOTHING RETURNING event_id",
            event["event_id"]
        )
        if not inserted:
            return  # duplicate delivery
        await db.execute(
            "UPDATE account_projection SET order_count = order_count + 1 "
            "WHERE customer_id = $1", event["customer_id"]
        )
# Commit Kafka offset only after the transaction succeeds.
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code.

Applied design walkthrough An order service publishes OrderPlaced. Billing, fulfilment, analytics and email each need the event. Use one Kafka topic with a stable order_id key so events for one order remain ordered in a partition. Each business function has a separate consumer group and independent offset. Billing processes with at-least-once delivery and deduplicates by event_id in the same transaction as its ledger update. Analytics can replay the retained log to rebuild a projection. Email uses bounded retries and a dead-letter path. State retention and replay limits explicitly.

### Interview questions and answer expectations

**Q. How are Kafka partitions related to consumer parallelism?**

**Q. What ordering does Kafka guarantee?**

**Q. Compare at-most-once and at-least-once.**

**Q. What does acks=all mean and what else affects durability?**

**Q. Why is exactly-once difficult with an external database?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Design a topic and key strategy for orders and order events.
- Write an idempotent consumer transaction using an inbox table.
- Estimate partition count for 20 MB/s with a tested target throughput per partition.

Operational metrics and signals

- Consumer lag and oldest unprocessed event age
- Produce/fetch rate and request latency
- Under-replicated partitions and ISR shrink events
- Broker disk usage and retention headroom
- Rebalance frequency and duration
- DLQ rate, retry count and duplicate event rate

Chapter mastery checklist

- Kafka is a retained partitioned log; Pub/Sub delivery semantics vary by system.
- Ordering is per partition; keys define locality.
- Consumer groups are independent readers with independent offsets.
- At-least-once plus idempotent effects is a practical default.
- Retention, schema evolution, lag and replay are part of the design.

## 🔬 Advanced Engineering Clinic

Kafka correctness lives at the boundaries between producer acknowledgement, partition ordering, consumer offset and external side effect. “Exactly once” must always name the boundary it covers. Producer idempotence + acks Kafka topic partition 0 / 1 / 2 key chooses partition Consumer group member assignments ordered log records DB transaction inbox dedupe + side effect process record Retry / DLQ workflow poison / exhausted retries Commit offset after durable effect commit after DB commit

Partition count, ordering and consumer parallelism Kafka ordering is guaranteed within a partition, not across an entire topic. Records with the same key are routed to the same partition under a stable partitioning scheme, which preserves per-key order. Choose a key that matches the ordering invariant, such as order_id or account_id. A key like country may create skew; a random key spreads load but destroys per-entity order. A consumer group assigns each partition to one active consumer at a time under the normal group model. Adding consumers beyond the partition count does not increase parallel processing for that topic. More partitions can increase parallelism but also increase metadata, file handles, leader management and operational overhead. Partition expansion can change key-to-partition mapping for future records, so ordering assumptions across the expansion boundary must be considered. Hot keys are a special challenge: a single celebrity account or tenant may dominate one partition. Splitting that key can improve throughput but may break ordering. Options include subkeys with merge semantics, dedicated hot-key pipelines or a different aggregation model. Choice Consequence Key = order_id Per-order ordering; possible hot order only in extreme case Key = user_id Per-user order; large user can become hot No key / round robin Better spread; no per-entity ordering More partitions More parallelism potential; more operational overhead

Offset commit and external side-effect boundary At-least-once processing is the common safe baseline: a message can be delivered more than once, so the consumer must be idempotent. If a consumer commits its offset before committing the database transaction, a crash can lose the side effect. If it commits the database first and crashes before offset commit, the record is replayed. The second sequence creates duplicates, which an inbox table can suppress. For a database side effect, insert the event_id into an inbox table and apply the business update in the same database transaction. A uniqueness constraint on event_id prevents duplicate application. Commit the Kafka offset only after that transaction commits. If the consumer crashes after DB commit but before offset commit, replay sees the inbox row and skips the effect. Kafka transactions and idempotent producers can provide strong guarantees within Kafka workflows, such as consuming offsets and producing output records transactionally. They do not automatically make an arbitrary external database, email provider or payment API exactly-once. For those, use idempotency keys, inbox/outbox patterns and reconciliation. Advanced implementation note # SQL transaction shape for a consumer side effect: # BEGIN; # INSERT INTO inbox(event_id) VALUES (:id) # ON CONFLICT DO NOTHING; # if row inserted: UPDATE orders ...; # COMMIT; # then commit Kafka offset. # The inbox insert and business update must share one DB transaction. Rebalances, poison records and operational readiness A rebalance changes partition ownership among consumer group members. During a rebalance, processing may pause or in-flight work may be duplicated depending on the consumer protocol and commit timing. Keep processing bounded, stop cleanly on revocation, commit only completed work and use idempotency to tolerate replay. Cooperative rebalancing can reduce disruption but does not remove the need for correct lifecycle handling. A poison record repeatedly fails due to deterministic invalid data or a bug. Blind retries can block later records in its partition. Define retry classes: transient failures may retry with backoff; permanent validation failures should be quarantined; unknown external outcomes require reconciliation. A DLQ needs an owner, alert, replay tool and retention policy. Monitor consumer

lag in offsets and in time, oldest unprocessed event age, rebalance rate, processing duration, retry count, DLQ volume, broker under-replicated partitions and produce error rate. Lag that grows continuously means arrival rate exceeds successful processing capacity. Failure point Expected outcome Crash before DB commit Replay; no committed business effect Crash after DB commit before offset Replay; inbox suppresses duplicate Offset before DB commit Potential lost business effect; avoid Poison record Bound retries and quarantine with context Consumer lag grows Scale to partition parallelism or fix downstream bottleneck

## ❓ Advanced Design-Review Questions

- A consumer commits a DB update and crashes before committing its Kafka offset. Walk through replay and

inbox deduplication.

- What happens to per-key ordering if you increase topic partitions and change the partitioner?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

PART III • ASYNCHRONOUS SYSTEMS • CHAPTER 12

# 12. Event-Driven Architecture (EDA)

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

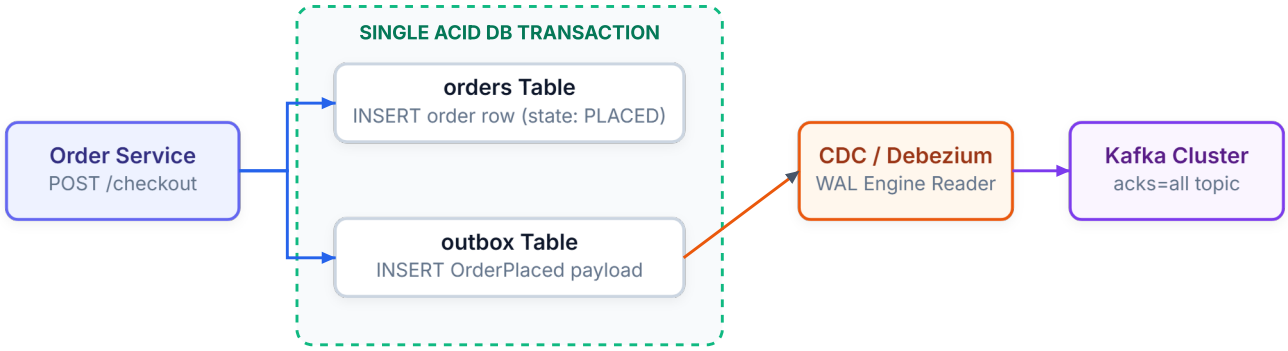
A town does not require every shop to call every other shop when something happens. A public notice board announces a fact, and interested organisations react independently. Event-driven systems use the same decoupling pattern, while needing careful contracts and delivery guarantees.

### 🤖 AI INFRASTRUCTURE LENS: TRANSACTIONAL OUTBOX FOR AGENT TOOL INVOCATION

**The Phantom Tool Call Disaster:** An LLM decides to execute an external tool: `book_flight(origin='SFO', dest='NYC', seat='12A')`. If the agent executes the HTTP request directly and encounters a network dropped packet before saving the transaction, the agent assumes the call failed and retries, booking two seats!

**Transactional Outbox Pattern for AI Agents:**

- The agent generates the tool call and writes the step intention into an ACID `agent_outbox` table in the local database: `INSERT INTO agent_outbox (agent_id, action, idempotency_key, payload, status) VALUES (...)`.
- A Debezium Change Data Capture (CDC) worker tails the database WAL and delivers the action event to an executor worker.
- The executor passes the unique `idempotency_key` to the external API. Even if the network drops or the agent container restarts, the external provider guarantees strictly-once tool execution.




## 🎯 Learning Outcomes

- Distinguish events, commands, queries and notifications.
- Compare choreography and orchestration.
- Design event contracts, outbox/inbox, idempotency and replay.

- Handle ordering, duplicate delivery, poison events and schema evolution.
- Know when synchronous request/response is simpler and better.

Order service DB transaction order + outbox Event broker retained / durable Payment consumer Fulfilment consumer Notification consumer Read-model consumer

broker decouples delivery time, but not the need for reliable contracts.

## 12.1 Events, commands and queries

An event is an immutable fact in the past: OrderPlaced, PaymentCaptured or DocumentPublished. A command asks a specific owner to perform an action: CapturePayment or ReserveInventory. A query asks for information and expects a response. Mixing these creates confusion: an event should not be phrased as an instruction to another service. Events are useful for broadcasting facts to multiple consumers. Commands are usually directed to one responsible handler, and should have a clear outcome or rejection. A notification is a message intended for a

human or external system; it may be generated as a consequence of an event. Distinguish domain events inside a bounded context from integration events published to other services.

## 12.2 Why event-driven architecture?

EDA decouples producers from consumers in time and often in deployment. The producer publishes a fact without knowing every downstream consumer. A new analytics or fraud consumer can subscribe without changing the producer, provided the event contract is stable. Consumers can scale independently and process asynchronously. The cost is that workflows become harder to trace. A request can trigger a chain of events across many services; failures may be delayed; data becomes eventually consistent; and operations need idempotency, replay and dead-letter handling. Event-driven is not automatically better than a direct API call. Use it when asynchronous decoupling, fan-out, buffering or replay provides a concrete benefit.

## 12.3 Choreography versus orchestration

In choreography, services react to events and emit subsequent events without a central workflow coordinator. It is loosely coupled for simple flows, but the overall process can become difficult to understand as branching and compensation grow. No single component may have a complete view of workflow state. In orchestration, a workflow coordinator issues commands and tracks state transitions. This makes the process and timeouts explicit, but the orchestrator becomes a critical component and can create central coupling. Use orchestration for long-running business workflows with clear steps and compensations; use choreography for simple independent reactions. Dimension Choreography Orchestration Control Distributed reactions to events Central workflow state machine Coupling Consumers know event contracts Participants know orchestrator commands/contracts Visibility Requires distributed tracing and event correlation Workflow state is explicit centrally Best fit Independent reactions and simple flows Long-running multi-step business process Risk Hidden cyclic dependencies Orchestrator complexity / central bottleneck

## 12.4 Event envelope and contract design

A useful event envelope includes event_id, event_type, schema_version, aggregate_id, occurred_at, producer, correlation_id, causation_id and payload. The event ID supports deduplication. Aggregate ID supports ordering and partitioning. Correlation ID connects a workflow; causation ID explains which prior event or command caused this one. Events should express domain facts and avoid leaking internal database table structure. Consumers should ignore unknown optional fields and reject incompatible breaking changes. Event payloads should be sized intentionally: large binary objects belong in blob storage, with a secure reference and content checksum in the event. Avoid putting secrets or data that consumers are not authorised to receive.

Order service Single DB transaction orders table outbox table Publisher poller Broker Consumer inbox + effect

still need deduplication.

## 12.5 Transactional outbox

The dual-write problem appears when a service commits business data to a database and separately publishes a message. If the DB commit succeeds and publish fails, the event is missing. If publish succeeds and DB commit fails, consumers observe a fact that never committed. The outbox pattern writes business state and an outbox record in one local database transaction. A separate publisher reads unsent outbox rows, publishes them, and marks them sent. It may publish twice if it crashes after publish but before marking sent, so consumers still need idempotency. The outbox turns an unreliable dual write into a reliable local transaction plus at-least-once publication.

## 12.6 Inbox and idempotent consumers

A consumer can receive the same event more than once because of broker redelivery, retries, consumer crash or publisher retry. Idempotency means repeated processing produces the same logical outcome as one processing. A durable inbox table with a unique event_id can record which event has been applied. The inbox insert and business side effect should occur in the same database transaction. If the event was already processed, skip the side effect. If the transaction rolls back, both the inbox claim and state update roll back. For external effects such as sending an email or charging a provider, use the external system's idempotency key or an additional workflow/outbox boundary.

## 12.7 Ordering, partitioning and replay

Global ordering across all events is expensive and often unnecessary. Define ordering per aggregate or entity, then use aggregate_id as a partition key. Include aggregate version or sequence number so consumers can detect gaps or duplicates. If a consumer sees version 14 before version 13, it can buffer, retry or rebuild from the source depending on the event contract. Replay supports rebuilding projections and debugging, but replay must not repeat external side effects. Separate projection consumers from effectful consumers or run replay mode with side-effect suppression. Store enough schema compatibility and reference data to make old events interpretable.

## 12.8 Error handling and poison messages

A poison message repeatedly fails because of malformed data or a deterministic bug. Blindly retrying it can block a partition or waste resources. Use bounded retries, exponential backoff with jitter and a dead-letter topic/queue that preserves the original payload, failure reason, attempt count and trace metadata. A dead-letter queue is an operational workflow, not a trash bin: alert and define ownership for reprocessing.

Classify failures: transient network timeout, dependency throttling, invalid business state, schema error and programming defect need different treatment. Do not retry a permanent validation error forever. Do not silently discard a financial or order event.

## 12.9 Event versioning and compatibility

Producers and consumers deploy independently, so compatibility is essential. Prefer additive optional fields, preserve existing semantics and use schema-registry compatibility checks. Consumers should tolerate unknown fields. If a field changes unit or meaning, introduce a new field or event version rather than silently changing it. Avoid creating a new event for every internal refactor. An event catalogue should describe owner, business meaning, schema, key, delivery expectation, retention, PII classification and consumers. Treat event schemas as public APIs within the organisation.

## 12.10 Python: durable outbox shape

This example shows the local transaction boundary. The publisher and consumer are separate processes. Database syntax varies by driver; the important property is that business row and outbox row commit atomically.

### 💻 Python / implementation sketch

```
async def place_order(db, order):
    async with db.transaction():
        await db.execute(
            "INSERT INTO orders(id, customer_id, status) "
            "VALUES ($1, $2, 'PLACED')",
            order.id, order.customer_id
        )
        await db.execute(
            "INSERT INTO outbox(event_id, event_type, aggregate_id, payload) "
            "VALUES ($1, 'OrderPlaced', $2, $3)",
            order.event_id, order.id, order.to_json()
        )
# Publisher reads pending rows, publishes with event_id, then
# marks sent. If it crashes after publish, it may publish again.
# Consumers deduplicate by event_id.
```

## 12.11 When not to use events

- Simple read-only request: a direct query is usually clearer.
- User needs immediate synchronous validation: use a request/response path, then emit events after commit if

needed.

- No durable replay or multi-consumer need: a simple background task may suffice.
- A team cannot operate broker, schemas, retries and tracing: reduce the number of asynchronous boundaries.
- Do not use an event as a remote command when the caller needs an immediate success/failure response.

### 💻 Extended Python lab

# Idempotent event handler using a database inbox.

```
async def handle_event(db, event):
    async with db.transaction():
        claimed = await db.fetchval(
            "INSERT INTO inbox(event_id) VALUES($1) "
            "ON CONFLICT DO NOTHING RETURNING event_id",
            event["event_id"]
        )
        if claimed is None:
            return "duplicate"
        await apply_business_change(db, event)
    return "applied"
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough An order workflow emits OrderPlaced. Billing reserves funds, inventory reserves stock, notification sends an email, and analytics updates a projection. The order service writes OrderPlaced into an outbox in the same transaction as the order record. Billing and inventory consumers deduplicate by event_id. A saga coordinator or explicit workflow state handles payment/inventory outcomes and compensation. Analytics can replay from the log; email uses bounded retries and dead-letter review. The order API returns after the local order transaction, while the user sees a pending/confirmed state as asynchronous steps complete.

### Interview questions and answer expectations

**Q. What is the difference between an event and a command?**

**Q. Explain outbox and inbox patterns.**

**Q. Why is at-least-once delivery common?**

**Q. Compare choreography and orchestration.**

**Q. How do you replay events without sending duplicate emails or payments?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Design an event envelope for OrderPlaced and PaymentCaptured.
- Implement a consumer inbox with a unique event_id constraint.
- Draw a choreography flow and an orchestrated flow for checkout.

Operational metrics and signals

- Outbox age and publish lag
- Consumer lag and oldest event age
- Duplicate event rate
- Retry and dead-letter volume
- Workflow completion time and stuck state count
- Schema compatibility failures and event volume by type

Chapter mastery checklist

- Events are facts; commands request actions.
- Outbox provides atomic state + event intent, not exactly-once delivery.
- Consumers must be idempotent.
- Choose choreography for simple reactions and orchestration for explicit workflows.
- Ordering, schema evolution, replay and dead-letter ownership are operational requirements.

## 🔬 Advanced Engineering Clinic

Event-driven architecture is a set of contracts and recovery processes, not merely a broker connection. This clinic distinguishes event facts from commands, transactional publication from best-effort publish, and choreography from hidden distributed coupling. Order service transaction Orders table Outbox row same local transaction CDC / publisher retry + checkpoint Durable event log Idempotent consumer inbox + projection

Event contract and envelope design An event is a fact that has happened: OrderPlaced, PaymentCaptured or ProfileChanged. A command asks a recipient to do something: CapturePayment or SendEmail. Naming facts as past-tense events discourages consumers from assuming they are being ordered to perform an action. The producer owns the meaning and schema of its event; consumers own their reaction. A robust envelope commonly includes event_id, event_type, schema_version, occurred_at, producer, aggregate_id, aggregate_version, tenant_id, correlation_id, causation_id and payload. IDs support deduplication and tracing. Aggregate version can help detect gaps or out-of-order processing. Keep secrets and unnecessary personal data out of event payloads; events are copied, retained and replayed.

Compatibility rules should be explicit. Additive optional fields are usually safer than renaming or changing field meaning. Consumers should tolerate unknown fields. Breaking changes may require a new event version or topic. Schema registry compatibility mode is a governance mechanism, not a substitute for careful semantic versioning. Field Why it matters event_id Deduplication and traceability aggregate_id + version Per-entity ordering and gap detection event_type + schema_version Interpretation and evolution correlation_id / causation_id Trace a business workflow occurred_at Business timestamp; not a total ordering clock Outbox, CDC and delivery semantics Directly updating a database and then publishing to a broker creates a dual-write failure window. If the DB commits and publish fails, consumers never hear about the change. If the event publishes first and the DB transaction rolls back, consumers react to a fact that did not happen. The transactional outbox writes the business row and an outbox row in one local transaction. A publisher or CDC connector reads committed outbox rows and emits events. Publication may be at-least-once: the publisher can crash after broker acknowledgement but before recording progress, causing a duplicate. Consumers must be idempotent. The

outbox needs cleanup/retention, lag monitoring, poison-row handling and partitioning at scale. An inbox stores processed event IDs at the consumer. Insert the inbox record and apply the local state change in one transaction. This gives effectively-once business effects within that database boundary under the stated assumptions; it is not a universal exactly-once guarantee across arbitrary external systems. Advanced implementation note event = { "event_id": "01J...", "event_type": "OrderPlaced", "schema_version": 2, "aggregate_id": "order-8421", "aggregate_version": 1, "correlation_id": "checkout-991", "causation_id": "command-774", "payload": {"order_id": "order-8421", "total_minor": 129900}, } # Store business row + serialized envelope in one DB transaction. Event sourcing is not the same as event-driven integration Event-driven integration publishes events so other components can react. The source service may still store current state in ordinary relational tables. Event sourcing uses the event log itself as the primary source of truth for aggregate state; current state is reconstructed by replaying events or loading snapshots plus later events. Event sourcing provides audit history and temporal reconstruction, but it adds schema evolution, replay, snapshot, event correction and privacy-erasure complexity. A published integration event is often a stable business fact, while internal event-sourcing events may be lower-level and change as the aggregate model evolves. Do not expose internal implementation events as public contracts without a deliberate compatibility layer.

Choose choreography for simple independent reactions and orchestration for workflows whose state, deadlines, compensation and ownership need a central view. Avoid using events where a caller needs an immediate validation result; use a synchronous command path and emit the fact after commit. Pattern Primary purpose Source of truth Event-driven integration Decouple service reactions Usually service DB; event is integration contract Event sourcing Store state changes as history Append-only event stream CQRS projection Optimised read model Rebuildable projection from source/events

## ❓ Advanced Design-Review Questions

- A service commits an order but crashes before publishing OrderPlaced. Show how an outbox resolves the

dual-write gap.

- When would orchestration be preferable to choreography for a multi-step workflow?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 13. Microservices Architecture

*CORE SYSTEM DESIGN*

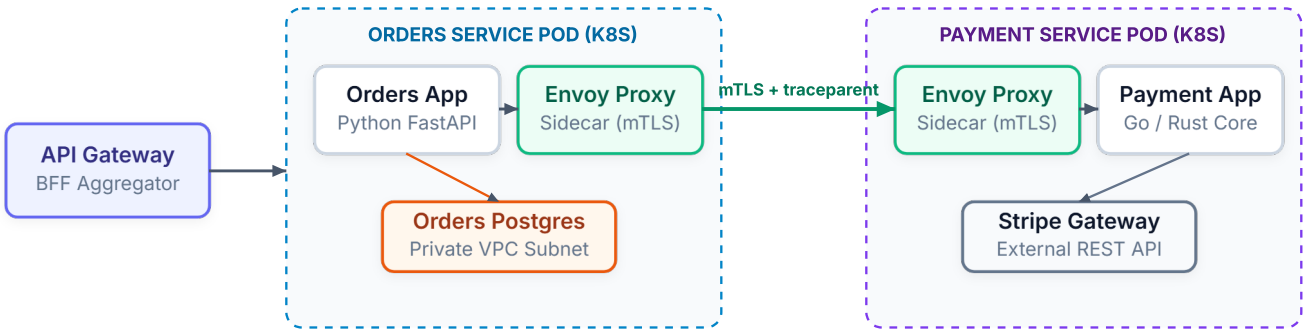
### 💡 INTUITIVE MENTAL MODEL

A company with one giant department can be simple at first, but every decision eventually goes through the same queue. Splitting into specialist teams helps only if responsibilities are clear and teams coordinate through well-defined contracts rather than sharing every internal detail.

### 🤖 AI INFRASTRUCTURE LENS: DECOUPLING GPU INFERENCE FROM BUSINESS MICROSERVICES

**Heterogeneous Architectural Domains:** A critical mistake in AI architecture is embedding LLM inference libraries (PyTorch/Transformers) directly inside CPU web application microservices (Django/FastAPI). The operational profiles are fundamentally incompatible:

- **Business Services:** Lightweight, CPU-bound, stateless, fast cold-starts (200ms), autoscale on HTTP request concurrency.
- **Inference Services (Triton / vLLM):** GPU-bound, massive memory-pinned CUDA allocations (80GB+ HBM), slow cold-starts (30–60s model weight loads), autoscale on GPU KV-cache saturation and queue wait time.
- **The Contract Boundary:** Microservices communicate with the dedicated Inference Cluster strictly via **high-performance gRPC** with Protobuf payloads, utilizing streaming responses and bidirectional backpressure to prevent CPU services from overloading GPU worker queues.




## 🎯 Learning Outcomes

- Compare a modular monolith and microservices.
- Choose service boundaries using business capabilities and domain concepts.
- Understand synchronous and asynchronous communication, data ownership and sagas.
- Manage deployment, observability, security and version compatibility.

- Recognise when microservices add more complexity than value.

Business services with owned data API gateway User service User DB Order service Order DB Inventory service Inventory DB reserve API Payment service Payment DB payment API Event broker OrderPlaced notification trigger projection / event

many boxes is not evidence of good boundaries.

## 13.1 What makes a service a microservice?

A microservice is a small, independently deployable service aligned to a business capability or bounded context. It owns a well-defined responsibility, exposes a contract and can evolve without requiring every other service to deploy in lockstep. Size is not measured only by lines of code or number of endpoints; the important property is autonomous ownership with manageable dependencies. A service should own its data model. Other services should not reach into its tables and depend on internal schema. They integrate through APIs or events. This creates freedom to change storage and implementation, but it also means cross-service queries and transactions need deliberate designs.

## 13.2 Modular monolith versus microservices

A modular monolith is one deployable application with strong internal modules and boundaries. It can use one database transaction, one process and one deployment while keeping domain logic separated. It is often the best starting point when the domain and team structure are still evolving. Microservices introduce network calls, partial failure, independent deployments, distributed tracing, service discovery, secrets, certificates, retries, versioning and eventual consistency. They are justified when independent scaling, team autonomy, release cadence, fault isolation or technology requirements outweigh the operational cost. Splitting a monolith without a business boundary creates a distributed monolith. Dimension Modular monolith Microservices Deployment One deployable unit Independent service deployments Transactions Can share a DB transaction Cross-service workflow needed Communication In-process calls Network APIs/events Scaling Often scale whole application Scale selected services Operations Simpler baseline Higher platform/observability burden Best fit Small/medium team, evolving domain Multiple teams and stable boundaries

## 13.3 Find boundaries with domain-driven design

Domain-driven design (DDD) helps identify bounded contexts: areas where a model and vocabulary have a consistent meaning. "Customer" in billing, support and identity may represent different concepts and rules. Forcing all teams to share one giant Customer model creates coupling. A bounded context owns its model and translates at integration boundaries. Start by mapping business capabilities and critical workflows. Identify commands, events, data ownership and invariants. A service boundary should keep changes that must be atomic together. If two components constantly need synchronous calls and shared transactions, the boundary may be wrong. Split where concepts, ownership or scaling needs differ, not merely by database table or technical layer.

## 13.4 Communication patterns

Synchronous HTTP/gRPC is appropriate when the caller needs an immediate response or query result. It creates runtime coupling: the caller depends on target availability, latency and contract. Set deadlines, timeouts, concurrency limits and retry rules. A chain of five synchronous services can multiply tail latency and failure probability. Asynchronous messaging is appropriate for durable work, fan-out and workflows that can complete later. It decouples time and lets consumers scale independently, but creates eventual consistency, duplicate delivery and harder tracing. Use events for facts and commands for directed work. Avoid request-response over a broker when an HTTP call is simpler.

Client Strangler router Legacy monolith remaining capabilities unmigrated routes New extracted service owns capability + data migrated routes Tracing / metrics / contract tests

must be planned alongside route migration.

## 13.5 Database per service and data ownership

Database-per-service means each service controls its schema and writes. It does not necessarily mean every service must run a separate database cluster; multiple schemas or logical databases can share managed infrastructure early, provided ownership and access boundaries are enforced. The essential rule is that one service should not depend on another service's private tables. Cross-service reads can use API composition, materialised views/projections or a dedicated analytical store. API composition calls multiple services and merges results but adds latency and failure coupling. A projection consumes events and maintains a read model; it is eventually consistent and needs replay and repair. Choose based on user latency and freshness requirements.

## 13.6 Distributed transactions and sagas

A local database transaction cannot atomically commit across independent service databases. A checkout may involve order, inventory, payment and shipment. Two-phase commit can coordinate resources but increases coupling and availability/latency costs and may not be supported by all participants. Sagas model the workflow as local transactions plus compensating actions. They require idempotency, durable state and timeout/reconciliation logic. Compensation is a business action, not a database rollback. Refunding a payment is not equivalent to erasing a charge; notifying a user may not be reversible. Design each step and compensation explicitly, and define what happens when compensation itself fails.

## 13.7 Deployment, compatibility and service evolution

Independent deployability requires backward-compatible contracts. Use expand-and-contract migrations: add new fields/endpoints first, deploy readers that understand both old and new forms, then switch producers, and remove old forms only after clients have migrated. A rolling deployment means multiple versions coexist, so compatibility must be deliberate. Use consumer-driven contract tests, schema compatibility checks, canary deployment and clear ownership. Avoid shared libraries that force every service to upgrade together for business logic changes. Common infrastructure libraries are useful, but they should not erase service boundaries.

## 13.8 Observability and resilience

Every request should carry trace/correlation context across gateway, services and asynchronous messages. Logs should identify service, version, request/trace ID and business correlation ID without leaking secrets. Metrics should show request rate, error rate, latency, saturation and dependency health. Distributed tracing helps find where a request spent its time; it does not replace metrics or logs. Each remote call needs a timeout and bounded retry budget. Circuit breakers prevent repeated calls to a failing dependency; bulkheads isolate thread pools, connection pools and concurrency. Avoid retrying every layer independently because retries multiply load. Define service-level SLOs and error budgets so teams can make trade-offs.

## 13.9 Security and platform concerns

Use service identity, mutual TLS where appropriate, least-privilege authorisation and secret rotation. Do not assume internal network traffic is trusted. Validate incoming requests at the boundary, and authorise at the service that owns the data. API gateway authentication is useful, but downstream services should still enforce critical object-level permissions where trust boundaries require it. A service platform may provide service discovery, configuration, certificate management, deployment, autoscaling, logging and policy enforcement. Kubernetes can orchestrate containers, but it does not automatically solve domain boundaries, distributed transactions or correct retries. Platform complexity must be justified by team and system scale.

## 13.10 The strangler migration

The strangler pattern incrementally moves capabilities out of a legacy monolith. An edge router sends selected routes to a new service while the rest remain in the monolith. The new service first may read from the existing source, then take ownership of writes, then migrate data and consumers. This reduces big-bang rewrite risk. Each extraction needs a clear data ownership transition. Avoid dual-writing two authoritative databases without a reconciliation protocol. Use shadow reads, comparison metrics, idempotent migration, backfill and rollback plans. Retire the old path only after traffic and data validation prove it is safe.

## 13.11 When microservices are the wrong choice

- A small team cannot operate many services reliably.
- Domain boundaries are uncertain and change weekly.
- Most operations require one large cross-module transaction.
- Service split is only by technical layer, such as separate CRUD services per table.
- The organisation lacks central observability, deployment automation and incident ownership.
- The main bottleneck is one database query that can be fixed inside a modular monolith.

### 💻 Extended Python lab

# A modular monolith starts with explicit Python boundaries.

```
class OrderService:
    def __init__(self, order_repository, inventory_port):
        self.orders = order_repository
        self.inventory = inventory_port
    async def place_order(self, command):
        # Domain workflow stays behind an explicit interface.
        reservation = await self.inventory.reserve(
            command.items, command.order_id
        )
        if not reservation.accepted:
            raise ValueError("inventory unavailable")
        return await self.orders.create(command)
# A future microservice can implement inventory_port over HTTP/gRPC
# without changing the domain-facing interface.
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough An e-commerce monolith has catalog, checkout, inventory, payments and notifications. The notification workload is spiky and can be delayed; extract it first behind an event contract because it has a clear boundary and does not need to block checkout. Next, separate inventory only after defining reservation semantics and data ownership. Keep checkout and order creation together until their transactional boundary is understood. Use the strangler router to migrate one capability at a time, with contract tests and a rollback path.

### Interview questions and answer expectations

**Q. When would you keep a modular monolith?**

**Q. What is a bounded context?**

**Q. Why should services own their data?**

**Q. How do you handle a cross-service transaction?**

**Q. How do you extract one service without a big-bang rewrite?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Draw a modular monolith for commerce and identify potential service boundaries.
- Design an API composition endpoint and a projection alternative.
- Plan a strangler migration of notifications from a monolith.

Operational metrics and signals

- Service-level RPS, p95/p99 and error budgets
- Dependency latency and timeout/retry volume
- Distributed trace coverage and trace completion
- Deployment frequency and rollback rate

- Cross-service call depth and synchronous fan-out
- Data ownership violations and schema compatibility failures

Chapter mastery checklist

- Microservices are business capabilities with independent ownership, not merely many deployables.
- A modular monolith is often the best starting point.
- Service boundaries should preserve cohesion and minimise chatty cross-boundary transactions.
- Events and sagas handle asynchronous workflows; they do not remove complexity.
- Observability, security, deployment and data ownership are prerequisites, not afterthoughts.

## 🔬 Advanced Engineering Clinic

Microservices are an organisational and data-ownership choice with runtime consequences. This clinic focuses on boundaries, independent deployability, synchronous call graphs and the cost of a distributed system. Edge / API gateway Order service owns order DB Inventory service owns inventory DB sync reservation Payment service owns payment DB sync or workflow command Event broker OrderPlaced Read projection / search async projection

Find boundaries from business invariants A service boundary is strongest when it groups data and behaviour that must change together and separates capabilities that can evolve independently. Domain-driven design uses bounded contexts: a model and its language are valid within a particular business boundary. “Customer” may mean a billing account in one context and a delivery recipient in another; forcing one shared model across all teams creates coupling. Look for transaction boundaries, ownership, team responsibility, change cadence and independent scaling. A service should own its write model and expose a contract; other services should not directly update its tables.

Sharing a database server is not necessarily a problem, but sharing tables and schema ownership makes deployments and invariants inseparable. A modular monolith is often the right starting point. It can enforce module boundaries in one process and one deployable unit while avoiding network failure and distributed transactions. Extract a service when there is evidence: independent scaling, release cadence, team autonomy, security boundary or failure isolation that outweighs the operational cost. Boundary signal Good sign Warning Data invariants One capability owns them Multiple services write same row Change cadence Independent changes are frequent Every release requires coordinated rollout Team ownership Clear team and on-call owner No one owns cross-service workflow Communication Small stable contracts Long synchronous call chains Synchronous call graph and tail latency If a request calls five services sequentially, their latency adds along the critical path. If it calls five services in parallel and all are required, the slowest branch determines completion. At scale, rare slow dependencies become common user-visible tail events. Each synchronous edge also adds an availability dependency: the caller cannot complete if a required callee fails. Keep synchronous chains shallow. Set an end-to-end deadline and allocate a budget to each hop. Use timeouts, bounded retries, circuit breakers and bulkheads. Avoid retries at every layer. Make optional enrichments asynchronous or return partial responses when the product contract allows it. For cross-service views, API composition is simple but fan-out can be slow and fragile. A materialised projection built from events adds eventual consistency but provides low-latency reads. Choose based on freshness and query complexity, and make projection rebuild/replay part of the operational design. Sequential dependencies: T_total ≈ T1 + T2 + ... + Tn Parallel required fan-out: T_total ≈ max(T1, ..., Tn) +

orchestration overhead Availability also compounds across required synchronous dependencies. Data ownership, migration and anti-patterns A distributed monolith has separate deployables but shared release coupling, shared database writes, chatty synchronous calls or coordinated schemas. It pays microservice network and operational costs without earning independent evolution. Avoid splitting by technical layer (UI service, business service, database service) when those are not business capabilities. The strangler approach routes a bounded capability to a new service while the old system remains available. Migrate one write path at a time, establish a single owner, replicate or project data deliberately, compare behaviour, then retire the legacy path. Avoid indefinite dual-write without a reconciliation plan. Platform capabilities become necessary as service count grows: service discovery, secrets, certificate rotation, deployment automation, tracing, service ownership catalogue, API/schema governance and incident response. A team that cannot operate these safely may be better served by fewer services. Anti-pattern Why it hurts Shared DB writes Hidden coupling and unclear ownership

Anti-pattern Why it hurts Chatty RPC Latency amplification and cascading failures Tiny services Deployment/observability overhead exceeds value Distributed monolith Independent deployability is only apparent No platform ownership Every team reinvents security, telemetry and release controls

## ❓ Advanced Design-Review Questions

- A proposed service split requires five synchronous calls to render one page. Explain the latency and

availability consequences.

- Define the ownership and migration plan for extracting a bounded context from a shared database monolith.

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 14. API Gateway

*CORE SYSTEM DESIGN*

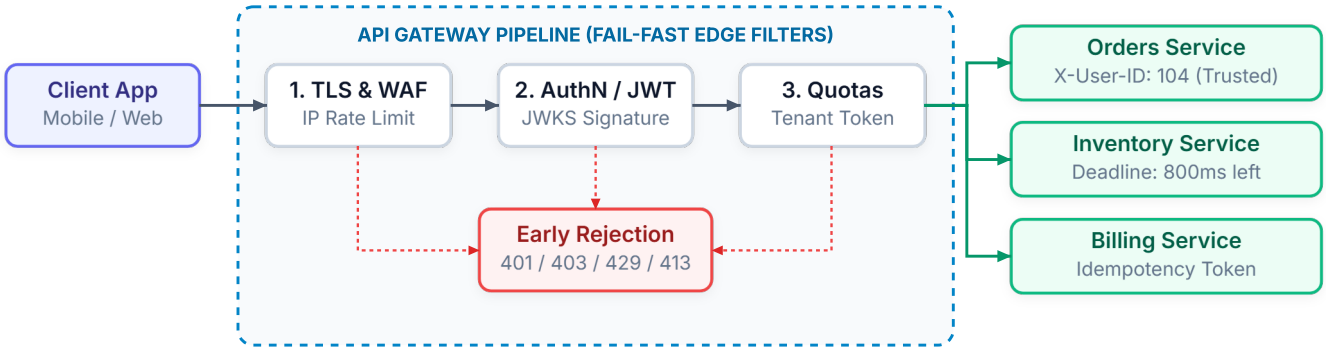
### 💡 INTUITIVE MENTAL MODEL

A large office has one reception desk that verifies visitors, checks badges, directs them to the right department and enforces building rules. The reception desk does not do the actual work of every department; it controls the boundary.

### 🤖 AI INFRASTRUCTURE LENS: LLM GATEWAYS, PROMPT GUARDS & DYNAMIC FALLBACKS

**Edge Fail-Fast for AI Traffic:** Sending prompt injection attacks or malformed queries into an 8x H100 inference engine wastes tens of thousands of expensive GPU matrix FLOPS. An AI API Gateway performs three fail-fast functions:

- **Prompt Injection & Jailbreak Edge Filters:** Lightweight ONNX classifiers and regex heuristics inspect incoming prompts at the edge (Envoy / Cloudflare Worker), dropping adversarial attacks before they consume GPU prefill compute.
- **Multi-Provider Semantic Fallbacks:** If Anthropic Claude 3.5 Sonnet returns an HTTP 429 (Rate Limit) or 503 (Overloaded), the gateway dynamically rewrites the schema and routes the prompt to OpenAI GPT-4o, or drops to an internal self-hosted vLLM Llama-3-70B cluster without user intervention.
- **Dynamic Token Quota Metering:** Verifies that the tenant has sufficient token balance before allowing the request to initiate token streaming.




## 🎯 Learning Outcomes

- Separate API gateway responsibilities from load balancing and reverse proxying.
- Understand routing, authentication, authorisation, quotas, transformation and aggregation.
- Design gateway high availability and avoid central business-logic overload.

- Apply BFF, versioning, timeouts, request limits and security controls.
- Implement a small FastAPI gateway proxy with safe timeout behaviour.

Internal service boundary Mobile / web / partner API Gateway TLS, auth, quotas, routing User API Order API Catalog API Identity provider / JWKS validate token Rate-limit store quota check

service that owns it.

## 14.1 Gateway versus reverse proxy versus load balancer

A reverse proxy forwards client traffic to upstreams and may provide TLS, buffering, caching and compression. A load balancer selects among backend instances. An API gateway adds API-consumer concerns: authentication, authorisation policy, quotas, request/response transformation, API versioning, routing by API contract, developer keys, aggregation and analytics. One product may perform all three functions, but the responsibilities remain conceptually distinct. A gateway usually operates at L7 because it understands HTTP semantics. A network load balancer may sit in front of it to distribute connections. Internal services may still use their own load balancing or service-mesh proxies. Avoid confusing the public API entry point with the service-to-service discovery layer. Component Core question answered Typical responsibility Reverse proxy Where should this request be forwarded? Forwarding, buffering, TLS, compression Load balancer Which healthy instance should receive it? Backend selection and health API gateway Is this API call allowed and which API contract handles it? Auth, quotas, API routes, transformation, aggregation Service mesh proxy How do service workloads communicate safely/reliably? mTLS, retries, telemetry, service routing

## 14.2 Gateway request lifecycle

A typical request arrives at the gateway, is normalised and size-checked, authenticated, authorised at the boundary, rate-limited, routed to an upstream, and then observed. The gateway may add trusted identity context and trace IDs, but downstream services must still enforce domain-specific permissions for sensitive resources. Order matters. Reject malformed or oversized requests before expensive authentication when possible; authenticate before applying user quotas; authorise before returning sensitive data; and set an overall deadline that downstream calls respect. Do not allow an upstream to extend the deadline indefinitely.

## 14.3 Authentication, authorisation and identity

propagation The gateway can validate JWT signatures, issuer, audience, expiry and key rotation using a trusted issuer/JWKS. API keys can identify a client but should not be treated as user authentication. OAuth scopes and claims can inform policy, but object-level authorisation belongs close to the data-owning service. A gateway that only checks "token is valid" may still allow a user to access another tenant's object if the service does not check ownership. When forwarding identity, use a trusted internal header or signed token that clients cannot spoof. Strip external copies of headers such as X-User-ID before injecting authenticated identity. Use service identity and mTLS where required; do not assume an internal network makes requests trustworthy.

## 14.4 Routing and API versioning

Routes can map hostnames, paths, methods, headers or API versions to upstreams. Path routing is simple for /v1/orders and /v2/orders; header-based routing can support canary release but should not create undocumented public contracts. Version for compatibility, not every internal code release. Prefer additive evolution where possible and publish deprecation timelines. The gateway should preserve important HTTP semantics: status codes, request IDs, content types, caching headers and retry signals. Transformations should be predictable and observable. Avoid rewriting error responses in ways that hide whether a failure came from gateway policy or an upstream business service. Incoming API request Normalize + size + schema check Authenticate / identify client Reject early 400 / 401 / 403 / 413 / 429 invalid Authorise + quota + WAF unauthenticated Route by host / path / version denied / quota Owning service Map response + headers + trace

## 14.5 Aggregation and the Backend-for-Frontend pattern

An API gateway can aggregate calls: a mobile screen may need profile, cart and recommendations. Aggregation reduces client round trips but creates a fan-out path. If three upstreams are called in parallel, total latency is approximately the slowest dependency plus overhead, not the sum, but any dependency failure can degrade the combined response. Use optional fields, partial responses or graceful degradation deliberately. Backend-for-Frontend (BFF) creates a tailored backend for each client type, such as mobile and web. It can reduce over-fetching and keep client-specific presentation logic out of domain services. A BFF is not a license to duplicate business invariants; those remain in owning services.

## 14.6 Timeouts, retries, circuit breakers and request

budgets The gateway should impose a request deadline and propagate a remaining budget downstream. If an end-to-end request has a 2 second budget, spending 1.8 seconds at the gateway leaves little time for the service. Avoid retries by default for non-idempotent operations. A retry policy needs a maximum attempt count, exponential backoff, jitter and an idempotency contract. The gateway may use a circuit breaker for an unstable upstream and enforce concurrency limits. However, a gateway-wide retry can multiply retries already configured in client and service code. Assign retry ownership to one layer where possible and record attempts. For payment and order creation, clients should supply idempotency keys and the service should persist the result.

## 14.7 Rate limiting, request validation and abuse protection

The gateway is a natural place to enforce coarse client/API quotas, payload-size limits, allowed methods, IP reputation, WAF policies and schema checks. Fine-grained domain limits may still belong in the service. Use trusted source IP extraction only behind known proxies. Return 429 for quota exceeded and a useful Retry-After when meaningful.

Validate content type and maximum body size before buffering large payloads. Do not log full credentials, payment data or personal payloads. Set safe CORS policy for browsers; CORS is not authentication. Apply separate limits for expensive endpoints, anonymous clients and authenticated tenants.

## 14.8 Gateway availability and blast radius

A gateway outage can make every API appear unavailable. Run multiple instances across failure zones, keep configuration rollout safe, and ensure the gateway has capacity for peak traffic. Configuration errors can be more dangerous than process crashes: a bad route or auth policy can break all requests. Use staged config validation, canaries and rollback. The gateway can become a bottleneck if it performs expensive transformations, calls multiple services serially or centralises business logic. Keep request processing bounded. Cache only safe responses. Avoid making every internal call hairpin through the public gateway unless that is an intentional security/operational boundary.

## 14.9 Python FastAPI gateway proxy sketch

This sketch demonstrates routing and a timeout. A production gateway requires streaming, header filtering, authentication, request IDs, body-size enforcement, retry semantics and robust connection pooling. A general-purpose gateway product is usually preferable to building all edge behaviour from scratch.

### 💻 Python / implementation sketch

```
import httpx
from fastapi import FastAPI, Request, Response, HTTPException
```

app = FastAPI() client = httpx.AsyncClient(timeout=httpx.Timeout(2.0))

```
@app.get("/api/catalog/{item_id}")
async def catalog_proxy(item_id: str, request: Request):
    try:
        upstream = await client.get(
            f"http://catalog-service/items/{item_id}",
            headers={"X-Request-ID": request.headers.get(
                "X-Request-ID", "generated-by-gateway"
            )}
        )
    except httpx.TimeoutException:
        raise HTTPException(504, "Upstream timeout")
    return Response(
        content=upstream.content,
        status_code=upstream.status_code,
        media_type=upstream.headers.get("content-type")
    )
```

## 14.10 Design boundaries and anti-patterns

- Putting order, payment or inventory business rules in gateway plugins.
- Using one global timeout for every endpoint regardless of workload.
- Blindly retrying POST requests.
- Trusting client-supplied identity headers.

- Letting the gateway call every dependency serially for screen composition.
- Treating CORS as an access-control mechanism.
- Routing internal traffic through the public gateway without a reason.

### 💻 Extended Python lab

# A gateway should pass a remaining deadline, not reset one.

```
import time
def remaining_budget_ms(started_at, total_budget_ms):
    elapsed = (time.monotonic() - started_at) * 1000
    remaining = total_budget_ms - elapsed
    if remaining <= 0:
        raise TimeoutError("request deadline exhausted")
    return int(remaining)
# Propagate this budget to each downstream call and set a timeout
# no larger than the remaining end-to-end budget.
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A mobile app needs profile, cart and recommendations on one screen. The gateway authenticates the user and enforces a client quota. A mobile BFF fetches profile and cart in parallel; recommendations are optional and can be omitted if slow. The cart service still validates ownership and inventory. The overall request has a 1.5 second budget; the BFF propagates remaining time to each dependency. A circuit breaker suppresses calls to a failing recommendation service without blocking checkout.

### Interview questions and answer expectations

**Q. How is an API gateway different from a load balancer?**

**Q. Where should object-level authorisation live?**

**Q. What is a BFF and when is it useful?**

**Q. How do you avoid retry amplification?**

**Q. How do you make gateway changes safe to deploy?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Design gateway routes and policies for mobile, web and partner APIs.
- Create a fan-out budget for three upstream services.
- List which security checks belong at gateway and at the domain service.

Operational metrics and signals

- Gateway request rate, p95/p99 and 4xx/5xx
- Auth failures, quota denials and WAF blocks

- Upstream latency, timeout and retry attempts
- Connection pool saturation and active requests
- Config rollout failures and route mismatch
- Payload size and rejected oversized requests

Chapter mastery checklist

- The gateway is the controlled API front door, not the place for all business logic.
- Authentication can be central; domain authorisation remains with the owning service.
- Propagate deadlines, limit retries and account for fan-out.
- BFF aggregation improves client fit but creates dependency latency/failure.
- Treat gateway availability and config rollout as critical system concerns.

## 🔬 Advanced Engineering Clinic

An API gateway is a policy enforcement and routing boundary. This clinic examines the request pipeline, trust propagation, backend-for-frontend composition and how to avoid turning the gateway into a monolithic business layer. Client Gateway listener TLS + request ID AuthN / token validation identity propagation Metrics • trace • audit Quota • WAF • validation body / header limits Route + deadline budget version / BFF Backend services

Trust boundaries and identity propagation TLS termination at the gateway does not make downstream traffic automatically trusted. The gateway should validate the external credential, derive a trusted identity context and propagate it over an authenticated internal channel. Backends should accept identity headers only from trusted gateway or service-mesh identities; clients must not be able to spoof them. mTLS, signed internal tokens or workload identity can establish the internal caller. Authentication answers who the caller is. Authorisation answers whether that identity may perform an action on a specific resource. The gateway can enforce coarse scopes and global policies, but resource-level rules often belong in the domain service because it owns the resource and invariant. Never rely solely on a client-provided tenant_id or role header.

Keep secrets out of logs and traces. Redact authorization headers, cookies, signed URLs and sensitive query parameters. Gateway access logs are high-volume and often broadly accessible, so data minimisation is part of the design. Concern Gateway responsibility Backend responsibility Authentication Validate external credentials Trust only authenticated internal identity Authorisation Coarse scopes / global policy Resource and business permission checks Tenant isolation Resolve/validate tenant context Enforce ownership in data access Audit Capture request identity and ID Record business decision and actor Aggregation and deadline propagation A backend-for-frontend can aggregate profile, cart and recommendation APIs into one client response. If calls run in parallel, response time is approximately the slowest required branch, but the failure probability increases with the number of required dependencies. If one branch is optional, return a partial response or omit that section rather than failing the whole page. Use an absolute deadline or remaining-time budget propagated to each upstream. Do not let every service start a new full timeout. Cancel work when the client disconnects if safe, and bound fan-out concurrency. Aggregation endpoints need caching and response-size limits because one request can amplify into many upstream requests. Keep business orchestration out of the gateway when it needs durable workflow state, compensation or domain decisions. A gateway should route and enforce cross-cutting edge policy; an order workflow belongs in the order domain or a workflow/orchestration service. Fan-out amplification: one gateway request × n upstream calls. 20,000 gateway RPS × 4 parallel dependencies can approach 80,000

upstream RPS before retries. Gateway resilience and deployment The gateway is on the critical path for nearly every API call. Run multiple instances across failure zones, size for peak traffic, and define behaviour when identity provider, policy store or service discovery is unavailable. Cache only safe policy/configuration and define how quickly revocation propagates. A stale allow-list can be a security issue; a stale non-critical route may be an availability issue. Configuration changes can break more traffic than a process crash. Validate routes, auth policy and upstream clusters before rollout; canary the config; monitor 4xx/5xx by route and tenant; keep a fast rollback. Separate control plane (configuration and discovery) from data plane (request processing) where the platform supports it. Avoid creating a "god gateway" with business rules, database access and long-running workflows. Keep gateway latency low and responsibilities bounded. Use separate BFFs when client experiences materially differ, but avoid one gateway per tiny endpoint. Gateway metric Why it matters Latency by filter Find expensive auth/policy/routing stage Upstream timeout rate Detect dependency degradation

Gateway metric Why it matters Request amplification Spot fan-out and retry multiplication Config rollout errors Catch blast-radius changes Auth denial by reason Separate invalid credentials from policy defects

## ❓ Advanced Design-Review Questions

- Which authorisation decisions belong in a gateway and which belong in the domain service?
- Design a BFF aggregator that can return a partial response without hiding a critical upstream failure.

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 15. Rate Limiter Design

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

A busy venue may let people enter at a controlled rate, permit a small queue, or allocate each group a quota. A rate limiter applies similar rules to API requests, protecting systems and enforcing fair usage.

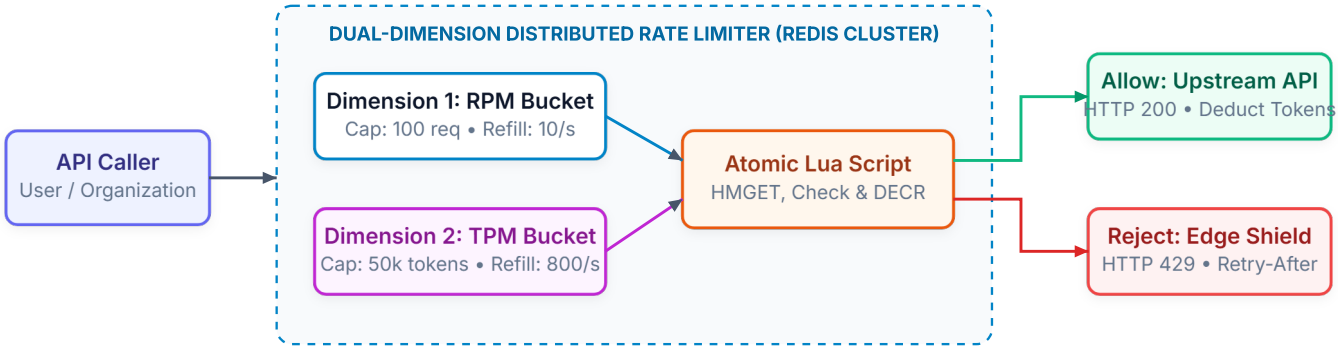
### 🤖 AI INFRASTRUCTURE LENS: DUAL-DIMENSION RATE LIMITING: RPM + TPM

**The 128k Context Window Rate-Limit Trap:** In classical web APIs, 1 request = 1 request. In modern AI APIs, Request 1 may contain a 200-token prompt, while Request 2 contains a 128,000-token prompt with 5 attached PDF documents. A rate limiter that only enforces 100 Requests Per Minute (RPM) will allow a user to submit 100 requests of 128k tokens, requiring 12.8 million tokens of prefill and instantly crashing GPU memory pools!

**Production Solution: Dual-Resource Token Bucket (RPM + TPM):**

- Every client is governed by two simultaneous bucket budgets: **Requests Per Minute (RPM)** and **Tokens Per Minute (TPM)**.
- An atomic Redis Lua script checks both buckets simultaneously:

```
-- Check RPM bucket
if redis.call('GET', rpm_key) < 1 then return {0, "RPM_EXCEEDED"} end
-- Estimate prompt tokens + requested max_tokens
if redis.call('GET', tpm_key) < estimated_tokens then return {0, "TPM_EXCEEDED"} end
-- Atomically decrement both buckets
redis.call('DECR', rpm_key)
redis.call('DECRBY', tpm_key, estimated_tokens)
return {1, "OK"}
```



### 🎯 Learning Outcomes

- Compare fixed window, sliding log, sliding counter, token bucket and leaky bucket.
- Choose limit keys and policy dimensions such as user, IP, tenant and endpoint.
- Implement atomic distributed decisions with Redis and Lua.
- Handle burst tolerance, distributed clocks, fail-open/closed and Retry-After.
- Design a scalable rate limiter for a multi-instance API.

API client Gateway / API server Derive identity key user / tenant / IP / route Atomic rate-limit decision Redis / local policy Forward request within budget 429 + Retry-After over budget Backend service

quota.

## 15.1 What rate limiting protects

Rate limiting controls how frequently a client or workload may invoke an operation. It protects infrastructure

```
from accidental spikes and abuse, enforces tenant fairness, manages paid quotas and shields expensive
```

dependencies. It is different from concurrency limiting: rate limiting controls arrivals over time, while concurrency limiting bounds simultaneous in-flight work. A request limit should be tied to a resource budget. A cheap cached GET and an expensive report generation endpoint should not necessarily consume the same quota. Some systems use weighted cost units: a simple lookup costs 1, a search costs 5 and a report costs 20. A tenant may have a sustained rate, a burst allowance and a daily quota.

## 15.2 Define the limiting key and scope

Common dimensions include source IP, authenticated user, API key, tenant, endpoint, region, resource ID and global dependency. IP-only limits can penalise users behind NAT and can be bypassed by distributed addresses. User-only limits do not protect an unauthenticated login endpoint. Use layered limits: coarse edge/IP limits, authenticated identity/tenant quotas and backend-specific concurrency limits. Key design must prevent accidental cross-tenant collisions. Include policy version and route class where necessary, such as rl:v3:tenant:acme:orders:create. Avoid high-cardinality keys without expiry. Decide how unauthenticated requests are grouped and how trusted forwarded IP information is obtained.

## 15.3 Fixed window counter

A fixed window counts requests in intervals, such as 100 requests per minute. The counter resets at the boundary. It is simple and memory-efficient, but allows boundary bursts: a client may send 100 requests just before the minute changes and 100 immediately after, producing 200 requests in a short interval. Use atomic increment with an expiry set on first creation. Separate INCR and EXPIRE calls can race if the process crashes between them, so use a script or transaction. Fixed windows are often acceptable for coarse quotas but not when strict burst smoothing is required.

### 💻 Python / implementation sketch

# Redis Lua: atomic fixed-window increment + TTL. FIXED_WINDOW_LUA = """ local count = redis.call('INCR', KEYS[1]) if count == 1 then redis.call('EXPIRE', KEYS[1], ARGV[1]) end return count """ # Call with key rl:{tenant-42}:minute:timestamp and window seconds. # Allow request when returned count <= configured limit.

## 15.4 Sliding log

A sliding log stores each request timestamp and removes timestamps outside the rolling window. It gives accurate rolling-window enforcement. For a limit of 100 requests per 60 seconds, the limiter removes entries older than 60 seconds, counts remaining events and allows only if the count is below 100. The cost is memory and command work proportional to request volume. At very high rates, storing every timestamp is expensive. Use atomic scripts and unique member IDs; clock source and timestamp precision must be considered. Sliding logs fit moderate-volume, high-accuracy quotas or high-value operations. Request with cost c Bucket state tokens + last_refill Refill by elapsed x rate cap at bucket capacity Enough tokens? Allow; subtract cost yes Reject 429 Retry-After no new state

## 15.5 Sliding-window counter

A sliding-window counter approximates a rolling window using the current fixed-window count and previous-window count weighted by how much of the previous window overlaps. If 30 percent of the current minute has elapsed, the estimate is current_count + previous_count x 0.70. This smooths fixed-window boundary bursts with much lower memory than a per-request log. It is an approximation. A sudden concentrated burst in the previous window is treated as evenly distributed by the weighting assumption. The method is often a useful balance for API quotas. The implementation must ensure the two counters and expiry are updated consistently.

## 15.6 Token bucket

A token bucket has capacity B and refills at rate r tokens/second. Each request consumes a cost c. If enough tokens exist, allow and subtract c; otherwise reject or wait. The bucket allows bursts up to B while controlling long-run average rate near r. It is intuitive for APIs where a short burst is acceptable but sustained abuse is not. A bucket state has token count and last-refill time. Refill is min(B, tokens + elapsed x r). Use a monotonic clock in a single process; for a distributed Redis implementation, use a consistent server-side time source or carefully controlled timestamps and perform read-modify-write atomically. Define how clock jumps are handled. tokens_now = min(capacity, tokens_previous + elapsed_seconds x refill_rate) Allow if tokens_now >= request_cost; then tokens_now -= request_cost.

## 15.7 Leaky bucket

A leaky bucket processes requests at a near-constant output rate. Incoming work is queued; the queue drains at a configured rate. If the queue is full, new work is rejected. It smooths bursts rather than allowing them to reach the backend. The queue creates waiting latency, so it needs a maximum queue size and deadline policy. Token bucket and leaky bucket solve related but different problems. Token bucket allows bounded bursts at the service boundary; leaky bucket smooths output. A rate limiter can reject immediately rather than queue. For expensive APIs, a bounded concurrency semaphore may be more useful than a queue. Algorithm State Burst behaviour

Cost / caveat Fixed window Counter + window Boundary burst Simple, coarse Sliding log Timestamps per request Accurate rolling window Memory-heavy Sliding counter Current and previous counts Smoothed approximation Approximate distribution Token bucket Tokens + last refill Allows bounded burst Atomic time-based update Leaky bucket Queue / drain state Smooth output Adds wait; needs queue bounds

## 15.8 Distributed rate limiting with Redis

If each API instance has its own counter, 10 instances each enforcing 100 requests/minute can collectively allow about 1,000 requests/minute. A distributed limiter stores shared state in Redis or another coordination service. The update must be atomic: two requests cannot both observe the same remaining token and both spend it. Redis Lua scripts are useful because a script runs atomically on one Redis node. For Redis Cluster, all keys touched by one script must be in the same hash slot; hash tags such as {tenant-42} can co-locate related keys. If the limiter is a hard global quota across regions, cross-region coordination may dominate latency. Often use local burst limits plus a regional or central quota with an explicit overshoot bound.

## 15.9 Failure policy and client response

Fail-open means requests proceed when the rate-limit store is unavailable. This favours availability but risks overload and quota bypass. Fail-closed rejects requests when the limiter cannot decide, protecting the backend but reducing availability. A hybrid policy can fail-open for low-risk public reads and fail-closed for expensive or abuse-sensitive operations. HTTP 429 Too Many Requests is the common response. Retry-After can provide a delay in seconds or an HTTP date. RateLimit and related header standards are evolving; define your public contract clearly. Include remaining quota and reset information where safe. Do not reveal internal customer quotas to unauthenticated clients if that creates an abuse signal.

## 15.10 Python token-bucket implementation

This in-process implementation is useful for learning and a single-process limiter. It is not globally correct across multiple workers or machines. For production, use an atomic shared store or a dedicated rate-limit service.

### 💻 Python / implementation sketch

```
import time
from dataclasses import dataclass
@dataclass
class Bucket:
    capacity: float
    refill_per_second: float
    tokens: float = 0.0
    updated_at: float = 0.0
    def __post_init__(self):
        self.tokens = self.capacity
        self.updated_at = time.monotonic()
    def allow(self, cost: float = 1.0) -> bool:
        now = time.monotonic()
        elapsed = max(0.0, now - self.updated_at)
        self.tokens = min(
            self.capacity,
            self.tokens + elapsed * self.refill_per_second
        )
        self.updated_at = now
        if self.tokens < cost:
            return False
        self.tokens -= cost
        return True
```

bucket = Bucket(capacity=10, refill_per_second=2) print(bucket.allow())

## 15.11 Fairness and hierarchical limits

A single request may need to pass multiple policies: global service capacity, per-tenant quota, per-user abuse limit and endpoint-specific expensive-operation quota. Apply cheap local checks early, then shared checks where global correctness matters. A hierarchical design can protect the whole system and stop one tenant

```
from consuming all capacity.
```

Fairness requires more than a token bucket. Large tenants may buy higher quotas; noisy-neighbour isolation may require reserved capacity or weighted scheduling. Use separate queues or bulkheads for critical and best-effort work. Be explicit whether a quota is per second, per minute, per day, or a concurrency cap.

## 15.12 Testing and operations

- Test boundary bursts, sustained load, concurrent requests and clock skew.
- Verify atomicity with many workers hitting the same key simultaneously.
- Measure limiter latency separately from backend latency.
- Alert on denied rate, Redis errors and hot limiter keys.
- Test fail-open/fail-closed behaviour during Redis outage.

- Use bounded cardinality and TTL for per-client keys.
- Ensure trusted IP extraction strips spoofed headers at the edge.

### 💻 Extended Python lab

# Atomic token bucket is usually implemented in a Redis Lua script. # Pseudocode for one script execution: # 1. Read tokens and last_refill from one hash. # 2. Compute elapsed using a trusted time source. # 3. Refill and cap at capacity. # 4. If tokens >= cost: subtract and ALLOW; else DENY. # 5. Persist tokens + timestamp with TTL. # 6. Return allowed, remaining tokens and retry delay. # All steps must be one atomic operation. Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A public API serves 20,000 tenants. Each tenant may make 100 reads/second with a burst of 200, while report generation is limited to 2 concurrent jobs per tenant and 50 globally. Use a token bucket for read rate and a separate concurrency semaphore for reports. Enforce an IP-level unauthenticated limit at the edge, then tenant quotas after authentication. Use Redis atomic scripts for shared quota state, a local emergency limit to protect a region, and clear 429/Retry-After responses. Define what happens if Redis is unavailable.

### Interview questions and answer expectations

**Q. Compare fixed window and token bucket.**

**Q. Why is a local limiter not a global quota?**

**Q. How do you make a Redis rate-limit decision atomic?**

**Q. When should a limiter fail open or closed?**

**Q. Rate limit versus concurrency limit: what is different?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Implement sliding-window counter in Python.
- Write Lua token-bucket logic and test concurrent calls.
- Design hierarchical limits for IP, tenant and endpoint.

Operational metrics and signals

- Allowed/denied requests per policy and tenant
- Limiter decision latency and Redis command errors
- Token exhaustion and retry-after distribution
- Hot keys and key cardinality
- Backend saturation before and after limiting
- Queue depth and concurrent expensive jobs

Chapter mastery checklist

- Rate limiting controls arrivals; concurrency limiting controls in-flight work.

- Token bucket supports bounded bursts and sustained rate.
- Distributed state updates must be atomic.
- Use layered keys and hierarchical policies for fairness.
- Failure policy, response headers and abuse resistance are part of the design.

## 🔬 Advanced Engineering Clinic

Rate limiting is a distributed admission-control problem. The algorithm is only one part: the key, clock, state placement, fairness, failure mode and client contract determine whether the limiter is useful. Incoming request Identity + scope user / tenant / API / IP Atomic shared state Redis script / local counter Allow + remaining quota within policy 429 + Retry-After limit exceeded Protected service

Compare algorithms mathematically Fixed window uses a counter keyed by window start. It is cheap and easy to distribute, but a client can send near the full limit just before a boundary and again just after it, creating a boundary burst. Sliding log records timestamps and is accurate but stores one entry per request. Sliding-window counter estimates the moving window using current and previous bucket counts, trading exactness for bounded state. Token bucket stores a balance that refills at rate r up to capacity B. A request of cost c is allowed when tokens ≥ c, after which c tokens are deducted. It permits bursts up to B while bounding long-run average rate near r. Leaky bucket drains at a configured rate and smooths output; a strict queueing version adds waiting, while a policing version rejects overflow. Choose based on the protected resource. An API quota may count requests; a compute endpoint should charge weighted cost; an upload endpoint may limit bytes; a tenant may need both request rate and concurrent jobs. A single “100 requests/minute” policy is not meaningful if endpoints have radically different costs. Algorithm State cost Burst behaviour Accuracy Fixed window O(1) per key/window Boundary burst possible Exact per fixed bucket Sliding log O(requests in window) Smooth exact window High, expensive state Sliding counter O(1) per key Approximate smooth window Approximation at boundary Token bucket O(1) per key Allows bounded burst Good rate + burst control Leaky bucket O(1) or queue state Smooth output Good shaping Distributed token bucket with atomic update A token bucket update must be atomic: read prior balance and timestamp, refill, decide, deduct, and write new state as one operation. If two application instances perform those steps separately, they can both observe the same token and overspend the quota. Redis Lua scripts or an equivalent atomic primitive keep the state transition server-side. Use a time source with consistent semantics. Redis TIME can provide a server-side timestamp within a script; client clocks can drift. Clamp elapsed time to non-negative and cap tokens at capacity. Store state with an expiry longer than the refill horizon so unused keys do not live forever. Decide how to handle backward clock movement and Redis failover state loss. Hierarchical limits often apply simultaneously: user limit, tenant limit, API-key limit and global provider budget. All relevant checks must be coordinated or have a deliberate approximation. If one key passes but another fails, define whether token deductions are rolled back atomically; otherwise users can spend quota from a bucket even when another policy rejects the request. Advanced implementation note # Lua-style atomic token bucket core (illustrative): # tokens = min(capacity, tokens + (now - last) * rate) # if tokens >= cost: tokens = tokens - cost; allowed = 1 # else: allowed = 0; retry_after = (cost - tokens) / rate # Redis script updates tokens and last timestamp atomically. # Production code also validates arguments and manages expiry.

Fairness, failure policy and HTTP contract IP-based limiting is useful for anonymous abuse protection but NAT can put many legitimate users behind one address. User or API-key limits are fairer after authentication. Tenant-level budgets prevent a noisy neighbour. Weighted cost limits protect CPU, database and vendor quotas more accurately than raw request count. When the limiter is unavailable, fail-open preserves API availability but can

overload the protected system. Fail-closed protects capacity but can reject legitimate traffic. A hybrid may use a conservative local emergency limit when Redis is unavailable. The policy should be explicit per endpoint and security risk. Return HTTP 429 Too Many Requests when the caller exceeds a policy. Include Retry-After where meaningful and document rate-limit headers. Distinguish caller quota from global service overload (often 503 with a retry hint). Do not expose internal quota keys or other tenants' usage. Policy scope Example key Purpose Anonymous abuse Trusted client IP / subnet Protect public endpoints Authenticated user user_id + route group Per-user fairness Tenant tenant_id + resource class Noisy-neighbour control API credential api_key_id Contractual quota Global dependency provider / operation Protect vendor or DB capacity

## ❓ Advanced Design-Review Questions

- Implement a Redis token bucket with atomic refill/deduct and describe clock and failover assumptions.
- How would you apply per-user, per-tenant and global provider limits without allowing one noisy tenant to

monopolise capacity? Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 16. Bloom Filters

*CORE SYSTEM DESIGN*

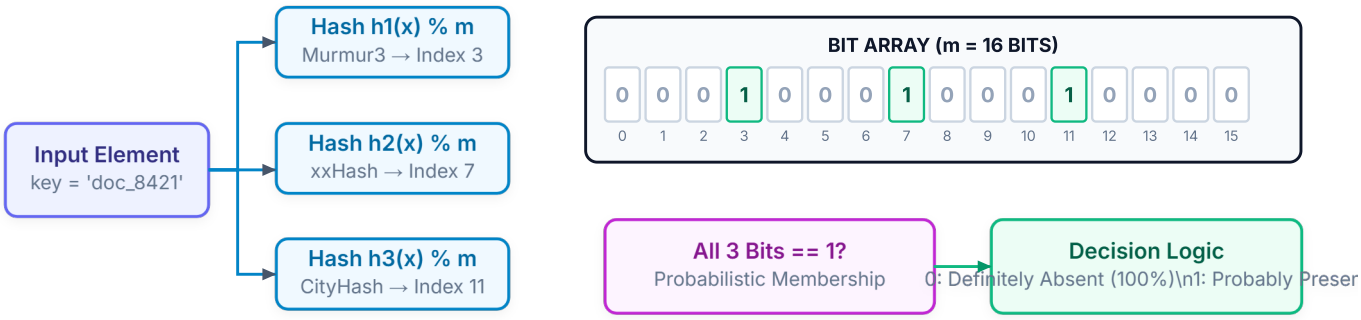
### 💡 INTUITIVE MENTAL MODEL

A guard at a venue has a compact checklist designed to answer "is this person definitely not on the guest list?" If the checklist says no, the person is definitely absent. If it says yes, the guard checks the authoritative list because the compact checklist can produce a false alarm.

### 🤖 AI INFRASTRUCTURE LENS: STRICT SEPARATION OF BLOOM FILTERS, MINHASH & LSH

**Deduplicating Billions of Training Tokens in Pre-Training Pipelines:** Pre-training corpora (Common Crawl, FineWeb, RedPajama) contain massive quantities of duplicate web pages and boilerplate text. Training on duplicates degrades reasoning and increases memorization. Distributed architectures use strictly separated probabilistic primitives:

- **Bloom Filter (Exact Set Membership):** A bit array with $k$ independent hash functions used to test exact membership ($x \in S$?) with zero false negatives. Used for $O(1)$ duplicate URL filtering and exact document SHA-256 hash checks. *A standard Bloom filter cannot measure semantic similarity or Jaccard overlap.*
- **MinHash (Jaccard Similarity Estimation):** Documents are tokenized into $n$-gram character shingles. An array of $M$ independent hash permutations creates an $M$-dimensional min-hash signature. The mathematical probability that two signatures match at index $i$ equals their exact Jaccard similarity: $P(h_i(A) = h_i(B)) = J(A, B) = \frac{|A \cap B|}{|A \cup B|}$.
- **Locality-Sensitive Hashing (LSH Banding):** Partitions the $M$ hashes into $b$ bands of $r$ rows ($M = b \times r$). Candidate document pairs sharing an identical hash tuple in at least one band are gathered into hash buckets, reducing candidate pair discovery from $O(N^2)$ to sub-quadratic time.
- **Exact Verification:** Candidate pairs retrieved from LSH buckets are explicitly compared against the threshold ($J(A, B) \geq 0.8$) using true set intersection before pruning duplicates.
- **Negative Query Cache for Vector Retrieval:** A Bloom filter populated with known unindexed query hashes or out-of-domain queries fast-fails invalid requests before issuing expensive vector graph traversals.




### 🎯 Learning Outcomes

- Explain bit arrays, hash functions and false positives.
- Derive the optimal bit count and number of hash functions.
- Implement a Bloom filter in Python and test its observed false-positive rate.
- Understand counting, scalable and partitioned variants.
- Use Bloom filters as a database/cache optimisation without treating them as truth.

Lookup key Bloom filter bit array + k hashes Definitely absent return not found any bit = 0 Possibly present all bits = 1 Authoritative cache / DB lookup

authoritative lookup.

## 16.1 The problem Bloom filters solve

Large systems often spend significant time checking whether a key exists. A database index or cache lookup can be expensive at high volume, especially for repeated nonexistent keys. A Bloom filter is a memory-efficient pre-check that can reject many definitely absent keys before touching the expensive store. The trade-off is probabilistic. A standard Bloom filter has false positives: it may say an item is possibly present when it was never inserted. It does not have false negatives for inserted items, assuming the bit array is intact and the same hash/probe configuration is used. A positive result must still be checked against the source.

## 16.2 Bit array and hashing process

A Bloom filter has an m-bit array initially set to zero and k hash-derived positions for each item. To insert an item, compute k positions and set those bits to one. To test an item, compute the same positions. If any bit is zero, the item is definitely absent. If all are one, it is possibly present. Bits can be set by multiple inserted items, so the filter cannot tell which item caused a bit to become one. Standard Bloom filters do not support deletion: clearing a bit could make another inserted item appear absent. Counting Bloom filters replace bits with small counters and can decrement on deletion, at a higher memory and correctness cost.

## 16.3 False-positive probability and sizing

For n inserted items, m bits and k independent uniform hash positions, the approximate false-positive probability is p = (1 - exp(-kn/m))^k. The optimal number of hashes is k = (m/n) ln 2. The approximate optimal bit count for target false-positive rate p is m = -n ln(p)/(ln 2)^2. These formulas assume a good hashing model and no saturation beyond the design capacity. For one million items and a 1 percent target false-positive probability, m is approximately 9.59 million bits, or about 1.14 MiB, with k around 7. This is much smaller than storing a million full keys. If inserted items exceed the design n, false-positive rate rises. Monitor capacity and rebuild or scale before saturation. m = -n ln(p) / (ln 2)^2 k = (m/n) ln 2 Example: n=1,000,000 and p=0.01 → about 9.59 bits/item and k≈7.

## 16.4 Hashing strategy and double hashing

Computing k independent hashes per lookup can be expensive. Double hashing derives positions from two hash values: g_i(x) = h1(x) + i x h2(x), modulo m. This approximates multiple independent positions with fewer hash computations. The hash function should distribute adversarial or patterned keys well and be deterministic across processes. In Python, hashlib provides deterministic digests. Avoid the built-in hash() for distributed Bloom filters because string hash randomisation can produce different positions in different processes. If the filter is shared, persist the bit array plus format version, m, k, hash algorithm and seed. Every reader must use the same configuration. Insert item x k hash positions 0 1 0 0 1 1 0 1 … Check all k positions Lookup y Any zero → definitely absent zero All one → maybe present query source all set

## 16.5 Python implementation

The following implementation uses SHA-256 double hashing and a bytearray. It is suitable for a learning lab and moderate test sizes. Production implementations should consider a native library, concurrent access, persistence, merge compatibility and adversarial input.

### 💻 Python / implementation sketch

```
import hashlib
import math
class BloomFilter:
    def __init__(self, capacity: int, false_positive_rate: float):
        if capacity <= 0 or not 0 < false_positive_rate < 1:
            raise ValueError("invalid capacity or probability")
        self.capacity = capacity
        self.m = math.ceil(
            -capacity * math.log(false_positive_rate) / (math.log(2) ** 2)
        )
        self.k = max(1, round((self.m / capacity) * math.log(2)))
        self.bits = bytearray((self.m + 7) // 8)

    def _positions(self, item: str):
        digest = hashlib.sha256(item.encode()).digest()
        h1 = int.from_bytes(digest[:16], "big")
        h2 = int.from_bytes(digest[16:], "big") | 1
        for i in range(self.k):
            yield (h1 + i * h2) % self.m
    def add(self, item: str):
        for pos in self._positions(item):
            self.bits[pos // 8] |= 1 << (pos % 8)
    def might_contain(self, item: str) -> bool:
        return all(
            self.bits[pos // 8] & (1 << (pos % 8))
            for pos in self._positions(item)
        )
```

f = BloomFilter(100_000, 0.01) f.add("customer:101") assert f.might_contain("customer:101") print(f.might_contain("customer:999")) # usually False

## 16.6 Variants and lifecycle

A counting Bloom filter uses counters instead of bits and supports deletion when counters are maintained correctly. A scalable Bloom filter layers filters as capacity grows, choosing error budgets for each layer so the total false-positive rate remains bounded. A partitioned Bloom filter divides the bit array into k slices and uses one position per slice, which can simplify parallelism and analysis. Bloom filters must be updated when the source changes. If the filter is built from a snapshot and new keys are inserted into the database but not the filter, it can create false negatives from the system perspective: the filter says definitely absent for a real item. Therefore the filter update path must be reliable, or the filter must be used only as a hint with a fallback that preserves correctness.

## 16.7 Distributed and persisted filters

A Bloom filter may be stored in Redis, a local file or a specialised service. If each application instance has its own filter, all instances need consistent updates or periodic rebuilds. A shared filter avoids divergence but adds network latency. A common approach is to load a compact filter locally and refresh it from a versioned snapshot, with a safe fallback to the database if freshness is uncertain. Merging two standard Bloom filters is

straightforward only when they have the same m, k, hash configuration and compatible bit positions: bitwise OR yields a filter representing the union. Filters built with different parameters cannot be safely merged without conversion/rebuild.

## 16.8 Where Bloom filters are useful

Use cases include preventing unnecessary database lookups for absent keys, checking whether a URL or file may have been seen, storage-engine membership filters, deduplication pre-checks and distributed cache miss reduction. They are especially useful when the set is large, membership checks are frequent and a small false-positive rate is acceptable. Do not use them as an authorisation list, as proof that a username is available, or as the sole source for financial uniqueness. A positive answer is only "maybe". A filter that is stale or not updated reliably can produce incorrect negatives at the application level.

## 16.9 Operational and security concerns

- Track inserted count and estimated false-positive rate.
- Rebuild when capacity is exceeded or filter saturation rises.
- Version hash algorithm, seed, m, k and serialization format.
- Protect against adversarial key patterns that create hot or saturated bits.
- Use a reliable update protocol or a safe authoritative fallback.
- Do not expose a positive result as proof of existence.
- Test union/merge only for compatible filter configurations.

### 💻 Extended Python lab

# Verify the observed false-positive rate empirically. known = [f"known:{i}" for i in range(100_000)] for item in known: f.add(item) assert all(f.might_contain(x) for x in known) unknown = [f"other:{i}" for i in range(100_000)] fp = sum(f.might_contain(x) for x in unknown) / len(unknown) print(f"Observed false-positive rate: {fp:.3%}") # It varies around the configured target; finite samples fluctuate. Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A key-value service has 200 million records and receives millions of random key lookups, many for absent keys. A database index is expensive under the miss pattern. Build a Bloom filter from a consistent snapshot, publish a versioned immutable filter and load it into each API instance. For a negative result, return not-found without touching the database only if the update protocol guarantees no newly inserted record is absent from the filter; otherwise fall back or use a freshness watermark. For a positive result, query the authoritative store. Rebuild when target capacity is reached.

### Interview questions and answer expectations

**Q. Why does a Bloom filter have false positives but no false negatives?**

**Q. Derive optimal m and k for n and p.**

**Q. Why can't a standard Bloom filter delete an item?**

**Q. How do you keep a distributed filter fresh?**

**Q. Can a Bloom filter prove a username is available?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Calculate memory and k for 10 million keys at 0.1% false-positive rate.
- Implement a counting Bloom filter and discuss counter overflow.
- Simulate saturation beyond configured capacity.

Operational metrics and signals

- Inserted item count versus designed capacity
- Estimated/observed false-positive rate
- Filter age and version distribution across nodes
- Bit density / saturation
- Source queries avoided and fallback rate
- Rebuild duration and update lag

Chapter mastery checklist

- Bloom filters cheaply rule out absent values.
- A positive means maybe present, never proof.
- Size the bit array and hash count from n and target p.
- Use deterministic hashing and versioned filter metadata.
- Reliable updates are essential to avoid application-level false negatives.

## 🔬 Advanced Engineering Clinic

A Bloom filter trades a controlled false-positive probability for compact memory and fast membership checks. It is a negative-lookup accelerator, not a replacement for the authoritative set. Candidate key x k hash positions h1(x) ... hk(x) Bit array m bits set positions on insert Membership test Any bit = 0 Definitely absent zero All bits = 1 Possibly present all set Authoritative lookup only on maybe

Derive memory and hash count For n inserted items, m bits and k independent uniform hash positions, the approximate false-positive probability is p ≈ (1 - e^(-kn/m))^k. The optimal number of hash functions for a fixed m/n is k ≈ (m/n) ln 2. The corresponding optimal bit budget is m ≈ -n ln(p)/(ln 2)^2. Lower false-positive targets cost memory logarithmically; reducing p from 1% to 0.1% requires materially more bits per item. Example: for n=10 million expected keys and target p=1%, the optimal bit count is approximately 95.9 million bits, or about 11.4 MiB before object/runtime overhead. The optimal k is about 6.64, so choose 7 hash probes. This is much smaller than storing ten million full identifiers, but a false positive still triggers the normal lookup. The model assumes good hash distribution and expected cardinality. If the filter is overfilled beyond planned n, false-positive probability rises. Monitor insert count and estimated p; rebuild or grow before the target is violated. Use a stable, well-distributed hash strategy and double hashing to derive multiple probes efficiently. p ≈ (1 - e^(-kn/m))^k k_opt ≈ (m/n) ln 2 m_opt ≈ -n ln(p) / (ln 2)^2 For n=10,000,000 and p=0.01: m≈95.9M bits; k≈6.64 → 7. Deletion,

counting and scalable variants A standard Bloom filter cannot safely delete an item because multiple items may have set the same bit. Clearing one bit could turn another inserted item into a false negative. A counting Bloom filter replaces bits with small counters, incrementing/decrementing counters for insert/delete. Counter overflow, hash collisions and concurrency must be handled; memory use is higher. A scalable Bloom filter adds layers as the set grows, allocating a false-positive budget to each layer so the combined false-positive probability remains bounded. A blocked Bloom filter improves cache locality by placing a key's probes into one cache-line-sized block. Cuckoo filters are another approximate membership structure that can support deletion and may be more space-efficient at some target false-positive rates. Bloom filters can become stale. If an item is deleted from the source but remains in the filter, the filter produces a false positive and the source lookup safely says absent. If a newly created item is not added to the filter, a standard Bloom filter can return "definitely absent" incorrectly. Therefore insertion updates must be reliable, or the filter must only be used as a hint with a safe fallback. Structure Deletion Strength Trade-off Bloom filter No Compact, simple False positives; no deletion Counting Bloom Yes Supports deletion More memory; counter lifecycle Scalable Bloom Layered growth Avoids fixed capacity Multiple lookup layers Cuckoo filter Yes Compact and deletable Insertion can fail at high load Use it at the right boundary A common use is avoiding disk/database lookups for keys that are definitely absent, such as checking whether a URL may exist in a distributed store. Another is a cache front-end: a filter can avoid expensive cache/backend probes for keys known not to exist. It is not appropriate to answer "yes, this user exists" for an authorisation decision without checking the authoritative system.

An attacker may deliberately target false positives or force excessive insertions. Protect filter updates, use bounded input cardinality, rebuild from an authoritative snapshot and version the filter. For a distributed system, document how new items reach every filter replica and what happens during rebuild or region failover.

- Define n and target false-positive rate before allocating memory.
- Treat positive as "maybe", never as proof.
- Ensure inserts propagate before relying on a negative result.
- Rebuild when capacity or hash quality assumptions are violated.

## ❓ Advanced Design-Review Questions

- Calculate m and k for 50 million keys at 0.5% false-positive probability, and state the consequences of

overfilling.

- Why can a standard Bloom filter not safely delete an item? Compare counting Bloom and Cuckoo filters.

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 17. Circuit Breaker Pattern

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

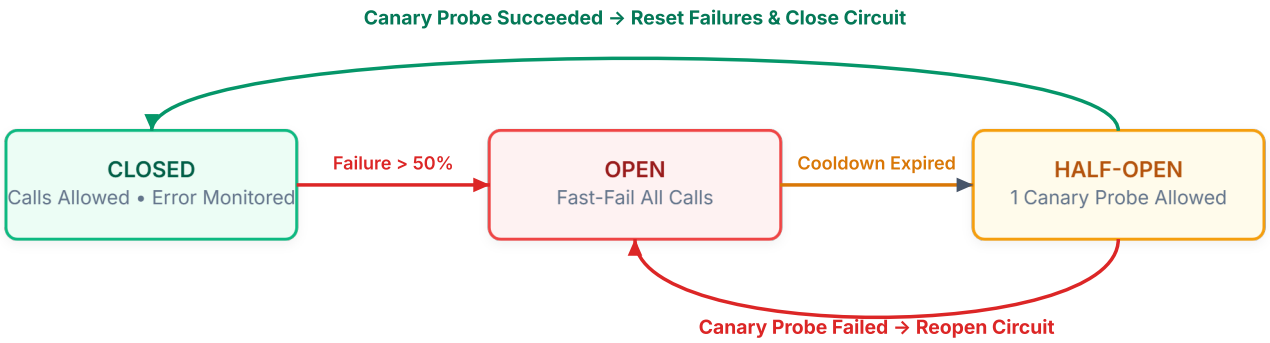
A circuit breaker in an electrical system interrupts current when a fault is detected, protecting the wider circuit. A software circuit breaker stops calls to an unhealthy dependency so that failures do not consume every request thread and spread through the system.

### 🤖 AI INFRASTRUCTURE LENS: CASCADING AI CIRCUIT BREAKERS & RETRY BUDGETS

**The Thundering Herd of LLM API Retries:** Upstream foundation model APIs frequently experience intermittent surges and return HTTP 429 (Rate Limit) or HTTP 504 (Gateway Timeout). When hundreds of enterprise worker threads simultaneously execute naive exponential backoff, they synchronize retry storms that sustain the provider's overload.

**AI Circuit Breaker Architecture:**

- **Dynamic Circuit State:** If error rates exceed 30% over a 10-second rolling window, the circuit trips to **OPEN**.
- **Semantic Degradation:** Rather than failing user queries, the circuit breaker activates a **local degraded pipeline**: (1) returns a cached semantic response if available; (2) routes to a local quantized 8B model; or (3) shortens context window to minimal few-shot examples to conserve provider tokens.
- **Canary Probing with Full Jitter:** In **HALF-OPEN** state, only 1 probe request is allowed every $T_{\text{wait}} = \text{random}(0, 2^{\text{attempt}} × 1000\text{ms})$ to safely test upstream recovery without causing retry stampedes.




## 🎯 Learning Outcomes

- Explain closed, open and half-open states.
- Distinguish circuit breakers from timeouts, retries, bulkheads and rate limits.
- Choose failure thresholds, rolling windows and recovery probes.

- Implement an async Python circuit breaker with clear caveats.
- Design fallback, metrics and per-dependency policies.

API service Circuit breaker Unstable payment provider CLOSED: allow call Fallback / degraded response OPEN: fail fast success/failure outcome

healthy; it limits the damage and probes recovery.

## 17.1 The cascading failure problem

When a dependency slows down, callers hold threads, event-loop tasks, sockets and database connections while waiting. As the dependency degrades, timeouts increase, retries add traffic and caller queues grow. A service that was healthy can fail because it has exhausted its own resources waiting for an unhealthy neighbour. A circuit breaker observes failures and temporarily stops sending ordinary calls to the dependency. Calls fail fast or use a fallback. After a cool-down period, a small number of probe requests are allowed. If they succeed, the circuit closes; if they fail, it opens again. This protects capacity and gives the dependency time to recover.

## 17.2 The three states

CLOSED is the normal state: calls flow to the dependency and outcomes are measured. OPEN means the failure threshold has been crossed: calls are rejected locally without attempting the dependency. HALF-OPEN allows a limited number of test calls after the open interval. Success can close the circuit; failure reopens it.

The state machine needs a rolling failure window or consecutive-failure threshold, an open duration, a limit on half-open probes and a rule for which outcomes count as failures. A 400 validation response should not normally count as dependency failure; a timeout, connection refusal or selected 5xx might. Use operation-specific classification.

## 17.3 Thresholds and windows

A circuit that opens after one failure may flap during transient noise. A circuit that requires 100 failures may be too slow to protect callers. A rolling window can count failed calls in the last N calls or last T seconds, often with a minimum sample size. A percentage threshold is useful when traffic volume varies; a consecutive-failure threshold can be useful for low-volume dependencies. Open duration can be fixed or exponentially increased after repeated failures. Half-open probes should be limited: allowing 100 probes at once can recreate the overload. Different endpoints may need different breakers if their dependency costs or correctness requirements differ.

## 17.4 Circuit breaker versus timeout, retry and bulkhead

Mechanism Controls Failure it addresses Timeout / deadline How long one call may wait Hung or slow dependency Retry Whether to attempt again Transient failure, if operation is safe Circuit breaker Whether calls should be attempted at all Repeated dependency failure Bulkhead How much concurrent capacity one dependency can consume Resource exhaustion / noisy neighbour Rate limiter How quickly requests may arrive Overload and abuse CLOSED requests pass track failures OPEN fail fast / fallback threshold crossed HALF-OPEN limited probe calls cool-down elapsed probe succeeds probe fails

workload.

## 17.5 Fallbacks and graceful degradation

A fallback must preserve the product contract. A recommendation service can fall back to a cached list or omit recommendations. A profile page might display a last-known non-sensitive value. A payment authorisation cannot safely return success merely because the provider is unavailable. For critical operations, fail with an explicit retryable state and reconcile the outcome. Fallbacks can become failure sources themselves. A cache fallback may be unavailable at the same time as the DB; a static default may be semantically misleading. Document whether fallback is stale, partial, approximate or unavailable. Track fallback rate and make degraded states visible to operations.

## 17.6 Python async circuit breaker

The following educational breaker supports closed/open/half-open state in one process. It is not a distributed breaker and does not include a rolling time window, concurrent probe lease or cancellation-safe production instrumentation. A real implementation can use a library or a carefully tested state machine.

### 💻 Python / implementation sketch

```
import asyncio
import time
from enum import Enum
from typing import Callable, Coroutine, Any, Optional

class State(Enum):
    CLOSED = "closed"
    OPEN = "open"
    HALF_OPEN = "half_open"

class CircuitOpen(Exception):
    # Raised when calls are rejected because dependency circuit is open or probing.
    pass

class CircuitBreaker:
    # Thread/Async-safe Circuit Breaker with Atomic Single-Probe HALF_OPEN semantics.
    def __init__(self, threshold: int = 5, reset_after: float = 10.0):
        self.threshold = threshold
        self.reset_after = reset_after
        self.failures = 0
        self.opened_at: Optional[float] = None
        self.state = State.CLOSED
        self._lock = asyncio.Lock()
        self._probe_lock = asyncio.Lock() # Permits exactly ONE canary probe in HALF_OPEN

    async def call(self, operation: Callable[..., Coroutine[Any, Any, Any]], *args, **kwargs) -> Any:
        async with self._lock:
            if self.state is State.OPEN:
                if self.opened_at and (time.monotonic() - self.opened_at < self.reset_after):
                    raise CircuitOpen("Dependency circuit is OPEN; fast-failing traffic")
                # Cooldown expired: transition to HALF_OPEN
                self.state = State.HALF_OPEN

            if self.state is State.HALF_OPEN:
                # Concurrent HALF_OPEN safety: fail fast if another canary probe is already in-flight
                if self._probe_lock.locked():
                    raise CircuitOpen("CircuitBreaker HALF_OPEN: Canary probe in flight; fast-failing concurrent
                await self._probe_lock.acquire()

        try:
            result = await operation(*args, **kwargs)
            async with self._lock:
                if self.state is State.HALF_OPEN:
                    self.state = State.CLOSED
                    self.failures = 0
                    self.opened_at = None
                    if self._probe_lock.locked():
                        self._probe_lock.release()
            return result
        except Exception:
            async with self._lock:
                self.failures += 1
                self.state = State.OPEN
                self.opened_at = time.monotonic()
```

```
                if self._probe_lock.locked():
                    self._probe_lock.release()
            raise
```

## 17.7 Concurrency and distributed breakers

In a multi-process deployment, each process may maintain its own breaker. This is often acceptable: the purpose is to protect each caller instance and reduce pressure. It also means one process can be half-open while another remains open. A globally shared breaker introduces coordination latency and can itself become a dependency. Use shared state only when fleet-wide coordination is required and justified. Half-open concurrency must be bounded. In the sample above, the lock protects state transitions but two calls may race after the state becomes half-open. Production code should admit a limited probe count or use a single-flight permit. Cancellation, timeout and task scheduling must be included in testing.

## 17.8 Retries and breakers must be designed together

Retries can defeat a breaker if every layer retries independently. If a client retries 3 times, gateway retries 2 times and service retries 3 times, one original request can create up to 18 downstream attempts. Put a total deadline and retry budget around the call chain. Use exponential backoff and jitter, and retry only transient failures and operations safe to repeat. A breaker should usually observe the final outcome of the dependency attempt at the layer that owns the call. A timeout should count as failure; a caller cancellation may not indicate dependency health. Do not count local validation errors or intentional 4xx responses as dependency failures.

## 17.9 Testing the state machine

- Inject connection refusal, slow response, timeout and 5xx responses.
- Verify the breaker opens after the configured threshold/window.
- Verify calls fail fast while open.
- Verify only a bounded number of half-open probes run.
- Verify success closes the breaker and resets relevant counters.
- Verify failure reopens it with correct timing.
- Test process restart, concurrency, cancellation and metrics.

## 17.10 Operational guidance

Use a separate breaker per dependency and often per operation class. A payment status read and payment capture may have different failure semantics. Configure timeouts at the HTTP client as well as breaker level. A breaker cannot interrupt a call that has no timeout. Expose state transitions and last-open reason to dashboards and logs. Circuit breakers are not a substitute for capacity, bulkheads or retries. They are a protection mechanism around a remote dependency. Tune them from observed latency and error distributions, then validate under fault injection.

### 💻 Extended Python lab

# Use a timeout and breaker together.

```
import asyncio
async def call_provider(client, breaker, payload):
    async def operation():
        return await asyncio.wait_for(
            client.post("/charge", json=payload),
            timeout=1.5
        )
    return await breaker.call(operation)
# Do not retry this POST blindly. Use provider idempotency keys
# and reconcile unknown outcomes after timeouts.
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code.

Applied design walkthrough A checkout service calls a recommendation service and a payment provider. Recommendation is optional: if its circuit is open, return checkout without recommendations. Payment capture is critical: if the provider times out, do not assume failure and retry with a new operation. Use a stable provider idempotency key, query or reconcile status, and keep a bounded timeout. Give each dependency a separate breaker; a recommendation failure must not open the payment circuit.

### Interview questions and answer expectations

**Q. Explain closed, open and half-open.**

**Q. How is a circuit breaker different from a timeout?**

**Q. What failures should count toward opening the breaker?**

**Q. Why can retries cause cascading failure?**

**Q. Should breaker state be globally shared across all instances?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Implement a rolling-window breaker with a minimum request count.
- Add a semaphore limiting half-open probes to one.
- Use fault injection to test slow and failing dependencies.

Operational metrics and signals

- Breaker state and state transition count
- Failure rate by dependency and operation
- Open duration and half-open probe result
- Timeout and retry counts

- Fallback/degraded response rate
- Bulkhead queue and concurrency saturation

Chapter mastery checklist

- The breaker stops repeated calls to an unhealthy dependency.
- Closed, open and half-open are explicit states.
- Timeout, retry, breaker and bulkhead solve different problems.
- Fallbacks must preserve business correctness.
- Bound retries and probe concurrency; observe and test state transitions.

## 🔬 Advanced Engineering Clinic

A circuit breaker is a local control loop around a dependency. It prevents repeated calls from consuming resources while a dependency is failing, but it must be paired with deadlines, bulkheads and a useful fallback.

Caller CLOSED allow calls + observe outcomes OPEN reject fast / fallback failure threshold Remote dependency normal call HALF-OPEN limited probes cooldown elapsed probe succeeds probe fails

Choose what counts as a failure Count failures that indicate the dependency cannot currently serve: connection errors, timeouts, selected 5xx responses and overload signals. Do not normally count caller validation errors, authorization denials or domain-level 4xx responses as dependency failures. A breaker that counts every non-200 response can open because clients submit bad input. Use a rolling time window or count-based window with a minimum sample size. A threshold such as 50% failures over the last 20 calls is more meaningful than opening after one failure in a quiet service. The exact values depend on traffic and recovery characteristics. Low-volume services may need a longer window or explicit consecutive-failure logic. Half-open limits probe concurrency. If 1,000 callers all probe simultaneously when cooldown expires, the breaker itself creates a recovery spike. Permit one or a small number of probes, release the rest to fallback or fast failure, and use jitter across instances so every breaker does not reopen or probe in synchrony. Outcome Typical breaker interpretation Connect timeout Failure Read timeout Failure Dependency 503 / 5xx Often failure, by policy Caller 400 validation error Not dependency failure Business “insufficient funds” Not dependency failure Local cancellation due to caller disconnect Usually not dependency failure

Timeout, retry, breaker and bulkhead are complementary A timeout bounds how long one call may wait. A retry decides whether to attempt again. A circuit breaker decides whether to attempt at all during a failure period. A bulkhead limits concurrent calls so one dependency cannot consume the entire process. None replaces the others. Use a strict overall deadline and small retry budget. A breaker should observe the final outcome of the attempt policy or be placed at a clearly documented boundary; otherwise retries may inflate failure counts or hide a dependency that is nearly failing. Do not retry beyond the caller’s remaining deadline. Use backoff and jitter for retries, not for immediate user-facing retries that have no time budget. Fallbacks must preserve semantics. Returning cached product recommendations may be acceptable; pretending a payment succeeded is not. A fallback should be explicit, observable and ideally include a degraded status. Never convert a failed write into a success response merely to keep the circuit green. Advanced implementation note # Dependency policy sketch policy = { "timeout_seconds": 0.35, "max_attempts": 2, "retry_on": ["connect_timeout", "selected_503"], "breaker_window_seconds": 20, "minimum_samples": 20, "failure_ratio_to_open": 0.5, "half_open_probes": 2, } # Tune from measurements and document the end-to-end deadline. Per-process breaker and observability

Breakers are commonly per process or per endpoint instance. If every application replica shares one distributed breaker state, a transient issue can become a global control-plane dependency and add latency to every call. Local breakers react quickly to their own observations but may differ between instances. Choose the scope deliberately; a fleet-level rate/admission policy is a separate mechanism. Monitor state transitions, open duration, half-open probe results, rejected calls, fallbacks and dependency latency. If the breaker stays open, the dependency may never receive traffic to recover; the system needs an operator view and a safe probe policy. If breakers flap, increase hysteresis, change windows or fix timeout noise. Test the state machine with deterministic time and injected dependency outcomes. Include a successful call after cooldown, a failed probe, a dependency that recovers slowly, cancellation, concurrent callers and a process restart.

- Do not count business rejections as infrastructure failures.
- Use minimum sample count and a rolling window.
- Limit half-open probes and stagger recovery.
- Expose degraded responses and breaker state.
- Keep breaker, timeout, retry and bulkhead budgets consistent.

## ❓ Advanced Design-Review Questions

- Which failures should count toward a breaker threshold? Explain why a 400 response should not usually open it.
- How do timeout, retry, breaker and bulkhead interact under a cascading failure?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 18. Saga Pattern

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

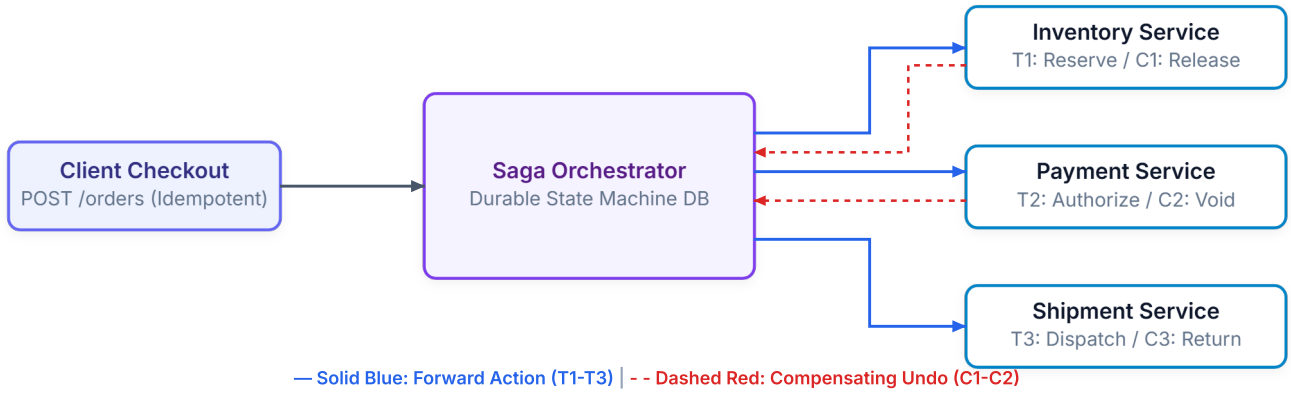
Planning a large event involves booking a venue, arranging catering and hiring transport. There is no single undo button. If transport cannot be arranged, the organiser cancels the venue and catering reservations according to agreed compensation rules.

### 🤖 AI INFRASTRUCTURE LENS: MULTI-STEP AGENT SAGAS & REVERSIBLE TOOL WORKFLOWS

**Compensating Hallucinations in Multi-Tool Execution:** An autonomous software engineering agent attempts to resolve a Jira bug through a 4-step tool workflow: (1) Fork Git Repo $ ightarrow$ (2) Provision Sandbox Cloud VM $ ightarrow$ (3) Apply Patch $ ightarrow$ (4) Execute End-to-End Test Suite.

- If Step 4 fails because the LLM generated invalid code, the workflow cannot be abandoned halfway—the VM will leak cloud costs, and the Git branch will pollute the repository.
- **Saga Orchestrator for Agents:** A durable state machine engine (Temporal / AWS Step Functions) tracks the agent's progress. Upon step failure or hallucination detection, the orchestrator executes the compensating chain in reverse order:

**Compensate: Tear Down VM (C2) → Delete Git Branch (C1) → Revert PR (C0)**




## 🎯 Learning Outcomes

- Explain why a local ACID transaction cannot atomically span independent services.
- Compare two-phase commit and sagas.
- Design choreography and orchestration variants.
- Build forward actions, compensations, timeouts, retries and idempotency.

- Handle ambiguous external outcomes and stuck workflow recovery.

Checkout request Saga orchestrator workflow state Order service create pending order CreateOrder Inventory service reserve stock ReserveStock Payment service authorise payment AuthorisePayment Shipping service create shipment CreateShipment success / failure

compensation is not a physical rollback.

## 18.1 Why distributed transactions are difficult

In a monolith with one relational database, checkout can update order, inventory and payment-ledger rows within one ACID transaction. In a microservice architecture, each service owns its database. There is no shared transaction manager that can atomically commit every database and third-party provider by default.

Two-phase commit coordinates participants through prepare and commit phases, but it can hold resources while waiting, requires participant support and introduces coordinator/availability complexity. A saga instead commits each local transaction and uses business-level compensations if a later step fails. It supports long-running workflows where a short database transaction is not practical.

## 18.2 Saga anatomy

A saga has a sequence of local transactions T1, T2, ... Tn. Each transaction commits locally and emits/returns a result. If a later step fails, the workflow may execute compensating actions C(n-1), ... C1 in reverse logical order. A compensation is a new business action: release a reservation, refund a payment or cancel a shipment. It may fail and need retry or manual intervention. Some steps are not compensable. A notification email cannot be unsent; a physical item may already have shipped; a payment refund is not equivalent to erasing a capture. Model the business outcome as states such as PENDING, CONFIRMED, CANCELLATION_REQUESTED, REFUND_PENDING and MANUAL_REVIEW rather than pretending every action can be rolled back.

## 18.3 Choreography versus orchestration

Choreography uses events: OrderPlaced triggers inventory reservation; InventoryReserved triggers payment authorisation; PaymentAuthorised triggers shipment. Each service knows the next reaction. It is suitable for a small number of independent steps, but branching, retries and compensation can become difficult to visualise. Orchestration uses a saga coordinator that stores workflow state and sends commands. It provides a visible state machine, central timeout management and explicit compensation order. It can become a critical service, so persist state, make command delivery idempotent and ensure the orchestrator can resume after restart. Dimension Choreography Orchestration Control flow Implicit in event subscriptions Explicit state machine Coupling Event contract dependencies Participants expose command/result contracts Observability Correlate many events Workflow state is central Best fit Simple, independent reactions Complex branching and compensation Failure risk Hard-to-see cycles Coordinator availability and complexity

## 18.4 Define the state machine before code

Write the workflow states and transitions first. Example: ORDER_PENDING → INVENTORY_RESERVED → PAYMENT_AUTHORIZED → SHIPMENT_CREATED → ORDER_CONFIRMED. Failure branches might be

INVENTORY_REJECTED → ORDER_CANCELLED, PAYMENT_FAILED → RELEASE_INVENTORY → ORDER_CANCELLED, or SHIPMENT_FAILED → REFUND_PAYMENT → RELEASE_INVENTORY → ORDER_CANCELLED. Every transition should define the command, expected result event, timeout, retry policy, idempotency key, compensation and terminal state. Include ambiguous outcomes: payment provider timeout may mean the charge succeeded but the response was lost. Query status by idempotency key before issuing a new capture.

Order pending T1 Reserve inventory T2 Authorise payment T3 Create shipment C1 Release inventory inventory/payment branch fails? Order confirmed C2 Void / refund payment shipment fails

compensation result.

## 18.5 Compensation is not rollback

A database rollback erases uncommitted changes within a transaction. Compensation is another committed transaction that attempts to restore an acceptable business state. A refund has different ledger, fee and audit consequences than never charging. Releasing inventory can fail if the reservation expired or was already fulfilled. Compensation must be idempotent and have a stable reference to the original action. It should not blindly reverse unrelated newer state. A compensation can itself be retried, escalated or placed in manual review. Some actions need forward recovery rather than compensation: retry shipment creation or choose an alternative carrier.

## 18.6 Timeouts, retries and idempotency

Each participant should accept a saga_step_id or idempotency key. Repeated ReserveInventory commands for the same order should return the original reservation result rather than reserve stock twice. Store command outcomes durably. Use deadlines and bounded retries with exponential backoff and jitter; separate transient infrastructure failures from permanent business rejection. A timeout is an unknown outcome, not proof that a remote operation did not occur. For payment, create a stable provider idempotency key, then query or reconcile if the response is lost. For inventory, use reservation IDs and expiry. For shipment, query by order reference before creating another shipment.

## 18.7 Durable saga state and recovery

The orchestrator should persist workflow ID, current state, completed steps, step attempts, deadlines, input references, compensation state and correlation IDs. On restart, it reloads incomplete workflows and resumes. State transitions should be transactional with the outbox event/command record so the coordinator does not advance state without reliably recording the next action. Operational tooling should list stuck sagas by age and state, show last error and allow safe replay or manual resolution. Do not manually edit database state without a supported recovery action. Retain audit history for financial and fulfilment operations.

## 18.8 Python: explicit saga coordinator sketch

This teaching example shows forward and compensation order. It omits durable persistence, outbox, retries and ambiguity reconciliation; these are mandatory for production use.

### 💻 Python / implementation sketch

```
async def checkout(command, inventory, payments, shipping):
    reservation = None

    payment = None
    try:
        reservation = await inventory.reserve(
            command.order_id, command.items,
            idempotency_key=f"reserve:{command.order_id}"
        )
        payment = await payments.authorise(
            command.order_id, command.amount,
            idempotency_key=f"authorise:{command.order_id}"
        )
        shipment = await shipping.create(
            command.order_id,
            idempotency_key=f"ship:{command.order_id}"
        )
        return {"status": "CONFIRMED", "shipment": shipment.id}
    except Exception:
        # Compensate in reverse order of completed actions.
        if payment is not None:
            await payments.void_or_refund(
                payment.id, idempotency_key=f"refund:{command.order_id}"
            )
        if reservation is not None:
            await inventory.release(
                reservation.id, idempotency_key=f"release:{command.order_id}"
            )
        raise
```

## 18.9 Sagas, outbox and event delivery

Saga commands and events should be delivered at least once and deduplicated. A participant can use an inbox table to make command handling idempotent and an outbox table to publish result events. The orchestrator should update its state and record the next command in one local transaction. This pattern makes the workflow resumable across crashes. Keep payloads small and include workflow_id, step_id, causation_id and idempotency_key. Use a stable partition key such as order_id when per-order order matters. Events should state what happened; commands should state what the orchestrator wants the participant to do.

## 18.10 Failure matrix

Failure point Possible reality Required behaviour Inventory timeout Reservation may have succeeded Query by idempotency key before retry/release Payment timeout Provider may have captured funds Reconcile status; never blindly charge again Shipment creation timeout Shipment may exist Lookup by order reference before retry Compensation fails Business state partially reserved/charged Retry, alert and manual review state Orchestrator crash Workflow stopped mid-step Reload durable state and resume safely Duplicate event Same result arrives again Inbox/deduplication and idempotent transition

## 18.11 When a saga is not appropriate

- A short operation contained in one database: use a local ACID transaction.
- A strict atomic invariant across resources that truly support a suitable distributed transaction: evaluate that

protocol explicitly.

- A process with no meaningful compensation or forward recovery: model the irreversibility honestly.
- A high-frequency operation where coordination cost dominates: redesign aggregate ownership or transaction

boundary.

### 💻 Extended Python lab

# Idempotency key rule for a participant.

```
async def reserve_inventory(order_id, items, db):
    key = f"reserve:{order_id}"
    async with db.transaction():
        existing = await db.fetchrow(
            "SELECT result_json FROM command_results WHERE idempotency_key=$1",
            key
        )
        if existing:
            return existing["result_json"]
        result = await perform_reservation(db, order_id, items)
        await db.execute(
            "INSERT INTO command_results(idempotency_key, result_json) "
            "VALUES($1, $2)", key, result
        )
        return result
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough Checkout reserves inventory, authorises payment, creates shipment and confirms an order. Inventory failure means no payment should be attempted. Payment timeout is ambiguous; query provider state using the same idempotency key. If shipment fails after payment authorisation, void the authorisation; if capture already occurred, issue a refund and wait for confirmation. Release inventory only after payment compensation reaches a defined safe state. A durable orchestrator records every transition and can resume after process crashes. The customer sees PENDING until the final confirmation.

### Interview questions and answer expectations

**Q. Why use a saga instead of one database transaction?**

**Q. Why is compensation not rollback?**

**Q. Compare choreography and orchestration.**

**Q. How do you handle a payment timeout with unknown outcome?**

**Q. What information must be persisted to resume a saga?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Draw a saga for hotel booking with room, payment and confirmation.
- Define compensation for a flight + hotel bundle.
- Implement participant idempotency and simulate duplicate commands.

Operational metrics and signals

- Saga completion time and age by state
- Compensation count and failure rate
- Pending/ambiguous payment outcomes
- Retry attempts and stuck workflow age
- Duplicate command/event rate
- Manual intervention queue and resolution time

Chapter mastery checklist

- A saga coordinates local commits across independent services.
- Compensation is a new business action, not rollback.
- Orchestration makes workflow state explicit; choreography distributes reactions.
- Persist state, use idempotency and treat timeouts as ambiguous outcomes.
- Every compensation needs retries, observability and a manual recovery path.

## 🔬 Advanced Engineering Clinic

A saga is a durable business workflow with explicit intermediate states and compensating actions. The difficult part is not calling services; it is recovering correctly from unknown outcomes, retries and compensation failure. Order accepted Reserve inventory Capture payment Arrange shipment Compensate release stock / refund known failure Order complete failure after payment Manual repair / reconciliation compensation fails

Model the saga as durable state, not nested try/except Write the state machine before writing orchestration code. Each step needs a stable step ID, input, attempt count, deadline, outcome, idempotency key and compensation definition. Persist the current state and every transition. A process restart should resume from durable state rather than replaying arbitrary in-memory code. For an order workflow, states might be PENDING, INVENTORY_RESERVED, PAYMENT_PENDING, PAYMENT_CAPTURED, SHIPMENT_CREATED, COMPLETED, COMPENSATING, COMPENSATED and MANUAL_REVIEW. Transitions should be guarded by expected current state/version so two workers cannot advance the same saga inconsistently. Use an outbox to publish commands/events after state transitions and an inbox/idempotency key at participants. A command may be delivered more than once. The participant should return the same logical result for the same command ID rather than repeating a debit or reservation. Field Purpose saga_id Durable workflow identity current_state + version Concurrency control and recovery step_id / command_id Idempotent participant invocation deadline / attempt

Bound retries and workflow age compensation status Track refund/release completion correlation_id Trace across services Unknown outcomes and compensation semantics A timeout after sending a payment request does not mean payment failed. The provider may have captured funds but the response was lost. Do not immediately issue a second capture. Query the provider by idempotency key or transaction reference, reconcile, and only then decide whether to retry or compensate. Compensation is not database rollback. A shipped parcel cannot be “unshipped”; the business may create a return label or refund. A seat reservation may be released if still held, but if the reservation expired and was sold to another customer, compensation may become a customer-support process. Compensation can fail, be delayed or require a new decision. Distinguish retryable, terminal and unknown outcomes. Retry network timeouts with the same idempotency key. Do not retry validation failures. Unknown provider outcomes require reconciliation. If compensation repeatedly fails, move to MANUAL_REVIEW with an alert and auditable operator actions instead of looping forever. Outcome Action Known success Persist success and advance Known transient failure Retry with same idempotency identity and bounded backoff Known business rejection Start defined compensation or terminal failure Unknown outcome / timeout Reconcile by external reference before retry Compensation failure Persist stuck state; alert and support repair

Choreography versus orchestration at scale Choreography lets services react to events without a central coordinator. It works for a small set of independent reactions, but workflow state becomes distributed across consumers and tracing the overall process is harder. Hidden cycles can form when services react to each other’s events indefinitely. Orchestration uses a coordinator that owns workflow state and sends commands to participants. It improves visibility, deadlines and compensation handling, but the orchestrator is a critical component and must be replicated, durable and idempotent. The orchestrator should coordinate business steps, not directly write participant databases. Choose choreography for loose fan-out such as analytics and notifications. Choose orchestration when there is a sequence with conditional branches, timeout handling, compensation and a need to answer “where is this order stuck?” in one place.

- Persist before sending the next command.
- Use stable idempotency keys per saga step.
- Reconcile unknown external outcomes.
- Treat compensation as a new business operation.
- Define manual repair and audit paths.

## ❓ Advanced Design-Review Questions

- A payment provider times out after possibly capturing funds. What should the saga do before retrying or

compensating?

- Define compensation for a shipment that has already left the warehouse. Why is it not a rollback?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

PART V • RESILIENCE & PRIMITIVES • CHAPTER 19

# 19. Blob and Object Storage

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

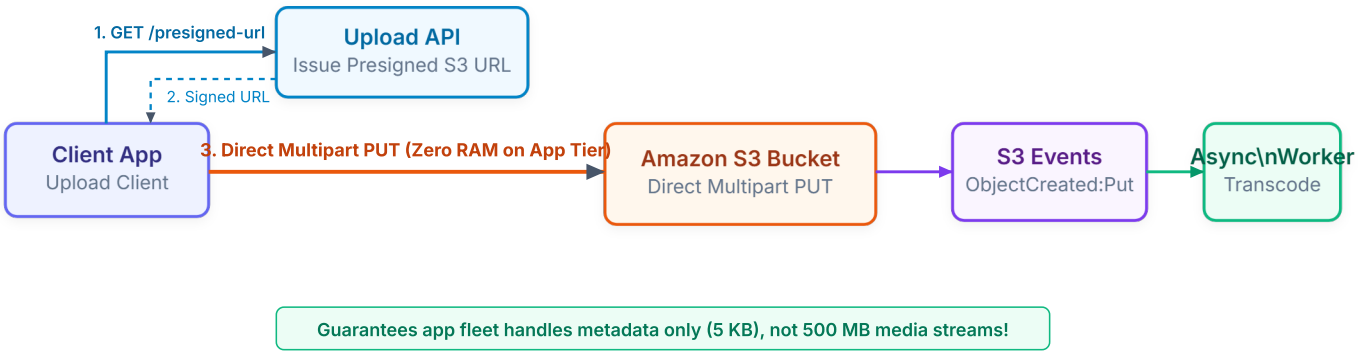
A warehouse stores parcels by object identifier rather than placing every item inside the office filing cabinet. The catalog keeps metadata and location; the warehouse handles large physical objects. Blob storage separates binary payloads from transactional metadata.

### 🤖 AI INFRASTRUCTURE LENS: SAFETENSORS, MEMORY MAPPING & GPUDIRECT STORAGE (GDS)

**Why Legacy PyTorch Checkpoints Fail at Scale:** Traditional PyTorch `.pt` / `.bin` checkpoints serialize weights using Python's `pickle` module. Loading 140 GB with pickle forces Python object deserialization, CPU memory copies, and exposes severe arbitrary remote code execution vulnerabilities.

**Safetensors, Host Memory Mapping (`mmap`), and GPU DMA:**

- **Safetensors Serialization:** Pure binary format storing contiguous tensor byte arrays aligned to page boundaries, preceded by a lightweight JSON header defining tensor shapes, data types, and byte offsets. Eliminates executable code risks.
- **Host Memory Mapping (`mmap`):** The OS system call `mmap(2)` maps file blocks directly into the process's *host virtual memory (CPU DRAM)*, avoiding user-space buffer copies. **Crucially, `mmap` maps into host CPU address space, NOT GPU memory.**
- **Host-to-Device Transfer:** Mapped host memory is page-locked (pinned via `cudaHostRegister`), allowing the GPU DMA controller to stream weights across the PCIe bus (e.g. PCIe Gen4 ×16 at $pprox 24 ext{ GB/s}$ sustained) via `cudaMemcpyAsync`.
- **GPUDirect Storage (GDS):** Bypasses host CPU DRAM bounce buffers entirely by establishing direct DMA transfers between NVMe SSDs and GPU VRAM over the PCIe switch. Requires NVMe drives, NVIDIA GPUs, the `nvidia-fs.ko` kernel module, and the `cuFile` library.
- **Realistic Load Timing:** Loading time is strictly bounded by hardware throughput: $T_{ ext{load}} = rac{ ext{Model Size (GB)}}{ ext{Sustained Throughput (GB/s)}}$. For a 140 GB 70B model in FP16: over single NVMe SSD ($6 ext{ GB/s}$), transfer requires $pprox 23 ext{ seconds}$; over 4x NVMe RAID0 with GDS ($26 ext{ GB/s}$), transfer requires $pprox 5.4 ext{ seconds}$; over cloud network storage ($1.5 ext{ GB/s}$), loading takes $pprox 93 ext{ seconds}$.




### 🎯 Learning Outcomes

- Distinguish object storage from file systems and block storage.
- Design object keys, metadata, multipart upload and direct client upload.
- Understand consistency, versioning, durability, lifecycle and replication.
- Secure access using IAM, bucket policies, signed URLs and encryption.
- Combine object storage with CDN and database metadata.

Browser / mobile API: auth + upload session request upload Object / blob storage large binary payload direct multipart upload CDN edge download signed upload URL Metadata DB owner, key, size, checksum create pending record Async scanner / transcoder object-created event origin fetch / signed access mark ready / metadata

consuming application-server bandwidth.

## 19.1 Object storage versus block and file storage

Block storage exposes fixed-size blocks to a host, which builds a filesystem or database volume on top. File storage exposes a hierarchical filesystem with directories, paths and file operations. Object storage exposes objects addressed by keys, commonly with metadata and HTTP APIs. It is designed for large-scale unstructured data and is accessed as whole objects or byte ranges rather than as a locally mounted POSIX disk in the general case. Blob/object stores are suitable for images, video, documents, backups, logs and model artefacts. They are not a drop-in replacement for a low-latency transactional database. Object listing and rename semantics differ

```
from a filesystem; a rename may be copy plus delete. Applications should use stable object IDs and metadata
```

records rather than treating bucket paths as a relational schema. Storage type Access abstraction Typical fit Trade-off Block Raw block devices / volumes Database disks, VM volumes Attached to compute; filesystem managed separately File Directories and file paths Shared team files, legacy apps Metadata and directory scale characteristics Object/blob Bucket + object key + metadata Images, video, backups, documents API/HTTP semantics; not a POSIX filesystem

## 19.2 Object model, keys and metadata

An object commonly has a key, payload bytes, metadata, size, content type, checksum, version and storage class. Buckets or containers provide namespace and policy boundaries. Object keys should be stable and opaque enough to avoid exposing user identity or private structure. A common pattern is tenant-id / random-object-id / version, but access policy must not rely on an unguessable key alone. Store application metadata in a database: owner, upload state, object key, checksum, content type, dimensions, moderation state, retention and visibility. The database controls business relationships; object storage contains bytes. Avoid trying to update a database row and upload object as one atomic transaction. Use an explicit state machine such as PENDING_UPLOAD, UPLOADED, SCANNING, READY, REJECTED and DELETED.

## 19.3 Direct-to-object-storage upload

A scalable upload path is: client asks API for an upload session; API authenticates and authorises, creates a pending metadata row and issues a short-lived signed URL or multipart capability; client uploads bytes directly to object storage; object-created event or completion callback triggers verification; a worker checks checksum, content type and malware; metadata is marked READY only after validation. This avoids proxying gigabytes through API workers. The API still controls who may upload, maximum size, allowed media type, destination key and expiry. The object store must restrict the signed capability to the intended operation, key and time window. Never accept a client claim that upload completed without verifying object metadata/checksum.

## 19.4 Multipart and resumable upload

Large uploads can be split into parts. The client initiates a multipart session, uploads parts independently, retries failed parts, then completes the upload with part identifiers/checksums. Multipart upload improves resilience to unreliable networks and allows parallelism. Incomplete multipart sessions consume storage and must be expired or aborted by lifecycle policy. Resumable upload requires a stable upload ID and a record of uploaded parts. Clients need retry-safe semantics and must not complete with missing or mismatched parts. For very large files, use checksums at part and whole-object levels, and verify content length before marking the object ready. Client API authorises + creates PENDING Private object storage quarantine prefix direct upload Short-lived signed URL Object created event Verify checksum / size malware scan / transform Metadata READY CDN / signed download

until verification completes.

## 19.5 Durability, availability and consistency

Durability is the probability that stored data is not lost over time; availability is the ability to access it when requested. High durability does not guarantee zero downtime or instant access. Providers often use internal replication or erasure coding, but applications still need deletion protection, versioning, replication policy and backup strategy for critical objects. Object consistency semantics differ by provider and operation. Many modern object stores provide strong read-after-write consistency for common object operations, but do not assume identical behaviour for every provider, listing path, replication target or cross-region copy. Check the service contract for overwrite, delete, list and replication semantics. Versioning helps recover prior objects but can increase storage cost.

## 19.6 Security: IAM, signed URLs and encryption

Use least-privilege IAM roles for services and separate roles for upload, read, lifecycle and administration. Bucket/container policies should block public access by default. Signed URLs are time-limited bearer capabilities: anyone who obtains one may use it until expiry, so scope them to one object, one method and a short lifetime. Do not log full signed query strings. Use TLS in transit and encryption at rest. Customer-managed keys may be needed for control, rotation or compliance. Validate uploads against content type and actual file signature; a client-supplied MIME type is not proof. Malware scan, size limits and decompression-bomb protection may be required. Treat files as untrusted input.

## 19.7 CDN integration and download flow

For public assets, a CDN can cache objects at edge locations. For private objects, the CDN can validate signed URLs/cookies or fetch from a protected origin. Cache keys must include the relevant authorisation variant. A private object must never become publicly cached because a cache-key policy ignored a signature or tenant dimension. Use content-hashed keys for immutable assets and long TTLs. Mutable objects can use version IDs or short TTL plus purge. Support byte-range requests for video seeking and large downloads. If an object is transformed, store derived variants under versioned keys and include transformation parameters in the key.

## 19.8 Lifecycle, storage classes and deletion

Lifecycle policies move objects to lower-cost tiers, expire temporary uploads and remove abandoned multipart parts. Cold storage has retrieval delay and retrieval fees; do not move an object to an archive tier if a user journey expects instant access. Model retention and legal hold explicitly. Deletion is often a workflow: mark metadata as deleted, revoke access, delete or transition object versions, remove CDN copies, and verify retention requirements. With versioning, a delete marker may hide an object while older versions remain. For regulated data, deletion must account for replicas, backups, logs and derived thumbnails according to policy.

## 19.9 Blob storage failure modes

- Upload succeeds but metadata update fails: reconciliation detects orphan objects.
- Metadata exists but object is missing: mark unavailable and repair or fail clearly.
- Duplicate completion callback: make finalisation idempotent.
- Signed URL leaks: short expiry, narrow scope, no sensitive logs.
- Malicious file: quarantine, scan and publish only after validation.
- Incomplete multipart upload: lifecycle abort and quota limits.
- Region outage: choose replication/backup based on RPO/RTO and cost.

## 19.10 Python: issue a signed upload capability

The example uses an S3-compatible client API. Exact SDK parameters can vary by provider. The API should authenticate the user, generate an opaque object key, create a pending metadata record and scope the signed URL to a specific method/key and short expiry.

### 💻 Python / implementation sketch

```
import boto3
from uuid import uuid4
```

s3 = boto3.client("s3", region_name="ap-south-1") BUCKET = "private-media"

```
def create_upload_url(user_id: str, content_type: str):
    object_key = f"uploads/{uuid4().hex}"
    # Persist PENDING_UPLOAD metadata before returning URL.
    url = s3.generate_presigned_url(
        ClientMethod="put_object",
        Params={
            "Bucket": BUCKET,
            "Key": object_key,
            "ContentType": content_type,
        },
        ExpiresIn=300,
    )
    return {"object_key": object_key, "upload_url": url}
# After upload, verify object existence, size, checksum and ownership
# before changing metadata state to READY.
```

## 19.11 Capacity and cost estimation

Estimate new objects per day x average size to obtain raw daily growth. Multiply by retention period for raw capacity, then add versions, derived renditions, replicas and backups. Estimate egress from views/downloads and average bytes transferred. For video, bitrate x viewing hours is often a better estimate than object count. Include request costs, retrieval fees, lifecycle transitions and cross-region replication. Use compression only when it reduces total cost or improves user experience; already-compressed JPEG, MP4 and ZIP files may gain little. Deduplication may help for identical files but adds hash computation and privacy considerations. Store checksums to verify integrity and detect accidental corruption.

### 💻 Extended Python lab

# Idempotent finalisation concept.

```
async def finalise_upload(upload_id, head_object, db):
    record = await db.get_upload(upload_id)
    if record.status == "READY":
        return record  # duplicate callback is safe
    meta = await head_object(record.object_key)
    if meta.size != record.expected_size:
        raise ValueError("size mismatch")
    if meta.checksum != record.expected_checksum:
        raise ValueError("checksum mismatch")
    # Transition with compare-and-set so duplicate workers cannot
    # publish a rejected or incomplete object.
    return await db.mark_ready_if_pending(upload_id, meta)
```

Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough A user uploads a 2 GB video from a mobile connection. The API authenticates the user, checks quota and creates a PENDING_UPLOAD row. It returns a signed multipart upload capability. The client uploads chunks directly to object storage and retries failed parts. Completion triggers a worker that verifies total size and checksum, scans for malware and transcodes renditions. Only then is the video marked READY and exposed through a CDN. If the worker crashes, a durable event or reconciliation job resumes processing. If the client abandons the upload, lifecycle cleanup removes incomplete parts.

### Interview questions and answer expectations

**Q. Compare object, file and block storage.**

**Q. Why upload directly to object storage?**

**Q. What is a signed URL and what risks does it introduce?**

**Q. How do you make multipart completion idempotent?**

**Q. How do you estimate storage and egress for a video platform?**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Design upload state transitions for a private document service.
- Estimate storage for 5 million photos/month at 3 MB average with 3-year retention and 2 derived thumbnails.
- Design a secure download path using CDN and signed access.

Operational metrics and signals

- Upload completion rate and multipart abandonment
- Object size/checksum mismatch
- Time from upload to READY
- Orphan object and missing-object reconciliation count
- Egress bytes and CDN hit ratio
- Storage by class, age and tenant
- Signed URL denials and malware/quarantine outcomes

Chapter mastery checklist

- Object storage is for unstructured bytes; metadata and business ownership belong in a database.
- Direct upload reduces API-server bandwidth and connection pressure.
- Use signed capabilities, verification and explicit upload states.
- Durability is not availability; versioning is not a full backup strategy.
- Plan lifecycle, CDN, cost, security and orphan reconciliation.

## 🔬 Advanced Engineering Clinic

Object storage design combines a metadata state machine with a byte-transfer path. The application should authorise and coordinate uploads without proxying every large byte through the API fleet. Client Upload API auth + quota + metadata create upload Quarantine object prefix direct multipart PUT Short-lived upload capability return URL Metadata state PENDING READY / REJECTED PENDING Checksum • malware scan content validation Published object versioned key + CDN status transition

Object identity, metadata and immutability An object is usually addressed by bucket/container plus key, optionally with a version ID. The key should be opaque and non-guessable where exposure matters; do not treat an unguessable key as authorisation. Store ownership, content type, declared size, actual size, checksum, encryption state, lifecycle and workflow status in a metadata database. Prefer immutable or versioned object keys for content that is distributed through a CDN. If a file changes, write a new object key and update metadata to point to the new version. This avoids stale CDN copies and makes

audit/recovery clearer. Metadata and object storage are separate systems, so model states such as PENDING_UPLOAD, UPLOADED, SCANNING, READY, REJECTED and DELETED. The database row is often the authority for whether an object is visible. An object existing in storage does not mean it is approved for download. Publish only after checksum, content-type sniffing, malware scanning and business validation succeed. A failed upload can leave an orphan object; lifecycle rules and reconciliation jobs clean these up. Metadata field Why it matters object_key + version Stable identity and versioning owner_id / tenant_id Authorisation and isolation declared + actual size Quota and integrity checksum Transfer verification state Visibility and workflow control encryption key reference Key management and rotation Multipart upload and direct transfer For large objects, multipart upload divides bytes into parts uploaded independently. The client can retry failed parts, upload in parallel within a limit, and resume using an upload ID. Completion assembles parts into the final object. Abort incomplete uploads and set lifecycle cleanup for abandoned sessions so storage is not consumed forever. The API creates an upload record, checks quota and content policy, then returns a short-lived signed URL or part URLs. The client uploads directly to object storage. On completion, the API verifies the object metadata and checksum, transitions to SCANNING and publishes a processing event. This keeps large byte flows away

```
from application servers.
```

Signed URLs are bearer capabilities. Scope them to a single object or part, method, size/checksum constraints where supported and short expiry. Never log the full URL. A client must not be able to choose a public bucket or overwrite another tenant’s key. Use generated keys and server-side ownership mapping. Advanced implementation note # State transition sketch; actual signature generation is provider-specific.

```
def create_upload(owner_id: str, declared_bytes: int):
    if declared_bytes <= 0 or declared_bytes > 2_000_000_000:
        raise ValueError("invalid object size")
    object_key = f"quarantine/{owner_id}/{new_uuid()}"
    upload = persist_upload(owner_id, object_key, "PENDING_UPLOAD")
    capability = sign_put(object_key, expires_in_seconds=600)
    return {"upload_id": upload.id, "url": capability}
# Completion must verify ownership, checksum and object existence.
```

Serving, lifecycle and recovery For public immutable content, a CDN can cache the object for a long time. For private content, enforce authorisation using signed URL/cookie or an authenticated edge function before returning cached bytes. Be careful that cache keys do not accidentally share private content across users. Range requests are important for video/audio and large downloads.

Lifecycle policies transition cold objects to cheaper storage classes, expire temporary uploads and delete retained data after policy deadlines. A storage class may have retrieval latency and minimum-duration charges; cost estimation should include requests, retrieval, egress, replication and lifecycle transition fees, not only GB-month. Durability is achieved through provider-managed redundancy and operational controls, but accidental deletion, compromised credentials and bad lifecycle policy can still destroy data. Enable versioning or retention where appropriate, apply least-privilege IAM, separate quarantine and published permissions, test restore and monitor replication/processing state. Failure Recovery Client upload interrupted Resume remaining multipart parts API crashes after object upload Reconciler finds object and advances or expires record Malware scan fails Keep quarantined; mark REJECTED; alert if needed Orphan object Periodic metadata-to-storage reconciliation CDN stale object Versioned key or targeted invalidation Credential leakage Short TTL, scope, revoke/rotate and audit

## ❓ Advanced Design-Review Questions

- Design a private multipart upload flow that handles abandoned uploads, malware scanning and CDN delivery.
- How do metadata state and object bytes recover if the object upload succeeds but the metadata update fails?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

# 20. Back-of-the-Envelope Estimation

*CORE SYSTEM DESIGN*

### 💡 INTUITIVE MENTAL MODEL

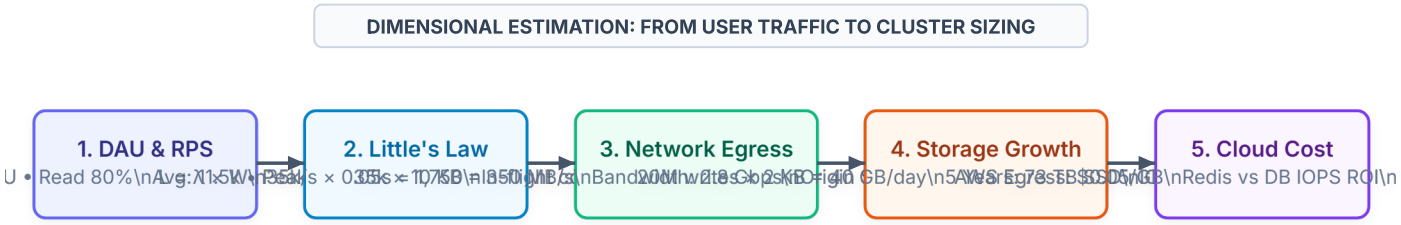
Before building a bridge, an engineer estimates the weight, span and material. The first estimate is not a final simulation; it identifies whether the design is in the right order of magnitude and which assumptions deserve testing.

### 🤖 AI INFRASTRUCTURE LENS: LLM MEMORY MATH, GPU SIZING & CLOUD ECONOMICS

**The Exact Mathematical Formula for LLM Memory Footprint:** To size GPU clusters for model inference, an engineer must calculate three distinct memory pools:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{KV-cache}} + M_{\text{activations}}$$

- **Model Weights ($M_{\text{weights}}$):** For a model with *P* billion parameters: $$M_{\text{weights}} = P \times \text{precision bytes}$$ Example: Llama-3-70B in 16-bit (FP16/BF16, 2 bytes) requires $70 \times 2 = 140\text{ GB}$. In 8-bit (INT8/FP8, 1 byte) it requires $70\text{ GB}$. In 4-bit (AWQ/GPTQ, 0.5 bytes) it requires $35\text{ GB}$.
- **KV-Cache Memory ($M_{\text{KV-cache}}$):** For context length *S*, batch size *B*, number of layers *L*, number of key-value heads $H_{\text{KV}}$, and head dimension *D*: $$M_{\text{KV}} = 2 \times B \times S \times L \times H_{\text{KV}} \times D \times \text{precision bytes}$$ For Llama-3-70B ($L=80, H_{\text{KV}}=8, D=128$) at FP16 (2 bytes): each token in context consumes $2 \times 80 \times 8 \times 128 \times 2 = 327,680\text{ bytes} \approx 320\text{ KB/token}$. A concurrency of 64 requests with 8,192 tokens consumes: $$64 \times 8192 \times 320\text{ KB} = 167.7\text{ GB of GPU HBM just for the KV cache!}$$
- **GPU Hardware Sizing:** To serve Llama-3-70B at FP16 with 64 concurrent 8k sessions: Total VRAM needed $= 140\text{ GB (weights)} + 168\text{ GB (KV)} + 12\text{ GB (overhead)} = 320\text{ GB}$. We require an **8x H100 80GB SXM5 node** (640 GB total HBM), running Tensor Parallelism *TP*=8.



### 🎯 Learning Outcomes

- Estimate DAU, requests/second, peak load, storage, bandwidth and memory.
- Use unit conversions and Little's Law correctly.
- Translate cache hit ratio and replication into capacity implications.
- Estimate Kafka throughput, partitions and object growth.
- Communicate assumptions, ranges and sensitivity rather than false precision.

Product assumptions DAU / actions / payload Workload model RPS / read-write / peak factor Capacity model CPU / DB / queue / cache Storage model bytes/day x retention x copies Bandwidth model requests/s x bytes x 8 Architecture + headroom validate with load tests

bottlenecks, not predict exact production numbers.

## 20.1 The estimation workflow

Start by clarifying scope: what counts as a user, request, object, event or transaction? Define geography, peak period, retention and availability target. Write assumptions in the open. Use round numbers so arithmetic is easy, and produce a low/base/high range when inputs are uncertain. Convert product behaviour into workload. Daily actions become average RPS by dividing by 86,400 seconds. Apply a peak factor or traffic profile. Add internal request amplification: one page load may trigger five API calls and each API call may query several services. Separate external RPS from internal service QPS.

## 20.2 Users, DAU and request rate

If 5 million daily active users perform 20 reads/day, daily reads = 100 million. Average read RPS = 100,000,000 / 86,400 = about 1,157. With a 5x peak factor, peak is about 5,787 RPS. This is only the read endpoint rate, not internal database QPS. If each request creates three downstream calls, internal service traffic can be much higher.

For a peak factor, justify the value using business patterns: morning commute, scheduled class start, sale campaign, sporting event or batch job. A single multiplier hides burst duration. A 10x peak for five minutes has a different autoscaling and queueing problem from a 3x peak for eight hours. Average RPS = DAU x actions per user per day / 86,400 Peak RPS = average RPS x peak factor (or use a time-series profile).

## 20.3 Throughput, concurrency and Little's Law

Use Little's Law L = lambda x W to estimate average in-flight requests. At 6,000 RPS and 180 ms average latency, L is about 1,080 concurrent requests. This is not a direct worker count: asynchronous servers can manage many waiting sockets, while CPU-heavy work may need a much smaller process pool. Distinguish request throughput from data throughput. A service processing 2,000 RPS with 4 KB requests and 20 KB responses transfers about 48 MB/s of application payload before protocol overhead: 2,000 × 24 KB = 48,000 KB/s. Convert bytes to bits by multiplying by 8 when estimating network bandwidth.

## 20.4 Storage growth and retention

For object storage, compute new objects/day x mean size. Multiply by retention days. Then add versions, derived media, replication, backup and metadata overhead. For database records, estimate row payload plus indexes, MVCC/engine overhead, replicas and backups. The raw data estimate is a lower bound, not a full provisioned capacity number. Example: 2 million images/day x 1.5 MB = 3 TB/day decimal. At 365 days, raw retained payload is roughly

## 1.095 PB. If three renditions add another 30 percent, and there are two independent copies, storage cost can

approach 2.85 PB before metadata and backup policy. State whether you use decimal TB or binary TiB. Product assumptions DAU / actions / payload / retention Traffic model average + peak RPS Compute / concurrency CPU, workers, pools Data model DB rows, indexes, cache Storage + network TB/day, Gbit/s Async work events/s, lag, retention Capacity range + headroom validate by load test

calculations.

## 20.5 Bandwidth and egress

Bandwidth = request rate x average bytes per response x 8 bits/byte. If 5,000 responses/s average 80 KB, payload bandwidth is 400,000 KB/s, or about 3.2 Gbit/s using decimal units. Add request traffic, headers, TLS overhead, retries and protocol effects. For media, estimate bitrate x concurrent viewers rather than average file size.

CDN byte hit ratio determines what portion reaches origin. At 92 percent byte hit ratio, origin egress is roughly 8 percent of edge-served bytes in a simple model. A request hit ratio is insufficient if large objects miss more often than small objects.

## 20.6 Cache memory and miss load

Cache memory is item count x average stored key/value size plus data-structure overhead and fragmentation. Add a safety margin and distinguish logical data bytes from resident memory. If 30 million keys average 1.2 KB, raw payload is 36 GB decimal before key names, allocator overhead and replicas. Cache miss RPS = source read RPS x (1 - hit ratio), assuming one source read per miss. At 25,000 read RPS and 94 percent hit ratio, source receives about 1,500 miss reads/s. Stampede and hot-key effects can make instantaneous miss load much higher, so size from peak and failure behaviour.

## 20.7 Database capacity and connection pools

Database QPS should include reads and writes from all services, not just public requests. A request may perform a user lookup, inventory check, audit write and event outbox insert. Query amplification can dominate. Estimate query count per endpoint, then validate with traces and database statistics. Connection count is not equal to query throughput. A connection can be idle or execute a long transaction. If 40 app instances each create 100 database connections, the database may see 4,000 connections even when average RPS is modest. Use

pooling, bounded concurrency and transaction timeouts. Pool size should be based on database concurrency capacity, not simply CPU core count on every app instance.

## 20.8 Kafka and event throughput

Estimate events/second from user actions and event fan-out. If 3,000 order events/s average 1.5 KB each, ingress payload is about 4.5 MB/s before protocol and replication overhead. With replication factor 3, broker disk/network work is larger than producer payload. Retention storage is ingress bytes/s x seconds retained x replication factor, adjusted for compression and compaction. Partition count is determined by parallelism, throughput per partition, consumer group needs and operational overhead. More partitions do not automatically make the cluster faster. Benchmark producer throughput, consumer processing rate and recovery/rebalance time. Ensure consumer capacity exceeds long-run arrival rate or lag grows without bound.

## 20.9 Availability and error budgets

Availability percentages translate into downtime budgets. 99.9 percent availability permits about 0.1 percent unavailable time: roughly 43.8 minutes in a 30.4-day month. 99.99 percent permits roughly 4.38 minutes/month. These are arithmetic budgets, not guarantees, and the definition of an unavailable request must be precise. For multi-region systems, account for failure-domain redundancy and surviving capacity. A two-zone design with each zone at 50 percent capacity may fail when one zone disappears. If one zone must carry the full peak, provision each zone accordingly or accept reduced service during failure.

## 20.10 Worked complete example: short-video platform

Assume 8 million DAU, 15 video starts/user/day, average watched duration 45 seconds, 1.8 Mbit/s average delivered bitrate, 500 KB metadata per thousand starts (aggregate API payload estimate to be refined), and 120-day retention for uploaded videos. Average starts/day = 120 million; average starts/s = about 1,389; at 6x peak = about 8,333 starts/s. Concurrent viewers by Little's Law using average session duration 45 seconds at peak start rate is a rough upper working estimate of 375,000 concurrent starts/view sessions if start rate is sustained; real session length and peak profile must be modelled. Video egress is driven by concurrent viewers x bitrate. At 250,000 concurrent viewers and 1.8 Mbit/s, payload egress is about 450 Gbit/s. This immediately suggests a CDN and careful regional capacity planning, not an API server streaming every segment. If 300,000 videos are uploaded/day at 80 MB average, raw ingest is 24 TB/day and 120-day raw retention is 2.88 PB before renditions, replication and backup. The estimate tells us dominant components: CDN egress and object storage, upload pipeline, metadata APIs, recommendation/read models and event processing. The exact design should be refined with measured video bitrate, watch duration, cache hit ratio, encoding renditions, geographic mix and peak concurrency.

## 20.11 Communicating uncertainty and sensitivity

An estimate should not pretend uncertain inputs are exact. Give a range and identify the parameters that dominate. If average video size could be 50-100 MB, show both. If peak factor could be 4-8x, size the base case and stress case. Sensitivity analysis helps focus product measurement: daily active users may matter less than watch duration or average bitrate. Use consistent units: KB versus KiB, MB versus MiB, GB versus GiB, decimal

Gbit/s versus binary GiB/s. State whether bandwidth is payload or line rate. Include replication, index and backup multipliers separately so reviewers can challenge assumptions without rebuilding the whole calculation.

## 20.12 Interview estimation checklist

- Clarify what is being counted and the time horizon.
- Write assumptions before arithmetic.
- Convert per-user activity to average and peak rates.
- Calculate payload bandwidth and storage growth.
- Apply cache hit ratio to source load, not to total requests blindly.
- Use Little's Law for in-flight work.
- Add replicas, indexes, retention and backup overhead.
- Identify the dominant cost/bottleneck and test the sensitivity of assumptions.

### 💻 Extended Python lab

```
from math import ceil
```

DAU = 8_000_000 actions_per_user_day = 15 peak_factor = 6 seconds_per_day = 86_400 avg_rps = DAU * actions_per_user_day / seconds_per_day peak_rps = avg_rps * peak_factor read_hit_ratio = 0.94 source_miss_rps = peak_rps * (1 - read_hit_ratio) mean_latency_s = 0.18 in_flight = peak_rps * mean_latency_s print(f"average RPS: {avg_rps:,.0f}") print(f"peak RPS: {peak_rps:,.0f}") print(f"source miss RPS: {source_miss_rps:,.0f}") print(f"in-flight estimate: {in_flight:,.0f}") Run the lab with a local test environment. Add type checking, tests, dependency timeouts, structured logging and metrics before treating it as deployable code. Applied design walkthrough An interviewer asks for a design that supports 10 million DAU. Do not start with technologies. Ask what users do, calculate average and peak requests, derive response bytes and storage growth, estimate concurrency, calculate cache miss load, and identify which data requires transactional consistency. Present a base and stress case. Then draw a minimal architecture and scale only the components the arithmetic identifies as bottlenecks. Finish with a load-test plan that validates the most uncertain assumptions.

### Interview questions and answer expectations

**Q. Estimate RPS for 6 million DAU with 24 actions/day and 5x peak.**

**Q. Estimate storage for 4 million objects/day at 750 KB for 180 days.**

**Q. Estimate bandwidth for 12,000 RPS and 36 KB response.**

**Q. Use Little's Law for 3,500 RPS and 220 ms mean latency.**

**Q. Explain how cache hit ratio changes database read load.**

A strong answer defines the requirement, explains the mechanism, draws the data/request path, states at least one failure mode, and justifies the trade-off. Avoid giving only a product name.

### Hands-on exercises

- Estimate a photo-sharing platform with uploads, feed reads and 2-year retention.
- Estimate Kafka ingress, retention storage and replication for 5,000 events/s at 2 KB.
- Build a spreadsheet or Python script that accepts DAU, actions, peak factor, object size and retention.

Operational metrics and signals

- Forecast versus actual DAU and peak profile
- External RPS and internal service QPS amplification
- Bytes/request, egress and storage growth
- Cache hit ratio and database miss RPS
- Concurrency, pool waits and queue age
- Capacity headroom under zone/region failure

Chapter mastery checklist

- Estimates are assumption chains, not exact forecasts.
- Convert daily actions to average RPS, then model peak and internal amplification.
- Calculate storage, bandwidth, memory and concurrency with consistent units.
- Include cache misses, replication, indexes, retention and backups.
- Use ranges and sensitivity to expose what must be measured next.

## 🔬 Advanced Engineering Clinic

Estimation is a chain of explicit assumptions. The purpose is not to predict the exact production number; it is to identify dominant resources, expose uncertainty and decide what must be measured. Product assumptions DAU • actions • payload • retention Traffic model average + peak RPS Compute + concurrency workers • pools • headroom Data capacity storage • indexes • replicas Bandwidth + egress bytes/s • CDN offload Events + queue rate • retention • lag Sensitivity + load test base / peak / failure case

Use dimensional analysis and unit discipline Always carry units through the calculation. DAU × actions/user/day gives actions/day; divide by 86,400 seconds/day to get average actions/second. Multiply by a peak factor only after stating how the peak is defined. Requests/second multiplied by bytes/request gives bytes/second. Event rate multiplied by average event size gives ingress bytes/second. Use decimal and binary units correctly. Storage vendors often quote decimal GB (10^9 bytes), while operating systems may display GiB (2^30 bytes). Network bandwidth is bits/second; divide bytes/second by 8 to compare with link capacity. State whether your “MB” means 10^6 bytes or 2^20 bytes.

Calculate average and peak separately. A daily total divided by 86,400 hides burstiness. A peak factor of 5× is an assumption, not a law. For flash sales, notifications or scheduled jobs, peak may be much higher and concentrated in a short interval. Model diurnal pattern and fan-out amplification where relevant. Average RPS = DAU × actions per user per day / 86,400 Peak RPS = average RPS × peak factor Bandwidth (bytes/s) = RPS × average response bytes Bandwidth (bits/s) = bytes/s × 8 Worked end-to-end example: photo service Assume 2 million daily active users upload 0.2 photos/day and view 30 photos/day. Uploads/day = 400,000; views/day =

60 million. Average upload rate ≈ 4.63/s and average view rate ≈ 694/s. With a 6× peak factor, peak uploads ≈ 28/s and peak views ≈ 4,167/s. This separates write traffic from read traffic instead of using one blended RPS. Assume average original image is 3 MB and retained for 365 days. Raw original storage/day = 1.2 TB; one-year raw retention approaches 438 TB before thumbnails, metadata, replication, versioning and cleanup. If thumbnails add 20% and storage has 2 copies in separate failure domains, a rough capacity envelope is 438 ×

### 1.2 × 2 = 1,051 TB. This is a planning estimate; lifecycle compression and actual retention distributions can

change it. Assume each view returns 150 KB of image data after resizing. At 4,167 peak views/s, image bandwidth is about 625 MB/s or 5 Gbit/s before protocol overhead. If the CDN hit ratio is 95%, origin image fetch rate is about 208 views/s, while edge egress still serves the user traffic. CDN offload reduces origin load, not total user-facing egress. Calculation Result Assumption Upload average 4.63/s 400,000 uploads / day View average 694/s 60M views / day Peak view rate 4,167/s 6× peak factor Raw storage/day

### 1.2 TB

400k × 3 MB 1-year raw originals 438 TB 365 days retained Peak image egress ~5 Gbit/s 4,167 × 150 KB × 8 Origin image fetch ~208/s 95% CDN hit ratio Sensitivity, confidence and bottleneck selection A useful estimate has a base case and a plausible range. If average object size is uncertain between 1 MB and 5 MB, storage and egress vary 5×. If peak factor is uncertain between 3× and 10×, compute capacity changes accordingly. Identify the assumption with the greatest effect and measure it first. Avoid stacking optimistic assumptions. A high cache hit ratio, low average payload and low peak factor may not occur together during a launch spike. Model a stress case: cache cold, one zone unavailable, provider latency elevated and retry volume increased. The system must have a defined response even if it cannot preserve the full normal SLO.

Turn the estimate into a test plan. Measure per-instance safe RPS, database query capacity, Redis miss load, queue processing rate and bytes per response. Compare forecast to production metrics and recalibrate. An estimate is a hypothesis with explicit units and uncertainty, not a precise promise.

- Show all assumptions and units.
- Separate read, write, upload and background traffic.
- Calculate peak, concurrency, storage, egress and replication overhead.
- Model cache miss and fan-out amplification.
- Run base, stress and failure scenarios.
- Identify the highest-sensitivity assumption and validate it.

## ❓ Advanced Design-Review Questions

- Estimate capacity for 5 million DAU with 18 reads/day, 2 writes/day, 7x peak and 80 KB average read

response.

- Run a sensitivity analysis for object size, CDN hit rate and retention period. Which assumption dominates cost?

Self-test: close the book and explain the architecture, the primary trade-off and two failure paths in three minutes.

### AI INFRASTRUCTURE LENS: HYBRID SEARCH (BM25 + HNSW) & SEARCH COMPLEXITY REALITIES

**Why Pure Vector Search Fails in Enterprise Retrieval:** Dense vector search (bi-encoders) excels at semantic understanding ("automobile malfunctions" matches "car engine issues"), but notoriously fails at exact keyword retrieval: product SKUs (`X-9042-B`), API error codes (`HTTP_504_TIMEOUT`), and proper nouns.

- **Hybrid Dual-Retrieval:** Retrieve top-100 sparse matches via BM25 (Elasticsearch/Lucene) and top-100 dense vector matches via HNSW (Milvus/Qdrant) in parallel.
- **Reciprocal Rank Fusion (RRF):** Merges ranks without calibrating arbitrary score distributions:

$$RRF(d) = \sum_{m \in \{\text{BM25, Dense}\}} 1 / (60 + r_m(d))$$

- **HNSW Complexity Clarification:** In HNSW graphs, logarithmic search scaling ($O(\log N)$) is an *empirical average-case property* observed in well-distributed vector spaces, not an unconditional worst-case mathematical guarantee. Under skewed data distributions, high intrinsic dimensionality, or restrictive boolean filters, graph traversal can degrade toward $O(N)$ unless *efSearch* beam widths are dynamically expanded.

PART VII • CHAPTER 21 (NEW)

# 21. Distributed Search & Inverted Indexes

*Building lightning-fast full-text retrieval and high-dimensional vector search engines.*

> **INTUITIVE MENTAL MODEL**
>
> If you want to find every mention of the word "distributed" in an 800-page book, reading page 1 to 800 (a full table scan) takes hours. Instead, you flip to the back of the book to the alphabetical index. The word "distributed" points directly to pages [14, 55, 70, 92]. An Inverted Index maps tokens to posting lists of matching document IDs.

## 21.1 Inverted Indexes, BM25 Scoring & Hybrid Vector Search

Full-text search engines (Elasticsearch, OpenSearch) decompose documents into normalized tokens during ingestion and calculate relevance scores using **BM25 (Best Matching 25)**:

- **Term Frequency (TF):** How often a term appears in the document (with diminishing returns).
- **Inverse Document Frequency (IDF):** Penalizes ubiquitous stop-words (e.g., "the", "and") while elevating rare, highly specific terms.
- **Field-Length Normalization:** A match in a 5-word title is scored higher than the same match in a 5,000-word essay.

Modern production systems combine keyword BM25 with **Vector Search (Hierarchical Navigable Small World - HNSW)** using Reciprocal Rank Fusion (RRF) to capture both exact keyword precision and deep semantic intent:

> ⚠️ **HNSW ALGORITHMIC COMPLEXITY & FILTERED ANN RECALL COLLAPSE**
>
> **Empirical Average-Case vs. Worst-Case Guarantees:** In HNSW graph indexes, logarithmic search scaling ($O(\log N)$) is an *empirical average-case property* achieved on uniformly distributed metric vectors, NOT an unconditional worst-case mathematical guarantee. Under severe data skew, high intrinsic dimensionality (the curse of dimensionality), or adversarial clustering, greedy graph routing can degrade toward $O(N)$ unless graph beam width (*efSearch*) is dynamically expanded.
>
> **The Filtered ANN Recall Collapse Trap:** When users execute filtered vector searches (e.g. finding similar documents where `department = 'compliance'`), evaluating boolean filters *during* HNSW greedy routing (single-stage filtered search) prunes graph links, isolating subgraphs and causing catastrophic 30% to 60% drops in recall. Production architectures solve this through ACORN multi-hop indexing or hybrid coarse pre-filtering + dense candidate generation + authoritative post-retrieval verification.

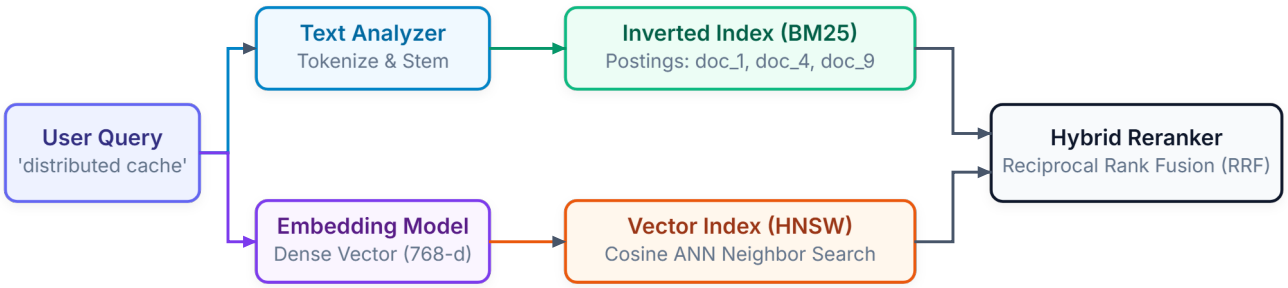



Figure 11. Symmetrical Dual-Retrieval Pipeline: Inverted BM25 keyword index fused with HNSW dense vector embeddings.

## 21.2 Production-Grade Python: Inverted Index with Postings Intersection

```
import math
import re
from typing import Dict, List, Set

class ProductionInvertedIndex:
    """Minimal Inverted Index supporting BM25 ranking and fast postings intersection."""
    def __init__(self, k1: float = 1.2, b: float = 0.75):
        self.k1 = k1
        self.b = b
        self.index: Dict[str, List[int]] = {} # token -> list of doc_ids
        self.doc_lengths: Dict[int, int] = {} # doc_id -> token count
        self.doc_count = 0
        self.avg_doc_len = 0.0

    def tokenize(self, text: str) -> List[str]:
        return [word.lower() for word in re.findall(r'\w+', text)]

    def add_document(self, doc_id: int, text: str):
        tokens = self.tokenize(text)
        self.doc_lengths[doc_id] = len(tokens)
        self.doc_count += 1
        self.avg_doc_len = sum(self.doc_lengths.values()) / self.doc_count

        seen_in_doc: Set[str] = set()
        for token in tokens:
            if token not in self.index:
                self.index[token] = []
            if token not in seen_in_doc:
                self.index[token].append(doc_id)
                seen_in_doc.add(token)

    def search_boolean_and(self, terms: List[str]) -> List[int]:
        """Executes fast posting list intersection."""
        cleaned_terms = [t.lower() for t in terms if t.lower() in self.index]
        if not cleaned_terms or len(cleaned_terms) != len(terms):
            return []

        # Sort by shortest posting list first for minimum set comparison
        cleaned_terms.sort(key=lambda t: len(self.index[t]))
        result = set(self.index[cleaned_terms[0]])
        for term in cleaned_terms[1:]:
            result.intersection_update(self.index[term])
        return sorted(list(result))

index = ProductionInvertedIndex()
index.add_document(1, "Consistent hashing distributes keys across nodes evenly")
index.add_document(2, "Kafka partitioned log guarantees FIFO ordering per partition")
index.add_document(3, "Consistent hashing ring maps keys clockwise to nodes")

print("Docs containing 'consistent' AND 'keys':", index.search_boolean_and(["consistent", "keys"])) # [1, 3]
```

**Chapter 21 Mastery Checklist**

- Shard search indices by document ID (`routing=doc_id`) to balance write ingestion evenly.
- For multi-tenant SaaS, route queries using `routing=tenant_id` to restrict search to a single physical shard.
- Never use search engines as primary transactional stores; search indices are eventually consistent read projections.
- Combine BM25 keyword matching with dense HNSW vector search to eliminate vocabulary mismatch in queries.

### AI INFRASTRUCTURE LENS: RAFT CONSENSUS IN GPU INFERENCE CLUSTER COORDINATION

**Deciding Master GPU Worker Membership via Raft:** In multi-node distributed LLM serving (e.g., serving a 405B parameter model split across 4 nodes of 8x H100 GPUs via Pipeline + Tensor Parallelism), the cluster requires microsecond coordination for pipeline orchestration and model weight sharding.

- Ray and vLLM clusters use etcd (Raft-backed) to manage GPU worker node heartbeats, master rank assignment, and dynamic worker replacement.
- If an InfiniBand switch drops, Raft quorums guarantee that only one majority partition can schedule forward tensor execution, preventing split-brain GPU memory corruption.

PART VII • CHAPTER 22 (NEW)

# 22. Distributed Consensus & Raft

*Reaching agreement across imperfect networks without single points of failure or split-brain.*

> **INTUITIVE MENTAL MODEL**
>
> A jury of five people must reach a verdict. Even if two jurors fall sick or cannot communicate, the remaining three jurors form a majority quorum (3 of 5). Because any two majorities of 5 must share at least one juror in common, it is mathematically impossible for two conflicting verdicts to ever be decided simultaneously.

## 22.1 The Raft Consensus Algorithm Mechanics

Raft decomposes distributed consensus into three clean subproblems:

1. **Leader Election:** Heartbeats with randomized election timers (150–300 ms). If a follower misses heartbeats, it transitions to Candidate, increments its Term, and requests votes. A candidate wins if it receives votes from a majority of nodes.
2. **Log Replication:** Clients send writes only to the Leader. The leader appends the command to its local log and broadcasts `AppendEntries` RPCs. Once a majority of nodes persist the log entry, the leader marks it **COMMITTED** and applies it to its state machine.
3. **Safety Invariant:** If a candidate's log is less up-to-date than a voter's log (evaluated by higher term or longer log index at same term), the voter *rejects* the vote! This guarantees that any elected leader is guaranteed to contain all previously committed entries.

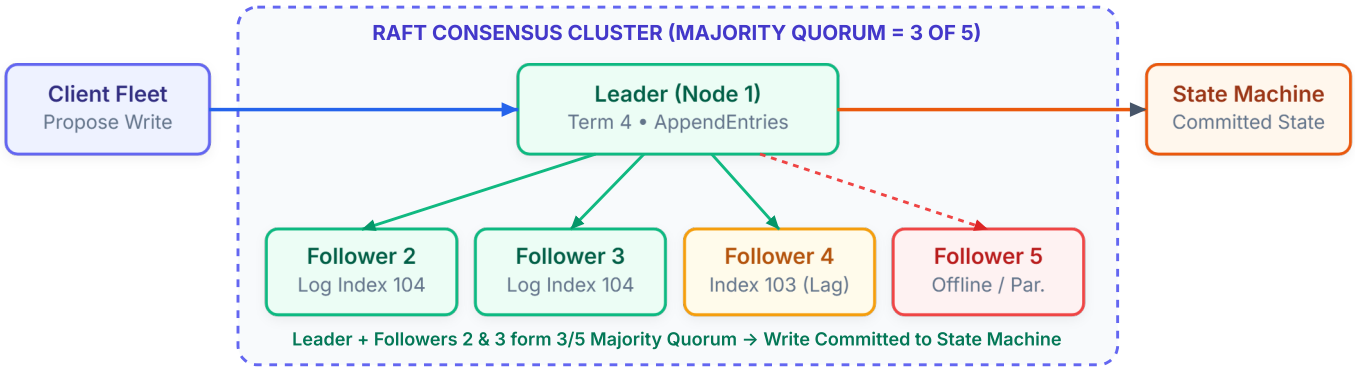



Figure 12. Symmetrical Raft Consensus Cluster: Leader replicating append-only entries to a majority quorum of followers.

### ARCHITECTURAL DILEMMA: PAUSE & REASON

**Scenario:** Why should a production Raft or etcd cluster always consist of an **odd number of nodes** (e.g. 3, 5, or 7)? What is the failure tolerance difference between a 4-node cluster and a 3-node cluster?

**Analysis:** Both a 3-node cluster and a 4-node cluster can tolerate exactly **1 node failure**! Majority for 3 nodes is 2 (3 - 2 = 1 failure tolerated). Majority for 4 nodes is 3 (4 - 3 = 1 failure tolerated). Adding a 4th node adds network overhead and an extra point of failure without increasing failure tolerance by even a single node! Always run an odd number of consensus nodes!

**Chapter 22 Mastery Checklist**

- Consensus clusters require an odd number of nodes (2f + 1) to survive f independent server failures.
- Never use Redis Redlock for correctness-critical distributed locks; use consensus-backed leases (etcd) with fencing tokens.
- A Raft leader commits an entry only after a strict majority of follower nodes acknowledge durable disk persistence.
- Randomize election timers to prevent perpetual split-vote stalemates during leader re-elections.

### AI INFRASTRUCTURE LENS: FLINK STREAMING FOR REAL-TIME RAG CONTEXT & FEEDBACK

**Continuous Online Feature Stores for Real-Time RAG:** In real-time AI applications (fraud detection, algorithmic trading, live sports agents), documents update millisecond by millisecond.

- Batch embedding ingestion every 6 hours results in stale answers. Apache Flink consumes document CDC updates from Kafka, chunks text, calls GPU embedding workers in mini-batches, and streams vector upserts directly to the HNSW vector database with sub-second end-to-end freshness.
- Sliding window analytics in Flink track real-time user feedback (thumbs down, prompt retry frequency) to trigger automatic model fallback alerts.

PART VII • CHAPTER 23 (NEW)

# 23. Stream Processing & Stateful Aggregations

*Real-time event-driven analytics, out-of-order event handling, and Chandy-Lamport snapshots.*

**INTUITIVE MENTAL MODEL**

Batch processing is like collecting all letters mailed over a week, sorting them on Sunday, and counting delivery times. Stream processing is standing next to the conveyor belt, reading each postmark as it zips past, and updating rolling hourly metrics in real time—even if some letters from Friday arrive on Monday due to a blizzard.

## 23.1 Event Time, Processing Time, and Watermarks

| Time Concept | Definition | Operational Consequence |
|---|---|---|
| **Event Time** | The wall-clock instant the user action actually occurred on the device (timestamp: 12:01:05). | Deterministic and reproducible across historical replays. Tolerates network delays. |
| **Processing Time** | The instant the stream processing server (Flink/Spark) evaluates the event. | Non-deterministic: pipeline lag or worker restarts will assign identical events to different hourly windows. |
| **Watermarks** | A temporal metric asserting: *"We assume all events older than time T have now arrived."* | Allows streaming windows to finalize results and advance state while bounding late-event wait times. |

## 23.2 Production-Grade Python: Event-Time Sliding Window with Watermarks

```
from dataclasses import dataclass
from typing import List, Dict

@dataclass
class TradeEvent:
    timestamp_ms: int
    ticker: str
    price: float

class EventTimeWindowProcessor:
    """Simulates a Flink-style tumbling event-time window with bounded out-of-order watermarking."""
    def __init__(self, window_size_ms: int = 10_000, max_lateness_ms: int = 2_000):
        self.window_size = window_size_ms
        self.max_lateness = max_lateness_ms
        self.current_watermark = 0
        self.windows: Dict[int, List[TradeEvent]] = {} # window_start -> events

    def process_event(self, event: TradeEvent) -> List[str]:
        output_results = []
        # Update watermark monotonically: max seen timestamp minus allowed lateness
        self.current_watermark = max(self.current_watermark, event.timestamp_ms - self.max_lateness)

        # Drop event if it arrived later than the finalized watermark
        if event.timestamp_ms < self.current_watermark:
            return [f"LATE EVENT DROPPED: {event.ticker} at {event.timestamp_ms} < WM {self.current_watermark}"]

        # Assign event to its tumbling window bucket
        w_start = (event.timestamp_ms // self.window_size) * self.window_size
        if w_start not in self.windows:
            self.windows[w_start] = []
        self.windows[w_start].append(event)

        # Trigger and close any windows whose boundary is older than current watermark
        closed_windows = [w for w in self.windows if (w + self.window_size) <= self.current_watermark]
        for w in closed_windows:
            events = self.windows.pop(w)
            avg_price = sum(e.price for e in events) / len(events)
            output_results.append(f"WINDOW [{w} - {w + self.window_size}]: Avg {events[0].ticker} = ${avg_price:.

        return output_results

processor = EventTimeWindowProcessor(window_size_ms=5000, max_lateness_ms=1000)
# Feed trades with out-of-order timestamps
print(processor.process_event(TradeEvent(1000, "AAPL", 150.0)))
print(processor.process_event(TradeEvent(4000, "AAPL", 152.0)))
print(processor.process_event(TradeEvent(7000, "AAPL", 155.0))) # Advances watermark to 6000! Finalizes Window 0-
```

**Chapter 23 Mastery Checklist**

- Always base financial metrics and aggregations on Event Time, never Processing Time.
- Tune watermarks to balance output latency against the probability of dropping late-arriving mobile data.
- Use the Chandy-Lamport distributed snapshotting algorithm to achieve true exactly-once state consistency in streaming graphs.
- Separate side-effect operations from idempotent aggregations during stateful stream replays.

### AI INFRASTRUCTURE LENS: ENFORCING ZANZIBAR REBAC PERMISSIONS IN ENTERPRISE RAG

**The Leaky Enterprise RAG Problem:** An enterprise indexes all company Google Docs, Notion pages, and Slack channels into a unified vector store. When an employee asks: *"What are the upcoming executive salary bonuses?"*, pure vector search retrieves confidential CFO documents because they are semantically identical, creating severe data leakage risks.

- **Why Vector Metadata Pre-Filtering Alone Fails:** Storing ACLs inside vector database metadata tags suffers from two fatal flaws: (1) *ACL Staleness:* When Alice is removed from a group, updating ACL metadata across millions of existing chunk vectors causes massive re-indexing write amplification; (2) *Filtered ANN Recall Collapse:* Applying restrictive boolean filters during HNSW traversal prunes valid graph pathways, causing 30–60% recall collapse.
- **Two-Stage Authoritative Enforcement Architecture:**
  1. **Stage 1 (Coarse Pre-Filtering):** Filter vector candidate retrieval by coarse tenant boundaries (e.g., `tenant_id = 'acme_corp'`) to bound the search space.
  2. **Stage 2 (Authoritative Zanzibar Evaluation):** Hybrid retrieval returns the top-50 candidate document chunks. Before any chunk text enters the LLM prompt context window, the query service issues batch `check(user, "viewer", doc_id)` requests to the authoritative Google Zanzibar ReBAC tuple store. Chunks failing evaluation are discarded, guaranteeing that retrieved content never reaches the model context without passing authoritative authorization policy.

PART VII • CHAPTER 24 (NEW)

# 24. Distributed Auth & Google Zanzibar

*Scaling fine-grained authorization, relationship graphs, and zero-trust identity across services.*

**INTUITIVE MENTAL MODEL**

A driver's license proves *who you are* (Authentication). But when you enter an office building, the receptionist looks at an organization directory to see if your team has permission to access the server room on the 4th floor (Authorization). In distributed systems, authentication is verified at the front gate, but authorization must traverse relationship graphs.

## 24.1 ReBAC: Google Zanzibar Architecture

Traditional Role-Based Access Control (RBAC) breaks down when access depends on recursive relationships (e.g., *"User Alice can view Doc X because Alice is a member of Team A, which owns Folder B, which contains Doc X"*).

Google Zanzibar models authorization as an evaluation over an append-only graph of **Object-Relation-User Tuples**:

🔬 **AUTHORITATIVE SECURITY AUTHORITY VS. VECTOR METADATA OPTIMIZATIONS**

In AI and search systems, candidates frequently make the dangerous claim that *"we just put ACLs in vector database metadata tags to handle authorization."* In production, this creates severe security vulnerabilities:

- **Stale Permissions & Write Amplification:** When user Alice is removed from a group, updating ACL metadata across millions of individual document chunk vectors causes massive re-indexing write storms.
- **Time-of-Check / Time-of-Use (TOCTOU) Exploits:** Vector metadata lags behind real-time permission revocations, leaking confidential documents during prompt context assembly.
- **Zookies & Bounded Staleness:** True Zanzibar ReBAC implementations issue cryptographic consistency tokens (Zookies) encoding Spanner snapshot timestamps to prevent the *"new enemy" problem* (where an authorized user shares a document, but the recipient's read request fails because the replica hasn't observed the ACL write). Vector metadata should only be used for coarse tenant partition pruning; the authoritative Zanzibar service must verify all candidates before prompt injection.

```
(namespace:object_id)#relation@(user_id | group_namespace:group_id#member)
```

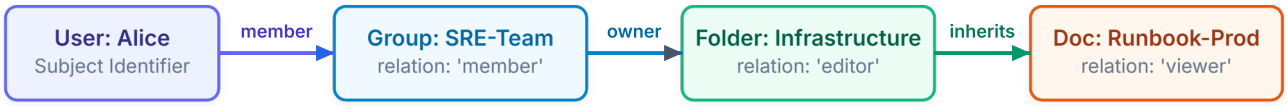



Figure 13. Symmetrical Relationship-Based Access Control (ReBAC) Graph evaluating multi-hop permission inheritance.

**Chapter 24 Mastery Checklist**

- Authenticate clients at the API Gateway using asymmetric JWT verification against a cached JWKS endpoint.
- Never pass raw client JWTs to internal microservices; terminate at edge and propagate a signed internal identity header.
- Enforce object-level authorization at the service that owns the data; the gateway only validates coarse scopes.
- Adopt Zanzibar-style ReBAC tuple stores to solve complex multi-tenant sharing without database join explosion.

### AI INFRASTRUCTURE LENS: LSM-TREES IN HIGH-THROUGHPUT VECTOR STORES & MEMORY TABLES

**Append-Only Writes for Vector Metadata & Embeddings:** Real-time AI systems ingest tens of thousands of vectors and metadata attributes per second (e.g. autonomous vehicle telemetry or security logs).

- Traditional B+ Trees experience catastrophic random write amplification because every new vector insertion forces 4KB disk page splits and random I/O.
- Modern vector engines (e.g. Milvus, LanceDB, Qdrant) utilize **LSM-Tree-inspired append-only storage**: incoming vectors are appended to a sequential WAL and buffered in an in-memory SkipList (MemTable). Periodically, immutable columnar vector segments are flushed to disk and compacted in background runs, maximizing sustained write throughput.

PART VII • CHAPTER 25 (NEW)

# 25. Storage Engines: B-Trees vs. LSM-Trees

*Under the hood of database disks: Page splits, random I/O, write amplification, and SSTable compaction.*

**INTUITIVE MENTAL MODEL**

A B-Tree is like neatly erasing and rewriting words in an alphabetical physical notebook: reads are immediate because you know the exact page, but writing requires constantly erasing and shifting sentences (random I/O). An LSM-Tree is writing down every update on sticky notes in an append-only inbox, and periodically hiring an assistant to sort and file the notes into permanent cabinets (sequential writes + background compaction).

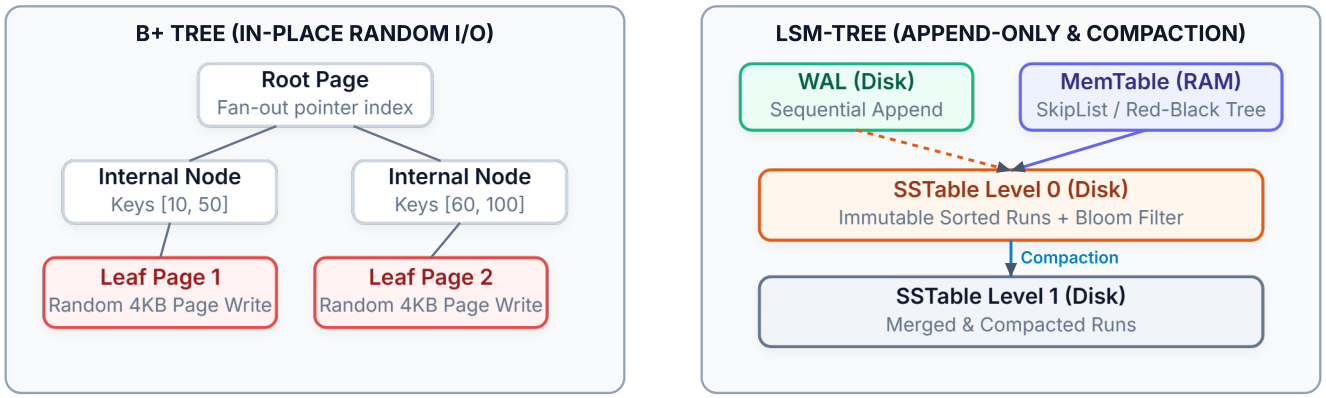



Figure 14. Storage Engine Architecture: B+ Tree Random Disk Page In-Place Overwrites vs. LSM-Tree Sequential Append with SSTables.

| Dimension | B+ Tree (Postgres, MySQL InnoDB) | LSM-Tree (Cassandra, RocksDB, ScyllaDB) |
|---|---|---|
| **Write Path** | Random I/O: Writes in-place to 4KB/8KB disk pages. May trigger page splits. | Sequential Append: Writes to memory MemTable and appends to disk WAL. Zero random disk writes! |
| **Read Path** | Deterministic O(log N) point lookups. Highly efficient for point reads and range scans. | Must search MemTable, then check Bloom filters across multiple on-disk SSTable levels. |
| **Write Amplification** | High: Modifying a 20-byte row requires writing an entire 8KB page to disk. | Low initially on write; amortized during background Leveled/Size-Tiered compaction. |
| **Optimal Workload** | Read-heavy OLTP databases requiring strong transactional ACID guarantees. | Write-heavy workloads: telemetry, IoT metrics, distributed event logs, time-series. |

**Chapter 25 Mastery Checklist**

- Choose LSM-Tree engines (RocksDB/Cassandra) when write throughput dominates and writes are append-heavy.
- Choose B+ Tree engines (Postgres/MySQL) when fast point reads and tight multi-row ACID transactions are required.
- Tune LSM compaction strategies: Size-Tiered for high-write ingest; Leveled for predictable read latency.
- Deploy Bloom filters in front of SSTables to avoid reading disk blocks for keys that do not exist.

### PRIMARY ENGINEERING LITERATURE & AUTHORITATIVE REFERENCES


- **Ch 21 (HNSW):** Malkov, Y. A., & Yashunin, D. A. (2018). *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. IEEE TPAMI, 42(4), 824-836.
- **Ch 21 (BM25):** Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333-389.
- **Ch 22 (Raft):** Ongaro, D., & Ousterhout, J. (2014). *In Search of an Understandable Consensus Algorithm*. USENIX Annual Technical Conference (ATC '14), 305-319.
- **Ch 23 (Flink):** Carbone, P., et al. (2015). *Apache Flink: Stream and Batch Processing in a Single Engine*. IEEE Data Engineering Bulletin, 38(4), 28-38.
- **Ch 24 (Zanzibar):** Pang, R., et al. (2019). *Zanzibar: Google's Consistent, Global Authorization System*. USENIX ATC '19, 923-935.
- **Ch 25 (LSM-Trees):** O'Neil, P., et al. (1996). *The Log-Structured Merge-Tree (LSM-Tree)*. Acta Informatica, 33(4), 351-385.

PART VIII • WORKSHOP A

# Workshop A: Global E-Commerce & Flash Sale Platform

*Handling 100k checkout RPS, zero overselling, and graceful payment reconciliation.*

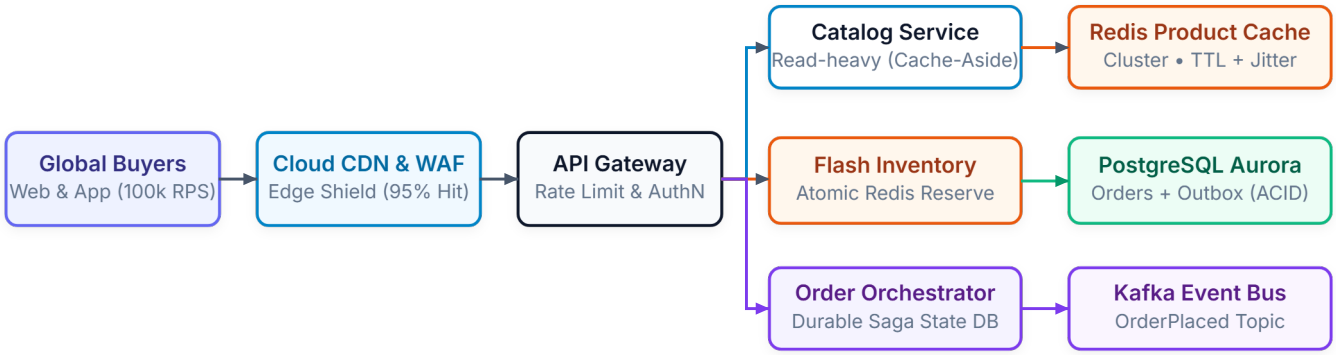



Figure 15. Symmetrical E-Commerce Flash Sale Topology: Edge WAF, atomic Redis reservation, and transactional outbox.

## A.1 The Flash Sale Concurrency Challenge

During high-profile flash sales, thousands of users compete for limited inventory (e.g., 1,000 units of an item). Direct SQL updates like `UPDATE inventory SET stock = stock - 1 WHERE item_id = ? AND stock > 0` create catastrophic row-level lock contention on the relational database primary, causing connection pools to saturate within milliseconds.

## A.2 Architecture & Data Flow

- **Atomic Reservation in Redis:** Decouple inventory decrement from relational database transactions. Use an in-memory Redis Lua script to atomically verify and decrement stock.
- **Idempotency Token Generation:** Every checkout attempt receives a client-generated UUID idempotency key stored in Redis with a short TTL (10 minutes). Duplicate network submissions return the cached reservation result.
- **Transactional Outbox & Asynchronous Sagas:** Once Redis reserves the item, the order is persisted in the PostgreSQL `orders` table together with an outbox record in a single ACID transaction. Debezium tails the Postgres WAL and publishes an `OrderPlaced` event to Kafka for payment capture.

## A.3 Atomic Redis Inventory Reservation Script

The following Lua script executes atomically on the Redis master shard responsible for the item key, guaranteeing zero overselling without database locks:

### ⚠️ REDIS CLUSTER MULTI-KEY HASH SLOT RULE & DURABILITY TRADE-OFFS

In Redis Cluster, multi-key Lua scripts fail with `(error) CROSSSLOT Keys in request don't hash to the same slot` unless all keys map to the exact same hash slot (0 to 16383). We enforce this by using **Redis Hash Tags**: `{item:item_id}:stock` and `{item:item_id}:reserve:order_id`. Only the substring inside `{...}` is hashed.

**Authoritative Persistence:** Redis asynchronous replication means that if the primary crashes before replicating an acknowledged reservation to replicas, a promoted replica may lose reservations. For global financial correctness, Redis acts as a fast reservation cache; the PostgreSQL transactional database acts as the durable authority, synchronized via idempotent outbox events and background reconciliation workers.

```
# flash_sale_inventory_lifecycle.py
import redis
from typing import Dict, Any

# Production Redis Lua Script with Hash Tags, Validation, and Idempotency
RESERVE_INVENTORY_LUA = """
local stock_key = KEYS[1]
local reserve_key = KEYS[2]
local order_id = ARGV[1]
local user_id = ARGV[2]
local quantity = tonumber(ARGV[3])
local ttl_seconds = tonumber(ARGV[4] or "600")

-- 1. Input Validation: Reject zero or negative quantities
if not quantity or quantity <= 0 then
    return -2 -- INVALID_QUANTITY
end

-- 2. Idempotency Check: Return existing reservation state if already processed
local existing_status = redis.call('HGET', reserve_key, 'status')
if existing_status then
    if existing_status == 'HELD' then
        return -1 -- ALREADY_HELD (Duplicate submission)
    elseif existing_status == 'CONFIRMED' then
        return -3 -- ALREADY_CONFIRMED
    end
end

-- 3. Stock Check: Verify available stock
local current_stock = tonumber(redis.call('GET', stock_key) or "0")
if current_stock < quantity then
    return 0 -- INSUFFICIENT_STOCK
end

-- 4. Atomically decrement stock and record reservation state machine
redis.call('DECRBY', stock_key, quantity)
redis.call('HMSET', reserve_key,
    'status', 'HELD',
    'user_id', user_id,
    'quantity', quantity,
    'created_at', redis.call('TIME')[1]
)
redis.call('EXPIRE', reserve_key, ttl_seconds)
return 1 -- SUCCESS_HELD
"""

# Lua script to release expired or cancelled reservations safely
RELEASE_INVENTORY_LUA = """
local stock_key = KEYS[1]
local reserve_key = KEYS[2]

local status = redis.call('HGET', reserve_key, 'status')
if status == 'HELD' then
    local qty = tonumber(redis.call('HGET', reserve_key, 'quantity') or "0")
    if qty > 0 then
        redis.call('INCRBY', stock_key, qty)
```

```
    end
    redis.call('HSET', reserve_key, 'status', 'RELEASED')
    redis.call('PERSIST', reserve_key)
    return 1 -- RELEASED_AND_RESTORED
end
return 0 -- NOT_IN_HELD_STATE
"""

class ProductionInventoryManager:
    # Production Flash Sale Manager enforcing full HELD -> CONFIRMED / RELEASED lifecycle.
    def __init__(self, redis_client: redis.Redis):
        self.r = redis_client
        self._reserve_script = self.r.register_script(RESERVE_INVENTORY_LUA)
        self._release_script = self.r.register_script(RELEASE_INVENTORY_LUA)

    def reserve_stock(self, item_id: str, order_id: str, user_id: str, qty: int = 1) -> str:
        # Enforce Redis Cluster hash tags to guarantee same hash slot
        stock_key = f"{item:{item_id}}:stock"
        reserve_key = f"{item:{item_id}}:reserve:{order_id}"

        result = self._reserve_script(keys=[stock_key, reserve_key], args=[order_id, user_id, qty, 600])
        status_map = {
            1: "RESERVED_HELD",
            0: "OUT_OF_STOCK",
            -1: "ALREADY_PROCESSED_HELD",
            -2: "INVALID_QUANTITY",
            -3: "ALREADY_CONFIRMED"
        }
        return status_map.get(result, "UNKNOWN_ERROR")

    def release_reservation(self, item_id: str, order_id: str) -> bool:
        stock_key = f"{item:{item_id}}:stock"
        reserve_key = f"{item:{item_id}}:reserve:{order_id}"
        return self._release_script(keys=[stock_key, reserve_key]) == 1
```

## A.4 Relational Schema & Transactional Outbox

```
-- Orders table stores canonical state
CREATE TABLE orders (
    order_id UUID PRIMARY KEY,
    user_id UUID NOT NULL,
    item_id UUID NOT NULL,
    amount NUMERIC(10, 2) NOT NULL,
    status VARCHAR(32) NOT NULL DEFAULT 'PENDING_PAYMENT',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Transactional Outbox ensures message dispatch is atomic with database mutations
CREATE TABLE order_outbox (
    outbox_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    aggregate_type VARCHAR(64) NOT NULL,
    aggregate_id UUID NOT NULL,
    event_type VARCHAR(64) NOT NULL,
    payload JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

PART VIII • WORKSHOP B

# Workshop B: Real-Time Distributed Chat & Presence Engine

*Architecting 50 million concurrent persistent WebSocket connections with sub-100ms message delivery.*

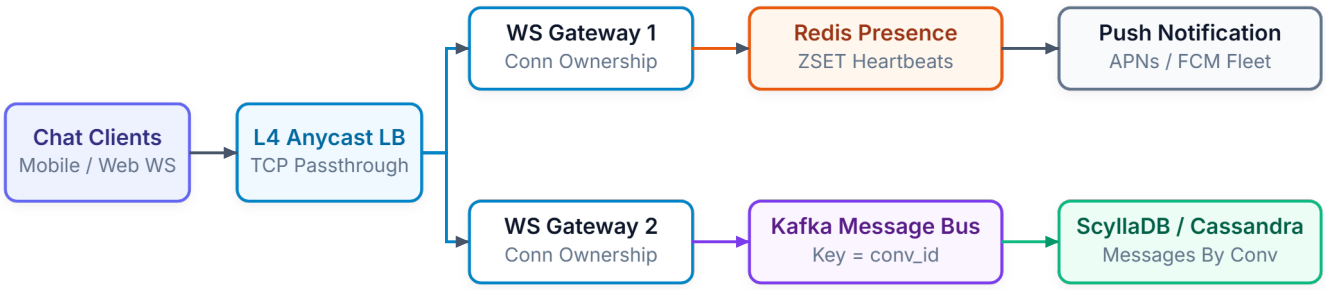



Figure 16. Symmetrical Real-Time Chat & Presence Topology: Decoupled WebSocket gateways, Redis heartbeats, and ScyllaDB message logs.

## B.1 ScyllaDB / Cassandra Data Model for Message History

In chat systems, queries always ask for the last N messages of a specific conversation sorted by time. Using a wide-column store ensures sequential disk reads:

```
CREATE KEYSPACE chat_system WITH replication = {'class': 'NetworkTopologyStrategy', 'us-east': 3};

CREATE TABLE chat_system.messages_by_conversation (
    conversation_id UUID,
    message_id TIMEUUID, -- Combines timestamp and uniqueness; gives natural reverse-time sort
    sender_id UUID,
    payload TEXT,
    PRIMARY KEY ((conversation_id), message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);
```

**ARCHITECTURAL DILEMMA: CELEBRITY CHANNELS & FAN-OUT**

In 1-on-1 chats, fan-out on write (pushing message to recipient's inbox queue) is optimal. But in a group channel with 250,000 members, writing 250,000 message copies per second destroys cluster disk IOPS! **Solution:** Use **Fan-out on Read** for large channels. Store one master message in `messages_by_conversation`, and broadcast live messages over ephemeral WebSocket pub/sub channels without creating 250,000 database rows!

PART VIII • WORKSHOP C

# Workshop C: Global Video Streaming & Processing Platform

*Direct-to-S3 uploads, async distributed transcoding, HLS/DASH chunking, and multi-terabit CDN offload.*

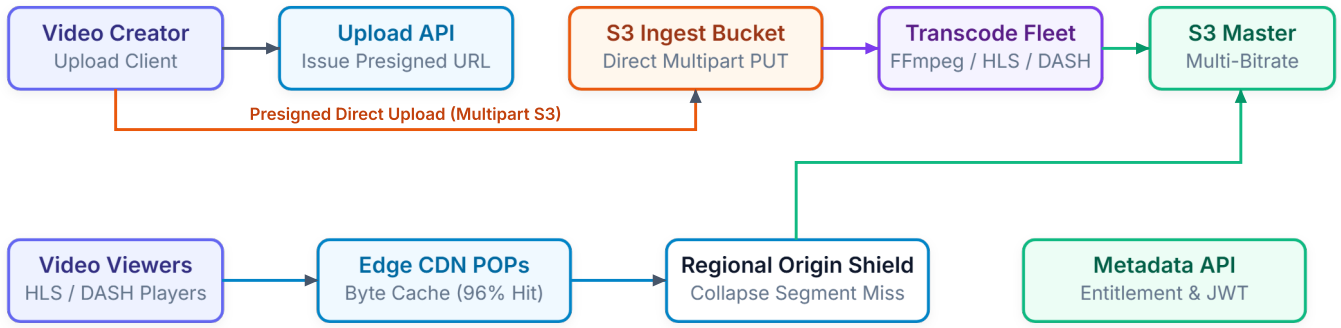



Figure 17. Symmetrical Video Streaming Blueprint: Decoupled multipart ingest pipeline and multi-tier CDN delivery.

## C.1 Mathematical Bandwidth & Cloud Economics Breakdown

- **Peak Viewers:** 1,000,000 concurrent streaming sessions at an average 1080p bitrate of 2.5 Mbit/s.
- **Total Peak Egress:** 1,000,000 × 2.5 Mbit/s = **2.5 Tbit/s** ≈ 312.5 GB/s.
- **CDN Offload:** With a 96% edge cache hit ratio, origin egress is reduced to only 0.04 × 312.5 = **12.5 GB/s**, completely protecting origin object storage from bandwidth collapse.

PART VIII • WORKSHOP D

# Workshop D: Distributed Ride-Hailing & Geospatial Matching Engine

*Ingesting 250k location pings/sec, Uber H3 hexagonal spatial indexing, and atomic dispatch state machines.*

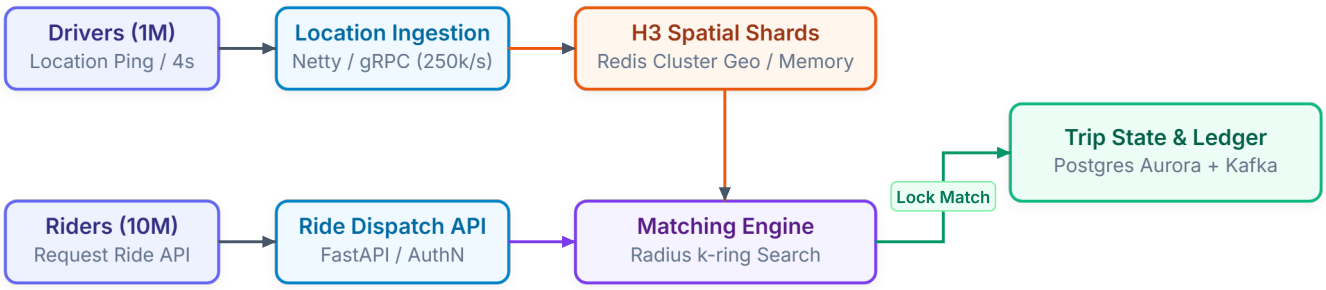



Figure 18. Symmetrical Ride-Hailing Matching Topology: High-throughput Netty ingestion, H3 hexagonal spatial partitioning, and transactional dispatch.

## D.1 Spatial Partitioning: Why Uber H3 Hexagons Win Over Geohashes

Traditional square grid partitioning (and rectangular Geohashes) suffers from severe distortion: distance from the center to a corner is $\sqrt{2} \approx 1.414$ times longer than distance to an edge, creating uneven neighbor search boundaries.

Uber's **H3 Hexagonal Spatial Index** ensures that every neighboring cell is at the exact same invariant distance from the center, enabling fast radius searches using concentric rings (k-ring). Driver locations are indexed into Redis sorted sets keyed by H3 Cell ID, allowing sub-5 ms proximity lookups across 100,000 active drivers per city!

PART VIII • WORKSHOP E (AI INFRASTRUCTURE)

# Workshop E: High-Throughput Distributed LLM Serving & Inference Platform at Scale

*vLLM architecture, PagedAttention block manager, Continuous Batching scheduler, Prefix KV-Cache router, and Tensor Parallelism.*

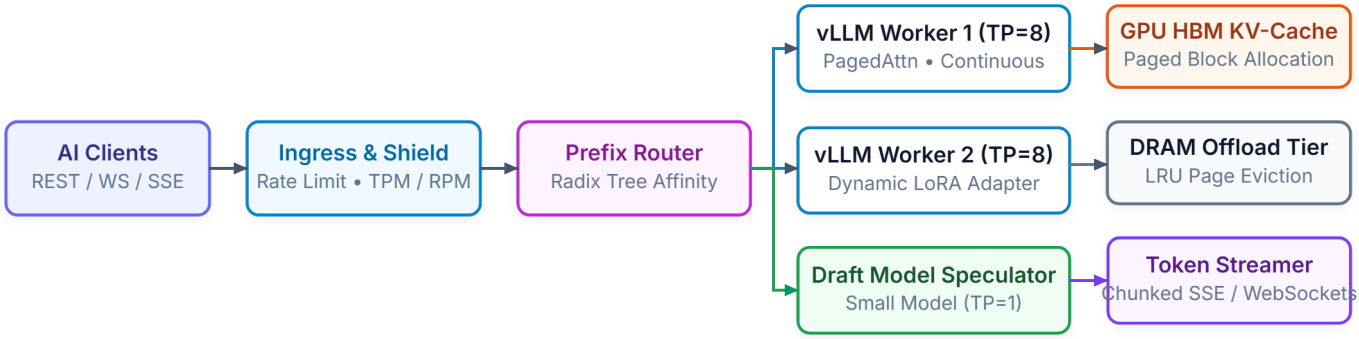



Figure 19. Symmetrical Distributed LLM Serving Topology: Dual RPM/TPM rate shielding, Radix prefix routing, 8-way Tensor Parallelism over NVLink, PagedAttention virtual memory, and chunked SSE streaming.

## E.1 System Requirements & Performance Objectives

- **Sustained Throughput:** 10,000 generation requests/minute with prompt context lengths varying from 512 tokens to 32,768 tokens.
- **Latency SLOs:** TTFT (Time To First Token) ≤ 180 ms (p95); TPOT (Time Per Output Token) ≤ 25 ms/token (equivalent to 40 tokens/sec decode throughput).
- **Multi-Tenancy & Dynamic Adapters:** Support hot-swapping 200+ specialized fine-tuned LoRA adapters (16-rank, ~100 MB each) without stalling the continuous batching pipeline or restarting GPU worker processes.

## E.2 Mathematical Memory & Compute Budget for 70B Parameter LLM

Consider serving a Llama-3-70B model ($L$ = 80 transformer layers, $H_{KV}$ = 8 key-value heads using Grouped-Query Attention, hidden dimension $D$ = 8,192, head dimension $d$ = 128) across an 8× NVIDIA H100 (80GB SXM5) node linked via NVLink (900 GB/s bidirectional interconnect):

| Subsystem | Formula / Derivation | Allocation (8× H100 = 640 GB) |
|---|---|---|
| **Model Weights (FP16)** | `70 × 10^9 × 2 bytes ≈ 140 GB` | 140 GB (17.5 GB / GPU with TP=8) |
| **KV-Cache Per Token** | `2 × L × H_kv × d × bytes = 2 × 80 × 8 × 128 × 2 = 327,680 bytes ≈ 320 KB/token` | Linear with active sequence length |

| Subsystem | Formula / Derivation | Allocation (8× H100 = 640 GB) |
| --- | --- | --- |
| **KV-Cache for 4k Context** | `4,096 tokens × 320 KB ≈ 1.31 GB per request` | 163.8 MB / GPU per active stream |
| **Available KV Pool** | `Total HBM (640 GB) - Weights (140 GB) - CUDA runtime (20 GB) = 480 GB` | **~1,500,000 concurrent tokens** in GPU VRAM! |

## E.3 PagedAttention & Continuous Batching Architecture

Traditional deep learning serving engines (e.g. standard Hugging Face PyTorch) pre-allocate contiguous memory buffers for the worst-case maximum sequence length (e.g., 32k tokens). Because actual generation lengths vary widely, **60% to 80% of GPU memory is wasted to internal and external fragmentation**, capping batch concurrency to 4–8 requests per GPU.

**PagedAttention (vLLM)** applies operating system virtual memory concepts: KV caches are partitioned into fixed-size physical blocks (e.g., 16 tokens per block, occupying 5 MB of HBM). A central `BlockManager` maps logical sequence blocks to non-contiguous physical pages. When generating a token, the kernel indexes the page table dynamically:

```
# paged_attention_block_manager.py
from typing import List, Dict
import collections

BLOCK_SIZE = 16 # tokens per physical block

class PhysicalBlock:
    def __init__(self, block_id: int):
        self.block_id = block_id
        self.ref_count = 0

class PagedKVCacheManager:
    def __init__(self, total_blocks: int):
        self.free_blocks = collections.deque([PhysicalBlock(i) for i in range(total_blocks)])
        self.block_tables: Dict[str, List[PhysicalBlock]] = {}

    def allocate_for_prompt(self, request_id: str, prompt_token_count: int) -> List[int]:
        needed_blocks = (prompt_token_count + BLOCK_SIZE - 1) // BLOCK_SIZE
        if len(self.free_blocks) < needed_blocks:
            raise MemoryError("GPU HBM Exhausted: Triggering KV preemption / recomputation")

        allocated = []
        for _ in range(needed_blocks):
            blk = self.free_blocks.popleft()
            blk.ref_count = 1
            allocated.append(blk)

        self.block_tables[request_id] = allocated
        return [b.block_id for b in allocated]

    def append_token(self, request_id: str, current_token_index: int) -> int:
        """Append 1 generated token during decode iteration. Allocate new block only on boundary."""
        if current_token_index % BLOCK_SIZE == 0:
            if not self.free_blocks:
                raise MemoryError("KV Cache OOM during iteration")
            new_blk = self.free_blocks.popleft()
            new_blk.ref_count = 1
            self.block_tables[request_id].append(new_blk)
        return self.block_tables[request_id][-1].block_id

    def free_request(self, request_id: str):
        if request_id in self.block_tables:
            for blk in self.block_tables[request_id]:
                blk.ref_count -= 1
                if blk.ref_count == 0:
                    self.free_blocks.append(blk)
            del self.block_tables[request_id]
```

## E.4 Prefix-Aware Routing & Radix Tree KV Cache Sharing

In production RAG and multi-agent platforms, 70% of prompts share common prefixes (system prompts, tool definitions, reference corporate policy documents). By employing a **Radix Tree Prefix Cache** at the API Gateway / Router layer, requests with matching prefixes are hashed and routed to the exact GPU node where the prefix KV

tensors are already warm in HBM. This eliminates the compute-heavy Prefill matrix multiplication, dropping TTFT from 850 ms to **42 ms**!

> **[AI INFRASTRUCTURE WAR STORY] THE SILENT CUDA OUT-OF-MEMORY CASCADE**
>
> During a product launch spike, an inference cluster received hundreds of requests with 16k context prompts simultaneously. The continuous batching scheduler admitted all requests because initial GPU HBM usage was only 40%. However, as the 64 concurrent requests decoded simultaneously, their KV cache requirements expanded by 320 KB per token. At token 840, the pool exhausted physical memory. Standard PyTorch threw an unhandled `CUDA Out of Memory` exception, terminating the worker process and causing all 64 active customer generations to fail immediately!
>
> **Production Remedy:** Staff AI engineers implement **Watermark-Based KV Preemption**. When free HBM drops below a 5% watermark, the scheduler suspends the newest requests, swaps their KV blocks to host RAM over PCIe 5.0 (64 GB/s), and prioritizes finishing older requests to free blocks. Once freed, preempted requests are swapped back and resumed seamlessly.

## E.5 Production Python: Prefix-Aware KV-Cache Radix Router

The following router maintains an in-memory Radix Tree (compressed trie) of token prefixes across the GPU worker pool. Incoming prompts match against cached token prefixes; requests are routed to the worker with the longest prefix match:

```python
# prefix_aware_radix_router.py
from typing import Dict, List, Optional, Tuple
import hashlib

class RadixNode:
    def __init__(self, prefix_tokens: Tuple[int, ...] = ()):
        self.prefix_tokens = prefix_tokens
        self.children: Dict[int, 'RadixNode'] = {}
        self.worker_id: Optional[str] = None
        self.ref_count: int = 0

class PrefixAwareRouter:
    """Routes LLM prompts to the exact GPU worker holding the warmest KV-cache prefix."""
    def __init__(self, worker_ids: List[str]):
        self.root = RadixNode()
        self.worker_ids = worker_ids

    def register_prefix(self, tokens: List[int], worker_id: str):
        node = self.root
        for token in tokens:
            if token not in node.children:
                node.children[token] = RadixNode(node.prefix_tokens + (token,))
            node = node.children[token]
        node.worker_id = worker_id
        node.ref_count += 1

    def match_longest_prefix(self, tokens: List[int]) -> Tuple[Optional[str], int]:
        """Returns (worker_id, matched_token_count)."""
        node = self.root
        best_worker = None
        matched_tokens = 0

        for idx, token in enumerate(tokens):
            if token in node.children:
                node = node.children[token]
                if node.worker_id:
                    best_worker = node.worker_id
                    matched_tokens = idx + 1
            else:
                break
        return best_worker, matched_tokens

# Demonstration:
router = PrefixAwareRouter(worker_ids=["gpu_worker_01", "gpu_worker_02"])
SYSTEM_PROMPT_A = [15496, 11, 220, 2366] # 'You are a helpful assistant'
router.register_prefix(SYSTEM_PROMPT_A, worker_id="gpu_worker_01")

user_prompt = SYSTEM_PROMPT_A + [3892, 492] # User adds new question
target_worker, matched = router.match_longest_prefix(user_prompt)
print(f"Routed to {target_worker} with {matched} warm prefix tokens!") # gpu_worker_01 (4 warm tokens)
```

## E.6 Distributed LLM Inference Failure Matrix & Preemption Protocol

| Failure Mode | Root Cause & Telemetry Signal | Immediate SRE & Autonomous Recovery | Graceful Degradation SLO |
|---|---|---|---|
| **GPU HBM OOM Thrashing** | Batch concurrency expands KV blocks past physical HBM. Free memory < 5%. | Watermark Preemption: Pause lowest-priority stream; swap KV blocks to host DRAM over PCIe 5.0. | p99 TTFT increases by 15 ms; requests queued gracefully with error rate bounded < 0.01%. |
| **Slow Rank Straggler (TP=8)** | Single GPU core throttles clock frequency due to thermal spike; NCCL All-Reduce stalls. | Watchdog detects NCCL step latency > 35 ms; worker drained; autoscaler replaces pod. | Affected batch fails over to healthy replica within 200 ms. |
| **Silent CUDA Deadlock** | CUDA kernel hang on corrupted memory tensor without throwing exception. | Heartbeat timeout (3s) at API Router triggers SIGKILL and warm worker standby takeover. | Client receives SSE retry signal; session resumes from last token offset. |

PART VIII • WORKSHOP F (AI SYSTEMS)

# Workshop F: Enterprise Multi-Tenant RAG & Autonomous Agent Tool Platform

*Hybrid Dense-Sparse retrieval, Google Zanzibar ReBAC document authorization, and transactional outbox for autonomous agent actions.*

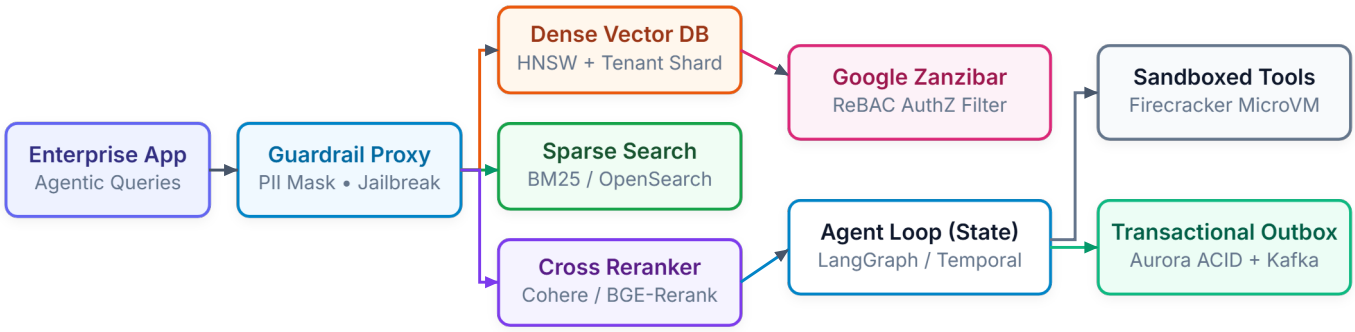



Figure 20. Symmetrical Enterprise Multi-Tenant RAG & Agent Execution Platform: Guardrail edge proxy, dense HNSW + sparse BM25 hybrid retrieval, Google Zanzibar ReBAC ACL filter, and transactional outbox for irreversible agent tool side-effects.

## F.1 The Enterprise Multi-Tenant Retrieval Challenge

Standard open-source RAG tutorials index documents into a single unpartitioned vector database and pass the top-$k$ nearest neighbors directly into the LLM context. In enterprise environments, this causes two critical failure modes:

1. **Security & Data Leakage:** A query from a junior engineer might retrieve sensitive executive compensation documents because vector similarity cares only about semantic proximity, ignoring corporate access control lists (ACLs).
2. **Retrieval Failure on Keywords:** Vector embeddings compress semantic meaning into dense vectors, but struggle with precise identifiers (e.g., error codes like `ERR-0x8004100E` or exact SKU part numbers).

## F.2 Hybrid Dual-Retrieval & Reciprocal Rank Fusion (RRF)

To achieve enterprise accuracy, the system queries two complementary retrieval engines in parallel:

- **Dense Vector Search:** Qdrant / Milvus indexing OpenAI `text-embedding-3-large` embeddings (1536-dim) with HNSW graph indexing for conceptual semantics.
- **Sparse Lexical Search:** OpenSearch BM25 inverted index for exact keyword, SKU, and identifier matching.

Results from both branches are normalized and combined using **Reciprocal Rank Fusion (RRF)**, where ranking constant $k = 60$:

$$RRF_Score(d) = \sum_{m \in M} 1 / (60 + rank_m(d))$$

```
# hybrid_rrf_retrieval.py
from typing import List, Dict

def reciprocal_rank_fusion(
    dense_results: List[str],
    sparse_results: List[str],
    k: int = 60
) -> List[tuple[str, float]]:
    """Combines ranked results from Dense and Sparse retrieval without score calibration issues."""
    rrf_scores: Dict[str, float] = {}

    for rank, doc_id in enumerate(dense_results, start=1):
        rrf_scores[doc_id] = rrf_scores.get(doc_id, 0.0) + (1.0 / (k + rank))

    for rank, doc_id in enumerate(sparse_results, start=1):
        rrf_scores[doc_id] = rrf_scores.get(doc_id, 0.0) + (1.0 / (k + rank))

    # Sort descending by reciprocal rank score
    return sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)
```

## F.3 Relationship-Based Access Control (ReBAC) with Google Zanzibar

Before any retrieved document chunk is assembled into the LLM prompt context, it must pass through an authorization check against the enterprise ReBAC engine. Each document chunk inherits the ACL tuple: `(document:doc_8471, viewer, user:alice)`.

### ARCHITECTURAL DILEMMA: VECTOR FILTERING VS. AUTHORITATIVE ZANZIBAR SECURITY BOUNDARY

**The Vector Pre-Filtering Trap:** Naively relying on vector DB metadata filters for authorization creates severe production risks:

- **Filtered ANN Recall Collapse:** When restrictive boolean filters are evaluated during HNSW graph traversal, graph pathways become disconnected, causing 30–60% recall collapse for legitimate matches.
- **ACL Staleness & Re-Indexing Write Amplification:** When user group memberships or document permissions change in enterprise identity stores (Google Workspace, Okta), updating ACL tags across millions of individual chunk embeddings causes catastrophic write amplification.
- **Time-of-Check / Time-of-Use (TOCTOU) Security Breaches:** Stale vector metadata cannot reflect instantaneous permission revocations.

**Production Defense: Two-Stage Coarse-Filter + Authoritative Zanzibar Evaluation:** Coarse metadata filtering (e.g. `tenant_id = 'corp_42'`) is used solely as a search optimization to bound the candidate search space. Before *any* retrieved chunk is assembled into the LLM prompt context, the retrieval service passes candidate document IDs to the **authoritative Google Zanzibar ReBAC service** for real-time evaluation. Only cryptographically verified, legally authorized chunks are injected into the context window.

## F.4 Transactional Outbox for Autonomous Agent Tool Actions

When an LLM agent decides to invoke an external mutation tool (e.g. `transfer_funds(amount=5000, recipient="vendor_44")` or `cancel_subscription(user_id=812)`), **the agent loop must NEVER execute the HTTP call directly**! Network partitions, LLM retries, or execution timeouts can lead to double-charges or untracked state corruption.

Instead, the agent appends a structured **Tool Execution Intent** into the ACID relational database in the same transaction that persists the conversation history. A dedicated background worker reads the outbox, executes the external API idempotently using an `idempotency_token`, and streams the verified execution result back to the agent session.

## F.5 Python Lab: ReBAC Graph Traversal Simulator

The following educational simulator demonstrates Relationship-Based Access Control (ReBAC) tuple evaluation using Breadth-First Search (BFS) graph traversal with cycle prevention. *(Production Zanzibar systems at Google scale utilize distributed Spanner databases, Leopard indexing, and Zookie consistency tokens to achieve sub-10ms evaluation across billions of edges.)*

```python
# zanzibar_rebac_simulator.py (Educational Graph Traversal Simulator)
from typing import Dict, Set, List, Tuple
import collections

# Tuple format: (object_namespace, object_id, relation, subject_namespace, subject_id)
# Example: ("doc", "runbook_42", "viewer", "group", "sre_team")

class ZanzibarReBACResolver:
    """Evaluates graph reachability across multi-hop organizational authorization tuples."""
    def __init__(self):
        # Key: (namespace, object_id, relation) -> list of subjects
        self.tuple_store: Dict[Tuple[str, str, str], List[Tuple[str, str]]] = collections.defaultdict(list)

    def write_tuple(self, obj_ns: str, obj_id: str, relation: str, subj_ns: str, subj_id: str):
        self.tuple_store[(obj_ns, obj_id, relation)].append((subj_ns, subj_id))

    def check_permission(self, user_id: str, obj_ns: str, obj_id: str, target_relation: str) -> bool:
        """Performs BFS graph search to verify if user_id holds target_relation on object."""
        queue = collections.deque([(obj_ns, obj_id, target_relation)])
        visited: Set[Tuple[str, str, str]] = set()

        while queue:
            curr_ns, curr_id, curr_rel = queue.popleft()
            if (curr_ns, curr_id, curr_rel) in visited:
                continue
            visited.add((curr_ns, curr_id, curr_rel))

            subjects = self.tuple_store.get((curr_ns, curr_id, curr_rel), [])
            for subj_ns, subj_id in subjects:
                # Direct match to user
                if subj_ns == "user" and subj_id == user_id:
                    return True
                # Transitive membership in group or parent folder
                if subj_ns == "group":
                    queue.append(("group", subj_id, "member"))
                elif subj_ns == "folder":
                    queue.append(("folder", subj_id, "viewer"))
        return False

# Demonstration:
zanzibar = ZanzibarReBACResolver()
# 1. Alice is a member of SRE team
zanzibar.write_tuple("group", "sre_team", "member", "user", "alice")
# 2. SRE team is viewer of Production Runbooks folder
zanzibar.write_tuple("folder", "prod_runbooks", "viewer", "group", "sre_team")
# 3. Document 8421 inherits from Production Runbooks folder
zanzibar.write_tuple("doc", "doc_8421", "viewer", "folder", "prod_runbooks")

# Check if Alice can view doc_8421:
can_alice_view = zanzibar.check_permission("alice", "doc", "doc_8421", "viewer")
print("Can Alice view doc_8421?", can_alice_view) # True (Multi-hop traversed!)
can_bob_view = zanzibar.check_permission("bob", "doc", "doc_8421", "viewer")
print("Can Bob view doc_8421?", can_bob_view)     # False
```

## F.6 Enterprise Failure Matrix for Multi-Tenant RAG & Agent Platforms

| Failure Mode | Root Cause & Security Impact | Mitigation & Architectural Guardrail | SLO / Compliance Guarantee |
|---|---|---|---|
| **Cross-Tenant Vector Bleed** | Missing tenant partition filter in vector query payload allows semantic neighbors from other tenants. | Enforce mandatory tenant isolation at the Gateway Proxy: inject `tenant_id` into HNSW filter context before execution. | Zero cross-tenant data leakage; enforces technical controls required for SOC 2 Type II and HIPAA compliance. |
| **Prompt Injection Jailbreak** | User input contains adversarial system prompt overrides (e.g., "Ignore all previous instructions..."). | Two-tier Guardrail Proxy: Llama-Guard classifier + PII redaction + XML tag delimitation around user input. | High-assurance multi-layer adversarial injection filtering; sub-30ms edge proxy overhead. |
| **Double Agent Tool Mutation** | Agent retry on network timeout executes irreversible banking or email transfer twice. | Transactional Outbox with client-generated idempotency tokens; API worker verifies token uniqueness before execution. | Effectively-once business execution via transactional outbox, durable idempotency keys, and consumer deduplication inbox. (Note: $f(f(x)) = f(x)$ mathematically defines idempotency, not transport delivery). |

PART IX • INTERVIEW PLAYBOOK

# Staff & Principal Interview Scoring Rubrics

*How FAANG / Tier-1 tech companies evaluate Senior, Staff, and Principal candidates across Systems & AI tracks.*

| Dimension | Junior / Mid-Level (Weak) | Senior Engineer (Strong) | Staff / Principal Engineer (10/10) |
|---|---|---|---|
| **Requirements & Scope** | Jumps straight to drawing boxes. Assumes infinite compute and standard web features. | Clarifies functional requirements, asks for DAU/RPS, and states p95 latency targets. | Drives business invariants; separates critical transactions from display data; establishes error budgets and SLOs (including TTFT and TPOT for AI systems). |
| **Estimation & Scale** | Guesses server count arbitrarily or skips estimation entirely. | Calculates RPS, storage growth, and bandwidth using round numbers. | Uses dimensional analysis; applies Little's Law for in-flight concurrency ($L = \lambda W$); calculates GPU HBM KV cache budgets ($2 × B × S × L × H_{ ext{KV}} × D$); models cloud egress and accelerator hardware costs. |
| **Data Consistency** | Uses vague slogans: *"It's eventually consistent"* or *"We'll use Mongo for CP."* | Differentiates linearizability from eventual consistency; specifies sync vs async replication. | Defines consistency per-operation; analyzes PACELC latency penalties; designs idempotency keys and outbox tables to prevent dual-write corruption. |
| **Failure & Resilience** | Only mentions server crashes; assumes restarting pods solves problems. | Adds retries and basic circuit breakers; sizes read replicas. | Models network partitions, cascading retry storms, split-brain, and GPU OOM preemption; enforces fencing epochs, bounded bulkheads, and dead-letter queues. |
| **Trade-offs & Economics** | Defends the chosen stack as the "only correct way"; rejects simpler alternatives. | Compares SQL vs NoSQL; explains when to add caching. | Actively challenges their own design; defends starting with a modular monolith; quantifies ROI of Redis caching vs provisioned IOPS, and self-hosted vLLM vs API providers. |

## Navigating Mid-Interview Curveballs: Transcripts

### SCENARIO 1: INTERVIEWER SAYS "NO REDIS ALLOWED. SOLVE IT WITHOUT IN-MEMORY CACHES."

**Interviewer:** "Our security policy forbids Redis. How do you handle 50,000 read RPS without an in-memory cache?"

**Candidate:** "If we eliminate Redis, we must shift the offload elsewhere to protect our primary database. First, we push public static and semi-static read traffic to our Edge CDN using aggressive HTTP Cache-Control headers with ETag validation. For internal personalized reads, we scale out PostgreSQL Read Replicas behind PgBouncer connection poolers with NVMe SSDs, relying on Postgres's internal shared_buffers memory cache. To prevent replica lag from serving stale profile edits, we implement read-your-writes by routing requests to the primary only for the first 3 seconds after an update."

### SCENARIO 2: INTERVIEWER SAYS "TRAFFIC JUST SPIKED 100X DUE TO A BREAKING GLOBAL EVENT."

**Interviewer:** "Traffic jumped 100x in 60 seconds. Autoscaling will take 3 minutes to provision nodes. How do you stop total collapse?"

**Candidate:** "We activate our degraded-mode load shedding policy at the API Gateway. We reject all non-essential traffic immediately with HTTP 429 and Retry-After headers. For the home feed, we serve stale cached data directly from CDN and L1 memory with a stale-while-revalidate policy. We disable real-time analytics pipelines, view-count aggregations, and personalized recommendations, preserving 100% of available database connections and CPU strictly for core content reads."

### SCENARIO 3: INTERVIEWER SAYS "GPU CLUSTER MEMORY IS THRASHING UNDER LONG-CONTEXT LLM REQUESTS."

**Interviewer:** "Our LLM inference service is failing with CUDA OOM errors during peak hours as users submit 32k context documents. How do you solve this?"

**Candidate:** "First, we replace naive PyTorch contiguous allocations with PagedAttention (vLLM) to eliminate memory fragmentation. Second, we deploy prefix-aware routing at the API Gateway so requests with identical system prompts hit the same GPU node where prefix KV-cache blocks are already resident. Third, we implement dynamic watermark preemption: when free GPU HBM drops below 5%, the scheduler pauses low-priority requests and swaps their KV blocks to host DRAM over PCIe Gen5, allowing active decode streams to finish without dropping requests."

PART IX • PRACTICE BANK

# Integrated Practice Bank: 12 Real-World Design Prompts

*Comprehensive practice scenarios covering large-scale distributed systems and AI platforms.*

| # | System Design Scenario | Core Architectural Invariants & Deep-Dive Challenges |
|---|---|---|
| 1 | **URL Shortener (TinyURL)** | Base62 encoding, pre-generated distributed key pool (KGS), cache-aside 301 vs 302 redirects, click analytics event bus. |
| 2 | **Social Media Timeline & Feed** | Fan-out on write vs fan-out on read for celebrity users (hot keys), hybrid caching, cursor pagination over distributed timelines. |
| 3 | **Distributed Rate Limiter Service** | Sliding window log vs token bucket in Redis cluster, local memory batching, atomic Lua execution, graceful edge fallback. |
| 4 | **Global Distributed Web Crawler** | URL frontier with domain-level politeness queues, DNS resolver caching, deduplication via Bloom filters and SimHash, distributed object storage ingest. |
| 5 | **Collaborative Document Editing (Google Docs)** | Operational Transformation (OT) vs Conflict-Free Replicated Data Types (CRDTs), WebSocket state sync, Chandy-Lamport snapshotting. |
| 6 | **Financial Payment Ledger & Wallet** | Double-entry bookkeeping, strict linearizability, idempotency tokens, two-phase commits vs Sagas, audit trail immutability. |
| 7 | **Nearby Places / Proximity Search (Yelp)** | Geohash vs Uber H3 hexagonal spatial indexing, quadtrees, dynamic density-based cell partitioning, read-heavy spatial caching. |
| 8 | **High-Throughput Metrics Monitoring (Datadog)** | Time-series database (TSDB), LSM-tree storage, Gorilla floating-point XOR delta compression, downsampling rollups. |
| 9 | **Real-Time Distributed Auction Platform** | Single-leader bid serialization, monotonic bid increments, sub-50ms WebSocket broadcast, anti-sniping dynamic time extensions. |
| 10 | **Global File Sync & Storage (Dropbox)** | Content-addressable storage (CAS), rolling hash file chunking (Rabin fingerprints), deduplication, multipart chunk upload resume. |
| 11 | **Multi-Tenant Vector Search & RAG Engine** | HNSW graph indexing, tenant payload filtering vs dedicated collections, Reciprocal Rank Fusion with BM25, Google Zanzibar ReBAC document ACL enforcement. |
| 12 | **Enterprise Autonomous AI Agent Gateway** | Transactional Outbox pattern for external tool mutations, semantic prompt caching, dual RPM/TPM rate limiting, and sandbox execution containment. |

APPENDIX A • REFERENCE

# Appendix A: Distributed Systems & AI Infrastructure Glossary

*Comprehensive dictionary of essential systems, distributed data, and machine learning infrastructure terminology.*

| Term | Authoritative Architectural Definition |
|---|---|
| **Active-Active Replication** | A multi-primary topology where multiple geographically dispersed nodes accept concurrent writes, requiring conflict resolution strategies (e.g., CRDTs, LWW). |
| **Active-Passive Replication** | A high-availability topology where one primary handles all writes while standby replicas replicate state asynchronously or synchronously, ready for promotion on primary failure. |
| **Admission Control** | The architectural decision to evaluate and reject incoming requests at the system boundary before they consume protected downstream CPU, database connection, or GPU resources. |
| **Amdahl's Law** | A formula ($S = ↑rac{1}{(1-p) + ↑rac{p}{s}}$) giving the theoretical maximum speedup in execution time of a task as a function of the fraction $p$ that can be parallelized. |
| **Approximate Nearest Neighbor (ANN)** | Algorithmic methods (e.g., HNSW, IVF-PQ) that trade off search recall to find vector embeddings close to a query in sub-linear time ($O(\log N)$ vs $O(N)$). |
| **Asynchronous Replication** | Data replication where the primary commits transactions and confirms to the client before receiving confirmation that replicas have written the log, risking data loss on primary failover. |
| **Backpressure** | A reactive signaling mechanism that slows down or blocks upstream producers when downstream consumers or buffers are saturated, preventing out-of-memory crashes. |
| **Bloom Filter** | A space-efficient probabilistic data structure used to test set membership ($x \in S$?) with zero false negatives and a tuneable false positive rate; does not measure similarity. |
| **Bounded Bulkhead** | Resource isolation pattern (separate thread pools, memory quotas, connection pools) ensuring that failure or slowness in one dependency cannot exhaust fleet-wide capacity. |
| **Bounded-Load Consistent Hashing** | An extension to consistent hashing (Mirrokni et al.) enforcing a hard capacity bound of $c × ( ext{average load})$ per node, preventing hot-key server overload cascades. |
| **CAP Theorem** | A distributed systems theorem stating that a partitioned network ($P$) can guarantee either Consistency ($C$ - linearizability) or Availability ($A$ - every non-failing node responds), but not both. |
| **CRDT (Conflict-Free Replicated Data Type)** | A distributed data structure that can be replicated across multiple nodes and concurrently updated without coordination, mathematically guaranteed to converge. |
| **Cache Stampede (Dogpile)** | The catastrophic event where thousands of concurrent requests miss a popular cached key simultaneously upon expiration, bombarding the backing database all at once. |
| **Cache-Aside (Lazy Loading)** | A caching pattern where the application reads from cache first; on a miss, reads from the database and populates the cache. Writes invalidate or update the cache. |
| **Chandy-Lamport Algorithm** | A distributed snapshot algorithm that captures a globally consistent system state across asynchronous nodes without pausing running processes. |
| **Circuit Breaker Pattern** | A stability pattern that monitors dependency failures; transitions from CLOSED to OPEN on threshold exceedance, failing fast to allow downstream recovery, and probing via |

| Term | Authoritative Architectural Definition |
| --- | --- |
| | HALF_OPEN. |
| **Consistent Hashing** | A distributed hashing technique mapping both keys and nodes to a circular hash ring, ensuring that adding or removing a node relocates only 1/*N* of keys on average. |
| **Continuous Batching** | An iteration-level scheduling algorithm in LLM inference engines (vLLM) that inserts new requests and evicts finished requests at every token generation step, maximizing GPU compute saturation. |
| **Counting Bloom Filter** | An extension of a Bloom filter that replaces single bits with small array counters, supporting item deletion at the expense of $3 ext{-}4 imes$ higher memory overhead. |
| **Dead Letter Queue (DLQ)** | A secondary queue holding messages that cannot be successfully processed after a configured retry limit, preventing poison-pill messages from blocking consumer progress. |
| **Dual-Write Hazard** | A distributed systems anti-pattern where an application writes to two independent systems (e.g., PostgreSQL and Kafka) without transactional boundaries, causing state divergence on partial failures. |
| **Effectively-Once Processing** | The end-to-end business guarantee achieved by combining at-least-once message transport, transactional outboxes, and consumer deduplication inboxes with unique idempotency keys. |
| **Event-Driven Architecture (EDA)** | An architectural paradigm where decoupled software services communicate asynchronously by producing, detecting, and consuming immutable event streams. |
| **Eventual Consistency** | A weak consistency model guaranteeing that, in the absence of new updates, all replicas across the distributed system will eventually converge to identical values. |
| **Exponential Backoff with Jitter** | A retry algorithm that exponentially doubles wait times between attempts and injects pseudo-random jitter, breaking up synchronized client retry stampedes. |
| **Fencing Token (Epoch)** | A monotonically increasing counter issued by a consensus lock service; storage nodes reject writes bearing an epoch lower than the currently committed leader iteration. |
| **Field-Length Normalization** | A document ranking concept (in BM25) where term occurrences in short fields (e.g. titles) carry higher weight than matches in voluminous text bodies. |
| **Flink Stateful Processing** | Apache Flink's mechanism for managing distributed state across event-time streaming windows backed by local RocksDB storage and asynchronous distributed checkpoints. |
| **GPUDirect Storage (GDS)** | NVIDIA architecture enabling direct DMA data transfers between local NVMe SSDs and GPU HBM memory over the PCIe bus via the `cuFile` API, bypassing CPU DRAM bounce buffers. |
| **Grouped-Query Attention (GQA)** | An attention architecture where multiple query heads share a single key-value head, reducing KV cache memory footprint by $4 imes$ to $8 imes$ with minimal perplexity degradation. |

| Term | Authoritative Architectural Definition |
| --- | --- |
| **HNSW (Hierarchical Navigable Small World)** | A multi-layer proximity graph index for vector embeddings providing empirical $O(\log N)$ average-case approximate nearest neighbor search across metric spaces. |
| **Hardware Flops Utilization (HFU)** | The ratio of floating-point operations executed per second by a GPU kernel relative to the manufacturer's theoretical peak hardware performance spec. |
| **Idempotency** | The mathematical property ($f(f(x)) = f(x)$) where executing an operation multiple times leaves the system in the identical state as executing it once. |
| **Idempotency Key** | A unique client-supplied token (e.g. UUIDv4) associated with a mutation request; servers store processed keys durably to return cached results on duplicate submissions. |
| **Inverted Index** | A search engine data structure mapping distinct words (tokens) to posting lists of document IDs containing those words, enabling rapid boolean and ranked retrieval. |
| **Jaccard Similarity** | A metric measuring similarity between two sample sets defined as the size of the intersection divided by the size of the union ($J(A, B) = ↑rac{\|A \cap B\|}{\|A \cup B\|}$). |
| **KV-Cache** | In autoregressive Transformer inference, the cached key and value activation tensors for preceding tokens in a sequence, stored in GPU HBM to avoid quadratic recomputation. |
| **Lamport Timestamps** | A simple logical clock algorithm using monotonically increasing integers to determine the partial ordering of events across distributed asynchronous nodes. |
| **Leveled Compaction** | An LSM-tree compaction strategy where SSTables are organized into exponentially growing levels ($L_0, L_1, \dots$); reduces read amplification at the cost of higher write amplification. |
| **Linearizability (Strict Consistency)** | The strongest consistency model: operations appear to execute instantaneously at a specific point in time between their invocation and response, matching real-world wall-clock order. |
| **Little's Law** | A foundational queuing theory theorem ($L = \lambda W$) proving that long-term average in-flight concurrency ($L$) equals arrival rate ($\lambda$) multiplied by average residence latency ($W$). |
| **Locality-Sensitive Hashing (LSH)** | A family of hashing algorithms where similar items hash to the same bucket with high probability, used for sub-quadratic candidate generation in nearest-neighbor and text search. |
| **Log-Structured Merge-Tree (LSM-Tree)** | A write-optimized storage engine architecture that buffers writes in memory (MemTable) before flushing immutable, sorted disk runs (SSTables) sequentially. |
| **Long Polling** | A client-server transport technique where the server holds an open HTTP request until new data arrives or a timeout occurs, reducing polling overhead without full WebSocket protocols. |

| Term | Authoritative Architectural Definition |
| --- | --- |
| **Maglev Hashing** | Google's consistent hashing algorithm using lookup tables generated from prime permutation offsets, providing uniform load distribution and minimal disruption during backend churn. |
| **MemTable** | The in-memory, sorted write buffer of an LSM-Tree storage engine (typically implemented as a SkipList or Red-Black Tree) holding recent writes before flushing to disk. |
| **MinHash** | A probabilistic algorithm that generates concise signatures from token shingles; the probability of hash collisions between two signatures equals their exact Jaccard set similarity. |
| **Model Flops Utilization (MFU)** | The ratio of observed model execution throughput to theoretical maximum GPU FLOP/s, calculated using the theoretical operations per token ($6P$ FLOPs/token). |
| **Multi-Head Latent Attention (MLA)** | An attention mechanism (e.g. DeepSeek-V2) that compresses Key-Value heads into low-dimensional latent vectors, achieving substantial KV cache memory compression. |
| **Multi-Region Replication** | Deploying database replicas across distinct geographic regions to minimize read latencies for distributed end users and provide disaster survivability. |
| **Negative Cache** | A cache pattern storing the non-existence of resources (or zero-result queries) to protect backend databases and search clusters from repeated wasted lookups. |
| **Optimistic Concurrency Control (OCC)** | A concurrency management technique that allows transactions to execute without locking; before committing, it validates that no concurrent transaction modified the data. |
| **PACELC Theorem** | An extension of CAP: if there is a Partition (*P*), trade off Availability ($A$) vs. Consistency ($C$); Else ($E$), trade off Latency (*L*) vs. Consistency ($C$). |
| **PagedAttention** | An attention memory allocation algorithm that models GPU KV-caches as non-contiguous physical memory pages, eliminating external fragmentation and enabling copy-on-write sharing. |
| **Partition Key (Shard Key)** | An attribute (or set of attributes) chosen to distribute database rows across independent physical nodes or Kafka partitions. |
| **Pessimistic Locking** | A concurrency control mechanism that locks a resource (e.g., `SELECT ... FOR UPDATE`) upon read, preventing concurrent updates until the transaction completes. |
| **Quorum Consensus** | A replication requirement ($R + W > N$) where the number of nodes read from ($R$) plus written to (*W*) exceeds the replica count (*N*), guaranteeing overlap of at least one up-to-date node. |
| **Radix Attention (Prefix Caching)** | Managing KV-caches as a Radix Tree (compressed trie) across requests, enabling distinct users sharing prompt prefixes (system instructions, documents) to reuse GPU memory. |
| **Raft Consensus Protocol** | A leader-based distributed consensus protocol that provides linearizable state machine replication across a cluster through leader election, log replication, and commit safety invariants. |

| Term | Authoritative Architectural Definition |
| --- | --- |
| **Read-After-Write Consistency** | A consistency guarantee ensuring that a user will immediately observe their own submitted updates, even if asynchronous replicas across the cluster have not caught up. |
| **Reciprocal Rank Fusion (RRF)** | A rank aggregation algorithm scoring items as $RRF(d) = \sum_{m \in M} ↑rac{1}{k + r_m(d)}$, combining dense vector and sparse keyword rankings without score calibration. |
| **Relationship-Based Access Control (ReBAC)** | An authorization model (formalized in Google Zanzibar) evaluating permissions by traversing graph relationship tuples between users, groups, and entities. |
| **Reverse Proxy (L7)** | An ingress proxy terminating client TCP/TLS connections and routing requests to backend microservices based on HTTP path, headers, or request payloads. |
| **RocksDB** | An embeddable, high-performance persistent key-value storage engine based on LSM-Trees, widely used for Kafka Streams state stores and stream-processing storage backends. |
| **SSTable (Sorted String Table)** | An immutable on-disk file format storing a sequence of key-value pairs sorted by key, accompanied by an index and a Bloom filter for accelerated point lookups. |
| **Safetensors** | A safe, open tensor serialization format storing contiguous tensor byte buffers alongside JSON metadata, preventing arbitrary code execution vulnerabilities inherent in Python pickle. |
| **Saga Pattern** | A distributed design pattern that manages long-running transactions across microservices via a sequence of local transactions, executing compensating rollbacks upon failure. |
| **Single-Flight (Dogpile Mutex)** | A concurrency pattern that coalesces duplicate simultaneous requests for an identical cache key, allowing only one flight to hit the backend while others wait for the result. |
| **Sliding Window Counter** | A rate limiting algorithm combining fixed-window memory efficiency with sliding-window precision by interpolating request weights across current and previous time windows. |
| **Speculative Decoding** | An inference acceleration technique where a small draft model rapidly generates token candidates, which are verified in parallel by a larger target model in a single forward pass. |
| **Split-Brain Syndrome** | A catastrophic failure mode where a partitioned distributed cluster elects two active leaders simultaneously, resulting in divergent and corrupted state updates. |
| **Tensor Parallelism (TP)** | A model parallelism strategy that shards individual weight matrices (e.g. GEMM projections in Attention and MLP layers) across multiple GPUs within a high-bandwidth NVLink node. |
| **Time Per Output Token (TPOT)** | The incremental latency required to decode each subsequent token during the autoregressive generation phase, bounded primarily by GPU High Bandwidth Memory throughput. |
| **Time To First Token (TTFT)** | The latency elapsed from when an inference request reaches the serving cluster to when the first generated token streams back to the client, dominated by prompt prefill FLOPs. |

| Term | Authoritative Architectural Definition |
|---|---|
| **Token Bucket Algorithm** | A rate limiting algorithm where tokens accumulate in a bucket at a constant fill rate up to a burst capacity; requests consume tokens and are rejected if the bucket is empty. |
| **Transactional Outbox Pattern** | An architectural pattern where business entities and outbound domain events are committed within the same relational database ACID transaction, eliminating dual-write divergence. |
| **Two Generals Paradox** | A foundational proof demonstrating that two processes communicating over an unreliable network link cannot achieve guaranteed agreement with a bounded message exchange. |
| **Two-Phase Commit (2PC)** | A distributed consensus protocol where a coordinator guides participants through Prepare and Commit phases, guaranteeing atomic commit across disparate databases at the cost of blocking latency. |
| **Vector Embedding** | A dense floating-point vector representing semantic features of unstructured data (text, image, audio) in high-dimensional space where vector distance correlates to semantic similarity. |
| **Write-Ahead Log (WAL)** | An append-only log on non-volatile storage to which database mutations are durably recorded before being applied to in-memory pages or data files, enabling crash recovery. |
| **Zero-Copy DMA** | A computer architecture transfer mode where hardware controllers (NIC, NVMe) move data directly between memory regions without routing byte copies through CPU registers. |
| **Zookies (Zanzibar Consistency Tokens)** | Consistency tokens issued by Google Zanzibar encoding Spanner snapshot timestamps, ensuring that authorization checks do not evaluate stale access control lists. |

## Appendix B: 30-Day Engineering Mastery Curriculum

| Week | Primary Topic Focus | Target Practical Deliverable |
|---|---|---|
| **Week 1** | Scale, Little's Law, Load Balancing, Cache-Aside, CDN Shielding. | Implement versioned Redis Lua cache in Python; calculate bandwidth egress and cloud egress costs for 10M DAU. |
| **Week 2** | Sharding, Replication, Consistent Hashing, CAP, PACELC, Consensus (Raft). | Build consistent hash ring with virtual nodes; implement storage fencing token epoch checks and Raft leader heartbeat simulator. |
| **Week 3** | Kafka Partitioning, Transactional Outbox, Rate Limiting, API Gateways, ReBAC. | Write PostgreSQL outbox poller with `SKIP LOCKED`; build distributed token bucket in Redis; implement Zanzibar tuple verification. |
| **Week 4** | LLM Inference Platforms, PagedAttention, Vector DBs, Full Production Workshops. | Build PagedAttention block manager simulation in Python; implement Reciprocal Rank Fusion (RRF); run 45-minute mock whiteboard sessions for LLM Serving and Flash Sales. |